

```
function 할일카드({ 제목, 끝났나, 눌렀을때 }) {  
  const [열림, 열기] = useState(false);  
  useEffect(() => { fetch("/api/할일").then((r) => r.json()); }, []);  
  return (  
    <li className={끝났나 ? "끝" : ""} onClick={눌렀을때}>  
      {제목}  
    </li>  
  );  
}
```

3 권 · 앱 만들기

React

화면을 컴포넌트로 쪼개고, state 로 바꾸고,
서버에서 받아 옵니다.

브라우저 화면이 어떻게 바뀌는지를 매번 적어 두었습니다.

5

단원

22

개념

102

직접 해보기

74

문제

차례

11	React Router	11
01	라우터 기본	12
02	Link 와 NavLink	15
03	URL 파라미터	25
04	중첩 라우트	34
05	이동과 없는 페이지	44
12	Context와 useReducer	67
01	props 내려주기의 한계	68
02	Context 기본	70
03	useReducer	77
04	Context 와 useReducer 함께	82
05	언제 쓰나	95
13	종합 프로젝트	129
01	할 일 목록	130
02	장바구니	139
03	사용자 검색	150
04	메모장	159
14	서버와 연동	165
01	내 서버에 붙기	166
02	폼을 useForm 으로	173
03	서버에 보내고 받기	181
04	파일 올리기	192
05	안 되는 경우들	205
15	부록 타입스크립트	223
02	기본 타입.tsx	232

03	props에 타입 붙이기.tsx	246
04	useState와 이벤트 타입.tsx	260
부록 가	연습문제·종합연습 정답	285
부록 나	심화문제 정답	355

여는 글

시작하기 전에

다 배우고 나면 이런 것을 직접 만듭니다. 지금은 무슨 코드인지 하나도 몰라도 됩니다.

이 책의 표시

블록마다 왼쪽 가장자리 표시가 다릅니다. 표시만 보고 “지금 내가 무엇을 해야 하는지” 알 수 있습니다.

```
console.log("안녕하세요");
```

— 직접 쳐 볼 코드입니다. 파일을 열고 그대로 따라 치세요.

출력 **안녕하세요**

— 실행하면 컴퓨터가 내놓는 값입니다. 내 화면과 같은지 맞춰 보세요.

▢ 직접 해보기 **1**

자기 이름을 출력해 보세요.

— **여러분 차례입니다.** 이 표시가 보이면 손을 움직이세요. 정답은 그 개념 맨 끝에 있습니다.

여기에 코드를 쓰세요

— 연필로 직접 적는 자리입니다.

이런 실수를 합니다

따옴표를 빠뜨림

```
console.log(안녕하세요);
```

따옴표가 없으면 컴퓨터는 ‘안녕하세요’라는 이름의 변수를 찾습니다.

— 여기서 많이 넘어집니다. 미리 보고 피해 가세요.

RUN node 개념01_console_log로_출력하기.js

— 개념마다 맨 위에 있습니다. 이대로 실행하면 됩니다.

준비물 — Node.js 하나

이 자료는 처음부터 끝까지 **실제 개발에서 쓰는 방식(Vite 프로젝트)** 으로 진행합니다. 그래서 첫 시간에 Node.js 를 확인하고 프로젝트를 한 번 켜는 것으로 시작합니다.

JS자료에서 이미 설치했다면 그대로 쓰면 됩니다. 없다면 <https://nodejs.org> 에서 **왼쪽 LTS** 버튼으로 받아 설치하세요. 설치 후 **터미널을 전부 닫고 새로 여세요.**

확인하는 법 — 터미널에 이렇게 칩니다.

```
node -v
npm -v
```

v22.14.0 / 11.16.0 처럼 숫자가 나오면 됩니다.

★ **Node 는 22 이상이어야 합니다.** 앞의 숫자가 22보다 작으면(예: v18.x, v20.x) 이 자료의 도구가 안 됩니다. <https://nodejs.org> 에서 LTS 를 새로 받아 설치하세요.

시작하는 다섯 줄

1. 터미널을 연다
2. React자료/실습프로젝트 폴더로 간다
3. npm install 라고 친다 ← 맨 처음 한 번만. 10초쯤 걸립니다
4. npm run dev 라고 친다
5. 브라우저에서 터미널에 뜬 주소(<http://localhost:5173/>)를 연다

3번은 맨 처음 한 번만 하면 됩니다. 이 자료가 쓰는 도구들을 인터넷에서 받아 오는 명령입니다. 받고 나면 node_modules 라는 폴더가 생기는데, 그게 도구 창고입니다. 그 뒤로는 4번(npm run dev)만 하면 됩니다.

인터넷이 필요합니다. 실습실에서 안 되면 강사에게 말하세요.

막히면 [실습프로젝트/src/00_개발환경_만들기/개념01_설치하고_켜기.md](#) 를 보세요. 터미널을 한 번도 안 써 본 사람 기준으로 썼습니다.

VS Code

코드를 쓰는 편집기입니다. <https://code.visualstudio.com> 에서 받으세요. **파일 → 폴더 열기**로 실습프로젝트 폴더를 여는 편이 훨씬 편합니다. **Ctrl + `** 로 터미널을 열면 이미 그 폴더에 서 있습니다.

3분만 만져 보고 오세요

화면이 떴으면 왼쪽 목록에서 [01_React_시작하기](#) / [개념01 왜 React를 쓰나](#) 를 고르세요.

같은 화면이 두 개 있습니다.

- ① 은 여러분이 JS자료에서 하던 방식으로 만든 것
- ② 는 React 로 만든 것

두 '답기' 버튼을 번갈아 눌러 보세요. **화면은 똑같이 움직입니다.** 그런데 F12 → Console 을 보면 이렇게 찍힙니다.

[JS 방식] 내가 직접 고친 곳: 3군데
[React 방식] 내가 직접 고친 곳: 0군데 (데이터만 바꿈)

이 차이가 이 자료의 전부입니다. 지금 코드가 하나도 안 읽혀도 괜찮습니다.

그리고 마지막(13단원)에는 **할 일 목록·장바구니·사용자 검색·메모장** 네 개를 직접 만듭니다. JS자료 13단원에서 만들었던 할 일 목록을, 이번에는 React 로 다시 만듭니다.

폴더 순서

단원	제목	어디에
00	개발환경 만들기	실습프로젝트 (읽는 문서 .md)
01	React 시작하기	실습프로젝트
02	JSX	실습프로젝트
03	컴포넌트와 props	실습프로젝트
04	state와 이벤트	실습프로젝트
05	조건부 렌더링과 리스트	실습프로젝트
06	폼과 입력	실습프로젝트
07	state 설계와 불변성	실습프로젝트
08	파일 나누기와 스타일링	실습프로젝트 (개념01은 읽는 문서)

09	useEffect와 API	실습프로젝트
10	useRef와 커스텀 훅	실습프로젝트
11	React Router	실습프로젝트
12	Context와 useReducer	실습프로젝트
13	종합 프로젝트	실습프로젝트
14	서버와 연동	실습프로젝트 (터미널 두 개)
15	부록 — 타입스크립트	실습프로젝트 (안 해도 됩니다)

앞 단원을 건너뛰면 뒤 단원이 안 읽힙니다. 순서대로 가세요.

14단원은 서버를 같이 켭니다

useForm 으로 폼을 만들고, FormData 로 파일을 올리고, 내가 만든 서버에 붙습니다. **PART 3(백엔드)**로 넘어가는 다리입니다.

```
터미널 ①  cd 실습프로젝트          → npm run dev
터미널 ②  cd 실습프로젝트/14단원_서버 → node 서버.js
```

서버는 설치할 것이 없습니다. Node 만 있으면 node 서버.js 로 바로 돍니다. 서버 코드는 읽지 않아도 됩니다. 만드는 법은 PART 3 에서 배웁니다.

자료는 전부 실습프로젝트/src 안에 있습니다

```
React자료/
├─ _검증도구/           ← 강사용. 학생은 볼 필요 없습니다
├─ 실습프로젝트/       ← ★ 자료는 전부 여기
│  ├─ 14단원_서버/     ← 14단원에서 같이 켜는 연습용 서버
│  └─ src/
│     ├─ _ui/           ← 자료가 쓰는 부품 (읽지 않아도 됩니다)
│     ├─ 00_개발환경_만들기/ ← 첫 시간에 읽는 문서 한 장 (.md)
│     ├─ 01_React_시작하기/
│     ├─ 02_JSX/
│     ├─ ...
│     └─ 15_부록_타입스크립트/
```

왼쪽 목록에 저절로 나타납니다. 단원 폴더에 .jsx 를 만들면 등록 없이 잡힙니다.

이렇게 진행합니다

[터미널] npm run dev 를 켜 둔 채로 그대로 둡니다 ← 건드리지 않습니다

[VS Code] src 안의 예제 파일을 열어 읽고 고칩니다

[브라우저] 왼쪽에서 예제를 고르고, 저장하면 저절로 바뀌는 화면을 봅니다

- **새로고침(F5)은 누르지 마세요.** 왼쪽에서 고른 예제가 풀립니다. 저장이면 충분합니다.
- **화면이 안 바뀌면 터미널부터 보세요.** 문법이 틀리면 브라우저까지 가지도 못합니다.

분량

단원	16개 (00 준비 + 01~14 + 부록 1개)
수업 파일	107개
개념 섹션	439개
 직접 해보기	367개
연습문제	220문항
표준 진행 시간	71.5시간

자세한 시간표와 강사용 안내는 [수업_진행_가이드.md](#) 에 있습니다.

React Router

OPEN 11_React_Router

```
function StateWayDemo() {  
  function showPage(name) {  
    setPage(name);  
  }  
  return (  

```

01 라우터 기본

02 Link 와 NavLink

03 URL 파라미터

04 중첩 라우트

05 이동과 없는 페이지

- 연습문제

라우터 기본

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

05단원에서 화면을 갈아 끼우는 법을 배웠습니다. 이런 코드였습니다.

```
{page === "home" ? <Home /> : <About />}
```

page 라는 state 를 바꾸면 화면이 바뀝니다. 잘 동작합니다. 그런데 이 방식으로 만든 앱에는 화면이 몇 개가 있든 주소가 하나뿐입니다. 주소가 하나라서 못 하는 일이 세 가지 생깁니다.

1. 지금 보고 있는 화면을 친구에게 링크로 보낼 수 없습니다.

주소를 복사해서 보내 봐야 상대는 첫 화면을 봅니다.

2. 브라우저의 뒤로 가기 버튼이 앱 안에서 동작하지 않습니다.

'소개'를 보다가 뒤로 가기를 누르면 앱 밖으로 나가 버립니다.

3. 새로고침하면 항상 첫 화면으로 돌아옵니다.

이 세 가지는 전부 "주소가 화면을 정하지 않기 때문"에 생깁니다. 이 단원에서는 반대로 만듭니다. 주소가 화면을 정하게 합니다. 그 일을 해 주는 도구가 react-router-dom 입니다.

★ 이 파일에서만 다른 점이 하나 있습니다. 진짜 앱은 BrowserRouter 를 앱 맨 바깥에 딱 한 번만 둡니다. 이 자료는 왼쪽 메뉴로 예제를 골라 보는 구조라서, 예제마다 하나씩 두었습니다. 자세한 이유는 섹션 5에 적어 두었습니다.

```
import { useState } from "react";
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";
import Summary from "../_ui/Summary.jsx";
```

지금까지의 방식 — state 로 화면 갈아 끼우기

섹션 1

아래는 05단원까지 배운 것만으로 만든 화면 전환입니다. 버튼을 누르면 page 가 바뀌고, page 에 따라 다른 문단이 그려집니다. 새로 배운 것은 하나도 없습니다. 무엇이 부족한지만 보면 됩니다.

```
function StateWayDemo() {
  const [page, setPage] = useState("home");

  function showPage(name) {
    setPage(name);
    console.log("[state 방식] 화면만 바꿨습니다. 주소는 그대로입니다.");
  }
}
```

F12 콘솔 [state 방식] 화면만 바꿨습니다. 주소는 그대로입니다.

```
}

return (
  <div className="demo"> <h3>① 지금까지의 방식 - state 로 갈아 끼우기
</h3> <button onClick={() => showPage("home")}>홈</button> <button onClick={() =>
showPage("about")}>소개</button> <button onClick={() => showPage("menu")}>메뉴
</button> <div className="output">
  {page === "home" && <p>홈 - 어서 오세요. 오늘도 좋은 하루 되세요.</p>}
  {page === "about" && <p>소개 - 2020년에 문을 연 작은 카페입니다.</p>}
  {page === "menu" && <p>메뉴 - 아메리카노 4000원 / 라떼 4500원 / 케이크 6000원
</p>}
</div> <p>
  화면은 잘 바뀝니다. 그런데 <strong>브라우저 주소창을 보세요.</strong> 한
글자도
  안 바뀝니다.
</p> </div>
);
}
```

▹ 직접 해보기

1

데모 ①의 버튼 세 개를 눌러 보세요. 누를 때마다 브라우저 주소창을 보고, 바뀌는지 확인하세요.

라우터를 들여오는 법

섹션 2

react-router-dom 은 React 가 기본으로 주는 것이 아닙니다. 따로 받아서 쓰는 라이브러리입니다. 진짜 프로젝트에서는 이렇게 받습니다.

```
npm install react-router-dom
// 터미널: added 5 packages ... 같은 줄이 나오면 끝입니다
```

이 실습프로젝트에는 이미 받아 두었습니다. 여러분은 아무것도 안 해도 됩니다. 그래서 이 파일 맨 위에서 바로 꺼내 쓸 수 있습니다.

```
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";
```

08단원에서 배운 이름 있는 import 그대로입니다. 다만 “./무엇.jsx” 같은 경로가 아니라 이름만 적었습니다. 경로 없이 이름만 적으면 “내가 만든 파일이 아니라 받아 둔 라이브러리”라는 뜻입니다.

꺼내 온 것이 하는 일은 이렇습니다.

BrowserRouter — 브라우저 주소를 지켜보는 상자입니다.

주소가 바뀌면 안에 있는 컴포넌트들에게 알려 줍니다.
이것으로 감싸지 않으면 아래 둘은 동작하지 않습니다.

Routes — 후보 중에서 하나만 고르는 곳입니다.

안에 든 Route 들을 보고, 지금 주소에 맞는 것 하나만 그립니다.
05단원의 조건부 렌더링을 대신한다고 보면 됩니다.

Route — 주소 하나와 화면 하나를 짝지어 둔 것입니다.

path 에 주소를, element 에 그릴 화면을 적습니다.

Link — 주소를 바꾸는 링크입니다. 개념02에서 자세히 배웁니다.

지금은 “누르면 저 주소로 가는 것” 정도로만 보세요.

▫ 직접 해보기

2

위 네 가지 중에서 ‘브라우저 주소가 바뀐 것을 알아채는’ 일을 맡은 것은 무엇일까요? (정답은 파일 맨 아래에)

BrowserRouter · Routes · Route 조립하기

섹션 3

화면 세 개를 그냥 컴포넌트로 만듭니다. 03단원에서 배운 그대로입니다. 라우터를 쓴다고 컴포넌트 만드는 법이 달라지지는 않습니다.

```
function HomePage() {  
  return <p>홈 - 어서 오세요. 오늘도 좋은 하루 되세요.</p>;  
}
```

화면 홈 - 어서 오세요. 오늘도 좋은 하루 되세요.

```
}  
  
function AboutPage() {  
  return <p>소개 - 2020년에 문을 연 작은 카페입니다.</p>;  
}  
  
function MenuPage() {  
  return (  
    <p>메뉴 - 맛있는 메뉴를 준비했습니다.</p>  
    <p>특집 메뉴 - 특별히 준비했습니다.</p>  
  )  
}
```

Link 와 NavLink

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

개념01에서 주소를 바꾸는 데 `<Link>` 를 썼습니다. 그때는 “누르면 저 주소로 가는 것” 이라고만 하고 넘어갔습니다.

그런데 주소를 바꾸는 방법은 이미 하나 알고 있습니다. HTML 의 `<a>` 입니다.

```
<a href="/r2/menu">메뉴</a>
<Link to="/r2/menu">메뉴</Link>
```

둘 다 누르면 `/r2/menu` 로 갑니다. 화면도 똑같이 나옵니다. 그러면 왜 `<Link>` 를 따로 쓸까요? 눈으로 보이는 결과가 같으니 말로만 들으면 와닿지 않습니다. 이 파일에서는 그 차이를 직접 재 봅니다.

재는 방법은 이렇습니다. 화면에 카운터를 하나 두었습니다. 이 카운터는 state 입니다. state 는 페이지가 새로 시작하면 처음 값으로 돌아갑니다. 그러니 카운터를 올려 둔 뒤 링크를 눌러 보면 됩니다. 숫자가 남아 있으면 페이지가 그대로인 것이고, 0이 되면 새로 시작한 것입니다.

★ 이 자료만의 사정: 진짜 앱은 `BrowserRouter` 를 앱 맨 바깥에 한 번만 둡니다. 이 자료는 왼쪽 메뉴로 예제를 골라 보는 구조라서 파일마다 하나씩 두었습니다. 그리고 파일마다 주소 앞머리가 다릅니다. 이 파일은 `/r2` 입니다.

```
import { useState } from "react";
import { BrowserRouter, Routes, Route, Link, NavLink } from "react-router-dom";
import Summary from "../_ui/Summary.jsx";
```

이 줄은 컴포넌트 밖에 있습니다. 그래서 화면을 다시 그릴 때는 실행되지 않습니다. 이 파일을 브라우저가 ‘처음 읽을 때’ 딱 한 번 실행됩니다. 즉 이 줄이 콘솔에 다시 찍힌다면, 앱이 통째로 새로 시작했다는 뜻입니다.

```
console.log("[개념02] 이 예제 파일을 새로 불러왔습니다");
```

```
F12 콘솔 [개념02] 이 예제 파일을 새로 불러왔습니다
```

<a> 로 이동하면 무슨 일이 일어나나

섹션 1

 를 누르면 브라우저는 이렇게 합니다.

- 1 지금 페이지를 버립니다.
- 2 서버에 /r2/menu 를 새로 달라고 요청합니다.
- 3 받아 온 것으로 페이지를 처음부터 다시 만듭니다.

HTML 을 다시 읽고, JS 파일을 다시 받고, React 를 다시 시작합니다.

3번에서 React 가 다시 시작하므로 state 가 전부 사라집니다. 담아 둔 장바구니도, 입력하던 글도, 스크롤 위치도 같이 사라집니다.

이 자료에서는 한 가지가 더 사라집니다. 왼쪽 메뉴의 선택입니다. 왼쪽 메뉴가 무엇을 고른 상태인지도 state 라서 함께 초기화됩니다. 그래서 <a> 링크를 누르면 이 예제에서 튕겨 나가 맨 위 예제가 열립니다. 고장난 것이 아닙니다. 왼쪽 메뉴에서 '개념02 Link와 NavLink' 를 다시 고르세요.

▹ 직접 해보기 1

데모 ① 에서 [+1] 을 세 번 눌러 카운터를 3으로 만드세요. 그다음 빨간 글씨의 <a> 링크를 누르고, 카운터와 왼쪽 메뉴를 보세요.

<Link> 로 이동하면 무슨 일이 일어나나

섹션 2

<Link to="/r2/menu"> 를 누르면 이렇게 됩니다.

1. Link 가 브라우저의 기본 동작(페이지 새로 받기)을 막습니다.

06단원에서 배운 e.preventDefault() 를 Link 가 대신 해 주는 것입니다.

- 2 주소창의 글자만 /r2/menu 로 바꿉니다.
- 3 BrowserRouter 가 그 변화를 알아채고 Routes 에게 알립니다.
- 4 Routes 가 맞는 Route 를 골라 그 자리만 다시 그립니다.

페이지를 새로 받지 않았으므로 React 는 계속 돌고 있습니다. 그래서 카운터도, 왼쪽 메뉴 선택도 그대로 남습니다.

화면이 깜빡이지 않는 것도 같은 이유입니다. 눌러 보면 <a> 는 화면이 한 번 하얘졌다 돌아오고, <Link> 는 글자만 바뀝니다.

▹ 직접 해보기 2

데모 ① 에서 [+1] 을 세 번 눌러 카운터를 3으로 만드세요. 그다음 파란 글씨의 <Link> 를 눌러 카운터를 보세요.

두 방식을 나란히 재 보기

섹션 3

아래 데모 ① 에 두 종류의 링크를 나란히 두었습니다. 가는 주소는 똑같습니다.

파란 글씨 = <Link to="..."> React 가 처리합니다 빨간 글씨 = 브라우저가 처리합니다

재 보는 것이 세 가지입니다.

[1] 왼쪽 메뉴 <Link> → 이 예제 그대로 / <a> → 맨 위 예제로 바뀜 [2] 카운터 <Link> → 숫자 그대로 / <a> → 돌아와 보면 0 [3] 콘솔 로그 <Link> → 안 찍힘 / <a> → 돌아오면 맨 위 줄이 다시 찍힘

[1]이 가장 먼저 눈에 띕니다. 빨간 링크를 누르면 이 예제가 화면에서 사라집니다. 왼쪽 메뉴에서 이 예제를 다시 고르면 [2]와 [3]을 확인할 수 있습니다.

[3]이 가장 확실한 증거입니다. 파일 맨 위의 `console.log` 는 컴포넌트 밖에 있어서 브라우저가 이 파일을 '처음 읽을 때' 만 실행됩니다. 그래서 <Link> 로 아무리 왔다 갔다 해도 다시 안 찍힙니다. 심지어 왼쪽 메뉴에서 다른 예제로 갔다가 돌아와도 안 찍힙니다. 이미 읽어 둔 파일이니까요. 그런데 <a> 로 이동한 뒤에는 다시 찍힙니다. 파일을 처음부터 다시 읽었다는 뜻입니다.

```
function HomeScreen() {
  return <p>홈 - 오늘의 커피는 아메리카노입니다.</p>;
}

function MenuScreen() {
  return (
    <ul> <li>아메리카노 4000원</li> <li>라떼 4500원</li> <li>케이크 6000원</li> </ul>
  );
}

function NoticeScreen() {
  return <p>공지 - 이번 주 화요일은 쉽니다.</p>;
}

function NotHereScreen() {
  return <p>이 예제가 아는 주소가 아닙니다. 위의 링크 중 하나를 눌러 시작하세요.</p>;
}

function CompareDemo() {
  const [count, setCount] = useState(0);

  function addOne() {
    setCount(count + 1);
    console.log("카운터를 눌렀습니다");
  }
}
```

F12 콘솔 카운터를 눌렀습니다

```

    }

    return (
      <div className="demo"> <h3>① Link 와 a 를 나란히</h3> <p>
        카운터: <strong>{count}</strong> <button onClick=
{addOne}>+1</button> </p> <nav> <p> <span style={{ color: "#2d6cdf" }}>Link 로 이동
</span> -{" "}
        <Link to="/r2">홈</Link> | <Link to="/r2/menu">메뉴</Link> |{" "}
        <Link to="/r2/notice">공지</Link> </p> <p> <span style={{ color: "#c00"
}}>a 로 이동 (누르면 앱이 다시 시작합니다)</span> -{" "}
        <a href="/r2">홈</a> | <a href="/r2/menu">메뉴</a> |{" "}
        <a href="/r2/notice">공지
</a> </p> </nav> <div className="output"> <Routes> <Route path="/r2" element=
{<HomeScreen />} />
        <Route path="/r2/menu" element={<MenuScreen />} />
        <Route path="/r2/notice" element={<NoticeScreen />} />
        <Route path="*" element={<NotHereScreen />} />
</Routes> </div> </div>
    );
  }

```

▸ 직접 해보기

3

콘솔을 비우고(휴지통 아이콘) 파란 링크를 세 번 눌러 보세요. “[개념02] ... 새로 불러왔습니다”가 다시 찍히는지 보세요.

NavLink — 지금 보고 있는 메뉴 표시하기

섹션 4

메뉴가 여러 개면 “지금 어디를 보고 있는지”를 표시해 주는 것이 친절합니다. Link 로도 할 수 있지만, 지금 주소를 직접 꺼내서 비교해야 합니다. NavLink 는 그 비교를 대신 해 줍니다.

```

<NavLink to="/r2/menu" className={({ isActive }) => (isActive ? "on" : "")}>
  메뉴
</NavLink>

```

NavLink 는 className 자리에 문자열 대신 함수를 넣을 수 있습니다. React 가 그 함수를 부르면서 { isActive: true } 같은 객체를 넘겨 줍니다. isActive 는 “지금 주소가 이 링크의 주소와 맞는가”입니다. 위 코드는 03단원에서 배운 매개변수 자리 구조분해로 그 값을 바로 꺼낸 것입니다.

여기서 쓴 “on” 은 이 자료의 공용 CSS 에 있는 클래스입니다(파란 배경 + 흰 글자). 08단원 개념05에서 본 그 index.css 안에 들어 있습니다.

— end 를 꼭 기억하세요

NavLink 는 기본적으로 '주소가 이 to 로 시작하면' 켜진 것으로 봅니다. 그래서 to="/r2" 인 홈 링크는 /r2/menu 에서도 켜져 버립니다. end 를 붙이면 '딱 이 주소일 때만' 으로 바뀝니다.

```
<NavLink to="/r2" end>홈</NavLink>
```

end 는 값을 안 적었습니다. 02단원에서 배운 불리언 속성이라 적으면 true 입니다.

```
function NavLinkDemo() {
```

함수로 넣기 때문에 이렇게 따로 빼서 이름을 붙여 둘 수도 있습니다.

```
function markActive({ isActive }) {
  return isActive ? "on" : "";
}

return (
  <div className="demo"> <h3>② NavLink - 보고 있는 곳에 색이 들어옵니다
</h3> <nav> <NavLink to="/r2" end className={markActive}>
  홈
  </NavLink>{" "}
  |{" "}
  <NavLink to="/r2/menu" className={markActive}>
  메뉴
  </NavLink>{" "}
  |{" "}
  <NavLink to="/r2/notice" className={markActive}>
  공지
  </NavLink> </nav> <div className="output"> <Routes> <Route path="/r2" element={<p>
지금 화면: 홈</p>} />
  <Route path="/r2/menu" element={<p>지금 화면: 메뉴</p>} />
  <Route path="/r2/notice" element={<p>지금 화면: 공지</p>} />
  <Route path="*" element={<p>지금 화면: 없음</p>} />
  </Routes> </div> <p>
  데모 ① 과 ② 는 같은 BrowserRouter 안에 있습니다. 그래서 어느 쪽 링크를
  눌러도 두
  상자가 함께 바뀝니다. 주소 하나가 화면 여러 곳을 동시에 정하는 것입니다.
  </p> </div>

);
}
```

▹ 직접 해보기 4

NavLinkDemo 의 홈 NavLink 에서 end 를 지우고 저장하세요. 그다음 '메뉴' 를 눌러 보세요. 홈에 색이 어떻게 되나요? 확인했으면 end 를 다시 붙이세요.

그러면 <a> 는 언제 쓰나

섹션 5

<Link> 가 다루는 것은 '이 앱 안의 주소' 입니다. 앱 밖으로 나가는 주소, 즉 다른 사이트로 갈 때는 <a> 가 맞습니다.

```
<a href="https://reactrouter.com">리액트 라우터 공식 문서</a>
```

다른 사이트는 우리 Routes 에 등록할 수 없습니다. 등록해 봐야 그런 화면이 없습니다. 그래서 나가는 링크는 <a> 로 쓰는 것이 의도가 분명합니다.

(참고 — <Link to="https://reactrouter.com"> 라고 써도 요즘 버전은 알아서 '바깥 주소' 로 알아채고 그냥 내보내 줍니다. 다만 읽는 사람이 헷갈리고 target="_blank" 같은 것도 직접 챙겨야 해서, 나갈 때는 <a> 를 쓰세요.)

정리하면 이렇습니다.

우리 앱 안의 주소 → <Link> 또는 <NavLink> 다른 사이트 주소 → 파일 내려받기
→

아래 데모 ③ 의 링크에는 target="_blank" 를 붙였습니다. 새 탭에서 열라는 뜻입니다. 지금 보던 것을 잃지 않아서 편합니다.

```
function OutsideLinkDemo() {
  return (
    <div className="demo"> <h3>③ 밖으로 나가는 링크
  </h3> <p> <a href="https://reactrouter.com" target="_blank" rel="noreferrer">
    리액트 라우터 공식 문서 (새 탭에서 열립니다)
  </a> </p> <p>
    이런 링크는 Routes 에 등록할 수 없습니다. 우리 앱의 화면이 아니기 때문입니다.
  </p> </div>
  );
}
```

▹ 직접 해보기 5

데모 ③ 에 <Link to="/r2/메뉴판"> 아직 없는 화면 </Link> 을 한 줄 넣고 눌러 보세요. 이 주소는 Routes 에 등록하지 않은 주소입니다. 화면이 어떻게 되나요? 확인 후 되돌리세요.

자주 하는 실수

섹션 6

⚠ [SyntaxError] 라고 적힌 것은 주석을 풀지 말고 눈으로만 보세요. 풀면 파일 전체가 멈춰서 화면이 통째로 비어 버립니다. 다시 // 를 붙이면 돌아옵니다.

이런 실수를 합니다

Link 에 href 를 쓴다 — [조용히 틀림]

```
<Link href="/r2/menu">메뉴</Link>
```

에러가 안 납니다. 글자는 그대로 나오고 눌러도 아무 일이 안 일어납니다. Link 가 아는 이름은 to 입니다. href 는 모르는 이름이라 그냥 무시합니다. “눌러도 반응이 없다” 면 to 인지 href 인지부터 보세요.

이런 실수를 합니다

a 를 그대로 두고 Link 로 안 바꾼다 — [조용히 틀림]

```
<a href="/r2/menu">메뉴</a>
```

화면은 잘 나옵니다. 그래서 다 됐다고 생각하기 쉽습니다. 하지만 누를 때마다 앱이 처음부터 다시 시작합니다. 장바구니에 담아 둔 것이 사라지고, 화면이 깜빡이고, 느립니다. 이 파일의 데모 ① 이 바로 그 차이를 재 보는 것입니다.

이런 실수를 합니다

BrowserRouter 밖에서 Link 를 쓴다 — [런타임 에러]

```
<div>
  <Link to="/r2">홈</Link>
</div>
<BrowserRouter>...</BrowserRouter>
```

화면이 빨간 에러 상자가 됩니다. "useHref() may be used only in the context of a <Router> component" 라고 나옵니다. Link 는 지금 주소를 알아야 동작하는데, 알려 줄 사람이 밖에는 없습니다.

이런 실수를 합니다

NavLink 의 className 에 함수 대신 값을 넣는다 — [조용히 틀림]

```
<NavLink to="/r2/menu" className={isActive ? "on" : ""}>메뉴</NavLink>
```

isActive 라는 변수는 이 자리에 없습니다. 바깥에 없으면 에러가 나고, 우연히 같은 이름의 변수가 있으면 에러도 없이 늘 같은 값이 됩니다. isActive 는 NavLink 가 함수를 부르면서 넘겨 주는 값입니다. 그래서 className={{ { isActive } } => ...} 처럼 함수로 받아야 합니다.

이런 실수를 합니다

Link 를 닫지 않는다 — [SyntaxError]

```
<Link to="/r2">홈
```

파일 전체가 안 돌아가고 화면이 통째로 빡니다. 02단원 개념04에서 배운 규칙 그대로입니다. 연 태그는 반드시 닫습니다. <a> 는 HTML 이라 브라우저가 봐주기도 하지만, JSX 는 봐주지 않습니다.

이런 실수를 합니다

end 를 안 붙여서 홈이 늘 켜져 있다 — [조용히 틀림]

```
<NavLink to="/r2">홈</NavLink>
```

/r2/menu 에서도 '홈' 에 색이 들어옵니다. 메뉴와 홈 둘 다 켜져 보입니다. NavLink 는 기본적으로 '시작이 같으면 켜짐' 이기 때문입니다. 가장 짧은 주소(보통 홈)에는 end 를 붙이세요.

정리

- 1 는 페이지를 서버에서 새로 받아 옵니다. React 가 다시 시작하면서 state 가 전부 사라집니다.
- 2 는 주소만 바꾸고 화면의 그 자리만 다시 그립니다. state 가 그대로 남습니다.
- 3 차이는 카운터·왼쪽 메뉴·콘솔 로그 세 가지로 직접 짤 수 있습니다.
- 4 Link 의 속성 이름은 href 가 아니라 to 입니다. href 를 쓰면 눌러도 아무 일이 없습니다.
- 5 NavLink 는 지금 주소와 맞으면 isActive 를 true 로 넘겨 줍니다. className 에 함수를 넣어 받습니다.
- 6 NavLink 는 '시작이 같으면 켜짐' 이라서, 홈처럼 짧은 주소에는 end 를 붙입니다.
- 7 다른 사이트로 나가는 링크는 Link 가 아니라 를 씁니다.

```
export default function Concept02LinkAndNavLink() {  
  return (  
    <div> <h1>개념 02 - Link 와 NavLink</h1> <p className="guide">  
      데모 ① 에서 <strong>카운터를 3까지 올린 뒤</strong> 파란 링크와 빨간 링크를  
      각각  
      눌러 보세요. 파란 링크로 이동하면 3이 그대로 남습니다.  
      <br /> <br /> <strong>빨간 링크를 누르면 앱이 통째로 다시 시작합니다.  
</strong> 왼쪽 메뉴 선택도  
      풀려서 이 예제가 화면에서 사라지고 맨 위 예제가 열립니다. 고장이 아닙니다.  
      왼쪽  
      메뉴에서 <strong>개념02 Link와 NavLink</strong> 를 다시 고르세요. 돌아와 보면  
      카운터가 0입니다. 같은 이유로 이 예제에서는{" "  
      <strong>새로고침(F5)도 하지 마세요.</strong> <br /> <br /> <strong>F12 →
```

```

Console</strong> 도 함께 열어 두세요. 파일 맨 위에서 찍는 줄이 다시
나오는지가 가장 확실한 증거입니다.
</p> <BrowserRouter> <CompareDemo /> <NavLinkDemo /> <OutsideLinkDemo
/> </BrowserRouter> </div>
);
}

```

직접 해보기 정답

1) 이 예제가 화면에서 사라집니다.

화면	카운터 3 → 빨간 링크 클릭 → 화면이 한 번 깜빡이고 전혀 다른 예제가 열립니다 왼쪽 메뉴의 파란 표시가 맨 위 예제로 옮겨 가 있습니다
----	---

→ 주소창은 /r2/notice 그대로입니다. 주소는 맞는데 화면이 다릅니다.

왼쪽 메뉴가 어디를 골랐는지는 주소가 아니라 state 에 들어 있기 때문입니다.
왼쪽 메뉴에서 '개념02 Link와 NavLink' 를 다시 고르면 돌아옵니다.
돌아와 보면 카운터는 0입니다.

2) 카운터가 3 그대로입니다.

화면	아래 상자의 내용만 바뀌고 카운터는 3 왼쪽 메뉴 선택도 그대로입니다
----	---

→ 페이지를 새로 받지 않았으니 React 가 계속 돌고 있습니다.

3) 파란 링크(<Link>)로는 한 번도 다시 찍히지 않습니다.

F12 콘솔	카운터를 눌렀습니다	← [+1] 을 눌렀을 때만
--------	------------	-----------------

→ 왼쪽 메뉴에서 다른 예제로 갔다가 돌아와도 다시 안 찍힙니다.

한 번 읽어 둔 파일은 다시 읽지 않기 때문입니다.

→ 반대로 빨간 링크를 누르면 콘솔이 통째로 지워집니다.

(콘솔이 지워지는 것부터가 페이지를 새로 받았다는 증거입니다)
그 뒤 왼쪽 메뉴에서 이 예제를 다시 고르면
"[개념02] 이 예제 파일을 새로 불러왔습니다" 가 다시 나타납니다.

4) '메뉴' 를 눌렀는데 '홈' 에도 파란 배경이 들어옵니다.

화면	홈과 메뉴 두 개가 동시에 켜져 보입니다
----	------------------------

→ /r2/menu 는 /r2 로 시작하기 때문입니다.

end 를 붙이면 주소가 딱 /r2 일 때만 켜집니다.

5) 주소창이 <http://localhost:5199/r2/메뉴판> 으로 바뀝니다.

화면 데모 ① 의 상자가 "이 예제가 아는 주소가 아닙니다" 로 바뀝니다.

페이지는 새로 열리지 않습니다. 카운터도 그대로입니다.

→ Link 는 to 에 적은 것을 '우리 앱 안의 주소' 로 봅니다.

그래서 등록 안 한 주소를 넣으면 `catchall(path="*")` 이 잡습니다.
바깥 사이트로 나가는 링크는 `<a href="...">` 를 쓰세요. 그게 의도가 분명합니다.

URL 파라미터

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

지금까지 만든 Route 는 주소를 하나씩 손으로 적었습니다.

```
<Route path="/r2/menu" element={<MenuScreen />} />
```

메뉴가 세 개면 Route 도 세 개 적으면 됩니다. 여기까지는 괜찮습니다. 그런데 메뉴가 100개라면 어떨까요? Route 를 100개 적을 수는 없습니다. 게다가 메뉴가 하나 늘 때마다 코드를 고쳐야 합니다.

그래서 라우터에는 '값이 들어오는 자리' 를 만드는 방법이 있습니다.

```
<Route path="/r3/menu/:id" element={<MenuDetail />} />
~~~~~
```

:id 는 "여기에는 아무 값이나 올 수 있다" 는 표시입니다. 이 Route 하나가 /r3/menu/1 도, /r3/menu/2 도, /r3/menu/999 도 다 받아 줍니다. 그리고 그 자리에 실제로 온 값을 화면에서 꺼내 쓸 수 있습니다. 그 값을 URL 파라미터라고 부릅니다.

★ 이 자료만의 사정: 진짜 앱은 BrowserRouter 를 앱 맨 바깥에 한 번만 둡니다. 이 자료는 예제를 왼쪽 메뉴로 골라 보는 구조라 파일마다 하나씩 두었습니다. 이 파일이 쓰는 주소 앞머리는 /r3 입니다. 그리고 이동한 뒤 새로고침(F5)을 하면 왼쪽 메뉴 선택이 풀립니다. 하지 마세요.

```
import { BrowserRouter, Routes, Route, Link, useParams } from "react-router-dom";
import Summary from "../_ui/Summary.jsx";
```

목록에서 상세로

섹션 1

거의 모든 앱에 있는 화면 짝입니다.

목록 화면 — 메뉴 이름이 죽 나열됩니다 상세 화면 — 그중 하나를 눌렀을 때 그 하나만 자세히 나옵니다

05단원 방식으로도 만들 수 있습니다. selected 라는 state 를 두면 됩니다. 하지만 그러면 개념01에서 본 문제가 그대로 생깁니다. "라떼 상세 화면" 을 친구에게 링크로 보낼 수 없습니다. 주소가 하나뿐이니까요.

주소를 이렇게 나눠 두면 그 문제가 사라집니다.

/r3/menu	→ 메뉴 목록
/r3/menu/1	→ 아메리카노 상세
/r3/menu/2	→ 라떼 상세

상세마다 주소가 다르니 그대로 복사해서 보내면 됩니다.

화면에 쓸 데이터입니다. 07단원까지 쓰던 것과 같은 모양입니다. id 는 숫자입니다. 이 점을 섹션 4에서 다시 봅니다.

```
const menuItems = [
  { id: 1, name: "아메리카노", price: 4000 },
  { id: 2, name: "라떼", price: 4500 },
  { id: 3, name: "케이크", price: 6000 },
  { id: 4, name: "삼각김밥", price: 1200 },
];

function MenuList() {
  return (
    <div> <p>메뉴를 눌러 보세요. 주소가 어떻게 바뀌는지 보세요.</p> <ul>
      {menuItems.map((item) => (
        <li key={item.id}>
          /* 백틱 문자열로 주소를 만듭니다. JS자료 02단원의 템플릿 리터럴입니다.
*/
          <Link to={`r3/menu/${item.id}`}>
            {item.name} {item.price}원
          </Link> </li>
        ))}
    </ul> </div>
  );
}
```

▣ 직접 해보기

1

데모의 '메뉴 목록' 으로 간 뒤 '라떼' 를 눌러 보세요. 주소창이 무엇으로 바뀌는지 적어 보세요.

:id — 값이 들어오는 자리 만들기

섹션 2

Route 의 path 에 콜론(:)을 붙이면 그 칸은 '아무 값이나 받는 칸' 이 됩니다.

```
<Route path="/r3/menu/:id" element={<MenuDetail />} />
```

이때 알아 둘 것이 세 가지입니다.

[1] 이름은 마음대로 정합니다.

:id 여도 되고 :menuId 여도 됩니다. 나중에 꺼낼 때 같은 이름을 쓰면 됩니다.

[2] 한 칸만 받습니다.

`/r3/menu/:id` 는 `/r3/menu/2` 와는 맞지만 `/r3/menu/2/price` 와는 안 맞습니다.
칸을 나누는 것은 `/` 입니다.

[3] 빈 값은 못 받습니다.

`/r3/menu/` 는 이 Route 와 맞지 않습니다. 값이 있어야 합니다.
그래서 목록 화면(`/r3/menu`)과 상세 화면은 서로 다른 Route 입니다.

▹ 직접 해보기 2

주소창에 직접 치지 말고, 아래 데모의 링크만 눌러서 `/r3/menu/4` 화면까지 가 보세요. 무슨 메뉴가 나오나요?

useParams 로 값 꺼내기

섹션 3

`:id` 자리에 들어온 값은 `useParams` 라는 훅으로 꺼냅니다.

```
const params = useParams(); // { id: "2" } 같은 객체를 돌려줍니다
const { id } = useParams(); // 09단원 구조분해로 바로 꺼내는 것이 편합니다
```

09:10단원에서 배운 훅들과 규칙이 같습니다. 컴포넌트 맨 위에서 부릅니다. 그리고 이 훅은 `BrowserRouter` 안쪽에서만 쓸 수 있습니다.

돌려주는 객체의 키 이름은 `path` 에 적은 이름과 같습니다. `path` 에 `:id` 라고 적었으면 `params.id` 로 꺼냅니다.

▹ 직접 해보기 3

상세 화면 맨 위의 `"useParams() 가 준 것"` 줄을 보세요. 목록에서 '케이크' 를 눌렀을 때 그 줄이 무엇으로 나오나요?

꺼낸 값은 언제나 문자열이다

섹션 4

여기가 이 파일에서 가장 중요한 곳입니다.

주소는 글자입니다. `/r3/menu/2` 에서 2 는 숫자처럼 보이지만 글자 `"2"` 입니다. 그래서 `useParams` 가 주는 값도 언제나 문자열입니다. 숫자가 아닙니다.

JS자료 11단원에서 똑같은 함정을 겪었습니다. `data-price="4000"` 을 꺼내면 숫자 4000이 아니라 문자열 `"4000"` 이라서 더하기가 이상해졌던 그 일입니다. 주소도 똑같습니다. 아래 세 줄로 먼저 확인해 봅시다.

```
const sampleId = "2"; // useParams 가 주는 값과 같은 모양입니다 console.log(typeof sampleId);
```

F12 콘솔 string

코드

```
console.log(sampleId === 2);
```

```
console.log(Number(sampleId) === 2);
```

▶ F12 콘솔

▶ false

▶ true

menuItems 의 id 는 숫자 1, 2, 3, 4 입니다. 그러니 문자열 "2" 로 찾으려 하면 영영 못 찾습니다.

```
menuItems.find((item) => item.id === id) // "2" 와 2 를 비교 → undefined  
menuItems.find((item) => item.id === Number(id)) // 2 와 2 를 비교 → 라떼
```

이 실수는 에러를 내지 않습니다. 그냥 '못 찾았다' 가 될 뿐입니다. 그래서 "링크는 맞는데 상세가 안 나온다" 로 나타납니다. 아래 데모에서 직접 보세요.

```
function MenuDetail() {  
  const params = useParams();  
  const { id } = params; // params.id 와 같습니다
```

틀린 방법 — 문자열과 숫자를 === 로 비교합니다

```
const wrongFound = menuItems.find((item) => item.id === id);
```

맞는 방법 — 숫자로 바꿔서 비교합니다

```
const found = menuItems.find((item) => item.id === Number(id));  
  
return (  
  <div> <h4>상세 화면</h4> <p>  
    useParams() 가 준 것:{" "  
    <strong>{JSON.stringify(params)}</strong>  
    /* JSON.stringify 는 값을 글자로 바꿔 보여 줍니다. JS자료 12단원에서  
    썼습니다. */  
    /* 화면: {"id":"2"} - 2 에 따옴표가 붙어 있습니다. 문자열이라는 뜻입니다.  
    */  
  </p> <p>  
    id 의 타입: <strong>{typeof id}</strong> / id === 2 의 결과:{" "  
    <strong>{String(id === 2)}</strong>  
    /* String(...) 으로 감싼 이유: JSX 는 true/false 를 화면에 안 그립니다  
(05단원). */  
    /* 화면: id 의 타입: string / id === 2 의 결과: false */  
  </p> <p>  
    문자열 그대로 찾은 결과:{" "  
    <strong>{wrongFound ? wrongFound.name : "못 찾았습니다"}</strong> </p> <p>  
    Number() 로 바꿔 찾은 결과:{" "  
    <strong>{found ? found.name + " " + found.price + "원" : "못 찾았습니다"}  
  </p>
```

```

</strong> </p> <p> <Link to="/r3/menu">- 목록으로</Link> </p> </div>
);
}

```

직접 해보기

4

상세 화면에서 '문자열 그대로 찾은 결과'와 'Number()로 바꿔 찾은 결과'를 비교해 보세요.

없는 값이 들어오면

섹션 5

:id는 아무 값이나 받습니다. 그래서 /r3/menu/99도 이 Route와 맞습니다. 라우터는 "주소가 맞으니 상세 화면을 그려라"라고만 합니다. menuItems에 99번이 있는지는 라우터가 모릅니다. 우리가 확인해야 합니다.

find는 못 찾으면 `undefined`를 돌려줍니다(JS자료 08단원). 그것을 확인하지 않고 `found.name`을 쓰면 이런 에러가 납니다.

```
Cannot read properties of undefined (reading 'name')
```

그래서 위 MenuDetail에서는 `found`가 있는지 삼항 연산자로 먼저 봤습니다(05단원). 화면 전체를 바꾸고 싶으면 05단원에서 배운 '일찍 `return`'이 더 깔끔합니다.

```

if (!found) {
  return <p>그런 메뉴가 없습니다.</p>;
}

```

없는 주소(Route 자체가 없는 것)와 없는 값(Route는 맞는데 데이터가 없는 것)은 다른 문제입니다. 앞의 것은 개념05의 `path="*"로`, 뒤의 것은 이렇게 처리합니다.

```

function NotHere() {
  return (
    <p>이 예제가 아는 주소가 아닙니다. 위의 '메뉴 목록'을 눌러 시작하세요.</p>
  );
}

function Demo() {
  return (
    <BrowserRouter> <div className="demo"> <h3>목록 → 상세
  </h3> <nav> <Link to="/r3/menu">메뉴 목록</Link> |{" "}
    <Link to="/r3/menu/2">라떼로 바로 가기</Link> |{" "}
    <Link to="/r3/menu/99">없는 메뉴 (99)
  </Link> </nav> <div className="output"> <Routes> <Route path="/r3/menu" element=
  {<MenuList />} />
    <Route path="/r3/menu/:id" element={<MenuDetail />} />
    <Route path="*" element={<NotHere />} />
  </Routes> </div> </div> </BrowserRouter>

```

```

    );
  }

  function Demo() {
    return (
      <BrowserRouter> <div className="demo"> <h3>목록 → 상세
</h3> <nav> <Link to="/r3/menu">메뉴 목록</Link> |{" "}
      <Link to="/r3/menu/2">라떼로 바로 가기</Link> |{" "}
      <Link to="/r3/menu/99">없는 메뉴 (99)
</Link> </nav> <div className="output"> <Routes> <Route path="/r3/menu" element=
{<MenuList />} />
      <Route path="/r3/menu/:id" element={<MenuDetail />} />
      <Route path="*" element={<NotHere />} />
</Routes> </div> </div> </BrowserRouter>
    );
  }

```

▫ 직접 해보기

5

'없는 메뉴 (99)' 를 눌러 보세요. 화면이 어떻게 되는지, 콘솔에 에러가 나는지 확인하세요.

▫ 직접 해보기

6

MenuDetail 안의 found 를 쓰는 줄을 이렇게 고쳐 보세요.

```
<strong>{found.name}</strong>
```

그리고 '없는 메뉴 (99)' 를 누르면 무슨 일이 나는지 보세요.
확인했으면 원래대로 되돌리세요.

자주 하는 실수

섹션 6

⚠ [SyntaxError] 라고 적힌 것은 주석을 풀지 말고 눈으로만 보세요. 풀면 파일 전체가 멈춰서 화면이 통째로 비어 버립니다. 다시 // 를 붙이면 돌아옵니다.

이런 실수를 합니다

문자열인 것을 잊고 숫자와 === 로 비교한다 — [조용히 틀림]

```
const found = menuItems.find((item) => item.id === id);
```

에러가 안 납니다. 그냥 undefined 가 나와서 “못 찾았습니다” 만 보입니다. 이 파일의 데모에서 두 결과를 나란히 보여 주는 이유입니다. 고치는 법은 Number(id) 로 바꾸거나, 데이터의 id 를 문자열로 통일하는 것입니다. 둘 중 하나만 고르세요. 섞으면 또 헷갈립니다.

이런 실수를 합니다

path 의 이름과 **useParams** 의 이름을 다르게 쓴다 — [조용히 틀림]

```
<Route path="/r3/menu/:menuId" element={<MenuDetail />} />
const { id } = useParams();    // menuId 라고 적어야 하는데 id 라고 적음
```

id 가 `undefined` 가 됩니다. 에러는 안 납니다. 화면에 "undefined" 가 나오거나 조용히 빈 값이 됩니다. 두 곳의 이름이 같은지부터 확인하세요.

이런 실수를 합니다

:id 앞에 콜론을 빠뜨린다 — [조용히 틀림]

```
<Route path="/r3/menu/id" element={<MenuDetail />} />
```

id 를 '값이 오는 자리' 가 아니라 그냥 글자로 봅니다. 그래서 /r3/menu/id 라는 주소일 때만 맞고, /r3/menu/2 와는 안 맞습니다. 링크를 눌러도 아무 화면이 안 나오면 콜론부터 확인하세요.

이런 실수를 합니다

find 결과를 확인하지 않고 바로 쓴다 — [런타임 에러]

```
<strong>{found.name}</strong>
```

있는 메뉴일 때는 잘 돌아갑니다. 그래서 다 됐다고 생각합니다. 없는 id 가 들어온 순간 "Cannot read properties of `undefined`" 로 화면이 터집니다. 주소는 사용자가 얼마든지 손으로 고칠 수 있습니다. 늘 없을 때를 대비하세요.

이런 실수를 합니다

Link 의 **to** 를 백틱 없이 쓴다 — [조용히 틀림]

```
<Link to="/r3/menu/${item.id}">
```

큰따옴표 안에서는 `${...}` 가 값으로 바뀌지 않습니다. 글자 그대로 들어갑니다. 주소가 /r3/menu/\${item.id} 같은 이상한 것이 됩니다. 값을 끼워 넣을 때는 백틱() 문자열을 쓰고, 중괄호로 감싸 `to={...}` 로 넘깁니다.

이런 실수를 합니다

중괄호 안에 주소를 따옴표 없이 쓴다 — [SyntaxError]

```
<Link to={/r3/menu/${item.id}}>
```

파일 전체가 안 돌아가고 화면이 통째로 빡니다. 중괄호 안은 자바스크립트 자리입니다. 거기에 /r3/menu/ 를 맨 채로 쓰면 자바스크립트가 나눗셈 기호로 읽으려다 막힙니다. 중괄호 안에 문자열을 넣을 때는 반드시 따옴표나 백틱으로 감쌉니다.

정리

- 1 path 에 :id 처럼 콜론을 붙이면 그 칸은 아무 값이나 받는 자리가 됩니다.
- 2 그 값은 `useParams()` 로 꺼냅니다. path 에 적은 이름과 같은 키로 들어 있습니다.
- 3 `useParams` 가 주는 값은 언제나 문자열입니다. 주소는 글자이기 때문입니다.
- 4 숫자 id 와 비교하려면 `Number(id)` 로 바꿔야 합니다. 안 바꾸면 에러 없이 못 찾습니다.
- 5 :id 는 아무 값이나 받으므로 없는 값도 들어옵니다. find 결과가 `undefined` 인지 꼭 확인하세요.
- 6 목록과 상세는 서로 다른 Route 입니다. `/r3/menu` 와 `/r3/menu/:id` 를 따로 적습니다.

```
export default function Concept03UrlParams() {
  return (
    <div> <h1>개념 03 - URL 파라미터</h1> <p className="guide">
      아래 데모에서 <strong>메뉴 목록 → 아무 메뉴</strong> 순서로 눌러 보세요. 상세
      화면마다 주소가 다릅니다. 그 주소를 복사해 두면 나중에 그 화면으로 바로 갈 수
      있습니다.
      <br /> <br /> <strong>가장 중요한 것은 상세 화면 위쪽 두 줄입니다.</strong>
      useParams 가 준 값이
      다음표에 싸여 있는 것을 보세요. 숫자처럼 보여도 문자열입니다.
      <br /> <br />
      이 예제에서는 <strong>새로고침(F5)</strong>을 하지 마세요.</strong> 왼쪽 메뉴 선택이
      풀려서
      다른 예제가 열립니다. 그때는 왼쪽 메뉴에서 다시 고르면 됩니다.
    </p> <Demo /> </div>
  );
}
```

직접 해보기 정답

1) `/r3/menu/2` 로 바뀝니다.

화면 상세 화면에 "라떼 4500원" 이 나옵니다.

→ 링크마다 주소가 다르니 그 주소를 복사해 두면 그 화면으로 바로 갈 수 있습니다.

2) 삼각김밥이 나옵니다.

화면 `Number()` 로 바꿔 찾은 결과 → 삼각김밥 1200원

→ 목록에서 '삼각김밥 1200원' 을 누르면 됩니다. Route 는 하나인데

네 가지 상세 화면이 전부 그려집니다. 이것이 `:id` 를 쓰는 이유입니다.

3) {"id": "3"} 으로 나옵니다.

화면 useParams() 가 준 것: {"id": "3"}

→ 키 이름 id 는 Route 의 path 에 적은 :id 에서 온 것입니다.

값 3 에 따옴표가 붙어 있는 것을 꼭 보세요. 문자열이라는 표시입니다.

4) 문자열 그대로 찾으면 늘 "못 찾았습니다" 입니다.

화면 문자열 그대로 찾은 결과: 못 찾았습니다
 Number() 로 바꿔 찾은 결과: 라떼 4500원

→ 같은 find 인데 결과가 다릅니다. "2" 와 2 는 === 로 보면 다른 값입니다.

콘솔에 찍힌 세 줄(string / false / true)이 그 이유를 그대로 보여 줍니다.

5) 상세 화면 틀은 나오고 두 결과 모두 "못 찾았습니다" 가 됩니다.

화면 useParams() 가 준 것: {"id": "99"}
 문자열 그대로 찾은 결과: 못 찾았습니다
 Number() 로 바꿔 찾은 결과: 못 찾았습니다

→ 콘솔에는 에러가 없습니다. 주소는 /r3/menu/:id 와 잘 맞기 때문입니다.

라우터가 보기에는 아무 문제가 없고, 데이터에만 없는 것입니다.

6) 화면이 빨간 에러 상자로 바뀝니다.

F12 콘솔 Cannot read properties of undefined (reading 'name')

→ found 가 undefined 인데 .name 을 꺼내려 했기 때문입니다.

있는 메뉴(1~4)를 누를 때는 멀쩡하게 동작해서 더 찾기 어렵습니다.
그래서 삼항으로 먼저 확인하거나, 일찍 return 으로 막아야 합니다.

중첩 라우트

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

지금까지는 주소가 바뀌면 화면을 통째로 갈아 끼웠습니다. 그런데 실제 앱을 보면 화면이 전부 바뀌는 일은 별로 없습니다.

가게 이름과 탭 메뉴는 위에 그대로 있고, 그 아래 내용만 바뀝니다.

쇼핑몰도, 은행 앱도, 이 자료의 왼쪽 메뉴도 그렇습니다. 이렇게 여러 화면이 함께 쓰는 껍데기를 '레이아웃'이라고 부릅니다.

레이아웃을 화면마다 복사해 붙이면 어떻게 될까요? 탭 이름 하나 고치는 데 여러 파일을 고쳐야 합니다. 01단원에서 본 "고칠 곳이 흩어진다" 문제가 그대로 돌아옵니다.

라우터에는 이것을 위한 방법이 있습니다. Route 안에 Route 를 넣는 것입니다.

★ 이 자료만의 사정: 진짜 앱은 `BrowserRouter` 를 앱 맨 바깥에 한 번만 둡니다. 이 자료는 예제를 왼쪽 메뉴로 골라 보는 구조라 파일마다 하나씩 두었습니다. 이 파일이 쓰는 주소 앞머리는 `/r4` 입니다. 이동한 뒤 새로고침(F5)을 하면 왼쪽 메뉴 선택이 풀립니다. 하지 마세요.

```
import { useState } from "react";
import { BrowserRouter, Routes, Route, Link, NavLink, Outlet } from "react-router-dom";
import Summary from "../_ui/Summary.jsx";
```

껍데기를 복사해 붙이면 생기는 일

섹션 1

중첩 라우트를 모르면 이렇게 씁니다. 화면마다 컴포넌트를 만들고, 그 안에 같은 머리말과 탭 메뉴를 각각 적습니다.

```
function OldMenu() {
  return (
    <div>
      <h4>민준이네 카페</h4>           ← 복사 1
      <nav>메뉴 | 장바구니 | 공지</nav> ← 복사 1
      <p>아메리카노 4000원 ...</p>
    </div>
  );
}
```

화면이 세 개면 같은 것을 세 번 적습니다. 지금은 견딜 만합니다. 화면이 열 개가 되고 탭이 하나 늘면 열 군데를 고쳐야 합니다. 그리고 한 군데를 빠뜨리면 그 화면에서만 탭이 다릅니다. 이런 버그는 그 화면에 직접 들어가 보기 전까지 아무도 모릅니다.

아래 데모 ① 이 딱 그 상태입니다. 일부러 한 군데를 빠뜨려 두었습니다.

화면 세 개가 각자 머리말과 탭을 들고 있습니다. 잘 보면 셋이 똑같지 않습니다.

```
function OldTabs({ onGo }) {
```

props 로 받은 함수를 부릅니다. 07단원 개념04에서 배운 방식입니다.

```
  return (
    <nav> <button onClick={() => onGo("menu")}>메뉴</button> <button onClick={() =>
onGo("cart")}>장바구니</button> <button onClick={() => onGo("notice")}>공지
</button> </nav>
  );
}
```

```
function OldWayDemo() {
  const [tab, setTab] = useState("menu");

  function goTab(name) {
    setTab(name);
    console.log("복불 방식 - 화면을 갈아 끼웠습니다");
  }
}
```

```
F12 콘솔    복불 방식 - 화면을 갈아 끼웠습니다
```

```

    }

    return (
      <div className="demo">
        <h3>① 복불 방식 - 화면마다 꺾데기를 따로 들고 있습니다</h3>

        {tab === "menu" && (
          <div className="output">
            <h4>민준이네 카페</h4>
            <OldTabs onGo={goTab} />
            <p>아메리카노 4000원 / 라떼 4500원 / 케이크 6000원</p>
          </div>
        )}

        {tab === "cart" && (
          <div className="output">
            <h4>민준이네 카페</h4>
            <OldTabs onGo={goTab} />
            <p>담은 것이 없습니다.</p>
          </div>
        )}

        {tab === "notice" && (
          <div className="output">
            {/* 이 화면만 머리말을 안 고쳤고 탭도 하나 빠졌습니다. 복불하다 생긴 일입니다. */}
            <h4>민준이네 커피</h4>
            <nav>
              <button onClick={() => goTab("menu")}>메뉴</button>
              <button onClick={() => goTab("cart")}>장바구니</button>
            </nav>
            <p>이번 주 화요일은 쉽니다.</p>
          </div>
        )}

        <p>
          '공지' 로 가 보세요. 가게 이름이 다르고 '공지' 탭이 사라져 있습니다. 꺾데기를
          세 번
          적었기 때문에 생긴 일입니다.
        </p>
      </div>
    );
  }

```

▹ 직접 해보기

1

데모 ① 에서 '공지' 를 눌러 보세요. 앞의 두 화면과 다른 점을 두 가지 찾아보세요.

Route 안에 Route 넣기

섹션 2

탭데기를 한 번만 쓰고 싶으면 Route 를 이렇게 겹쳐 씁니다.

```
<Route path="/r4" element={<ShopLayout />}>
  <Route path="menu" element={<ShopMenu />} /> <Route path="cart" element={<ShopCart />} />
</Route>
```

바깥 Route 를 부모, 안쪽 Route 를 자식이라고 부릅니다. 부모는 껍데기를, 자식은 내용을 말합니다.

— 주소는 이어 붙입니다

자식의 path 에는 슬래시를 붙이지 않습니다. 부모 주소 뒤에 이어 붙기 때문입니다.

```
부모 /r4 + 자식 menu → /r4/menu
부모 /r4 + 자식 cart → /r4/cart
```

자식에 path="/menu" 라고 슬래시를 붙이면 "앱의 맨 처음부터 /menu" 라는 뜻이 됩니다. 부모가 /r4 인데 자식이 /menu 면 서로 말이 안 맞아서 에러가 납니다. 이 실수는 섹션 6에서 다시 봅니다.

— 주소가 맞으면 둘 다 그립니다

주소가 /r4/menu 면 React Router 는 부모와 자식을 함께 그립니다. 부모 ShopLayout 을 그리고, 그 안에 자식 ShopMenu 를 넣습니다. 그러면 "그 안" 이 정확히 어디일까요? 그것을 정하는 것이 다음 섹션의 Outlet 입니다.

▹ 직접 해보기

2

아래 Demo 의 Routes 안에 자식 Route 를 하나 더 넣으세요. path 는 hours, element 는 <p>10시부터 22시까지 오픈하다</p> 입니다. 넣은 뒤 탭에도 <NavLink to="hours">영업시간</NavLink> 를 추가하세요.

Outlet — 자식이 그려질 자리

섹션 3

부모 컴포넌트 안에서 <Outlet /> 을 쓴 곳에 자식 화면이 들어갑니다.

```
function ShopLayout() {
  return (
    <div>
      <h4>민준이네 카페</h4>
      <nav>...</nav>
      <Outlet />      {/* ← 여기에 자식 화면이 들어옵니다 */}
    </div>
  );
}
```

03단원에서 배운 children 과 비슷하지만 다릅니다. children 은 부모가 쓸 때 직접 넣어 준 것이고, Outlet 은 '지금 주소에 맞는 자식 Route' 를 라우터가 골라서 넣어 주는 것입니다.

— 레이아웃은 다시 만들어지지 않습니다

탭을 옮기면 Outlet 안쪽만 바뀝니다. 바깥 껍데기는 그대로 있습니다. 그래서 껍데기가 들고 있는 state 도 그대로 남습니다. 아래 데모에 숫자 두 개를 두었으니 직접 비교해 보세요.

담은 개수 · 껍데기(ShopLayout)가 들고 있습니다 → 탭을 옮겨도 그대로

본 횟수 · 자식(ShopMenu)이 들고 있습니다 → 탭을 옮겼다 오면 0

```
function ShopLayout() {
  const [cartCount, setCartCount] = useState(0);

  function addToCart() {
    setCartCount(cartCount + 1);
  }
}
```

NavLink 의 className 에 넣을 함수입니다(개념02에서 배웠습니다).

```

function markActive({ isActive }) {
  return isActive ? "on" : "";
}

return (
  <div>
    <h4>민준이네 카페</h4> <p>
      담은 개수: <strong>{cartCount}</strong>{" "}
      <button onClick={addToCart}>담기</button> </p> <nav>
        {/* 이 링크만 절대 경로입니다. 슬래시로 시작하면 앱의 맨 처음부터 세는
주소입니다. */}
        <NavLink to="/r4" end className={markActive}>
          홈
        </NavLink>{" "}
        |{" "}
        {/* 아래 셋은 슬래시가 없습니다. 지금 부모(/r4) 뒤에 이어 붙습니다. */}
        <NavLink to="menu" className={markActive}>
          메뉴
        </NavLink>{" "}
        |{" "}
        <NavLink to="cart" className={markActive}>
          장바구니
        </NavLink>{" "}
        |{" "}
        <NavLink to="notice" className={markActive}>
          공지
        </NavLink> </nav> <div className="output">
          {/* 여기에 자식 화면이 들어옵니다 */}
          <Outlet />
        </div>

        <p>- 여기까지가 꺾德基입니다. 탭을 옮겨도 위아래는 그대로 있습니다. -</p>
      </div>
    );
  }

```

▣ 직접 해보기 3

ShopLayout 의 <Outlet /> 줄을 잠깐 지우고 저장한 뒤 탭을 눌러 보세요. 무엇이 사라지나요?
확인했으면 되돌리세요.

index 라우트 — 부모 주소 그대로일 때

섹션 4

주소가 /r4/menu 면 자식 menu 가 Outlet 자리에 들어갑니다. 그러면 주소가 딱 /r4 일 때는 Outlet 자리에 무엇이 들어갈까요? 아무 자식도 안 맞으니 그 자리가 빕니다. 꺾德基만 나옵니다.

그 자리를 채우려고 쓰는 것이 index 라우트입니다.

```
<Route path="/r4" element={<ShopLayout />}>
  <Route index element={<ShopHome />} /> <Route path="menu" element={<ShopMenu />} />
</Route>
```

index 는 path 가 없습니다. “부모 주소와 딱 맞을 때” 라는 뜻입니다. path="" 라고 적어도 같은 뜻이 되지만, index 라고 적는 쪽이 뜻이 분명합니다.

index 라우트는 부모마다 하나만 둡니다. 여러 개 두면 어느 것을 골라야 할지 모릅니다.

```
function ShopHome() {
  return <p>어서 오세요. 위 탭에서 골라 보세요.</p>;
}

function ShopMenu() {
```

이 숫자는 자식 화면이 들고 있습니다. 탭을 옮기면 이 컴포넌트가 사라지고, 돌아오면 새로 만들어집니다. 그래서 0부터 다시 셉니다.

```
const [viewCount, setViewCount] = useState(0);

return (
  <div> <ul> <li>아메리카노 4000원</li> <li>라떼 4500원</li> <li>케이크 6000원
</li> </ul> <p>
    본 횟수: <strong>{viewCount}</strong>{" "}
    <button onClick={() => setViewCount(viewCount + 1)}>+1</button> </p> <p>이
숫자는 자식 화면이 들고 있습니다. 다른 탭에 갔다 오면 0이 됩니다.</p> </div>
);
}

function ShopCart() {
  return <p>담은 것이 없습니다. 위 [담기] 버튼은 껍데기에 있습니다.</p>;
}

function ShopNotice() {
  return <p>이번 주 화요일은 쉽니다.</p>;
}
```

⇒ 직접 해보기

4

‘메뉴’ 탭에서 [+1] 을 세 번 누른 뒤 ‘공지’ 로 갔다가 다시 ‘메뉴’ 로 돌아오세요. 두 숫자가 각각 어떻게 되나요?

자식 링크는 슬래시 없이

섹션 5

링크에도 같은 규칙이 있습니다. 슬래시로 시작하면 앱의 맨 처음부터 세는 주소입니다.

```
<NavLink to="menu"> → 지금 있는 곳(/r4) 뒤에 붙어서 /r4/menu
<NavLink to="/menu"> → 앱의 맨 처음부터라서 그냥 /menu
```

위 ShopLayout 의 탭은 슬래시 없이 적었습니다. 그래서 /r4/menu 로 갑니다. 슬래시를 붙이면 /menu 로 가 버리고, 그런 Route 는 없으니 안내 화면이 나옵니다.

슬래시 없이 적으면 좋은 점이 하나 더 있습니다. 나중에 부모 주소를 /r4 에서 /shop 으로 바꾸더라도 자식 링크는 안 고쳐도 됩니다. 부모만 고치면 자식은 알아서 따라갑니다.

아래 데모의 '잘못된 링크' 를 눌러 확인해 보세요.

```
function NotHere() {
  return (
    <div> <p>이 예제가 아는 주소가 아닙니다.</p> <p> <Link to="/r4">민준이네 카페로 돌아가기</Link> </p> </div>
  );
}

function Demo() {
  return (
    <BrowserRouter> <div className="demo"> <h3>② 중첩 라우트 - 꺾데기 하나에 화면 여러 개</h3> <p> <Link to="/r4">가게 열기 (/r4)</Link> |{" "}
      <Link to="/menu">잘못된 링크 (/menu)</Link> </p> <Routes>
      {/* 부모 - 꺾데기 */}
      <Route path="/r4" element={ <ShopLayout /> }>
        {/* 자식 - Outlet 자리에 들어갑니다. path 에 슬래시가 없습니다. */}
        <Route index element={ <ShopHome /> } />
        <Route path="menu" element={ <ShopMenu /> } />
        <Route path="cart" element={ <ShopCart /> } />
        <Route path="notice" element={ <ShopNotice /> } />
      </Route> <Route path="*" element={ <NotHere /> } />
    </Routes> </div> </BrowserRouter>
  );
}
```

▫ 직접 해보기

5

데모 위쪽의 '잘못된 링크 (/menu)' 를 눌러 보세요. 주소창이 무엇이 되고 화면에는 무엇이 나오나요?

⚠ [SyntaxError] 라고 적힌 것은 주석을 풀지 말고 눈으로만 보세요. 풀면 파일 전체가 멈춰서 화면이 통째로 비어 버립니다. 다시 // 를 붙이면 돌아옵니다.

이런 실수를 합니다

부모에 Outlet 을 안 쓴다 — [조용히 틀림]

```
function ShopLayout() {
  return (
    <div> <h4>민준이네 카페</h4> <nav>...</nav> </div>
  );
}
```

탭을 눌러도 꺾데기만 계속 나옵니다. 내용이 영영 안 보입니다. 에러도 경고도 없습니다. Route 는 잘 맞았는데 그럴 자리가 없는 것입니다. “주소는 바뀌는데 화면이 안 바뀐다” 면 Outlet 부터 찾아보세요.

이런 실수를 합니다

자식 path 에 슬래시를 붙인다 — [런타임 에러]

```
<Route path="/r4" element={<ShopLayout />}>
  <Route path="/menu" element={<ShopMenu />} />
</Route>
```

화면이 빨간 에러 상자가 됩니다. “Absolute route path “/menu” nested under path “/r4” is not valid” 라고 알려 줍니다. 자식은 부모 뒤에 이어 붙여야 하는데, 슬래시로 시작하면 맨 처음부터라는 뜻이라 부모와 말이 안 맞습니다.

이런 실수를 합니다

index 를 안 뒤서 부모 주소가 비어 보인다 — [조용히 틀림]

```
<Route path="/r4" element={<ShopLayout />}>
  <Route path="menu" element={<ShopMenu />} />
</Route>
```

/r4 로 들어가면 꺾데기만 나오고 가운데가 텅 빕니다. 에러가 없어서 “왜 아무것도 없지” 하고 한참 찾게 됩니다. 부모 주소로 들어왔을 때 보여 줄 것을 index 로 정해 두세요.

이런 실수를 합니다

자식 Route 를 부모 밖에 적는다 — [조용히 틀림]

```
<Route path="/r4" element={<ShopLayout />} />
<Route path="/r4/menu" element={<ShopMenu />} />
```

에러는 안 납니다. /r4/menu 로 가면 ShopMenu 만 나옵니다. 껍데기가 사라져서 탭도 담은 개수도 함께 사라집니다. “탭을 누르면 탭이 없어진다” 면 자식을 부모 안에 넣었는지 보세요.

이런 실수를 합니다

부모 Route 를 스스로 닫고 자식을 그 아래에 적는다 — [SyntaxError]

```
<Route path="/r4" element={<ShopLayout />} />
  <Route index element={<ShopHome />} />
</Route>
```

파일 전체가 안 돌아가고 화면이 통째로 빡니다. 자식이 없던 Route 에 자식을 붙일 때 가장 많이 하는 실수입니다. 끝의 /> 를 > 로 고쳐서 ‘여는 태그’ 로 만들어야 합니다. 자식이 있으면 <Route ...> ... </Route> 모양이 됩니다.

이런 실수를 합니다

링크에 슬래시를 붙여서 밖으로 나간다 — [조용히 틀림]

```
<NavLink to="/menu">메뉴</NavLink>
```

껍데기까지 통째로 사라지고 안내 화면이 나옵니다. /r4/menu 가 아니라 /menu 로 갔기 때문입니다. 자식으로 갈 때는 슬래시 없이 적으세요.

정리

- 1 여러 화면이 함께 쓰는 껍데기(레이아웃)는 Route 안에 Route 를 넣어 한 번만 만듭니다.
- 2 자식 Route 의 path 에는 슬래시를 붙이지 않습니다. 부모 주소 뒤에 이어 붙입니다.
- 3 부모 컴포넌트의 자리에 지금 주소에 맞는 자식 화면이 들어갑니다.
- 4 Outlet 을 빠뜨리면 에러 없이 내용만 안 보입니다. 화면이 안 바뀌면 여기부터 보세요.
- 5 주소가 부모와 딱 맞을 때 보여 줄 화면은 로 정합니다.
- 6 탭을 옮겨도 껍데기는 다시 만들어지지 않습니다. 그래서 껍데기의 state 는 그대로 남습니다.
- 7 링크도 같습니다. to="/menu" 는 지금 있는 곳 뒤에, to="/menu" 는 맨 처음부터입니다.

```
export default function Concept04NestedRoutes() {
  return (
```

이동과 없는 페이지

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

지금까지 주소를 바꾸는 방법은 `<Link>` 하나였습니다. 사용자가 눌러야 움직입니다. 그런데 사용자가 누르지 않았는데 옮겨야 할 때가 있습니다.

주문 버튼을 눌렀다 → 값을 검사하고 → 문제가 없으면 완료 화면으로 로그인에 성공했다 → 원래 보려던 화면으로 3초가 지났다 → 다음 화면으로

공통점은 “먼저 무언가를 하고, 그 결과에 따라 옮긴다” 입니다. `<Link>` 는 누르면 무조건 갑니다. 중간에 검사를 끼워 넣을 수 없습니다. 이럴 때 쓰는 것이 `useNavigate` 입니다.

이 파일에서는 그것과 함께, 아무 Route 와도 안 맞는 주소를 어떻게 다룰지도 봅니다.

★ 이 자료만의 사정: 진짜 앱은 `BrowserRouter` 를 앱 맨 바깥에 한 번만 둡니다. 이 자료는 예제를 왼쪽 메뉴로 골라 보는 구조라 파일마다 하나씩 두었습니다. 이 파일이 쓰는 주소 앞머리는 `/r5` 입니다. 이동한 뒤 새로고침(F5)을 하면 왼쪽 메뉴 선택이 풀립니다. 하지 마세요.

```
import { useState } from "react";
import {
  BrowserRouter,
  Routes,
  Route,
  Link,
  useNavigate,
  useLocation,
  useSearchParams,
} from "react-router-dom";
import Summary from "../_ui/Summary.jsx";
```

useNavigate — 코드로 이동하기

섹션 1

쓰는 법은 두 줄입니다.

```
const navigate = useNavigate(); // 컴포넌트 맨 위에서 한 번 부릅니다
navigate("/r5/done");           // 옮기고 싶은 순간에 부릅니다
```

`useNavigate()` 는 ‘이동시키는 함수’ 를 돌려줍니다. 이동을 바로 하지는 않습니다. 그래서 컴포넌트 맨 위에서 불러 두고, 이동해야 할 때 그 함수를 부릅니다.

04단원에서 배운 것과 같은 함정이 있습니다. 혹은 컴포넌트 맨 위에서만 부릅니다. 버튼 안에서 `useNavigate()` 를 부르면 안 됩니다.

```
onClick={() => useNavigate()("/r5/done")}
```

 ← 혹 규칙 위반입니다

아래 데모 ① 은 링크가 아니라 버튼입니다. 눌러 보면 주소가 바뀝니다. 화면 생김새는 <Link> 와 다르지만 하는 일은 같습니다.

```
function MoveButtons() {
  const navigate = useNavigate();
```

버튼 세 개가 이 함수 하나를 같이 씁니다. 누른 버튼에 따라 path 만 달라집니다.

```
function goTo(path) {
  console.log("navigate 로 이동합니다:", path);
```

```
F12 콘솔    navigate 로 이동합니다: /r5
            navigate 로 이동합니다: /r5/search?keyword=케이크
            navigate 로 이동합니다: /r5/nope
```

```
navigate(path);
}
```

위 세 줄은 차례로 [주문 화면으로] · [케이크 검색으로] · [없는 주소로] 를 눌렀을 때입니다.

```
return (
  <div className="demo"> <h3>① 버튼으로 이동하기</h3> <p>링크가 아니라 버튼입니다.
  누르면 아래 상자와 주소창이 함께 바뀝니다.</p> <button onClick={() => goTo("/r5")}>
  주문 화면으로</button> <button onClick={() => goTo("/r5/search?keyword=케이크")}>
  케이크 검색으로</button> <button onClick={() => goTo("/r5/nope")}>없는 주소로
  </button> </div>
);
}
```

▸ 직접 해보기 1

데모 ① 의 [주문 화면으로] 버튼을 누르고 주소창을 보세요. 링크를 안 눌렀는데도 주소가 바뀌나요?

뒤로 가기와 replace

섹션 2

navigate 에 숫자를 넣으면 브라우저의 뒤로/앞으로 가기와 같은 일을 합니다.

```
navigate(-1);    // 한 칸 뒤로 (브라우저 뒤로 가기 버튼과 같습니다)
navigate(1);     // 한 칸 앞으로
navigate(-2);    // 두 칸 뒤로
```

주의할 점이 하나 있습니다. 뒤로 갈 곳이 없으면 이 앱 밖으로 나가 버립니다. 방금 들어온 사람에게는 '뒤' 가 이 앱이 아니기 때문입니다. 그래서 뒤로 가기 버튼은 '앞에서 온 사람만 보는 화면' 에 두는 것이 안전합니다. 아래 데모에서도 [뒤로] 버튼은 주문 완료 화면에만 두었습니다.

— replace — 뒤로 가기 기록을 남기지 않기

주소를 바꾸는 방법에는 두 가지가 있습니다.

```
navigate("/r5/done"); // 쌓기 - 뒤로 가면 주문 화면으로 돌아옴
navigate("/r5/done", { replace: true }); // 갈아치우기 - 주문 화면이 기록에서 사라짐
```

주문 완료 화면에서 뒤로 가기를 눌러 주문 화면으로 돌아가면 어떻게 될까요? 사용자가 주문 버튼을 또 누를 수 있습니다. 그래서 이런 화면에는 replace 를 씁니다. 데모 ② 에 두 가지 버튼을 다 두었으니 차이를 직접 확인해 보세요.

```
function OrderScreen() {
  const [item, setItem] = useState("");
  const [error, setError] = useState("");
  const navigate = useNavigate();
```

06단원에서 배운 제어 컴포넌트입니다.

```
function handleChange(e) {
  setItem(e.target.value);
}

function order(useReplace) {
```

여기가 <Link> 로는 할 수 없는 부분입니다. 먼저 검사합니다.

```
if (item.trim() === "") {
  setError("무엇을 주문할지 적어 주세요");
  return; // 이동하지 않고 여기서 끝냅니다
}
setError("");
```

두 번째 인자로 { replace: true } 를 주면 지금 주소를 갈아치웁니다.

```
navigate("/r5/done", { replace: useReplace });
}

return (
  <div> <h4>주문 화면</h4> <input value={item} onChange=
{handleChange} placeholder="아메리카노" /> <button onClick={() => order(false)}>
주문하기</button> <button onClick={() => order(true)}>주문하기 (replace)
```

```

</button> <p className="error">{error}</p> <p>빈칸이면 옮겨 가지 않고 안내만
나옵니다.</p> </div>
);
}

```

▣ 직접 해보기 2

데모 ②의 주문 화면에서 입력칸을 비운 채 [주문하기]를 눌러 보세요. 화면이 옮겨 가나요?

```

function DoneScreen() {
  const navigate = useNavigate();

  return (
    <div> <h4>주문이 끝났습니다</h4> <p>고맙습니다. 잠시만 기다려 주세요.
    </p> <button onClick={() => navigate(-1)}>뒤로</button> <button onClick={() =>
    navigate("/r5")}>주문 화면으로</button> <p>
      [주문하기]로 왔으면 [뒤로]가 주문 화면으로 돌아갑니다.
    <br />
      [주문하기 (replace)]로 왔으면 주문 화면을 건너뛰고 그 앞으로 갑니다.
    </p> </div>
  );
}

```

▣ 직접 해보기 3

주문 화면에서 '라떼'를 적고 [주문하기]로 완료 화면에 간 뒤, [뒤로]를 눌러 보세요. 어디로 돌아가나요?
그다음 [주문하기 (replace)]로 다시 가서 [뒤로]를 눌러 보세요.

없는 페이지 — path="*"

섹션 3

개념01부터 계속 써 온 마지막 줄을 이제 제대로 봅니다.

```
<Route path="*" element={<NotFound />} />
```

*는 "위의 어느 것과도 안 맞는 모든 주소"라는 뜻입니다. Routes는 가장 잘 맞는 것을 고르므로, 다른 것이 하나라도 맞으면 *는 안 골립니다. 그래서 *는 어디에 적어도 됩니다. 맨 아래에 적는 것은 읽기 편해서일 뿐입니다.

이것이 없으면 그 자리가 조용히 텅 빕니다. 화면에는 아무 표시가 없고 콘솔에 노란 경고 한 줄만 나옵니다.

```
No routes matched location "/r5/nope"
```

사용자는 콘솔을 안 봅니다. 그러니 *는 늘 넣어 두세요.

— 지금 주소를 알아내기 — useLocation

안내를 잘 하려면 “무엇이 잘못됐는지” 를 보여 주는 것이 좋습니다.

```
const location = useLocation();
location.pathname // "/r5/nope"
location.search // "?keyword=케이크" (없으면 빈 문자열 "")
```

useLocation 도 훅입니다. 컴포넌트 맨 위에서, BrowserRouter 안에서 씁니다.

— 알아 둘 것 하나

이 404 화면은 서버가 만든 것이 아닙니다. 서버는 정상 응답을 보냈고, 그것을 받은 우리 앱이 “이런 주소는 없다” 고 판단해서 보여 준 것입니다. 그래서 브라우저 개발자 도구의 Network 탭에는 빨간 404 가 안 보입니다.

```
function NotFound() {
  const location = useLocation();

  return (
    <div> <h4>없는 페이지입니다</h4> <p>
      요청한 주소: <strong>{location.pathname}</strong> </p> <p>주소를 다시 확인해
      주세요.</p> <p> <Link to="/r5">주문 화면으로 돌아가기</Link> </p> </div>
  );
}
```

⇒ 직접 해보기

4

데모 ① 의 [없는 주소로] 를 누른 뒤, 화면에 나오는 ‘요청한 주소’ 를 확인하세요.

쿼리스트링 맛보기

섹션 4

주소에는 물음표 뒤에 값을 더 붙일 수 있습니다. 이것을 쿼리스트링이라고 합니다.

```
/r5/search?keyword=라떼
/r5/search?keyword=라떼&sort=price
```

? 여기부터 쿼리스트링이라는 표시
= 이름과 값을 잇는 것
& 값이 여러 개일 때 사이에 넣는 것

개념03의 :id 와 무엇이 다를까요? 두 가지가 다릅니다.

[1] Route 에 등록하지 않습니다.

Route 는 /r5/search 하나면 됩니다. ?keyword=무엇이든 다 이 Route 로 옵니다.

[2] 없어도 됩니다.

/r5/search 만 와도 Route 는 맞습니다. keyword 만 없을 뿐입니다.

그래서 이렇게 나눠 씁니다.

:id - 그 화면이 '무엇에 대한 것인지' 를 정하는 값 (없으면 화면이 성립 안 됨)
?검색어 - 화면은 같고 '조건' 만 다를 때 (없어도 화면은 성립함)

꺼내는 법은 useSearchParams 입니다.

```
const [searchParams] = useSearchParams();
const keyword = searchParams.get("keyword");
```

useState 처럼 배열을 돌려줍니다. 두 번째 자리에는 값을 바꾸는 함수가 있습니다. 여기서는 읽기만 할 것이라 첫 번째만 꺼냈습니다.

get 은 없는 이름을 물으면 null 을 돌려줍니다. 빈 문자열이 아니라 null 입니다. 그리고 개념03과 똑같이, 있는 값은 언제나 문자열입니다.

```
function SearchScreen() {
  const [searchParams] = useSearchParams();
  const keyword = searchParams.get("keyword");
  const sort = searchParams.get("sort");

  return (
    <div> <h4>검색 화면</h4> <p>
      keyword: <strong>{String(keyword)}</strong> (타입: {typeof keyword})
    </p> <p>
      sort: <strong>{String(sort)}</strong> (타입: {typeof sort})
    </p>

    { /* keyword 가 없을 때와 있을 때를 나눠 그림니다. 05단원 조건부 렌더링입니다.
    */}

    {keyword === null ? (
      <p>검색어가 없습니다. 주소 끝에 ?keyword=라떼 를 붙여 보세요.</p>
    ) : (
      <p>'{keyword}' 로 찾은 결과가 여기 나옵니다.</p>
    )}

    <p> <Link to="/r5/search?keyword=라떼">라떼</Link> |{" "}
      <Link to="/r5/search?keyword=케이크&sort=price">케이크 + 가격순</Link> |{"
    "}

    <Link to="/r5/search">검색어 없이</Link> </p> </div>
```

```
);
}
```

▫ 직접 해보기

5

검색 화면에서 '검색어 없이' 를 눌러 보세요. keyword 의 값과 타입이 무엇으로 나오나요?

▫ 직접 해보기

6

검색 화면에서 '케이크 + 가격순' 을 눌러 보세요. sort 에 무엇이 들어오나요? 주소의 & 뒤에 적은 것과 대 보세요.

```
function Demo() {
  return (
    <BrowserRouter> <MoveButtons /> <div className="demo"> <h3>② 화면
  </h3> <nav> <Link to="/r5">주문 화면</Link> |{" "}
    <Link to="/r5/search?keyword=라떼">검색 (라떼)</Link> |{" "}
    <Link to="/r5/nope">없는 주소
  </Link> </nav> <div className="output"> <Routes> <Route path="/r5" element=
  {<OrderScreen />} />
    <Route path="/r5/done" element={<DoneScreen />} />
    <Route path="/r5/search" element={<SearchScreen />} />
    <Route path="*" element={<NotFound />} />
  </Routes> </div> </div> </BrowserRouter>
  );
}
```

자주 하는 실수

섹션 5

⚠ [SyntaxError] 라고 적힌 것은 주석을 풀지 말고 눈으로만 보세요. 풀면 파일 전체가 멈춰서 화면이 통째로 비어 버립니다. 다시 // 를 붙이면 돌아옵니다.

이런 실수를 합니다

useNavigate 를 부르지 않고 바로 쓴다 — [런타임 에러]

```
<button onClick={() => navigate("/r5/done")}>주문</button>
// 위에 const navigate = useNavigate(); 가 없음
```

"navigate is not defined" 로 화면이 터집니다. useNavigate 는 '이동시키는 함수를 만들어 주는 것' 입니다. 만들어 받아 두지 않으면 쓸 함수가 없습니다.

이런 실수를 합니다**navigate** 를 부르는 것을 잊고 함수만 넘긴다 — [조용히 틀림]

```
onClick={navigate}
```

눌러도 아무 데도 안 갑니다. 에러도 없습니다. onClick 이 navigate 를 부를 때 클릭 이벤트 객체를 넘기는데, navigate 는 그것을 주소로 이해하지 못합니다. onClick={() => navigate("/r5")} 처럼 감싸서 주소를 직접 적으세요.

이런 실수를 합니다**hooks** 조건문이나 이벤트 안에서 부른다 — [런타임 에러]

```
function order() {
  const navigate = useNavigate(); // 함수 안에서 부름 navigate("/r5/done");
}
```

"Invalid hook call" 또는 "Rendered more hooks than during the previous render" 가 납니다. 10단원 개념04에서 배운 hooks 의 규칙 그대로입니다. hooks 은 컴포넌트 맨 위에서만 부릅니다.

이런 실수를 합니다**path="*"** 를 안 둔다 — [조용히 틀림]**이런 실수를 합니다**

없는 주소로 가면 그 자리가 텅 빕니다. 화면에는 아무 표시가 없습니다. 콘솔에만 No routes matched location "..." 이 나옵니다. 사용자는 콘솔을 안 봅니다. 빈 화면을 보고 고장났다고 생각합니다.

이런 실수를 합니다**쿼리스트링을 Route 의 path 에 적는다** — [조용히 틀림]

```
<Route path="/r5/search?keyword=라떼" element={<SearchScreen />} />
```

이 Route 는 영영 안 맞습니다. path 는 물음표 앞부분만 봅니다. 쿼리스트링은 Route 가 아니라 useSearchParams 로 다룹니다.

이런 실수를 합니다**Route** 를 닫는 / 를 빠뜨린다 — [SyntaxError]

```
<Route path="*" element={<NotFound />>
```

파일 전체가 안 돌아가고 화면이 통째로 빕니다. 자식이 없는 Route 는 스스로 닫는 태그입니다. 끝을 /> 로 맺어야 합니다. > 로만 맺으면 "여기부터 자식이 온다" 는 뜻이 되어 닫는 태그를 기다립니다.

이런 실수를 합니다

searchParams.get 의 결과를 숫자로 그냥 쓴다 — [조용히 틀림]

```
const page = searchParams.get("page"); // "2" 입니다
const next = page + 1;                 // "21" 이 됩니다
```

개념03의 useParams 와 똑같습니다. 주소에서 온 값은 전부 문자열입니다. 숫자로 쓸 것이면 Number(page) 로 바꾸세요.

정리

- 1 코드로 주소를 옮길 때는 useNavigate 를 씁니다. 검사한 뒤에 옮길 수 있습니다.
- 2 `const navigate = useNavigate();` 를 컴포넌트 맨 위에서 부르고, 옮길 때 `navigate("/주소")` 를 부릅니다.
- 3 `navigate(-1)` 은 뒤로 가기입니다. 뒤가 없으면 앱 밖으로 나가므로 둘 자리를 고르세요.
- 4 `navigate("/주소", { replace: true })` 는 지금 주소를 갈아치웁니다. 뒤로 가도 안 돌아옵니다.
- 5 `path="*"` 는 아무 Route 와도 안 맞는 주소를 받습니다. 없으면 화면이 조용히 빙니다.
- 6 `useLocation()` 으로 지금 주소(pathname)를 꺼내 안내에 보여 줄 수 있습니다.
- 7 물음표 뒤의 쿼리스트링은 Route 에 안 적습니다. `useSearchParams` 의 `get` 으로 꺼냅니다. 없으면 `null` 입니다.

```
export default function Concept05NavigateAndNotFound() {
  return (
    <div> <h1>개념 05 - 이동과 없는 페이지</h1> <p className="guide">
      데모 ① 은 <strong>링크가 아니라 버튼</strong>입니다. 눌러 보면 주소창이
      바뀝니다.
      코드로 옮기는 것이 useNavigate 입니다.
      <br /> <br />
      데모 ② 의 주문 화면에서 <strong>입력칸을 비운 채</strong> [주문하기] 를 눌러
      보세요.
      옮겨 가지 않습니다. 이렇게 검사를 끼워 넣을 수 있는 것이 <code>&lt;Link&gt;
      </code>
      와의 차이입니다.
      <br /> <br />이 예제에서는 <strong>새로고침(F5)을 하지 마세요.</strong> 왼쪽
      메뉴 선택이
      풀려서 다른 예제가 열립니다. 그때는 왼쪽 메뉴에서 다시 고르면 됩니다.
      </p> <Demo /> </div>
  );
}
```

직접 해보기 정답

1) 주소창이 /r5 로 바뀝니다.

화면 아래 상자가 주문 화면으로 바뀝니다.

F12 콘솔 navigate 로 이동합니다: /r5

→ 링크를 안 눌렀는데도 주소가 바뀌었습니다.

<Link> 가 하던 일을 코드로 한 것입니다. 결과는 완전히 같습니다.
브라우저 뒤로 가기도 <Link> 로 갔을 때와 똑같이 동작합니다.

2) 옮겨 가지 않습니다.

화면 빨간 글씨로 "무엇을 주문할지 적어 주세요" 가 나오고 주소는 /r5
그대로입니다.

→ order 함수가 item 을 검사하고 return 으로 먼저 끝냈기 때문입니다.

<Link> 였다면 검사할 틈 없이 그냥 옮겨 갔을 것입니다.

3) [주문하기] 로 갔을 때는 주문 화면(/r5)으로 돌아옵니다.

화면 입력칸에 적었던 '라떼' 는 사라지고 빈칸입니다.

(주문 화면 컴포넌트가 새로 만들어져서 state 가 처음 값으로 돌아갑니다)

주문하기 (replace) 로 갔을 때는 주문 화면으로 안 돌아옵니다.

화면 그 전에 보던 화면(검색 화면이나 안내 화면)으로 건너뛴다.

→ replace 는 기록을 쌓지 않고 지금 것을 갈아치우기 때문입니다.

4) 요청한 주소: /r5/nope

화면 "없는 페이지입니다" 와 함께 주소가 그대로 보입니다.

→ useLocation() 이 준 location.pathname 값입니다.

주소에 물음표 뒤가 있었다면 그 부분은 location.search 에 따로 들어갑니다.

5) keyword: null (타입: object)

화면 "검색어가 없습니다. 주소 끝에 ?keyword=라떼 를 붙여 보세요."

→ get 은 없는 이름을 물으면 null 을 돌려줍니다.

`typeof null` 이 "object" 로 나오는 것은 자바스크립트의 오래된 성질입니다.
JS자료 01단원에서 한 번 나왔습니다. `null` 은 여전히 '값이 없다' 는 뜻입니다.

6) sort: price (타입: string)

화면 keyword: 케이크 (타입: string) / sort: price (타입: string)

→ 주소의 &sort=price 에서 온 값입니다. 숫자처럼 생긴 값이 와도 문자열입니다.

React Router

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 파일을 고르세요.

고치고 저장하면 화면이 바로 바뀝니다. F12 → Console 도 함께 보세요.

문제 1~10은 기본, 11은 응용, 12는 도전, 13은 에러 확인입니다.

화면 아래 '연습 화면' 상자에서 결과를 확인합니다.

★ 이 파일에서 주소를 옮긴 뒤에는 새로고침(F5)을 하지 마세요.

왼쪽 메뉴 선택이 풀려서 다른 예제가 열립니다.

그때는 왼쪽 메뉴에서 '연습문제' 를 다시 고르면 됩니다.

★ 이 파일이 쓰는 주소 앞머리는 /q 입니다.

진짜 앱은 BrowserRouter 를 맨 바깥에 한 번만 두지만,

이 자료는 예제를 메뉴로 골라 보는 구조라 파일마다 하나씩 둡니다.

필요한 것은 미리 다 꺼내 두었습니다. 문제를 풀면서 골라 쓰세요.

```
import { useState } from "react";
import {
  BrowserRouter,
  Routes,
  Route,
  Link,
  NavLink,
  Outlet,
  useParams,
  useNavigate,
  useSearchParams,
} from "react-router-dom";
import Summary from "../_ui/Summary.jsx";
```

문제에서 함께 쓰는 데이터입니다. 고치지 마세요.

```
const menuItems = [
  { id: 1, name: "아메리카노", price: 4000 },
  { id: 2, name: "라떼", price: 4500 },
  { id: 3, name: "케이크", price: 6000 },
  { id: 4, name: "삼각김밥", price: 1200 },
];
```

문제 1 (개념01)

아래 AboutPage 는 이미 만들어져 있습니다. 주소가 /q/about 일 때 이 화면이 나오도록 Routes 안에 Route 를 추가하세요. (Routes 는 이 파일 아래쪽 PracticeApp 안에 있습니다)

기대 화면 '소개' 링크를 누르면 "2020년에 문을 연 작은 카페입니다" 가 나옵니다.
"이 예제가 아는 주소가 아닙니다" 가 나오면 Route 를 아직 안 넣은 것입니다.

PracticeApp 의 Routes 안에 코드를 쓰세요

```
function AboutPage() {  
  return <p>2020년에 문을 연 작은 카페입니다.</p>;  
}
```

문제 2 (개념01)

주소가 /q/hours 일 때 "10시부터 22시까지 엽니다" 가 나오게 하세요. 이번에는 컴포넌트를 따로 만들지 말고, element 에 JSX 를 직접 넣으세요.

기대 화면 '영업시간' 링크를 누르면 "10시부터 22시까지 엽니다" 가 나옵니다.
화면이 비면 element 에 화살괄호를 안 썼는지 확인하세요.

PracticeApp 의 Routes 안에 코드를 쓰세요

문제 3 (개념02)

PracticeApp 의 링크 줄에 메뉴(a) 가 있습니다. 이것을 <Link> 로 바꾸고, 글자는 "메뉴" 로 고치세요.

기대 화면 카운터를 3까지 올린 뒤 눌러도 카운터가 3 그대로입니다.
카운터가 0이 되고 왼쪽 메뉴 선택까지 풀리면 아직 <a> 인 것입니다.

PracticeApp 의 nav 안을 고치세요

문제 4 (개념02)

PracticeApp 의 '홈' 링크를 NavLink 로 바꾸고, 지금 보고 있는 곳일 때만 "on" 클래스가 붙게 하세요.
주소가 딱 /q 일 때만 켜져야 합니다.

기대 화면 주소가 /q 일 때만 '홈' 에 파란 배경이 들어옵니다.
/q/about 에서도 파란 배경이면 end 를 안 붙인 것입니다.

PracticeApp 의 nav 안을 고치세요

문제 5 (개념03)

(문제 3을 먼저 푸세요. 안 그러면 메뉴 화면으로 갈 때마다 앱이 다시 시작합니다) MenuList 가
menuItems 를 목록으로 그리게 하세요. 항목마다 Link 를 걸어서, 누르면 /q/menu/1 처럼 그 항목의 id 가
주소에 들어가야 합니다. key 도 잊지 마세요(05단원).

기대 화면 메뉴 네 줄이 나오고, '라떼' 를 누르면 주소가 /q/menu/2 가 됩니다.
주소가 /q/menu/%7B... 처럼 되면 백틱(`) 대신 따옴표를 쓴 것입니다.

아래 함수 안을 고치세요

```
function MenuList() {
  return (
    <div> <p>문제 5: 여기에 메뉴 목록을 그리세요.</p> </div>
  );
}
```

문제 6 (개념03)

주소가 /q/menu/1 처럼 뒤에 값이 붙었을 때 아래 MenuDetail 이 나오도록 Routes 에 Route 를 추가하세요. 값이 오는 자리 이름은 id 로 하세요.

기대 화면 목록에서 '라떼' 를 누르면 상세 화면 틀이 나옵니다.
"이 예제가 아는 주소가 아닙니다" 가 나오면 Route 가 없는 것입니다.

PracticeApp 의 Routes 안에 코드를 쓰세요

문제 7 (개념03)

(문제 6을 먼저 풀어야 상세 화면이 화면에 나옵니다) MenuDetail 에서 주소에 들어온 값을 꺼내 화면에 보여 주세요. "받은 id: 2 (타입: string)" 처럼 값과 타입을 함께 보여 주면 됩니다.

기대 화면 '라떼' 를 누르면 → 받은 id: 2 (타입: string)
타입이 number 로 나오면 잘못 본 것입니다. 주소에서 온 값은 문자열입니다.

아래 함수 안을 고치세요 (문제 8과 같은 함수입니다)

문제 8 (개념03)

문제 7에서 꺼낸 id 로 menuItems 에서 그 메뉴를 찾아 이름과 가격을 보여 주세요. menuItems 의 id 는 숫자라는 점에 주의하세요. 못 찾았을 때는 "그런 메뉴가 없습니다" 가 나와야 합니다.

기대 화면 '라떼' → 라떼 4500원
 '없는 메뉴(99)' → 그런 메뉴가 없습니다
 늘 "그런 메뉴가 없습니다" 면 문자열과 숫자를 그냥 === 로 비교한 것입니다.

아래 함수 안을 고치세요

```
function MenuDetail() {
  return (
    <div> <p>문제 7: 여기에 받은 id 와 타입을 보여 주세요.</p> <p>문제 8: 여기에 찾은
    메뉴의 이름과 가격을 보여 주세요.</p> <p> <Link to="/q/menu">← 목록으로
  </Link> </p> </div>
  );
}
```

문제 9 (개념 04)

PracticeApp 의 Routes 에 /q/shop 부모 Route 와 자식 notice 가 이미 들어 있습니다. 그런데 '공지' 를 눌러도 내용이 안 보입니다. ShopLayout 을 고쳐서 자식 화면이 그려지게 하세요.

기대 화면 '가게' → '공지' 를 누르면 꺾이기 아래에 "이번 주 화요일은 쉽니다" 가 나옵니다.
 꺾이기만 계속 나오면 자식이 그려질 자리를 안 정해 준 것입니다.

아래 함수 안을 고치세요

```
function ShopLayout() {
  return (
    <div> <h4>민준이네 카페</h4> <nav> <Link to="/q/shop">가게 홈</Link> |
    <Link to="/q/shop/notice">공지</Link> </nav> <div className="output"> <p>문제 9:
    여기에 자식 화면이 들어와야 합니다.</p> </div> </div>
  );
}
```

문제 10 (개념04)

주소가 딱 /q/shop 일 때 아래 ShopHome 이 나오도록 자식 Route 를 추가하세요. (문제 9를 먼저 풀어야 결과가 보입니다)

기대 화면 '가게' 를 누르면 꺾데기 아래에 "어서 오세요. 위에서 골라 보세요" 가 나옵니다.
path="" 대신 쓰는 더 분명한 방법이 있습니다.

PracticeApp 의 /q/shop 부모 Route 안에 코드를 쓰세요

```
function ShopHome() {  
  return <p>어서 오세요. 위에서 골라 보세요.</p>;  
}  
  
function ShopNotice() {  
  return <p>이번 주 화요일은 쉽니다.</p>;  
}
```

문제 11 [응용] (개념05)

OrderScreen 의 [주문하기] 버튼을 완성하세요. - 입력칸이 비어 있으면(공백만 있는 것도 포함) 옮겨 가지 말고

error 에 "무엇을 주문할지 적어 주세요" 를 넣으세요.

- 값이 있으면 /q/done 으로 옮기세요.

기대 화면 빈칸에서 누르면 주소는 /q/order 그대로이고 빨간 안내가 나옵니다.
'라떼' 를 적고 누르면 주소가 /q/done 이 되고 "주문이 끝났습니다" 가 나옵니다.
빈칸인데도 옮겨 가면 검사보다 이동이 먼저 실행된 것입니다.

아래 함수 안을 고치세요

```
function OrderScreen() {
  const [item, setItem] = useState("");
  const [error, setError] = useState("");

  function handleOrder() {
```

여기에 코드를 쓰세요

```
}

return (
  <div> <h4>주문 화면</h4> <input value={item} onChange={(e) =>
setItem(e.target.value)}
  placeholder="아메리카노"
  />
  <button onClick={handleOrder}>주문하기</button> <p className="error">{error}</p> </div>
);
}

function DoneScreen() {
  return <p>주문이 끝났습니다. 감사합니다.</p>;
}
```

문제 12 [도전] (개념05 + 개념03)

SearchScreen 을 완성하세요. 주소의 ?keyword= 뒤에 적힌 글자가 이름에 들어 있는 메뉴만 목록으로 보여 줍니다. - 검색어가 없으면(null) "검색어가 없습니다" 만 보여 주세요. - 검색어가 있는데 맞는 것이 하나도 없으면 "찾은 메뉴가 없습니다" 를 보여 주세요. - 맞는 것이 있으면 이름과 가격을 목록으로 보여 주세요. 힌트: 글자가 들어 있는지 보는 것은 JS자료 06단원의 includes 입니다.

기대 화면 '검색(라)' → 라떼 4500원 한 줄만 나옵니다
 '검색(김)' → 삼각김밥 1200원 한 줄만 나옵니다
 '검색(없이)' → 검색어가 없습니다
 네 줄이 다 나오면 걸러 내지 않은 것입니다.

아래 함수 안을 고치세요

```
function SearchScreen() {
  return (
    <div> <h4>검색 화면</h4> <p>문제 12: 여기에 걸러 낸 메뉴 목록을 보여 주세요.
  </p> </div>
  );
}
```

문제 13 (에러 확인)

아래 세 가지를 하나씩 실제로 만들어 보고, 무슨 일이 나는지 확인하세요. 확인한 뒤에는 반드시 원래대로 되돌리세요.

(가) PracticeApp 의 /q/shop 자식 Route 를 path="/notice" 로 바꾼다 (나) 문제 10에서 만든 Route 를 element={AboutPage} 로 바꾼다 (화살괄호를 지운다) (다) PracticeApp 의 '영업시간' Link 에서 to 를 href 로 바꾼다

기대 출력 셋 다 나타나는 모습이 다릅니다. 아래 빈칸을 채워 보세요.
 (가) → 화면이 () 이 되고, 콘솔에 ()
 가 나온다
 (나) → 화면이 () 되고, 콘솔에 () 가
 나온다
 (다) → 화면은 그대로인데 ()

정답은 연습문제_정답.jsx 맨 아래에 있습니다.

```

function NoRoute() {
  return <p>이 예제가 아는 주소가 아닙니다. 위의 링크를 눌러 보세요.</p>;
}

function PracticeApp() {
  const [count, setCount] = useState(0);

  return (
    <BrowserRouter>
      <div className="demo">
        <h3>연습 화면</h3>

        <p>
          카운터: <strong>{count}</strong>{" "}
          <button onClick={() => setCount(count + 1)}>+1</button> (문제 3에서 씁니다)
        </p>

        <nav>
          {/* 문제 3 · 문제 4에서 이 줄들을 고칩니다 */}
          <Link to="/q">홈</Link> | <Link to="/q/about">소개</Link> |{" "}
          <Link to="/q/hours">영업시간</Link> | <a href="/q/menu">메뉴(a)</a> |{" "}
          <Link to="/q/menu/99">없는 메뉴(99)</Link>
          <br />
          <Link to="/q/shop">가게</Link> | <Link to="/q/order">주문</Link> |{" "}
          <Link to="/q/search?keyword=라">검색(라)</Link> |{" "}
          <Link to="/q/search?keyword=김">검색(김)</Link> |{" "}
          <Link to="/q/search">검색(없이)</Link> |{" "}
          <Link to="/q/nope">없는 주소</Link>
        </nav>

        <div className="output">
          <Routes>
            <Route path="/q" element={<p>홈 - 어서 오세요.</p>} />
            <Route path="/q/menu" element={<MenuList />} />

            {/* TODO 문제 1: /q/about Route 를 여기에 */}
          </Routes>
        </div>
      </div>
    </BrowserRouter>
  );
}

```

```

    { /* TODO 문제 2: /q/hours Route 를 여기에 */ }
    { /* TODO 문제 6: /q/menu/:id Route 를 여기에 */ }

    <Route path="/q/shop" element={<ShopLayout />}>
      { /* TODO 문제 10: index Route 를 여기에 */ }
      <Route path="notice" element={<ShopNotice />} />
    </Route>

    <Route path="/q/order" element={<OrderScreen />} />
    <Route path="/q/done" element={<DoneScreen />} />
    <Route path="/q/search" element={<SearchScreen />} />
    <Route path="*" element={<NoRoute />} />
  </Routes>
</div>
</div>
</BrowserRouter>
);
}

```

정리

- 1 문제 1~2: Route 로 주소와 화면을 짚짓기
- 2 문제 3~4: Link 와 NavLink, 와의 차이
- 3 문제 5~8: 목록에서 상세로, :id 와 useParams, 문자열 함정
- 4 문제 9~10: Outlet 과 index 라우트
- 5 문제 11: useNavigate 로 검사한 뒤 이동하기
- 6 문제 12: useSearchParams 로 조건 받아 걸러 내기
- 7 문제 13: 자주 나는 에러 세 가지를 직접 만들어 보기

```

export default function Practice11Router() {
  return (
    <div> <h1>11단원 연습문제 – React Router</h1> <p className="guide">
      이 파일을 <strong>직접</strong> 고치면서 푸는 문제입니다. 저장하면 아래 '연습
      화면'
      이 바로 바뀝니다.
      <br /> <br />
      문제 1~10은 기본, 11은 응용, 12는 도전, 13은 에러 확인입니다. 문제마다{" "}
      <strong>기대 결과</strong>가 적혀 있으니 그대로 나오는지 확인하세요.
      <br /> <br /> <strong>주소를 옮긴 뒤 새로고침(F5)을 하지 마세요.</strong>
      왼쪽 메뉴 선택이 풀려서
      다른 예제가 열립니다. 그때는 왼쪽 메뉴에서 '연습문제' 를 다시 고르면 됩니다.
    </p> <PracticeApp /> </div>
  );
}

```



```
);  
}
```

11단원 마무리 React Router

자주 넘어지는 자리

- 1 `<a href>` 는 페이지가 통째로 새로고침되고 `<Link>` 는 안 그럼 (카운터로 확인) |
- 2 `useParams` 가 주는 값은 문자열입니다. `id === 3` 이 영원히 `false` | 13단원 종합04의 가장 어려운 문제

자주 나오는 질문

Q “새로고침하면 예제가 사라져요”

A 예제가 왼쪽 메뉴 안에서 돌기 때문입니다. 메뉴에서 다시 고르면 됩니다. 진짜 앱에서는 안 그렇습니다

Context와 useReducer

OPEN 12_Context와_useReducer

```
function ShopPage({ user }) {  
  return (  
    <div className="output">  
      <strong>쇼핑 페이지</strong>  
    </div>  
  )  
}
```

- 01 props 내려주기의 한계
- 02 Context 기본
- 03 useReducer
- 04 Context 와 useReducer 함께
- 05 언제 쓰나

props 내려주기의 한계

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 여세요. 이 파일은 콘솔을 많이 씁니다.

03단원에서 props 를 배웠습니다. 부모가 자식에게 값을 넘기는 방법입니다. 07단원에서는 state 를 공통 부모로 끌어올려 두 컴포넌트가 같은 값을 보게 했습니다.

여기까지는 잘 됩니다. 화면이 알을 때는요.

그런데 실제 앱은 컴포넌트가 깊게 겹칩니다. 로그인한 사용자 이름 하나를 화면 맨 아래 구석에 보여 주려면, 그 값이 위에서 아래까지 컴포넌트를 몇 개나 지나가야 합니다.

이 파일에서는 그 불편함을 '눈으로' 봅니다. 아직 해결책(Context)은 나오지 않습니다. 개념02에서 나옵니다. 문제를 충분히 느끼는 것이 이 파일의 목표입니다.

```
import { useState } from "react";
import Summary from "../_ui/Summary.jsx";
```

4단계 깊이의 화면을 만들어 봅니다

섹션 1

로그인한 사용자입니다. 이 값 하나를 맨 아래까지 보내는 것이 목표입니다. (JS자료와 같은 등장인물입니다)

컴포넌트가 이렇게 겹쳐 있습니다.

ShopPage ← user 를 받는다

```
└ HeaderBar      ← user 를 받는다
  └ UserMenu      ← user 를 받는다
    └ UserBadge   ← 여기서 '드디어' user 를 쓴다
```

실제로 user 를 쓰는 곳은 맨 아래 UserBadge 하나뿐입니다. 나머지 셋은 받아서 그대로 아래로 넘기기만 합니다.

```
function ShopPage({ user }) {
  console.log("ShopPage 실행 - user 를 받았지만 쓰지 않습니다");
```

F12 콘솔 ShopPage 실행 - user 를 받았지만 쓰지 않습니다

```

return (
  <div className="output"> <strong>쇼핑 페이지</strong>
    { /* user 를 여기서 쓰지 않는데도 넘겨야 합니다 */ }
    <HeaderBar user={user} /> </div>
);
}

function HeaderBar({ user }) {
  console.log("HeaderBar 실행 - user 를 받았지만 쓰지 않습니다");
}

```

F12 콘솔 HeaderBar 실행 - user 를 받았지만 쓰지 않습니다

```

return (
  <div className="output"> <strong>헤더</strong> <UserMenu user={user} /> </div>
);
}

function UserMenu({ user }) {
  console.log("UserMenu 실행 - user 를 받았지만 쓰지 않습니다");
}

```

F12 콘솔 UserMenu 실행 - user 를 받았지만 쓰지 않습니다

```

return (
  <div className="output"> <strong>사용자 메뉴</strong> <UserBadge user={user}
/> </div>
);
}

function UserBadge({ user }) {
  console.log("UserBadge 실행 - 여기서만 user 를 씁니다");
}

```

F12 콘솔 UserBadge 실행 - 여기서만 user 를 씁니다

```

return (
  <div className="output">
    {user.name} 님 ({user.age}세)
  </div>
);
}

function DeepTreeDemo() {
  const [user, setUser] = useState({ name: "김민준", age: 20 });

  return (
    <div> <button onClick={() => setUser({ name: "이서연", age: 22 })}>
      이서연으로 로그인
    </div>
  );
}

```

Context 기본

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 여세요.

개념01에서 이런 문제를 봤습니다. - 값 하나를 깊은 곳에 보내려고 중간 컴포넌트 셋이 그 값을 들고 넘겼다
- 그렇다고 컴포넌트 밖 변수에 두면 화면이 다시 그려지지 않는다

Context 는 이 둘을 한 번에 해결합니다. - 중간 컴포넌트를 건너뛰고 값을 꺼낼 수 있습니다 - 값이 바뀌면 React 가 화면을 다시 그려 줍니다

쓰는 순서는 딱 세 단계입니다. 이 파일은 이 세 단계가 전부입니다.

- 1 `createContext(기본값)` 상자를 만든다 ← 컴포넌트 밖에서 한 번
- 2 `<상자.Provider value={값}>` 상자에 값을 넣는다 ← 위쪽 컴포넌트에서
- 3 `useContext(상자)` 상자에서 값을 꺼낸다 ← 아래쪽 컴포넌트에서

비유하면 '택배' 가 아니라 '방송' 입니다. props 는 옆집을 하나씩 거쳐 손으로 전달하는 택배이고, Context 는 위에서 한 번 방송하면 아래에 있는 누구나 듣는 것입니다.

비유는 여기까지입니다. 실제로 일어나는 일은 이렇습니다. `useContext` 를 부르면 React 가 그 컴포넌트에서 위로 거슬러 올라가며 같은 상자의 Provider 를 찾습니다. 처음 만난 Provider 의 value 를 돌려줍니다. 끝까지 못 찾으면 `createContext` 에 적어 둔 기본값을 돌려줍니다.

```
import { createContext, useContext, useState } from "react";
import Summary from "../_ui/Summary.jsx";
```

createContext — 상자를 만든다

섹션 1

`createContext` 는 컴포넌트 '밖' 에서 부릅니다. 파일 맨 위가 보통입니다. 인자로 준 값이 '기본값' 입니다. Provider 를 못 찾았을 때 쓰입니다.

이름은 보통 XxxContext 로 짓습니다. 대문자로 시작합니다. 상자 자체는 컴포넌트가 아닙니다. 값을 담는 통로일 뿐입니다.

```
const UserContext = createContext({ name: "손님", age: 0 });

console.log("UserContext 를 만들었습니다");
```

F12 콘솔 `UserContext` 를 만들었습니다

이 줄은 파일이 처음 불러질 때 한 번만 찍힙니다. 컴포넌트 밖에 있으니 화면을 다시 그려도 다시 찍히지 않습니다.

▸ 직접 해보기 1

기본값의 이름을 "손님" 에서 "비회원" 으로 바꾸면 아래 데모 ③ 의 글자가 어떻게 바뀔지 예상해 보세요.

Provider — 상자에 값을 넣는다

섹션 2

값을 넣고 싶은 범위를 `<UserContext.Provider>` 로 감쌉니다. `value` 속성에 넣은 값이 그 안쪽 전체에서 꺼내집니다.

```
<UserContext.Provider value={{ name: "김민준", age: 20 }}>
```

... 이 안쪽 어디서든 꺼낼 수 있습니다 ...

```
</UserContext.Provider>
```

`value` 의 종괄호가 두 겹인 이유는 02단원 개념03에서 본 것과 같습니다. 바깥 종괄호는 "JSX 에 자바스크립트 값을 넣는다", 안쪽 종괄호는 "객체 리터럴" 입니다.

※ React 19 부터는 `<UserContext value={...}>` 처럼 `.Provider` 를 빼고 써도 됩니다. 인터넷 예제에서 두 모양이 다 보일 겁니다. 이 자료는 어디서나 되는 `.Provider` 를 씁니다.

▸ 직접 해보기 2

아래 `ProviderDemo` 의 `value` 를 `{ name: "박지훈", age: 28 }` 로 바꾸고 화면을 확인해 보세요.

useContext — 어느 깊이에서든 꺼낸다

섹션 3

개념01의 네 단계 트리를 그대로 다시 만듭니다. 다른 점은 하나입니다. 중간 컴포넌트가 `props` 를 하나도 받지 않습니다.

```
function ShopPage() {
```

`user` 라는 글자가 아예 없습니다

```
  return (
    <div className="output"> <strong>쇼핑 페이지</strong> <HeaderBar /> </div>
  );
}
```

```
function HeaderBar() {
  return (
    <div className="output"> <strong>헤더</strong> <UserMenu /> </div>
  );
}

function UserMenu() {
  return (
    <div className="output"> <strong>사용자 메뉴</strong> <UserBadge /> </div>
  );
}

function UserBadge() {
```

여기서 상자를 열어 값을 꺼냅니다. 위에서 아무것도 안 받았습니다.

```
const user = useContext(UserContext);
console.log("UserBadge 가 꺼낸 이름: " + user.name);
```

F12 콘솔 UserBadge 가 꺼낸 이름: 김민준

```
return (
  <div className="output">
    {user.name} 님 ({user.age}세)
  </div>
);
}

function ProviderDemo() {
  return (
    <UserContext.Provider value={{ name: "김민준", age: 20 }}> <ShopPage
  /> </UserContext.Provider>
  );
}
```

화면 상자 네 겹 안에 "김민준 님 (20세)" 가 보입니다.

개념01과 화면은 똑같습니다. 코드가 다릅니다. 개념01: ShopPage · HeaderBar · UserMenu 가 user 를 받아서 넘겼습니다 개념02: 셋 다 user 라는 글자를 쓰지 않습니다

값을 쓰는 UserBadge 와 값을 주는 Provider 만 user 를 압니다. 그래서 중간에 컴포넌트를 하나 더 끼워 넣어도 고칠 것이 없습니다.

⇒ 직접 해보기 3

UserMenu 와 UserBadge 사이에 컴포넌트를 하나 더 끼워 보세요. (예: `functionCorner() { return <div className="output"><UserBadge /></div>; }`) 화면이 그대로 나오는지 확인하세요.

Provider 를 못 찾으면 어떻게 되나

두 경우가 있습니다. 결과가 다릅니다. 둘 다 에러가 안 납니다. 그래서 위험합니다.

경우 가 · Provider 가 아예 없다 → createContext 의 기본값이 나옵니다

```
function OutsideBadge() {
  const user = useContext(UserContext);
  console.log("Provider 밖에서 꺼낸 이름: " + user.name);
}
```

F12 콘솔 Provider 밖에서 꺼낸 이름: 손님

```
return <div className="output">{user.name} 님 ({user.age}세)</div>;
}
```

화면 손님 님 (0세)

위 UserBadge 와 코드가 똑같은데 결과가 다릅니다. Provider 안이냐 밖이냐만 다릅니다. 이름이 안 나오는 게 아니라 '영똥한 이름' 이 나온다는 점이 무섭습니다. 화면이 멀쩡해 보여서 한참 뒤에야 발견합니다.

경우 나 · Provider 는 있는데 value 가 undefined 다 → 기본값이 아니라 undefined 가 나옵니다

"value 가 없으면 기본값이 나오겠지" 라고 생각하기 쉽습니다. 아닙니다. Provider 를 만난 순간 기본값은 더 이상 쓰이지 않습니다. value 에 있는 것이 그대로 나옵니다.

실제로 이런 실수가 자주 납니다.

```
value={settings.theme}   ← settings 에 theme 이라는 속성이 없었다
```

이러면 value 가 undefined 가 되고, 아래에서는 기본값 "light" 가 아니라 undefined 를 받습니다.

```
const ThemeContext = createContext("light");

function ThemeLabel() {
  const theme = useContext(ThemeContext);
  console.log("꺼낸 테마: " + theme);
}
```

F12 콘솔 꺼낸 테마: light
 꺼낸 테마: undefined

← 아래 데모에서 같은 컴포넌트를 두 자리에 놓았기 때문에 두 줄이 찍힙니다

```
return <div className="output">테마: {String(theme)}</div>;
}
```

값을 구해 오다 실패한 상황을 흉내낸 것입니다. 실수로 이런 값이 들어갑니다.

```
const settings = { color: "blue" }; // theme 이라는 속성이  
없습니다 function DefaultValueDemo() {  
  return (  
    <div> <p>Provider 밖 (기본값이 나옵니다)</p> <ThemeLabel />  
    { /* settings.theme 은 없는 속성이라 undefined 입니다 */}  
    <ThemeContext.Provider value={settings.theme}> <p>Provider 안인데 value 가  
    undefined 입니다</p> <ThemeLabel /> </ThemeContext.Provider> </div>  
  );  
}
```

화면	테마: light	← Provider 밖이라 기본값
	테마: undefined	← Provider 는 만났는데 value 가 undefined

ऐ러도 경고도 안 납니다. 화면에만 이상한 값이 보입니다. "기본값을 적어 뒀으니 안전하겠지" 라는 생각이 통하지 않는 경우입니다.

String(theme) 로 감싼 이유가 있습니다. JSX 는 undefined 를 화면에 아무것도 안 그립니다(02단원). 그러면 빈칸만 보입니다. 여기서는 undefined 인 것을 눈으로 봐야 해서 글자로 바꿔 찍었습니다.

▫ 직접 해보기 4

DefaultValueDemo 의 Provider 를 value="dark" 로 고치고 화면이 어떻게 바뀌는지 보세요.

값이 바뀌면 화면도 바뀐다

섹션 5

지금까지 value 에 고정된 값을 넣었습니다. 그러면 개념01의 전역 변수와 다를 게 없습니다. Context 의 진짜 쓸모는 value 에 state 를 넣을 때 나옵니다.

value 에 state 를 넣으면 set 함수 호출 → Provider 를 가진 컴포넌트가 다시 실행 → value 가 새 값이 됨 → 그 값을 useContext 로 꺼내 쓰던 컴포넌트들이 다시 그려집니다.

값을 바꾸는 함수까지 같이 넣어 주면, 깊은 곳에서 로그인/로그아웃도 할 수 있습니다.

상자는 컴포넌트 밖에서 만듭니다. 기본값은 null 로 두었습니다. (항상 Provider 로 감쌀 계획이라 기본값을 쓸 일이 없습니다)

```
const AuthContext = createContext(null);  
  
function LoginButton() {
```

객체 안의 세 개를 한 번에 꺼냅니다(JS자료 09단원 구조분해)

```
const { login, logout } = useContext(AuthContext);  
return (  
  <div className="output"> <button onClick={() => login("이서연", 22)}>로그인
```

```

    </button> <button onClick={logout}>로그아웃</button> </div>
  );
}

function AuthStatus() {
  const { user } = useContext(AuthContext);
  return (
    <div className="output">
      {user === null ? "로그인하지 않았습니다" : user.name + " 님 환영합니다"}
    </div>
  );
}

```

두 컴포넌트를 감싸는 중간 컴포넌트입니다. 역시 props 가 없습니다.

```

function AuthPage() {
  return (
    <div className="output"> <strong>마이 페이지</strong> <AuthStatus /> <LoginButton
    /> </div>
  );
}

function AuthProviderDemo() {
  const [user, setUser] = useState(null);

  function login(name, age) {
    setUser({ name: name, age: age });
    console.log("로그인: " + name);
  }
}

```

F12 콘솔 로그인: 이서연

```

}

function logout() {
  setUser(null);
  console.log("로그아웃");
}

```

F12 콘솔 로그아웃

```

}

```

value 에 값과 함수를 함께 담아 내려보냅니다. 이렇게 하면 아래 어느 컴포넌트든 login() 을 부를 수 있습니다.

```
return (
  <AuthContext.Provider value={{ user: user, login: login, logout: logout
  }}> <AuthPage /> </AuthContext.Provider>
);
}
```

화면 "로그인하지 않았습니다" 와 [로그인] [로그아웃] 버튼

화면(누르면): [로그인] 을 누르면 "이서연 님 환영합니다" 로 바뀝니다. 화면(누르면): [로그아웃] 을 누르면 다시 "로그인하지 않았습니다" 가 됩니다.

AuthPage 는 user 도 login 도 모릅니다. 그냥 자식을 그릴 뿐입니다. 07단원의 state 끌어올리기와 비교해 보세요. 그때는 부모가 함수를 props 로 자식에게 내려보냈습니다(개념04). 여기서는 그 함수를 Provider 의 value 에 실어 보냈습니다. 중간을 건너뛰니다.

▹ 직접 해보기

5

login 을 부를 때 이름을 "박지훈", 나이를 28 로 바꿔 보세요.

상자는 여러 개 만들어도 됩니다

섹션 6

Context 는 하나만 쓰는 규칙이 없습니다. 성격이 다른 값은 상자를 나누는 편이 낫습니다. 이유는 개념05에서 자세히 다룹니다.

Provider 는 겹쳐서 쓸 수 있습니다. 아래처럼요.

```
const LangContext = createContext("ko");

function GreetingBox() {
  const user = useContext(UserContext);
  const lang = useContext(LangContext);
```

useContext 를 두 번 부르면 상자 두 개에서 각각 꺼냅니다.

```
const hello = lang === "ko" ? "안녕하세요" : "Hello";
return (
  <div className="output">
    {hello}, {user.name}
  </div>
);
}

function TwoContextDemo() {
  const [lang, setLang] = useState("ko");

  return (
```

useReducer

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 여세요.

이 파일은 Context 와 상관없습니다. state 를 다루는 또 다른 방법입니다. 개념04에서 둘을 합칩니다.

이름부터 봅시다. useReducer 의 reducer 는 JS자료 08단원 개념05의 reduce 입니다.

```
배열.reduce((누적값, 현재값) => 새누적값, 시작값)
      ^^^^^  ^^^^^  ^^^^^^^

reducer(현재state, action) => 새state
      ^^^^^^^  ^^^^^  ^^^^^^^
```

모양이 같습니다. 하는 일도 같습니다. reduce 는 '값들' 을 하나씩 접어서 최종 값 하나를 만듭니다.

useReducer 는 '사용자의 행동들' 을 하나씩 접어서 지금의 state 를 만듭니다.

섹션 3에서 실제로 reducer 를 reduce 에 넣어 돌려 봅니다. 그냥 돌아갑니다.

switch 문은 JS자료 03단원 개념04에서 배웠습니다. 여기서 다시 씁니다.

```
import { useState, useReducer } from "react";
import Summary from "../_ui/Summary.jsx";
```

먼저 useState 로 만들어 봅니다

섹션 1

아메리카노를 몇 잔 담았는지 세는 화면입니다. 규칙이 둘 있습니다. - 0잔 아래로는 안 내려간다 - 10잔을 넘지 않는다

```
function CounterWithState() {
  const [count, setCount] = useState(0);

  function handleAdd() {
```

규칙이 여기에 한 벌

```

if (count >= 10) return;
    setCount(count + 1);
}

function handleRemove() {

```

규칙이 여기에 또 한 벌

```

if (count <= 0) return;
    setCount(count - 1);
}

function handleClear() {
    setCount(0);
}

return (
    <div className="output"> <strong>useState 방식</strong> <p>아메리카노 {count}잔</p>
    <button onClick={handleAdd}>담기</button> <button onClick={handleRemove}>빼기</button>
    <button onClick={handleClear}>비우기</button> </div>
);
}

```

화면 아메리카노 0잔

화면(누르면): [담기] 를 누르면 1잔, 2잔... 10잔에서 멈춥니다.

잘 돌아갑니다. 문제도 없습니다. 그런데 한 가지가 눈에 걸립니다. “숫자를 어떻게 바꾸는가” 하는 규칙이 handleAdd · handleRemove 두 곳에 흩어져 있습니다.

여기에 “쿠폰을 쓰면 최대 20잔” 같은 규칙이 붙으면 두 곳을 다 고쳐야 합니다. 버튼이 다섯 개가 되면 다섯 곳이 됩니다. 개념01에서 본 것과 같은 종류의 불편함입니다. 이번엔 화면이 아니라 state 쪽입니다.

▹ 직접 해보기

1

CounterWithState 에 “5잔 담기” 버튼을 추가해 보세요. (10잔을 넘지 않게 하는 규칙도 그 안에 또 적어야 합니다)

같은 것을 useReducer 로

섹션 2

useReducer 는 “규칙을 함수 하나에 모으는” 방법입니다.

```
const [state, dispatch] = useReducer(reducer, 초기값);
```

state 지금 값. useState 의 첫 번째와 같습니다. dispatch “이런 일이 있었다” 고 알리는 함수입니다. reducer 그 알림을 받아 새 state 를 돌려주는 함수입니다.

dispatch 를 부르면 React 가 대신 reducer(지금state, 우리가준action) 을 부릅니다. 그 결과를 새 state 로 삼고 화면을 다시 그립니다.

action 은 "무슨 일이 있었는지" 를 담은 객체입니다. type 속성에 이름을 적는 것이 관례입니다. 다른 이름을 써도 동작은 합니다.

```
function counterReducer(state, action) {
```

state 는 지금 잔 수(숫자), action 은 { type: "..." } 모양의 객체입니다.

```
  switch (action.type) {
    case "add":
```

규칙이 여기 한 곳에만 있습니다

```
    if (state >= 10) return state;
    return state + 1;
    case "remove":
      if (state <= 0) return state;
      return state - 1;
    case "clear":
      return 0;
    default:
```

모르는 action 이 오면 아무 일도 없던 것으로 합니다

```
    return state;
  }
}
```

★ reducer 는 컴포넌트 '밖' 에 둡니다. 혹도 state 도 쓰지 않는 그냥 함수입니다. 값을 받아 값을 돌려줄 뿐입니다. 그래서 테스트하기 쉽습니다(섹션 3).

```
function CounterWithReducer() {
  const [count, dispatch] = useReducer(counterReducer, 0);

  return (
    <div className="output"> <strong>useReducer 방식</strong> <p>아메리카노 {count}잔</p>
    { /* setCount(...) 대신 dispatch({ type: "..." }) 를 부릅니다 */ }
    <button onClick={() => dispatch({ type: "add" })}>담기</button>
    <button onClick={() => dispatch({ type: "remove" })}>빼기</button>
    <button onClick={() => dispatch({ type: "clear" })}>비우기</button> </div>
  );
}
```

화면 아메리카노 0잔

화면(누르면): 왼쪽 `useState` 방식과 똑같이 동작합니다.

두 방식을 나란히 놓고 비교해 보세요.

useState · 버튼마다 “숫자를 어떻게 바꿀지” 를 적는다

→ 규칙이 화면 쪽 코드에 들어간다

useReducer · 버튼은 “무슨 일이 있었는지” 만 말한다

→ 규칙은 `reducer` 한 곳에 모인다

버튼은 `dispatch({ type: "add" })` 만 압니다. 10잔 제한이 있는 줄도 모릅니다. 규칙을 고칠 때 화면 코드를 열 필요가 없습니다.

⇒ 직접 해보기 2

`counterReducer` 에 case “addFive” 를 만들고 `CounterWithReducer` 에 “5잔 담기” 버튼을 붙여 보세요. 직접 해보기 1과 비교하면 고칠 곳이 어디가 다른가요?

reducer 는 그냥 함수입니다 — reduce 로 돌려 보기

섹션 3

`reducer` 는 React 와 아무 상관이 없는 순수한 함수입니다. 그래서 배열 메소드 `reduce` 에 그대로 넣을 수 있습니다. JS자료 08단원 개념05 그대로입니다.

```
const steps = [{ type: "add" }, { type: "add" }, { type: "add" }, { type: "remove" }];
```

```
console.log(steps.reduce(counterReducer, 0));
```

F12 콘솔 2

담기 3번, 빼기 1번 → 2잔입니다. `reduce` 의 콜백 자리에 `counterReducer` 를 그대로 넣었습니다. 아무것도 안 고쳤습니다.

한 바퀴씩 들여다봅시다. JS자료에서 `acc` 를 찍어 본 것과 같은 방법입니다.

```
steps.reduce((state, action) => {  
  const next = counterReducer(state, action);  
  console.log(`state=${state}, action=${action.type}, 결과=${next}`);  
  return next;  
}, 0);
```



```
F12 콘솔    state=0, action=add, 결과=1
            state=1, action=add, 결과=2
            state=2, action=add, 결과=3
            state=3, action=remove, 결과=2
```

읽는 법 reduce 에서 콜백이 return 한 값이 다음 바퀴의 acc 가 되었습니다. useReducer 에서도 reducer 가 return 한 값이 다음 state 가 됩니다. 다른 점은 '다음 바퀴' 가 언제 오는가입니다.

```
reduce      → 배열의 다음 칸에서 바로
useReducer  → 사용자가 버튼을 누를 때
```

그래서 useReducer 로 만든 화면은 이렇게 말할 수 있습니다. "지금 state 는, 처음 값에 사용자가 한 행동들을 차례로 접은 결과다"

★ 이 코드는 컴포넌트 밖에 있어서 파일을 처음 불러올 때 한 번만 실행됩니다. 화면을 다시 그려도 콘솔에 다시 찍히지 않습니다.

⇒ 직접 해보기 3

steps 뒤에 { type: "clear" } 를 하나 더 넣으면 최종 결과가 얼마일지 예상한 뒤 확인해 보세요.

action 에 값을 실어 보내기

섹션 4

지금까지 action 에는 type 만 있었습니다. 값을 같이 보낼 수도 있습니다. 속성 이름은 자유입니다. 관례로 payload(실어 보내는 짐) 를 많이 씁니다.

```
function amountReducer(state, action) {
  switch (action.type) {
    case "addMany":
```

action.payload 만큼 더합니다. 10잔은 여전히 최대입니다.

```
    if (state + action.payload > 10) return 10;
    return state + action.payload;
    case "clear":
      return 0;
    default:
      return state;
  }
}

function PayloadDemo() {
  const [count, dispatch] = useReducer(amountReducer, 0);

  return (
```

Context 와 useReducer 함께

RUN 실습프로젝트에서 npm run dev → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 여세요.

개념02에서 Context 를, 개념03에서 useReducer 를 배웠습니다. 서로 상관없는 둘이었습니다. 이제 합칩니다. 둘을 합치면 이런 것이 됩니다.

useReducer → 값이 어떻게 바뀌는지의 규칙을 한 곳에 모은다
Context → 그 값과 dispatch 를 어느 깊이에서든 꺼내 쓴다

합친 결과를 흔히 '전역 상태' 라고 부릅니다. 그런데 이 말은 오해를 부릅니다. 전역이 아니라 'Provider' 로 감싼 범위' 입니다. 섹션 6에서 직접 확인합니다.

이 파일에서 만들 것은 장바구니입니다.

```
import { createContext, useContext, useReducer } from "react";
import Summary from "../_ui/Summary.jsx";
```

무엇을 만드나

섹션 1

화면 조각이 넷입니다.

MenuList	메뉴를 보여 주고 [담기] 를 누르면 장바구니에 넣는다
CartList	담긴 것을 보여 주고 수량을 조절하거나 뺀다
CartTotal	총액을 보여 준다
CartBadge	화면 구석에서 담긴 개수만 보여 준다 (깊은 곳에 둡니다)

넷 다 같은 장바구니를 봅니다. 07단원의 state 끌어올리기로 만들면 공통 부모가 items 와 함수 넷을 들고 있다가 네 조각에 전부 내려보내야 합니다. 개념01에서 본 그 상황입니다.

장바구니 한 칸은 이렇게 생겼습니다.

```
{ id: 1, name: "아메리카노", price: 4000, count: 2 }
```

그리고 items 는 이런 칸들의 배열입니다. 처음엔 빈 배열입니다.

```
const MENU = [
  { id: 1, name: "아메리카노", price: 4000 },
  { id: 2, name: "라떼", price: 4500 },
  { id: 3, name: "케이크", price: 6000 },
  { id: 4, name: "삼각김밥", price: 1200 },
];
```

▹ 직접 해보기 1

MENU 에 { id: 5, name: "쿠키", price: 3000 } 을 추가해 보세요. 화면이 어떻게 바뀌는지 확인하세요.

reducer 를 먼저 만듭니다

섹션 2

순서가 중요합니다. 화면보다 규칙을 먼저 정합니다. reducer 는 컴포넌트 밖의 그냥 함수라서 화면 없이도 시험해 볼 수 있습니다.

```
function cartReducer(state, action) {
```

state 는 장바구니 배열입니다. 07단원 불변성 규칙을 그대로 지킵니다.

```
  switch (action.type) {
    case "add":
```

이미 담긴 메뉴면 수량만 1 올립니다. some 은 "조건에 맞는 게 하나라도 있나" 를 true/false 로 알려 줍니다(JS자료 08단원).

```
    if (state.some((item) => item.id === action.payload.id)) {
      return state.map((item) =>
        item.id === action.payload.id
          ? { ...item, count: item.count + 1 } // 그 칸만 새 객체로
          : item
      );
    }
  }
```

처음 담는 메뉴면 count 1 을 붙여 배열 끝에 넣습니다.

```
    return [...state, { ...action.payload, count: 1 }];

    case "decrease":
```

수량을 1 내리고, 0이 된 칸은 목록에서 뺍니다.

```
    return state
      .map((item) =>
```

```

    )

    .filter((item) => item.count > 0);

    case "remove":
      return state.filter((item) => item.id !== action.payload);

    case "clear":
      return [];

    default:
      return state;
  }
}

```

총액을 구하는 함수입니다. 이것도 컴포넌트 밖의 그냥 함수입니다. JS자료 08단원 개념05 섹션4에서 만든 것과 똑같습니다.

```

function getTotal(items) {
  return items.reduce((acc, item) => acc + item.price * item.count, 0);
}

```

화면 없이 시험해 봅시다. 개념03 섹션3과 같은 방법입니다.

```

const testSteps = [
  { type: "add", payload: MENU[0] }, // 아메리카노
  { type: "add", payload: MENU[0] }, // 아메리카노 한 잔 더
  { type: "add", payload: MENU[2] }, // 케이크
];

const testResult = testSteps.reduce(cartReducer, []);

console.log(testResult.map((item) => item.name + " " + item.count + "개").join(", "));

```

F12 콘솔 아메리카노 2개, 케이크 1개

```
console.log(getTotal(testResult));
```

F12 콘솔 14000

$4000 \times 2 + 6000 = 14000$ 입니다. 브라우저를 열기 전에 규칙이 맞는지 확인한 것입니다. reducer 를 컴포넌트 밖에 두면 이런 검산이 가능합니다. 이게 큰 장점입니다.

➤ 직접 해보기

2

testSteps 뒤에 { type: "decrease", payload: 1 } 을 넣고 콘솔 두 줄이 어떻게 바뀔지 예상한 뒤 확인해 보세요.

Provider 컴포넌트로 묶기

섹션 3

상자를 만듭니다. 기본값은 `null` 입니다. 이유는 섹션 4에서 설명합니다.

```
const CartContext = createContext(null);
```

Provider 를 직접 쓰지 않고 '감싸는 컴포넌트' 를 하나 만듭니다. children 은 03단원 개념04에서 배웠습니다. 태그 사이에 넣은 것이 그대로 옵니다.

```
function CartProvider({ children }) {
  const [items, dispatch] = useReducer(cartReducer, []);
```

값과 dispatch 를 한 객체에 담아 내려보냅니다.

```
return (
  <CartContext.Provider value={{ items: items, dispatch: dispatch }}>
    {children}
  </CartContext.Provider>
);
}
```

이렇게 컴포넌트로 감싸면 쓰는 쪽이 이렇게 짧아집니다.

```
<CartProvider>
```

```
<아무거나 />
```

```
</CartProvider>
```

useReducer 도, cartReducer 도, 초기값도 쓰는 쪽에서는 안 보입니다. 나중에 useReducer 를 useState 로 바꿔도 쓰는 쪽 코드는 그대로입니다.

▫ 직접 해보기

3

CartProvider 의 초기값 `[]` 을 `[{ id: 1, name: "아메리카노", price: 4000, count: 1 }]` 로 바꿔 보세요. 화면을 열자마자 한 잔이 담겨 있어야 합니다.

useCart 커스텀 훅

섹션 4

값을 꺼내는 쪽도 짧게 만듭시다. 커스텀 훅은 10단원 개념03에서 배웠습니다. use 로 시작하는 이름의 함수를 만들고 그 안에서 훅을 부르면 됩니다.

```
function useCart() {
  const cart = useContext(CartContext);
```

Provider 밖에서 부르면 여기서 바로 알려 줍니다.

```
if (cart === null) {
  throw new Error("useCart 는 <CartProvider> 안에서만 쓸 수 있습니다");
}

return cart;
}
```

혹으로 감싸면 좋은 점이 셋입니다.

1) 쓰는 쪽이 짧아집니다

```
const { items, dispatch } = useCart();
← useContext 도 CartContext 도 몰라도 됩니다
```

2) Provider 밖에서 쓴 실수를 바로 잡아 줍니다

개념02 [실수 1] 을 떠올려 보세요. Provider 없이 쓰면 기본값이 조용히 나왔습니다. 기본값을 null 로 두고 혹에서 검사하면, 조용히 틀리는 대신 그 자리에서 멈춥니다. 실습프로젝트에서는 `ErrorBox` 가 잡아서 빨간 상자에 이 문장을 보여 줍니다. "조용히 틀리는 것" 보다 "요란하게 멈추는 것" 이 고치기 훨씬 쉽습니다.

3) 나중에 안쪽을 바꿔도 쓰는 쪽이 안 바뀝니다

Context 를 둘로 쪼개도(개념05) `useCart()` 만 고치면 됩니다.

★ `throw` 는 "여기서 멈추고 이 에러를 알려라" 는 뜻입니다. JS자료 12단원에서 봤습니다. Provider 안에서만 쓰면 이 줄은 한 번도 실행되지 않습니다.

▫ 직접 해보기 4

파일 맨 아래 `Concept04CartApp` 에서 `<CartProvider>` 태그를 잠깐 지워 보세요. 빨간 상자에 어떤 문장이 뜨는지 읽고 다시 되돌리세요.

화면 조각 만들기

섹션 5

이제 네 조각을 만듭니다. 넷 다 props 를 하나도 받지 않습니다. 필요한 것은 각자 `useCart()` 로 꺼냅니다.

```
function MenuList() {
  const { dispatch } = useCart(); // items 는 안 쓰므로 dispatch 만 꺼냅니다 return (
```

```

<strong>메뉴</strong>
<ul>
  {MENU.map((menu) => (
    <li key={menu.id}>
      {menu.name} {menu.price}원{" "}
      <button
        onClick={() => {
          dispatch({ type: "add", payload: menu });
          console.log("담기: " + menu.name);

```

F12 콘솔 담기: 아메리카노

```

    })
  >
    담기
  </button>
</li>
  )})
</ul>
</div>
);
}

function CartList() {
  const { items, dispatch } = useCart();

  if (items.length === 0) {
    return <div className="output">장바구니가 비어 있습니다</div>;
  }

  return (
    <div className="output"> <strong>장바구니</strong> <ul>
      {items.map((item) => (
        <li key={item.id}>
          {item.name} × {item.count} = {item.price * item.count}원{" "}
          <button onClick={() => dispatch({ type: "add", payload: item })}>+
        </button> <button onClick={() => dispatch({ type: "decrease", payload: item.id })}>-
          -
          <button> <button onClick={() => dispatch({ type: "remove", payload:
item.id })}>
            빼기
          </button> </li>
        )})
      </ul> </div>

```

```

    );
  }

  function CartTotal() {
    const { items, dispatch } = useCart();

    return (
      <div className="output"> <strong>합계 {getTotal(items)}원</strong>{" "}
        <button onClick={() => dispatch({ type: "clear" })}>비우기</button> </div>
    );
  }
}

```

이 배지는 일부러 깊은 곳에 둡니다. 개념01의 4단계 트리와 같은 깊이입니다.

```

function CartBadge() {
  const { items } = useCart();

```

담긴 잔 수를 전부 더합니다. reduce 를 또 씁니다.

```

const count = items.reduce((acc, item) => acc + item.count, 0);
return <span>🛒 {count}</span>;
}

function CornerBox() {
  return (
    <div className="output"> <CartBadge /> </div>
  );
}

function SideBar() {
  return (
    <div className="output"> <strong>사이드바</strong> <CornerBox /> </div>
  );
}

function ShopLayout() {

```

ShopLayout 도 SideBar 도 CornerBox 도 장바구니를 모릅니다. props 가 하나도 없습니다.

```

return (
  <div className="output"> <MenuList /> <CartList /> <CartTotal /> <SideBar
/> </div>
);
}

```

화면 메뉴 네 줄, "장바구니가 비어 있습니다", "합계 0원", 사이드바 안에 🛒 0

화면(누르면): 아메리카노 [담기] 를 두 번 누르면

장바구니에 "아메리카노 × 2 = 8000원", 합계 8000원,  2

화면(누르면): [-] 를 한 번 누르면 × 1 로 줄고, 한 번 더 누르면 목록에서 사라집니다

▢ 직접 해보기 5

CartTotal 에 "담은 종류: N가지" 를 같이 보여 주세요. (힌트: items.length 입니다)

‘전역’ 이 아니라 ‘Provider 로 감싼 범위’ 입니다

섹션 6

Context 를 배우면 "이제 어디서나 쓸 수 있는 값" 이라고 생각하기 쉽습니다. 정확히는 "그 Provider 로 감싼 안쪽에서만" 입니다.

확인해 봅시다. CartProvider 를 두 개 두면 장바구니도 두 개가 됩니다.

```
function MiniShop({ title }) {
  const { items, dispatch } = useCart();
  const count = items.reduce((acc, item) => acc + item.count, 0);

  return (
    <div className="output"> <strong>{title}</strong> <p>
      {count}개 담김 / 합계 {getTotal(items)}원
    </p> <button onClick={() => dispatch({ type: "add", payload: MENU[0] })}>
      아메리카노 담기
    </button> <button onClick={() => dispatch({ type: "clear" })}>비우기
    </button> </div>
  );
}

function TwoProvidersDemo() {
  return (
    <div> <CartProvider> <MiniShop title="1번 가게"
  /> </CartProvider> <CartProvider> <MiniShop title="2번 가게"
  /> </CartProvider> </div>
  );
}
```

화면 두 상자 모두 "0개 담김 / 합계 0원"

화면(누르면): 1번 가게에서 담아도 2번 가게는 0개 그대로입니다.

MiniShop 코드는 하나뿐인데 장바구니는 둘입니다. useReducer 의 state 는 CartProvider 컴포넌트 '한 개' 가 들고 있기 때문입니다. CartProvider 를 두 번 그리면 state 도 두 벌이 됩니다.

그래서 이렇게 정리할 수 있습니다. Provider 를 앱 맨 위에 하나 두면 → 앱 전체가 하나를 공유합니다
Provider 를 화면마다 두면 → 화면마다 따로따로가 됩니다 어디에 두느냐가 곧 '공유 범위' 를 정하는 일입니다.

▫ 직접 해보기 6

TwoProvidersDemo 에서 <CartProvider> 하나로 MiniShop 둘을 함께 감싸 보세요. 그러면 어떻게 될까요?

자주 하는 실수

섹션 7

⚠ [SyntaxError] 라고 적힌 것은 주석을 풀지 말고 눈으로만 보세요. 풀면 파일 전체가 멈춰서 화면이 통째로 비어 버립니다. 다시 // 를 붙이면 돌아옵니다.

이런 실수를 합니다

CartProvider 안에서 useCart() 를 부름

```
function CartProvider({ children }) {  
  const { items } = useCart(); ← 자기 자신을 꺼내려 합니다
```

... } useContext 는 '자기 위' 를 찾습니다. 자기가 만든 Provider 는 자기 아래입니다. 그래서 못 찾고 null 이 나와 throw 로 멈춥니다. Provider 안의 값은 그냥 useReducer 가 준 items 를 쓰면 됩니다.

이런 실수를 합니다

Provider 로 감싸지 않고 화면 조각만 씀

<MenuList /> ← <CartProvider> 밖입니다 useCart 의 throw 가 걸려 빨간 상자가 뜹니다. "useCart 는 <CartProvider> 안에서만 쓸 수 있습니다" 에러 문장이 원인을 그대로 말해 줍니다. 이게 흑으로 감싼 이유입니다.

이런 실수를 합니다

reducer 에서 배열을 직접 고침

```
case "add":  
  state.push({ ...action.payload, count: 1 });  
  return state;
```

에러가 안 납니다. 화면만 안 바뀝니다. 07단원 개념01에서 본 그대로입니다. push 는 원본을 고치고 같은 배열을 돌려줍니다. React 는 같은 배열이면 다시 그리지 않습니다.

...state, 새칸 · 처럼 새 배열을 만드세요.

이런 실수를 합니다

map 안에서 칸을 직접 고침

```
return state.map((item) => {  
  if (item.id === action.payload) item.count = item.count + 1;  
  return item;  
});
```

map 이 새 배열을 돌려주니 괜찮아 보입니다. 아닙니다. 배열은 새것이지만 안의 객체는 그대로입니다. 이 예제에서는 화면이 바뀌긴 합니다. 배열이 새것이니까요. 하지만 그 객체를 다른 곳에서도 보고 있으면 조용히 함께 바뀌입니다. 07단원 개념02의 얇은 복사 함정입니다. { ...item, count: ... } 로 새 객체를 만드세요.

이런 실수를 합니다

key 를 안 붙임

```
{items.map((item) => <li>{item.name}</li>)}
```

화면은 나옵니다. 대신 콘솔에 경고가 뜹니다. Each child in a list should have a unique "key" prop. 05단원 개념03에서 배운 그대로입니다. key={item.id} 를 붙이세요.

이런 실수를 합니다

payload 모양을 액션마다 다르게 해 놓고 헛갈림

```
dispatch({ type: "add", payload: menu }) ← payload 가 객체  
dispatch({ type: "remove", payload: item.id }) ← payload 가 숫자
```

에러가 안 납니다. 잘못 넣으면 조용히 아무 일도 안 일어납니다. 이 파일도 위처럼 섞어 썼습니다. reducer 를 볼 때 payload 가 무엇인지 꼭 확인하세요. 헛갈리면 payload 대신 { type: "remove", id: item.id } 처럼 이름을 붙여도 됩니다.

정리

- 1 reducer 를 컴포넌트 밖에 먼저 만들고, 화면 없이 reduce 로 검산할 수 있습니다.
- 2 Provider 를 직접 쓰지 말고 CartProvider 같은 컴포넌트로 감싸면 쓰는 쪽이 짧아집니다.
- 3 children 을 받으면 그 안쪽 전체가 Provider 범위가 됩니다.
- 4 useCart 같은 커스텀 훅으로 감싸면 useContext 를 쓰는 쪽에서 몰라도 됩니다.
- 5 기본값을 null 로 두고 훅에서 검사하면 Provider 밖 사용을 그 자리에서 잡아 줍니다.
- 6 화면 조각들이 props 를 하나도 안 받습니다. 개념01의 중간 컴포넌트 문제가 사라집니다.
- 7 전역' 이 아니라 'Provider 로 감싼 범위' 입니다. Provider 를 둘 두면 값도 두 벌이 됩니다.

```
export default function Concept04CartApp() {
  return (
    <div> <h1>개념 04 – Context 와 useReducer 함께</h1> <p className="guide">
      아래 화면 조각들은 <strong>props 를 하나도 받지 않습니다</strong>. 필요한
      값은
      각자 <code>useCart()</code> 로 꺼냅니다.
      <br /> <strong>F12 → Console</strong> 에서 섹션 2의 검산 결과도 확인하세요.
      <p> <div className="demo"> <h3>① 장바구니 (섹션 3·4·5)
    </h3> <CartProvider> <ShopLayout
    /> </CartProvider> </div> <div className="demo"> <h3>② Provider 두 개 = 장바구니 두
    개 (섹션 6)</h3> <TwoProvidersDemo /> </div> </div>
  );
}
```

직접 해보기 정답

```
1) const MENU = [
```

```
  ...,
  { id: 5, name: "쿠키", price: 3000 },
```

```
];
// 화면: 메뉴가 다섯 줄이 됩니다
```

→ MenuList 는 MENU 를 map 으로 그리기만 하므로 고칠 곳이 없습니다.

```
reducer 도 그대로입니다. 메뉴가 무엇인지 모르고 payload 만 받기 때문입니다.
```

```
2) const testSteps = [
```

```
{ type: "add", payload: MENU[0] },
{ type: "add", payload: MENU[0] },
{ type: "add", payload: MENU[2] },
{ type: "decrease", payload: 1 },
```

```
];
// 콘솔: 아메리카노 1개, 케이크 1개
// 콘솔: 10000
```

→ payload 가 1 인 이유는 decrease 가 '아이디' 를 받기 때문입니다.

아메리카노의 id 가 1 입니다.

```
3) const [items, dispatch] = useReducer(cartReducer, [
```

```
{ id: 1, name: "아메리카노", price: 4000, count: 1 },
```

```
]);
// 화면: 처음부터 "아메리카노 × 1 = 4000원", 합계 4000원,  1
```

→ count 를 빠뜨리면 화면에 "아메리카노 × " 까지만 나오고 합계가 NaN 이 됩니다.

초기값도 reducer 가 만드는 모양과 같아야 합니다.

4) 빨간 상자에 이렇게 뜹니다. Error: useCart 는 <CartProvider> 안에서만 쓸 수 있습니다 → 화면이 통째로 사라지지 않고 이 예제만 빨간 상자가 됩니다. ErrorBox 가 잡아 준 것입니다.

만약 useCart 에 throw 가 없었다면 cart 가 null 이고
const { items } = null 에서 다른 에러가 났을 겁니다. 원인을 알기 어려웠겠지요.

```
5) function CartTotal() {
```

```
const { items, dispatch } = useCart();
return (
  <div className="output"> <strong>합계 {getTotal(items)}원</strong> (담은 종류:
  {items.length}가지){ " " }
  <button onClick={() => dispatch({ type: "clear" })}>비우기</button> </div>
);
```

```
}
// 화면: 합계 8000원 (담은 종류: 1가지)
```

→ 종류와 잔 수는 다릅니다. 아메리카노 2잔은 1가지에 2잔입니다.

```
6) function TwoProvidersDemo() {
```

```
  return (  
    <CartProvider> <MiniShop title="1번 가게" /> <MiniShop title="2번 가게"  
    /> </CartProvider>  
  );
```

```
}  
// 화면(누르면): 1번 가게에서 담으면 2번 가게 숫자도 같이 올라갑니다
```

→ 같은 Provider 안이니 같은 state 를 봅니다.

"값을 공유할 범위" 는 Provider 를 어디에 두느냐로 정해집니다.

언제 쓰나

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 을 반드시 여세요. 이 파일은 콘솔 없이는 볼 것이 없습니다.

개념01~04에서 Context 와 useReducer 로 장바구니를 만들었습니다. 편했습니다. 그래서 이런 생각이 듭니다.

"props 로 내려보내는 건 번거로우니까, 앞으로는 다 Context 에 넣자"

이 파일은 그 생각을 말리는 파일입니다.

Context 는 성능을 좋게 하는 도구가 아닙니다. 값을 '전달' 하는 도구입니다. 오히려 잘못 쓰면 화면을 더 많이 다시 그리게 만듭니다. 그리고 값이 어디서 바뀌는지 코드만 보고는 알 수 없게 만듭니다.

이 파일에서는 무엇이 얼마나 다시 그려지는지 콘솔로 직접 세어 봅니다. 짐작하지 말고 세어 보세요. 이 파일의 숫자는 전부 실제로 세어서 적은 것입니다.

★ 개발 중에는 StrictMode 때문에 컴포넌트가 두 번 실행됩니다. 그래서 콘솔에 같은 줄이 두 번씩 찍힙니다. 두 줄이 곧 '한 번 다시 그려짐' 입니다. 아래 설명의 '한 번' 은 전부 이 기준입니다.

```
import { createContext, useContext, useReducer, useState } from "react";
import Summary from "../_ui/Summary.jsx";
```

다시 그려지는 범위를 세어 봅니다

섹션 1

흔히 하는 설계입니다. 앱이 쓰는 값을 한 상자에 다 담았습니다.

```
const AppContext = createContext(null);

function UserName() {
  console.log("[한 상자] UserName 실행");
```

F12 콘솔 [한 상자] UserName 실행

```
const app = useContext(AppContext);
return <div className="output">사용자: {app.user}</div>;
}

function ThemeLabel() {
  console.log("[한 상자] ThemeLabel 실행");
```

F12 콘솔 [한 상자] ThemeLabel 실행

```
const app = useContext(AppContext);
return <div className="output">테마: {app.theme}</div>;
}
```

Provider 컴포넌트의 JSX 안에 '직접' 들어 있는 컴포넌트입니다. context 는 안 씁니다.

```
function InsideBox() {
  console.log("[한 상자] InsideBox 실행 (context 안 씀)");
```

F12 콘솔 [한 상자] InsideBox 실행 (context 안 씀)

```
return <div className="output">Provider 안쪽에 직접 놓인 상자</div>;
}
```

children 으로 넘어온 컴포넌트입니다. 역시 context 는 안 씁니다.

```
function FromChildren() {
  console.log("[한 상자] FromChildren 실행 (children 으로 넘어옴)");
```

F12 콘솔 [한 상자] FromChildren 실행 (children 으로 넘어옴)

```
return <div className="output">children 으로 넘어온 상자</div>;
}
```

```
function OneBoxProvider({ children }) {
  const [user, setUser] = useState("김민준");
  const [theme, setTheme] = useState("light");

  return (
    <AppContext.Provider value={{ user: user, theme: theme }}> <button onClick={() =>
setTheme(theme === "light" ? "dark" : "light")}>
      테마만 바꾸기
    </button> <button onClick={() => setUser(user === "김민준" ? "이서연" :
"김민준")}>
      사용자만 바꾸기
    </button> <InsideBox />
    {children}
  </AppContext.Provider>
  );
}
```

```
function OneBoxDemo() {
  return (
    <OneBoxProvider> <UserName /> <ThemeLabel /> <FromChildren /> </OneBoxProvider>
```



```

    );
  }

  function OneBoxDemo() {
    return (
      <OneBoxProvider> <UserName /> <ThemeLabel /> <FromChildren /> </OneBoxProvider>
    );
  }

```

콘솔을 비우고 [테마만 바꾸기] 를 한 번 눌러 보세요. 실제로 세어 본 결과입니다.

다시 그려짐 UserName ← 테마와 상관없습니다. user 만 읽는데도 그려집니다 다시 그려짐 ThemeLabel
← 이걸 당연합니다 다시 그려짐 InsideBox ← context 를 안 쓰는데도 그려집니다 안 그려짐
FromChildren ← context 를 안 쓰고 children 으로 넘어와서 안 그려집니다

두 가지를 알 수 있습니다.

[1] 상자 하나에 값을 여럿 담으면, 그중 하나만 바뀌어도

그 상자를 읽는 컴포넌트가 전부 다시 그려집니다.
React 는 "이 컴포넌트는 app.user 만 읽더라" 를 구분하지 못합니다.
value 객체가 통째로 바뀐 것만 압니다.

[2] Provider 를 가진 컴포넌트(OneBoxProvider)가 다시 실행되면

그 JSX 안에 직접 쓴 컴포넌트(InsideBox)도 같이 다시 그려집니다.
context 와는 상관없이, 부모가 다시 그려졌기 때문입니다.
반대로 children 으로 받은 것(FromChildren)은 다시 그려지지 않습니다.
그 자리에 들어갈 화면 조각을 '위에서 이미 만들어서 넘겨준' 것이라
내용이 그대로면 React 가 건너뛸니다.

[2] 는 그냥 알아 두면 좋은 정도이고, 문제가 되는 것은 [1] 입니다.

▸ 직접 해보기 1

콘솔을 지우고 [사용자만 바꾸기] 를 한 번 누르세요. ThemeLabel 이 다시 그려지는지 확인하세요.

상자를 쪼개면 범위가 줄어듭니다

섹션 2

같은 화면을 상자 두 개로 만들어 봅시다. user 상자, theme 상자.

```

const UserOnlyContext = createContext(null);
const ThemeOnlyContext = createContext(null);

function SplitUserName() {
  console.log("[쪼개 상자] SplitUserName 실행");
}

```

F12 콘솔 [쪼갠 상자] SplitUserName 실행

```
const user = useContext(UserOnlyContext);
return <div className="output">사용자: {user}</div>;
}

function SplitThemeLabel() {
  console.log("[쪼갠 상자] SplitThemeLabel 실행");
}
```

F12 콘솔 [쪼갠 상자] SplitThemeLabel 실행

```
const theme = useContext(ThemeOnlyContext);
return <div className="output">테마: {theme}</div>;
}

function SplitProvider({ children }) {
  const [user, setUser] = useState("김민준");
  const [theme, setTheme] = useState("light");

  return (
    <UserOnlyContext.Provider value={user}> <ThemeOnlyContext.Provider value=
{theme}> <button onClick={() => setTheme(theme === "light" ? "dark" : "light")}>
  테마만 바꾸기 (쪼갠 쪽)
    </button> <button onClick={() => setUser(user === "김민준" ? "이서연" :
"김민준")}>
  사용자만 바꾸기 (쪼갠 쪽)
    </button>
    {children}
  </ThemeOnlyContext.Provider> </UserOnlyContext.Provider>
  );
}

function SplitDemo() {
  return (
    <SplitProvider> <SplitUserName /> <SplitThemeLabel /> </SplitProvider>
  );
}
```

콘솔을 비우고 [테마만 바꾸기 (쪼갠 쪽)] 를 한 번 누른 결과입니다.

다시 그려짐 SplitThemeLabel 안 그려짐 SplitUserName ← 섹션 1에서는 그려졌습니다

상자를 나눈 것만으로 범위가 줄었습니다. 성격이 다른 값은 상자를 따로 두세요. 이것이 가장 쉬운 해결입니다.

값과 dispatch 를 나누는 것도 같은 이야기입니다. 개념04에서는 value={{ items, dispatch }} 로 함께 담았습니다. 그러면 items 가 바뀔 때 dispatch 만 쓰는 컴포넌트도 같이 다시 그려집니다.

```
function itemsReducer(state, action) {
  if (action.type === "add") return [...state, "아메리카노"];
  if (action.type === "clear") return [];
  return state;
}
```

```
const ItemsContext = createContext(null);
const DispatchContext = createContext(null);
```

```
function ItemsView() {
  console.log("[분리] ItemsView 실행");
}
```

F12 콘솔 [분리] ItemsView 실행

```
const items = useContext(ItemsContext);
return <div className="output">담긴 것: {items.length}개</div>;
}
```

```
function AddButton() {
  console.log("[분리] AddButton 실행");
}
```

F12 콘솔 [분리] AddButton 실행

```
const dispatch = useContext(DispatchContext);
return (
  <div className="output"> <button onClick={() => dispatch({ type: "add" })}>담기
</button> <button onClick={() => dispatch({ type: "clear" })}>비우기</button> </div>
);
}
```

```
function DispatchSplitProvider({ children }) {
  const [items, dispatch] = useReducer(itemsReducer, []);
  return (
    <DispatchContext.Provider value={dispatch}> <ItemsContext.Provider value={items}>
    {children}</ItemsContext.Provider> </DispatchContext.Provider>
  );
}
```

```
function DispatchSplitDemo() {
  return (
    <DispatchSplitProvider> <ItemsView /> <AddButton /> </DispatchSplitProvider>
  );
}
```

콘솔을 비우고 [담기] 를 한 번 누른 결과입니다.

다시 그려짐 ItemsView 안 그려짐 AddButton ← 목록이 바뀌어도 버튼은 그대로입니다

왜 AddButton 은 안 그려질까요? dispatch 는 화면을 다시 그려도 '같은 함수' 이기 때문입니다. React 가 그렇게 만들어 줍니다. 그래서 DispatchContext 의 value 는 한 번도 바뀌지 않습니다.

버튼처럼 '값은 안 보고 바꾸기만 하는' 컴포넌트가 많을수록 이 분리가 이득입니다.

▣ 직접 해보기 2

DispatchSplitProvider 를 개념04처럼 value={{ items, dispatch }} 상자 하나로 되돌려 보세요.
[담기] 를 눌렀을 때 AddButton 이 다시 그려지는지 확인하세요.

value 를 매번 새로 만들면 손해입니다

섹션 3

이건 실수인데 눈에 잘 안 띵니다. 세어 봐야 보입니다.

```
const InlineContext = createContext(null);
const FixedContext = createContext(null);
```

컴포넌트 밖에 한 번만 만든 객체입니다. 늘 같은 객체입니다.

```
const FIXED_VALUE = { shopName: "우리카페" };

function InlineReader() {
  console.log("[매번 새 객체] InlineReader 실행");
```

F12 콘솔 [매번 새 객체] InlineReader 실행

```
const info = useContext(InlineContext);
return <div className="output">매번 새 객체: {info.shopName}</div>;
}
```

```
function FixedReader() {
  console.log("[같은 객체] FixedReader 실행");
```

F12 콘솔 [같은 객체] FixedReader 실행

```
const info = useContext(FixedContext);
return <div className="output">같은 객체: {info.shopName}</div>;
}

function ValueIdentityDemo() {
  const [tick, setTick] = useState(0);

  return (
    <div> <p>아래 버튼은 Context 와 아무 상관 없는 state 를 바꿉니다.
    </p> <button onClick={() => setTick(tick + 1)}>상관없는 state 바꾸기 ({tick})
    </button>
```

```

    { /* value 자리에서 객체를 새로 만듭니다. 내용은 늘 같습니다. */
      <InlineContext.Provider value={{ shopName: "우리카페" }}> <InlineReader
    /> </InlineContext.Provider>

    { /* 밖에서 한 번 만든 객체를 그대로 씁니다. */
      <FixedContext.Provider value={FIXED_VALUE}> <FixedReader
    /> </FixedContext.Provider> </div>
  );
}

```

콘솔을 비우고 [상관없는 state 바꾸기] 를 한 번 누른 결과입니다.

다시 그려짐 InlineReader ← 화면에 보이는 글자는 하나도 안 바뀌었는데 그려집니다 안 그려짐 FixedReader

이유는 JS자료 09단원에서 배운 그대로입니다. { shopName: "우리카페" } 와 { shopName: "우리카페" } 는 내용이 같아도 다른 객체입니다. React 는 value 가 이전과 '같은 것' 인지만 봅니다. 내용을 하나씩 비교하지 않습니다. 그래서 매 렌더마다 새 객체를 만들어 넣으면 매번 "바뀌었다" 로 처리됩니다.

그럼 어떻게 하나요? 값이 정말 안 바뀌는 것이면 밖에 한 번만 만들어 두세요. 안에서 만들어야 하면 10단원 개념05의 useMemo 로 감쌉니다.

```

const value = useMemo(() => ({ items, dispatch }), [items]);
<CartContext.Provider value={value}>

```

이렇게 하면 items 가 바뀔 때만 새 객체가 됩니다. 실제로 세어서 확인했습니다.

★ 다만 개념04처럼 Provider 컴포넌트가 오직 그 state 때문에만 다시 그려진다면 value 가 어차피 매번 바뀌어야 맞습니다. 이럴 땐 useMemo 가 하는 일이 없습니다. 문제가 되는 것은 '상관없는 이유로 Provider 가 다시 그려질 때' 입니다. 10단원에서 배운 대로, 먼저 재지 말고 쓰지는 마세요.

▫ 직접 해보기 3

InlineContext 쪽도 밖에 만들어 둔 객체를 쓰도록 고치고 콘솔에서 InlineReader 가 사라지는지 확인해 보세요.

먼저 검토할 것 (1) state 끌어올리기

섹션 4

여기서부터가 이 파일의 진짜 요점입니다. Context 를 꺼내기 전에 먼저 볼 것이 둘 있습니다. 07단원에서 이미 배운 것들입니다.

첫째는 state 끌어올리기(07단원 개념03)입니다. 두 컴포넌트가 같은 값을 본다고 곧바로 Context 가 필요한 것이 아닙니다. 둘의 공통 부모가 가까이 있으면 거기에 state 를 두면 끝입니다.

```
function PriceInput({ price, onChange }) {
  return (
    <div className="output">
      가격{" "}
      <input value={price} onChange={(e) => onChange(Number(e.target.value))}
        type="number"
      />
    </div>
  );
}

function PriceResult({ price }) {
  return <div className="output">10잔이면 {price * 10}원입니다</div>;
}

function LiftingDemo() {
```

두 컴포넌트가 같은 값을 봅니다. 공통 부모가 바로 여기입니다.

```
const [price, setPrice] = useState(4000);

return (
  <div> <PriceInput price={price} onChange={setPrice} /> <PriceResult price={price}
  /> </div>
);
}
```

화면 가격 입력칸에 4000, 아래에 "10잔이면 40000원입니다"

화면(누르면): 숫자를 4500 으로 고치면 "10잔이면 45000원입니다" 가 됩니다.

Context 가 한 줄도 없습니다. 그리고 이게 더 좋습니다. · PriceInput 과 PriceResult 는 어디에 갖다 놔도 그대로 동작합니다 · 값이 어디서 오는지 코드에 그대로 보입니다 · Provider 로 감싸는 것을 잊을 일도 없습니다

props 로 한두 단계 내려보내는 것은 문제가 아닙니다. 정상입니다. 개념01에서 문제였던 것은 '쓰지도 않는 컴포넌트가 셋 이상 끼어 있을 때' 였습니다.

◀ 직접 해보기

4

LiftingDemo 에 "5잔이면 얼마" 를 보여 주는 컴포넌트를 하나 더 붙여 보세요. Context 없이 됩니까?

먼저 검토할 것 (2) children 으로 구멍 뚫기

섹션 5

둘째 방법입니다. 03단원 개념04의 children 을 쓰면 중간 단계를 건너뛸 수 있습니다.

개념01의 문제를 다시 봅시다. Layout 이 user 를 받아서 Sidebar 에 넘기고, Sidebar 가 UserPanel 에 넘겼습니다. Layout 과 Sidebar 는 user 를 쓰지도 않는데 말입니다.

그런데 왜 넘겨야 했을까요? UserPanel 을 Sidebar ‘안에서’ 만들었기 때문입니다. UserPanel 을 ‘바깥에서’ 만들어서 통째로 넘기면 됩니다.

```
function UserPanel({ user }) {
  return <div className="output">{user.name} 님 ({user.age}세)</div>;
}
```

sidebar 자리에는 ‘이미 다 만들어진 화면 조각’ 이 들어옵니다. Layout 은 그것이 무엇인지 몰라도 됩니다.

```
function Layout({ sidebar }) {
  return (
    <div className="output"> <strong>레이아웃
  </strong> <div className="output"> <strong>사이드바 자리</strong>
    {sidebar}
  </div> </div>
  );
}

function CompositionDemo() {
  const [user, setUser] = useState({ name: "김민준", age: 20 });

  return (
    <div> <button onClick={() => setUser({ name: "이서연", age: 22 })}>
      이서연으로 바꾸기
    </button>
    { /* UserPanel 을 여기서 만들어 통째로 넘깁니다 */ }
    <Layout sidebar={<UserPanel user={user} />} />
  </div>
  );
}
```

화면 레이아웃 상자 안에 사이드바 자리, 그 안에 "김민준 님 (20세)"

화면(누르면): 버튼을 누르면 "이서연 님 (22세)" 가 됩니다.

Layout 에는 user 라는 글자가 없습니다. Context 도 없습니다. props 로 넘기는 것이 ‘값’ 이 아니라 ‘이미 만들어진 화면’ 이라는 점만 다릅니다.

children 을 쓰면 <Layout><UserPanel user={user} /></Layout> 처럼 쓸 수도 있습니다. 자리가 여러 개면 위처럼 이름을 붙여(sidebar, header) props 로 넘깁니다.

이 방법으로 해결되는 경우가 생각보다 많습니다. 특히 "레이아웃 컴포넌트가 값을 나르고 있다" 면 거의 이 방법이 답입니다.

Layout 에 header 자리를 하나 더 만들고 <Layout header={...} sidebar={...} /> 로 넘겨 보세요.

그래서 언제 쓰나

섹션 6

순서대로 물어보세요. 위에서 걸리면 아래로 안 내려갑니다.

- 1 한두 단계만 내려가나? → props 를 쓰세요. 끝입니다.
- 2 공통 부모가 가까운가? → state 끌어올리기(07단원 개념03)
- 3 중간이 레이아웃 컴포넌트인가? → children / 화면 조각을 props 로 (섹션 5)
- 4 그래도 셋 이상 깊고 여러 곳에서 쓰나? → Context 를 씁니다

4)까지 왔다면 다음도 같이 보세요.

— Context 에 잘 맞는 값

- 앱 전체가 보고, 잘 안 바뀌는 값
- 로그인한 사용자, 테마, 언어 설정, 장바구니

— Context 에 넣으면 안 되는 값

- 자주 바뀌는 값. 입력 중인 글자, 마우스 위치, 스크롤 위치, 타이머 숫자
→ 섹션 1에서 봤듯이 그 상자를 읽는 컴포넌트가 전부 다시 그려집니다.
글자 한 자 칠 때마다 앱 절반이 다시 그려지는 화면이 이렇게 만들어집니다.
- 한 화면에서만 쓰는 값
→ 그 화면의 state 로 두세요.

전역 상태가 정답처럼 보이는 함정

Context 를 배우면 “전역 상태” 라는 말이 매력적으로 들립니다. 어디서나 꺼내 쓸 수 있으니 편할 것 같습니다. 실제로는 이런 일이 생깁니다.

함정 1 · 상자가 하나로 뭉칩니다

처음엔 user 만 넣었는데, 곧 theme 이 들어오고, 장바구니가 들어오고,
모달 열림 여부가 들어옵니다. 어느새 AppContext 하나에 전부 들어 있습니다.
그러면 섹션 1의 [1] 이 앱 전체 규모로 벌어집니다.

함정 2 · 값이 어디서 바뀌는지 안 보입니다

props 는 코드에 경로가 그대로 보입니다. 부모를 따라 올라가면 됩니다.
 Context 는 안 보입니다. 앱 어디에서든 dispatch 를 부를 수 있기 때문입니다.
 "이 숫자를 누가 바꿨지?" 를 찾는 데 시간이 걸립니다.
 그래서 개념03처럼 규칙을 reducer 한 곳에 모아 두는 것이 중요합니다.

함정 3 · 컴포넌트를 혼자 못 씁니다

useCart 를 쓰는 컴포넌트는 CartProvider 없이는 못 씁니다.
 '어디서나 쓸 수 있게' 하려다 '한 곳에서만 쓸 수 있게' 된 것입니다.
 재사용할 컴포넌트라면 값을 props 로 받는 편이 낫습니다.

함정 4 · 서버에서 받아온 데이터를 Context 에 쌓아 둡니다

09단원에서 fetch 로 받은 목록을 Context 에 넣고 캐시처럼 쓰고 싶어집니다.
 그러면 언제 다시 받아야 하는지, 낡은 값을 언제 버릴지를 직접 다 관리해야 합니다.
 이 자료의 범위를 넘어섭니다. 그런 일에는 따로 만들어진 도구들이 있습니다.

마지막으로 한 가지만 기억하세요. props 로 넘기는 것은 부끄러운 일이 아닙니다. 대부분의 화면은 props 로 충분합니다.

▫ 직접 해보기

6

아래 넷 중 Context 가 어울리는 것을 골라 보세요. (가) 검색창에 지금 입력 중인 글자 (나) 로그인한 사용자 정보 (다) 목록 한 줄에 보여 줄 상품 이름 (라) 지금 열려 있는 아코디언 항목 번호 (그 목록 안에서만 씀)

자주 하는 실수

섹션 7

⚠ [SyntaxError] 라고 적힌 것은 주석을 풀지 말고 눈으로만 보세요. 풀면 파일 전체가 멈춰서 화면이 통째로 비어 버립니다. 다시 // 를 붙이면 돌아옵니다.

이런 실수를 합니다**앱의 모든 state 를 상자 하나에 몰아넣음**

섹션 1에서 세어 본 그대로입니다. 에러는 안 납니다. 화면만 계속 다시 그려집니다. 성격별로 상자를 나누세요. 나누는 데는 비용이 거의 없습니다.

이런 실수를 합니다

value 자리에서 객체를 새로 만들

```
<CartContext.Provider value={{ items, dispatch }}>
```

섹션 3에서 세어 봤습니다. 상관없는 이유로 Provider 가 다시 그려질 때마다 value 가 새 객체가 되어, 읽는 컴포넌트가 전부 다시 그려집니다. 내용이 하나도 안 바뀌었는데도 그렇습니다. 고치려면 useMemo 로 감싸거나(10단원), 안 바뀌는 값은 밖에 만들어 두세요.

이런 실수를 합니다

memo 로 감싸면 Context 변경을 막을 수 있다고 생각함

```
const Row = memo(function Row() { const c = useContext(AppContext); ... });
```

막히지 않습니다. 실제로 세어서 확인했습니다. memo 는 'props 가 같으면 건너뛰다' 는 도구입니다 (10단원 개념05). useContext 로 읽는 값은 props 가 아닙니다. 그 값이 바뀌면 memo 여도 다시 그려집니다. memo 가 도움이 되는 것은 'context 를 안 쓰는데 부모 때문에 그려지는' 컴포넌트입니다.

이런 실수를 합니다

Context 를 성능 도구로 오해함

"props 를 많이 내려보내면 느리니까 Context 로 바꾸자" props 를 내려보내는 것 자체는 느리지 않습니다. 값을 넘기는 것뿐입니다. Context 로 바꾸면 오히려 다시 그려지는 범위가 넓어질 수 있습니다. Context 는 '코드를 정리하는 도구' 이지 '빠르게 만드는 도구' 가 아닙니다.

이런 실수를 합니다

Provider 를 조건부로 넣었다 뺐다 함

```
{로그인했다 && <CartProvider><Shop /></CartProvider>}
```

조건이 false 가 되면 CartProvider 가 화면에서 사라집니다. 컴포넌트가 사라지면 그 안의 state 도 같이 사라집니다. 다시 true 가 되면 장바구니가 빈 채로 새로 시작합니다. 에러가 안 나서 "왜 장바구니가 비었지?" 로만 보입니다. Provider 는 화면 맨 위에 항상 두고, 안쪽에서 조건을 거세요.

이런 실수를 합니다

상자를 쪼갠 뒤 Provider 순서를 헷갈림

```
<ItemsContext.Provider value={items}>  
<DispatchContext.Provider value={dispatch}>
```

순서는 결과에 영향이 없습니다. 겹치는 순서는 자유입니다. 다만 잘 안 바뀌는 것(dispatch)을 바깥에 두면 코드를 읽기 좋습니다. 진짜 실수는 '한쪽 Provider 를 빼먹는 것' 입니다. 그러면 그 상자만 null 이 됩니다.

정리

- 1 Context 값이 바뀌면 그 상자를 읽는 컴포넌트가 전부 다시 그려집니다. 어느 속성을 읽었는지는 구분되지 않습니다.
- 2 성격이 다른 값은 상자를 나누세요. 값 상자와 dispatch 상자를 나누면 버튼들은 다시 안 그려집니다.
- 3 value 자리에서 객체를 새로 만들면 내용이 같아도 '바뀐 것'이 됩니다. 밖에 두거나 useMemo 로 고정하세요.
- 4 memo 는 Context 변경을 막지 못합니다. memo 는 props 만 비교합니다.
- 5 Context 를 꺼내기 전에 props → state 끌어올리기 → children 순서로 먼저 검토하세요.
- 6 자주 바뀌는 값(입력 중인 글자 등)은 Context 에 넣지 마세요. 화면 절반이 매 글자마다 다시 그려집니다.
- 7 Context 는 정리하는 도구지 빠르게 만드는 도구가 아닙니다. props 로 충분한 화면이 대부분입니다.

```
export default function Concept05WhenToUse() {
  return (
    <div> <h1>개념 05 - 언제 쓰나</h1> <p className="guide"> <strong>F12 → Console 을
    꼭 여세요.</strong> 버튼을 누르기 전에 콘솔을 한 번
      지우고(🗑️ 아이콘) 누르면 무엇이 다시 그려졌는지 정확히 보입니다.
    <br />
      StrictMode 때문에 한 번 그려질 때 두 줄씩 찍힙니다. <strong>두 줄이 한 번
    </strong>입니다.
    <p> <div className="demo"> <h3>① 상자 하나에 값 둘 (섹션 1)</h3> <OneBoxDemo
    /> </div> <div className="demo"> <h3>② 상자를 쪼갬 경우 (섹션 2)</h3> <SplitDemo
    /> </div> <div className="demo"> <h3>③ 값 상자 / dispatch 상자 (섹션 2)
    </h3> <DispatchSplitDemo /> </div> <div className="demo"> <h3>④ value 를 매번 새로
    만들면 (섹션 3)</h3> <ValueIdentityDemo /> </div> <div className="demo"> <h3>⑤ 대안
    A - state 끌어올리기 (섹션 4)</h3> <LiftingDemo /> </div>

    <div className="demo">
      <h3>⑥ 대안 B - 화면 조각을 통째로 넘기기 (섹션 5)</h3> <CompositionDemo />
    </div>

  </div>
);
}
```

직접 해보기 정답

- 1) ThemeLabel 도 다시 그려집니다.

```
// 콘솔: [한 상자] UserName 실행
// 콘솔: [한 상자] ThemeLabel 실행
// 콘솔: [한 상자] InsideBox 실행 (context 안 씀)
```

→ 테마를 바꿨을 때와 완전히 같은 결과입니다. FromChildren 만 빠집니다.

한 상자에 담긴 이상 어느 값이 바뀌든 결과가 같습니다.

```
2) function DispatchSplitProvider({ children }) {
```

```
const [items, dispatch] = useReducer(itemsReducer, []);
return (
  <ItemsContext.Provider value={{ items: items, dispatch: dispatch }}>
    {children}
  </ItemsContext.Provider>
);
```

```
}
```

(ItemsView 는 useContext(ItemsContext).items, AddButton 은 .dispatch 를 쓰게 고칩니다)

```
// 콘솔: [분리] ItemsView 실행
// 콘솔: [분리] AddButton 실행
```

→ AddButton 이 다시 그려집니다. 상자를 합치면 손해가 그대로 돌아옵니다.

```
3) const INLINE_VALUE = { shopName: "우리카페" }; ← 컴포넌트 밖에 둡니다
<InlineContext.Provider value={INLINE_VALUE}>
// 콘솔: (InlineReader 줄이 안 나옵니다)
```

→ FixedReader 와 같은 결과가 됩니다. value 가 늘 같은 객체이기 때문입니다.

```
4) function PriceResultFive({ price }) {
```

```
return <div className="output">5잔이면 {price * 5}원입니다</div>;
```

```
}
```

LiftingDemo 안에 <PriceResultFive price={price} /> 를 한 줄 더 씁니다.

```
// 화면: 5잔이면 20000원입니다
```

→ 됩니다. 공통 부모가 바로 위라서 Context 가 필요 없습니다.

이 상황에서 Context 를 쓰면 코드만 길어집니다.

```

5) function Layout({ header, sidebar }) {

  return (
    <div className="output"> <div className="output">{header}
    </div> <div className="output"><strong>사이드바 자리</strong>{sidebar}</div> </div>
  );

}

<Layout header={<strong>우리카페</strong>} sidebar={<UserPanel user={user} />} />
// 화면: 위 상자에 "우리카페", 아래 상자에 "김민준 님 (20세)"

```

→ Layout 은 여전히 user 를 모릅니다. 자리만 빌려줄 뿐입니다.

6) (나) 로그인한 사용자 정보입니다. → (가) 는 글자를 칠 때마다 바뀝니다. 그 상자를 읽는 컴포넌트가 매 글자마다 다시 그려집니다.

검색창과 결과 목록의 공통 부모에 state 를 두세요(섹션 4).
 (다) 는 그 줄에서만 씁니다. props 로 충분합니다.
 (라) 는 그 목록 안에서만 씁니다. 목록 컴포넌트의 state 로 두세요.
 (나) 만 앱 전체가 보고, 자주 바뀌지도 않습니다.

연습문제 (13문항)

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 여세요.

- 1 아래로 내려가며 // TODO 를 찾아 코드를 고칩니다.
- 2 저장하면 화면이 저절로 바뀝니다(Vite). F5 를 누르지 않아도 됩니다.
- 3 각 문제의 '기대 결과' 와 화면을 비교합니다.
- 4 막히면 개념01~05 파일의 해당 섹션을 다시 보세요.

순서는 기본 10문제 → [응용] → [도전] → 에러 확인 입니다. 뒤로 갈수록 어렵습니다. 정답은 연습문제_정답.jsx 에 있습니다. 먼저 스스로 해 보세요.

★ 고치기 전 지금 화면에는 "(아직 안 고쳤습니다)" 같은 글자가 보입니다. 정상입니다.

```
import { createContext, useContext, useReducer, useState } from "react";
import Summary from "../_ui/Summary.jsx";
```

문제 1 (개념01)

props 를 세 단계로 내려보내 보세요. Q1Parent 가 nickname 을 가지고 있습니다. 이것이 Q1GrandChild 에 보여야 합니다. Q1Child 는 nickname 을 쓰지 않지만 받아서 넘겨야 합니다.

기대 화면 별명: 민준
 "별명: (아직 안 고쳤습니다)" 가 그대로면 아직 안 넘긴 것입니다.
 "별명: " 뒤가 비면 이름을 잘못 적어(nickName 등) undefined 가 된
 것입니다.

```
function Q1GrandChild() {
```

props 를 받아 nickname 을 화면에 보여 주세요



```
return <div className="output">별명: (아직 안 고쳤습니다)</div>;
}

function Q1Child() {
```

props 를 받아 Q1GrandChild 에 그대로 넘기세요

```
return (
  <div className="output"> <Q1GrandChild /> </div>
);
}

function Q1Parent() {
  const nickname = "민준";
```

Q1Child 에 nickname 을 넘기세요

```
return (
  <div className="output"> <strong>문제 1</strong> <Q1Child /> </div>
);
}
```

문제 2 (개념 02)

createContext 의 기본값을 "라이트" 로 지정하세요. 지금은 인자를 안 줬기 때문에 기본값이 `undefined` 입니다.

기대 화면 테마: 라이트
 "테마:" 뒤가 비어 있으면 아직 기본값을 안 준 것입니다.
 (JSX 는 `undefined` 를 화면에 아무것도 그리지 않습니다 - 02단원)

createContext 안에 “라이트” 를 넣으세요

```
const Q2Context = createContext();

function Q2Label() {
  const theme = useContext(Q2Context);
  return <div className="output">테마: {theme}</div>;
}
```

문제 3 (개념 02)

Q3Label 을 Provider 로 감싸서 “다크” 가 보이게 하세요. 상자와 라벨은 이미 만들어져 있습니다.
Q3Demo 만 고치면 됩니다.

기대 화면 테마: 다크
"테마: 라이트" 가 보이면 아직 Provider 로 안 감싼 것입니다(기본값이 나온 것).

```
const Q3Context = createContext("라이트");

function Q3Label() {
  const theme = useContext(Q3Context);
  return <div className="output">테마: {theme}</div>;
}

function Q3Demo() {
```

<Q3Label /> 을 <Q3Context.Provider value="다크"> 로 감싸세요

```
return (
  <div className="output"> <strong>문제 3</strong> <Q3Label /> </div>
);
}
```


문제 4 (개념02)

아래 Q4Demo 에는 같은 컴포넌트가 두 번 놓여 있습니다. 하나는 Provider 안, 하나는 Provider 밖입니다. 화면을 보기 전에 각각 무엇이 보일지 예상해서 아래 빈칸에 적어 보세요.

Provider 안

Provider 밖

기대 출력 먼저 예상해서 적은 뒤 화면과 비교하세요.
예상이 틀렸다면 개념02 섹션 4를 다시 보세요.

예상을 위 빈칸에 적은 뒤, 아래 코드는 고치지 말고 화면만 확인하세요.

```
const Q4Context = createContext("기본값입니다");

function Q4Label({ where }) {
  const value = useContext(Q4Context);
  return (
    <div className="output">
      {where}: {value}
    </div>
  );
}

function Q4Demo() {
  return (
    <div className="output"> <strong>문제
4</strong> <Q4Context.Provider value="Provider 가 준 값"> <Q4Label where="Provider
안" /> </Q4Context.Provider> <Q4Label where="Provider 밖" /> </div>
  );
}
```

문제 5 (개념03)

reducer 에 case "add" 와 case "remove" 를 만드세요. add 는 1 늘리고, remove 는 1 줄입니다. 0 아래로는 내려가지 않습니다.

기대 화면 [답기] 를 세 번 누르면 "아메리카노 3잔"

빼기 · 를 여러 번 눌러도 0잔에서 멈춥니다.

버튼을 눌러도 숫자가 안 변하면 case 가 default 로 빠진 것입니다.

```
function q5Reducer(state, action) {  
  switch (action.type) {
```

case "add" 와 case "remove" 를 여기에 쓰세요

```
    default:  
      return state;  
  }  
}
```

```
function Q5Demo() {  
  const [count, dispatch] = useReducer(q5Reducer, 0);  
  return (  
    <div className="output"> <strong>문제 5</strong> <p>아메리카노 {count}잔  
</p> <button onClick={() => dispatch({ type: "add" })}>답기</button> <button onClick=  
{() => dispatch({ type: "remove" })}>빼기</button> </div>  
  );  
}
```

문제 6 (개념03)

아래 reducer 에 case "reset" 을 추가해 0으로 되돌리세요. 그리고 Q6Demo 에 [비우기] 버튼을 만들어 dispatch 하세요. (add 는 이미 만들어져 있습니다)

기대 화면 [답기] 를 두 번 눌러 2잔이 된 뒤 [비우기] 를 누르면 0잔

비우기 · 를 눌렀는데 그대로면 type 이름이 서로 다른 것입니다.

```
function q6Reducer(state, action) {
  switch (action.type) {
    case "add":
      return state + 1;
```

case "reset" 을 여기에 쓰세요

```
default:
  return state;
}

function Q6Demo() {
  const [count, dispatch] = useReducer(q6Reducer, 0);
  return (
    <div className="output"> <strong>문제 6</strong> <p>아메리카노 {count}잔</p>
    <p><button onClick={() => dispatch({ type: "add" })}>담기</button>
    { /* TODO: [비우기] 버튼을 여기에 만드세요 */ }
    </div>
  );
}
```

문제 7 (개념03)

action 에 값을 실어 보내세요. case "addMany" 를 만들고 action.payload 만큼 늘리세요. 버튼 두 개는 각각 1과 3을 실어 보냅니다.

기대 화면 [3잔 담기] 를 두 번 누르면 "6잔"
숫자가 안 변하면 case 이름이 다르거나 payload 를 안 읽은 것입니다.
NaN 이 나오면 payload 를 안 보냈거나 이름을 잘못 읽은 것입니다.

```
function q7Reducer(state, action) {
  switch (action.type) {
```

case “addMany” 를 여기에 쓰세요

```
default:
    return state;
}

function Q7Demo() {
    const [count, dispatch] = useReducer(q7Reducer, 0);
    return (
        <div className="output"> <strong>문제 7</strong> <p>아메리카노 {count}잔</p> <button onClick={() => dispatch({ type: "addMany", payload: 1 })}>1잔 담기</button> <button onClick={() => dispatch({ type: "addMany", payload: 3 })}>3잔 담기</button> </div>
    );
}
```

문제 8 (개념03)

reducer 는 컴포넌트 밖의 그냥 함수라서 배열의 reduce 에 넣어 돌릴 수 있습니다. 아래 q8Steps 를 q8Reducer 로 접으면 결과가 얼마일까요? 먼저 종이에 예상해 보고, 그 다음 `console.log` 를 써서 확인하세요.

기대 콘솔 예상한 숫자와 같아야 합니다.
틀렸다면 한 바퀴씩 짊어 보세요(개념03 섹션 3의 방법).

```
function q8Reducer(state, action) {
    switch (action.type) {
        case "add":
            return state + 1;
        case "double":
            return state * 2;
        case "reset":
            return 0;
        default:
            return state;
    }
}
```

```
const q8Steps = [
  { type: "add" },
  { type: "add" },
  { type: "double" },
  { type: "add" },
  { type: "모르는것" },
];
```

아래 줄의 주석을 풀어 결과를 콘솔에서 확인하세요

```
console.log(q8Steps.reduce(q8Reducer, 0));
```

문제 9 (개념04)

아래 장바구니는 Provider 와 useCart 훅까지 다 만들어져 있습니다. Q9MenuList 의 [담기] 버튼이 아무 일도 하지 않습니다. dispatch 를 부르게 고치세요.

기대 화면 [담기] 를 누르면 아래에 "아메리카노 1개" 가 생깁니다.
한 번 더 누르면 "아메리카노 2개" 가 됩니다.
아무 변화가 없으면 dispatch 를 안 부른 것입니다.

```

const Q9_MENU = [
  { id: 1, name: "아메리카노", price: 4000 },
  { id: 2, name: "라떼", price: 4500 },
];

function q9Reducer(state, action) {
  switch (action.type) {
    case "add":
      if (state.some((item) => item.id === action.payload.id)) {
        return state.map((item) =>
          item.id === action.payload.id ? { ...item, count: item.count + 1 } : item
        );
      }
      return [...state, { ...action.payload, count: 1 }];
    default:
      return state;
  }
}

const Q9Context = createContext(null);

function Q9Provider({ children }) {
  const [items, dispatch] = useReducer(q9Reducer, []);
  return (
    <Q9Context.Provider value={{ items: items, dispatch: dispatch }}>
      {children}
    </Q9Context.Provider>
  );
}

function useQ9Cart() {
  const cart = useContext(Q9Context);
  if (cart === null) {
    throw new Error("useQ9Cart 는 <Q9Provider> 안에서만 쓸 수 있습니다");
  }
  return cart;
}

```

```

}

function Q9MenuList() {
  const { dispatch } = useQ9Cart();
  return (
    <div className="output"> <ul>
      {Q9_MENU.map((menu) => (
        <li key={menu.id}>
          {menu.name} {menu.price}원{" "}
          { /* TODO: 누르면 dispatch({ type: "add", payload: menu }) 가 되게 하세요
*/}

          <button>담기</button> </li>
        )))}
    </ul> </div>
  );
}

function Q9CartList() {
  const { items } = useQ9Cart();
  if (items.length === 0) {
    return <div className="output">장바구니가 비어 있습니다</div>;
  }
  return (
    <div className="output"> <ul>
      {items.map((item) => (
        <li key={item.id}>
          {item.name} {item.count}개
        </li>
      )))}
    </ul> </div>
  );
}

```

문제 10 (개념 04)

Q10Total 이 총액을 보여 주게 만드세요. reduce 를 쓰세요(JS자료 08단원 개념05). 한 칸의 금액은 price * count 입니다. (문제 9의 장바구니를 그대로 씁니다. 문제 9를 먼저 푸세요)

기대 화면 아메리카노 2개, 라떼 1개를 담으면 "합계 12500원"
 "합계 0원" 에서 안 변하면 아직 안 고친 것입니다.
 "합계 8500원" 이면 count 를 안 곱한 것입니다.

```

function Q10Total() {
  const { items } = useQ9Cart();

```

reduce 로 총액을 구해 total 에 담으세요

```
const total = 0;
return (
  <div className="output"> <strong>합계 {total}원</strong> </div>
);
}

function Q9Q10Demo() {
  return (
    <div className="output"> <strong>문제 9 · 10</strong> <Q9Provider> <Q9MenuList
/> <Q9CartList /> <Q10Total /> </Q9Provider> </div>
  );
}
```

문제 11 [응용] (개념04)

수량을 줄이는 기능을 만드세요. case "decrease" 를 추가해서 count 를 1 줄이되, 0이 되면 목록에서 아예 빼세요. (힌트: map 으로 줄인 다음 filter 로 거릅니다. 개념04 섹션 2와 같은 방법입니다)

기대 화면 [답기] 두 번 → "아메리카노 2개" → [-] → "1개" → [-] → 목록에서 사라짐
"아메리카노 0개" 가 남아 있으면 filter 를 안 한 것입니다.
숫자가 음수로 내려가면 filter 조건이 잘못된 것입니다.

```
const Q11_MENU = { id: 1, name: "아메리카노", price: 4000 };

function q11Reducer(state, action) {
  switch (action.type) {
    case "add":
      if (state.some((item) => item.id === action.payload.id)) {
        return state.map((item) =>
          item.id === action.payload.id ? { ...item, count: item.count + 1 } : item
        );
      }
      return [...state, { ...action.payload, count: 1 }];
  }
}
```


case "decrease" 를 여기에 쓰세요 (payload 는 id 입니다)

```
default:
  return state;
}
}

function Q11Demo() {
  const [items, dispatch] = useReducer(q11Reducer, []);
  return (
    <div className="output"> <strong>문제 11 [응용]</strong> <button onClick={() =>
dispatch({ type: "add", payload: Q11_MENU })}>담기</button>
    {items.length === 0 ? (
      <div className="output">비어 있습니다</div>
    ) : (
      <ul>
        {items.map((item) => (
          <li key={item.id}>
            {item.name} {item.count}개{" "}
            <button onClick={() => dispatch({ type: "decrease", payload: item.id
        ))}>
          -
        </button> </li>
        ))}
      </ul>
    )}
    </div>
  );
}
```

문제 12 [도전] (개념05)

아래는 상자 하나에 user 와 theme 을 같이 담은 코드입니다.

테마만 바꾸기 · 를 누르면 Q12UserName 도 같이 다시 그려집니다. 콘솔에서 확인하세요.

이것을 상자 두 개로 쪼개서, 테마를 바꿔도 Q12UserName 이 다시 그려지지 않게 만드세요. 고칠 곳은 세 군데입니다. 상자 만들기 / Provider / 꺼내 쓰는 곳.

기대 콘솔 콘솔을 지우고 [테마만 바꾸기] 를 한 번 누르면
 "[문제12] Q12ThemeLabel 실행" 만 나와야 합니다.
 "[문제12] Q12UserName 실행" 이 같이 나오면 아직 한 상자인 것입니다.
 ★ StrictMode 때문에 같은 줄이 두 번씩 찍힙니다. 두 줄이 한 번입니다.

상자를 Q12UserContext 와 Q12ThemeContext 두 개로 나누세요

```
const Q12Context = createContext(null);

function Q12UserName() {
  console.log("[문제12] Q12UserName 실행");
}
```

F12 콘솔 [문제12] Q12UserName 실행

user 만 담긴 상자에서 꺼내도록 고치세요

```
const app = useContext(Q12Context);
return <div className="output">사용자: {app.user}</div>;
}

function Q12ThemeLabel() {
  console.log("[문제12] Q12ThemeLabel 실행");
}
```

F12 콘솔 [문제12] Q12ThemeLabel 실행

theme 만 담긴 상자에서 꺼내도록 고치세요

```
const app = useContext(Q12Context);
return <div className="output">테마: {app.theme}</div>;
}

function Q12Demo() {
  const [user, setUser] = useState("김민준");
  const [theme, setTheme] = useState("light");
```

Provider 를 두 개로 겹쳐서 각각 user 와 theme 만 내려보내세요

```
return (
  <div className="output"> <strong>문제 12 [도전]
</strong> <Q12Context.Provider value={{ user: user, theme: theme }}> <button onClick=
{() => setTheme(theme === "light" ? "dark" : "light")}>
  테마만 바꾸기
</button> <button onClick={() => setUser(user === "김민준" ? "이서연" :
"김민준")}>
  사용자만 바꾸기
</button> <Q12UserName /> <Q12ThemeLabel /> </Q12Context.Provider> </div>
);
}
```

문제 13 에러 확인 (개념02 · 개념04)

아래 Q13Badge 는 기본값이 null 인 상자에서 값을 꺼냅니다. 그런데 Provider 로 감싸지 않았습니다.

(1) 아래 TODO 의 주석을 풀고 저장하세요. 화면에 빨간 상자가 뜹니다.

에러 문장을 읽고 무슨 뜻인지 생각해 보세요.

(2) 확인했으면 Q13Demo 에서 Provider 로 감싸 고치세요.

value 는 { name: "김민준" } 입니다.

(3) 고친 뒤에는 주석을 푼 채로 두어도 빨간 상자가 안 뜹니다.

기대 화면 고치기 전 → 빨간 상자에
TypeError: Cannot read properties of null (reading 'name')
고친 뒤 → 김민준 님
★ 이 예제만 빨간 상자가 되고 왼쪽 메뉴와 다른 예제는 멀쩡합니다.
ErrorBox 가 잡아 주기 때문입니다.

```
const Q13Context = createContext(null);

function Q13Badge() {
  const user = useContext(Q13Context);
```

아래 줄의 주석을 풀어 어떤 에러가 나는지 확인하세요

```
return <div className="output">{user.name} 님</div>;
```

```
return <div className="output">(아직 확인하지 않았습니다)</div>;
}

function Q13Demo() {
```

(2)에서 여기를 Q13Context.Provider 로 감싸세요

```
return (
  <div className="output"> <strong>문제 13 (에러 확인)</strong> <Q13Badge /> </div>
);
}
```

정리

- 1 문제 1~4: props 로 내려보내기와 Context 세 단계(createContext · Provider · useContext).
- 2 문제 5~8: reducer 를 만들고 dispatch 로 부르기. reducer 는 reduce 에 넣어 돌려볼 수 있습니다.
- 3 문제 9~11: Context 와 useReducer 를 합쳐 장바구니 만들기. 불변 갱신을 지켜세요.
- 4 문제 12: 상자를 쪼개면 다시 그려지는 범위가 줄어듭니다. 콘솔로 세어서 확인하세요.
- 5 문제 13: Provider 없이 useContext 를 쓰면 어떤 에러가 나는지 직접 보고 고칩니다.
- 6 다 풀었으면 연습문제_정답.jsx 와 비교해 보세요. 답이 하나만 있는 것은 아닙니다.

```

export default function Exercise12() {
  return (
    <div>
      <h1>12단원 연습문제</h1>

      <p className="guide">
        <strong>F12 → Console</strong> 을 함께 여세요. 문제 8과 12는 콘솔로
        확인합니다.
        <br />
        고치기 전에는 "(아직 안 고쳤습니다)" 같은 글자가 보입니다. 정상입니다.
        <br />
        저장하면 화면이 저절로 바뀝니다. F5 를 누르지 않아도 됩니다.
      </p>

      <div className="demo">
        <Q1Parent />
      </div>

      <div className="demo">
        <strong>문제 2</strong>
        <Q2Label />
      </div>

      <div className="demo">
        <Q3Demo />
      </div>

      <div className="demo">
        <Q4Demo />
      </div>

      <div className="demo">
        <Q5Demo />
      </div>

      <div className="demo">
        <Q6Demo />
      </div>
    </div>
  )
}

```

```

    </div>

    <div className="demo"> <Q7Demo /> </div> <div className="demo"> <strong>문제
8</strong> <p>콘솔에서 확인하는 문제입니다. 화면에는 아무것도 안 나옵니다.
</p> </div> <div className="demo"> <Q9Q10Demo
/> </div> <div className="demo"> <Q11Demo /> </div> <div className="demo"> <Q12Demo
/> </div> <div className="demo"> <Q13Demo /> </div>

    </div>
  );
}

```

12단원 마무리 Context와 useReducer

자주 넘어지는 자리

- 1 Context 값이 바뀌면 그 값을 쓰는 컴포넌트가 **전부** 다시 그려짐 | 남용 경고의 근거
- 2 `memo` 는 **Context** 변경을 못 막습니다 |

종합 프로젝트

OPEN 13_종합_프로젝트

```
const FIRST_TODOs = [  
  let nextId = 3;  
  return (  
    style={{ cursor: "pointer" }}  
  )  
];
```

01 할 일 목록

02 장바구니

03 사용자 검색

04 메모장

할 일 목록

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 여세요.

★ 이 앱은 JS자료 13단원 종합03과 '완전히 같은 앱' 입니다. 추가 · 완료 토글 · 삭제 · 필터 · 개수까지 요구사항이 하나도 다르지 않습니다. 같은 앱을 이번에는 React 로 다시 만듭니다.

— 두 코드는 무엇이 다른가

JS 판에서 우리는 화면을 '직접' 고쳤습니다. `createElement` 로 li 를 만들고, `appendChild` 로 붙이고, `innerHTML = ""` 로 지우고, `classList.add("done")` 로 취소선을 그었습니다. 데이터가 한 번 바뀔 때마다 "화면의 어느 부분을 어떻게 고칠지" 를 우리가 전부 적어 썼습니다. 그래서 `render()` 라는 함수를 직접 만들고, 무슨 일이 생길 때마다 그것을 잊지 않고 다시 불러야 했습니다(JS 판 문제 8). React 판에는 그런 코드가 한 줄도 없습니다. `createElement` 도, `appendChild` 도, `classList` 도, `render()` 를 다시 부르는 줄도 없습니다. 우리는 "todos 가 이러면 화면은 이렇게 생겼다" 만 적습니다. 데이터를 `setTodos` 로 바꾸면 화면을 고치는 일은 React 가 합니다. 01단원 개념01에서 "React 는 그 일을 대신해 줍니다" 라고 했던 이야기가 여기서 끝납니다. 같은 앱을 두 번 만들어 봤으니 이제 그 말이 무슨 뜻이었는지 코드로 확인할 수 있습니다.

쓰는 단원 · 04(state·이벤트) · 05(조건부 렌더링·리스트·key) ·

06(폼·제어 컴포넌트) · 07(불변 갱신·컴포넌트 쪼개기)

★ 이 실습에서 가장 어려운 것은 문제 5 입니다. 재미있게도 JS 판에서도 가장 어려운 것이 문제 5였습니다. 이유는 전혀 다릅니다. JS 판은 '이벤트 위임과 dataset 문자열' 이 어려웠고, React 판은 '배열도 새것, 그 안의 객체도 새것' 이라는 두 겹이 어렵습니다.

— 푸는 법

- 1 아래로 내려가며 // TODO 를 찾아 코드를 고칩니다.
- 2 저장하면 화면이 저절로 바뀝니다(Vite). F5 를 누르지 않아도 됩니다.
- 3 각 문제의 '기대 결과' 와 화면을 비교합니다.
- 4 막히면 07단원 개념01(추가·삭제·수정 세 줄)을 다시 보세요.

★ 아직 아무것도 안 고친 지금도 화면은 나옵니다. 목록 자리에 "여기에 목록이 나옵니다 (문제 2)" 같은 안내가 보이는 것이 정상입니다.

```
import { useState } from "react";
import Summary from "../_ui/Summary.jsx";
```

할 일 하나의 모양입니다. JS 판과 똑같습니다.

```
{ id: 1, text: "장보기", done: false }
```

```
const FIRST_TODOs = [
  { id: 1, text: "장보기", done: false },
  { id: 2, text: "설거지", done: true },
];
```

새 항목에 붙일 번호입니다. 화면에 보이는 값이 아니므로 state 가 아니어도 됩니다(07단원 개념05). 07단원 개념01의 nextCartId 와 같은 방식입니다.

```
let nextId = 3;
```

한 줄짜리 부품 두 개 (이미 만들어 두었습니다)

할 일 한 줄입니다. 03단원에서 배운 대로 props 로 값과 함수를 받습니다. 함수를 props 로 내려보내는 것은 07단원 개념04에서 배웠습니다.

```
function TodoItem({ todo, onToggle, onDelete }) {
  return (
    <li>
      { /* done 이면 취소선이 그어집니다. 05단원 개념01의 삼항 연산자입니다. */ }
      <span className={todo.done ? "done" : ""}
        style={{ cursor: "pointer" }}
        onClick={() => onToggle(todo.id)}
      >
        {todo.text}
      </span>{" "}
      <button onClick={() => onDelete(todo.id)}>삭제</button> </li>
    );
  }
}
```

필터 버튼 한 개입니다. 문제 7에서 이 안을 고칩니다.

```
function FilterButton({ value, label, filter, onChange }) {
```

문제 7

지금 고른 필터와 같은 버튼에만 "on" 클래스를 붙이세요. (className 이 "on" 이면 파란 배경이 됩니다. 05단원 개념01)

기대 화면 [전체] 만 파란색인 상태로 시작합니다.

미완료 · 를 누르면 [미완료] 만 파란색이 됩니다.

세 개가 다 파랑거나 다 회색이면 조건을 반대로 쓴 것입니다.
누른 버튼이 안 파래지면 value 와 filter 를 비교하지 않은 것입니다.

className 을 filter === value 일 때만 "on" 으로 만드세요

```
return (  
  <button className="" onClick={() => onChange(value)}>  
    {label}  
  </button>  
);  
}
```

여기서부터가 앱 본체입니다

```
function TodoApp() {  
  const [todos, setTodos] = useState(FIRST_TODOs);  
  const [text, setText] = useState(""); // 입력칸 (06단원 제어 컴포넌트) const  
  [error, setError] = useState(""); // 빨간 안내 문구 const [filter, setFilter] =  
  useState("all"); // "all" | "active" | "done"
```

문제 1 보여 줄 목록 고르기

filter 값에 따라 화면에 보여 줄 배열을 만드세요. "all" → 전부 "active" → done 이 false 인 것만 "done"
→ done 이 true 인 것만

★ 이것을 state 로 두지 마세요. todos 와 filter 로 언제든 계산할 수 있습니다. 계산할 수 있는 값을 state
로 두면 두 값이 어긋납니다(07단원 개념05).

기대 화면 [미완료] 를 누르면 "장보기" 한 줄만,

완료 · 를 누르면 "설거지" 한 줄만 남습니다.

필터를 눌러도 두 줄 그대로면 아직 todos 를 그대로 쓰는 것입니다.
(문제 2까지 해야 목록이 눈에 보입니다)

아래 줄을 filter 를 반영하도록 고치세요

```
const visibleTodos = todos;
```

문제 3 개수 세기

아래 세 값을 실제 개수로 채우세요. 필터와 상관없이 '전체 todos' 기준입니다. total 전체 개수
doneCount 완료한 개수 leftCount 남은 개수

★ 이것도 state 로 두지 않습니다. todos 하나에서 전부 나옵니다.

기대 화면 맨 아래 줄 → 할 일 2개 (완료 1개, 남은 일 1개)
필터를 [미완료] 로 바꿔도 이 줄은 그대로여야 합니다.
필터에 따라 숫자가 변하면 visibleTodos 를 세고 있는 것입니다.

아래 세 줄을 고치세요

```
const total = 0;
const doneCount = 0;
const leftCount = 0;
```

문제 4 추가

- 1 입력값 양옆 공백을 없앤다 (trim)
- 2 비어 있으면 error 에 "할 일을 입력해 주세요" 를 넣고 끝낸다
- 3 같은 내용이 이미 있으면 "이미 있는 항목입니다" 를 넣고 끝낸다

(some 은 "하나라도 있나" 를 알려 줍니다 - JS자료 08단원)

4) 통과하면 error 를 비우고, todos 에 새 항목을 추가한다

새 항목: { id: nextId, text: 다듬은글, done: false }
★ push 가 아니라 [...todos, 새것] 입니다 (07단원 개념01)

5) nextId 를 1 늘리고, 입력칸(text)을 비운다

e.preventDefault() 는 미리 넣어 두었습니다. 이 줄을 지우면 폼이 제출되면서 페이지가 통째로 새로고침되어 답아 둔 할 일이 전부 날아갑니다(06단원 개념04).

기대 화면 빈 칸에서 [추가] → 할 일을 입력해 주세요 (목록은 2줄 그대로)
"장보기" 넣고 [추가] → 이미 있는 항목입니다 (목록은 2줄 그대로)
"운동하기" 넣고 [추가] → 목록 3줄, 입력칸이 비워지고 빨간 글자도
사라집니다
상태줄 → 할 일 3개 (완료 1개, 남은 일 2개)
추가는 되는데 입력칸에 글자가 남아 있으면 5)를 빠뜨린 것입니다.
추가할 때마다 목록이 하나로 덮어써지면 [...todos, 새것] 의 ... 을 뺀
것입니다.

```
function handleSubmit(e) {  
  e.preventDefault();
```

여기에 코드를 쓰세요

```
}
```

문제 5 완료 토글 ★ 이 파일에서 가장 어렵습니다

누른 항목의 done 을 뒤집으세요.

두 가지를 동시에 지켜야 합니다. ① todos 배열이 '새 배열' 이어야 한다 → map 을 씁니다

② 바꾸는 그 항목도 '새 객체' 여야 한다 → { ...todo, done: !todo.done }

형태는 07단원 개념01 섹션5의 그 줄입니다.

```
setTodos(todos.map((todo) => (todo.id === 고칠id ? { ...todo, 바꿀것 } : todo)))
```

기대 화면 "장보기" 글자를 누르면 회색 취소선이 생깁니다.
상태줄 → 할 일 2개 (완료 2개, 남은 일 0개)
한 번 더 누르면 취소선이 없어집니다.
아무 반응이 없으면 setTodos 를 안 불렀거나, 배열을 직접 고친 것입니다.
누른 줄 말고 다른 줄까지 바뀌면 todo.id === id 조건을 빠뜨린 것입니다.

```
function handleToggle(id) {
```

여기에 코드를 쓰세요

```
}
```

문제 6 삭제

누른 항목을 목록에서 지우세요. filter 는 '남길 것' 을 고릅니다(07단원 개념01).

기대 화면 "설거지" 의 [삭제] 를 누르면 그 줄만 사라집니다.
상태줄 → 할 일 1개 (완료 0개, 남은 일 1개)
누른 것만 남고 나머지가 사라지면 !== 를 === 로 쓴 것입니다.

```
function handleDelete(id) {
```

여기에 코드를 쓰세요

```
}
```

문제 8 완료 항목 한꺼번에 지우기

done 이 true 인 항목을 전부 지우세요. 문제 6과 조건만 다릅니다.

기대 화면 처음 상태에서 [완료 항목 삭제] → "설거지" 가 사라지고
"장보기" 한 줄만 남습니다.
상태줄 → 할 일 1개 (완료 0개, 남은 일 1개)
한 번 더 누르면 아무 일도 안 일어나야 정상입니다.
목록이 통째로 비면 조건을 반대로 쓴 것입니다.

```
function handleClearDone() {
```

여기에 코드를 쓰세요


```

}

return (
  <div className="demo"> <h3>할 일 목록</h3>

    {/* 입력칸은 06단원의 제어 컴포넌트입니다. 이미 만들어 두었습니다. */}
    <form onSubmit={handleSubmit}> <input type="text" value={text} onChange={(e) =>
setText(e.target.value)}
      placeholder="할 일을 입력하고 Enter"
    />{" "}
    <button type="submit">추가</button> </form> <div className="error">{error}
</div> <div> <FilterButton value="all" label="전체" filter={filter} onChange=
{setFilter} /> <FilterButton value="active" label="미완료" filter={filter} onChange=
{setFilter} /> <FilterButton value="done" label="완료" filter={filter} onChange=
{setFilter} />{" "}
    <button onClick={handleClearDone}>완료 항목 삭제</button> </div>

    {/* —— 문제 2 —— 목록 그리기
      visibleTodos 를 map 으로 돌면서 <TodoItem /> 을 그리세요.
      TodoItem 에 넘겨야 하는 것 (위쪽 TodoItem 정의를 보세요)
      todo={todo} onToggle={handleToggle} onDelete={handleDelete}
      그리고 key 를 잊지 마세요. key 는 todo.id 를 씁니다(05단원 개념03).

      기대 결과 (화면): 두 줄이 나옵니다.
      장보기 [삭제]
      설거지 [삭제] ← 회색 + 취소선 (done 이 true 라서)
      key 를 빠뜨리면 화면은 나오지만 콘솔에 노란 경고가 뜹니다.

      Each child in a list should have a unique "key" prop.
      아무것도 안 나오면 map 이 JSX 를 돌려주지 않는 것입니다(return 확인).

      TODO: 아래 ul 안을 고치세요 */}
    <ul>
      <li>여기에 목록이 나옵니다 (문제 2)</li>
    </ul>

    {/* 비었을 때 안내입니다. 05단원 개념05에서 배운 && 입니다.
      length 를 그대로 && 앞에 쓰면 0이 화면에 찍히므로 === 0 으로 비교합니다.
    */}
    {visibleTodos.length === 0 && <div className="output">항목이 없습니다</div>}

    <div className="output">
      할 일 {total}개 (완료 {doneCount}개, 남은 일 {leftCount}개)
    </div>
  </div>
);
}

```

정리

- 1 이 앱은 JS자료 종합03과 같은 앱입니다. 요구사항이 하나도 다르지 않습니다.
- 2 JS 판에는 createElement · appendChild · innerHTML · classList 가 가득했습니다. React 판에는 한 줄도 없습니다.
- 3 JS 판에서 손으로 만들던 render() 와 '바뀔 때마다 다시 부르기' 가 통째로 사라졌습니다. setTodos 가 그 일을 대신합니다.
- 4 우리가 적는 것은 '데이터가 이러면 화면은 이렇게 생겼다' 하나뿐입니다.
- 5 화면에 보여 줄 목록과 개수는 state 가 아닙니다. todos 와 filter 에서 계산해 냅니다(07단원 개념05).
- 6 추가는 [...todos, 새것], 삭제는 filter, 수정은 map + 스프레드입니다(07단원 개념01).
- 7 목록을 map 으로 그릴 때는 key 를 붙입니다(05단원 개념03).

```
export default function Project01TodoApp() {
  return (
    <div> <h1>종합 01 - 할 일 목록</h1> <p className="guide"> <strong>JS자료 13단원
종합03과 같은 앱</strong>입니다. 이번에는 React 로 만듭니다.
    <br /> <br />
문제는 <strong>8개</strong>입니다. 위에서부터 순서대로 푸세요. 문제 1·2를
풀면
    목록이 눈에 보이기 시작합니다.
    <br /> <br /> <strong>가장 어려운 것은 문제 5(완료 토글)</strong> 입니다.
배열도 새것, 그 안의
    객체도 새것 - 두 겹을 지켜야 합니다.
    <br /> <br />
    막히면 <strong>종합01_할일목록_정답.jsx</strong> 를 보세요. 먼저 스스로 해
보고
    나서 보는 것이 훨씬 남습니다.
    </p> <TodoApp /> </div>
  );
}
```

장바구니

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 여세요.

카페 주문 화면을 만듭니다. 담기 · 수량 조절 · 빼기 · 총액 · 비우기까지 합니다.

— 왜 Context 와 useReducer 를 쓰나

화면 조각이 넷입니다. 메뉴판 · 주문표 · 합계줄 · 머리띠의 장바구니 배지. 넷 다 같은 장바구니를 봅니다. 07단원의 state 끌어올리기로 만들면 공통 부모가 장바구니와 함수들을 들고 있다가 넷에 전부 내려보내야 합니다. 특히 배지는 화면 맨 위 깊은 곳에 있어서 중간 컴포넌트들이 쓰지도 않는 값을 손에서 손으로 넘겨야 합니다. 12단원 개념01에서 본 그 불편함입니다.

그리고 장바구니가 바뀌는 방법이 다섯 가지나 됩니다. 담기 · 수량 올리기 · 수량 내리기 · 빼기 · 비우기. 이렇게 '바뀌는 방법이 여러 가지' 일 때가 useReducer 를 쓸 자리입니다(12단원 개념03).

쓰는 단원 · 07(불변 갱신) · 12(Context · useReducer · 커스텀 훅)

★ 이 실습에서 가장 어려운 것은 문제 1 입니다. '담기' 하나가 상황에 따라 완전히 다른 두 가지 일을 하기 때문입니다. 처음 담는 메뉴면 새 칸을 만들고, 이미 담긴 메뉴면 수량만 올립니다.

— 푸는 법

- 1 문제 1~3(규칙) → 문제 4(저장소) → 문제 5~8(화면) 순서로 푸세요.
- 2 문제 1~3은 화면 없이 콘솔로 먼저 확인할 수 있습니다. 아래 '검산' 을 보세요.
- 3 저장하면 화면이 저절로 바뀝니다(Vite).

★ 아직 아무것도 안 고친 지금도 화면은 나옵니다. 장바구니에 아메리카노 2잔이 담겨 있고 버튼만 안 먹는 것이 정상입니다.

```
import { createContext, useContext, useReducer } from "react";
import Summary from "../_ui/Summary.jsx";
```

메뉴판입니다. 07단원까지 쓰던 것과 같은 모양입니다.

```
const MENU = [
  { id: 1, name: "아메리카노", price: 4000 },
  { id: 2, name: "라떼", price: 4500 },
  { id: 3, name: "케이크", price: 6000 },
  { id: 4, name: "삼각김밥", price: 1200 },
];
```

장바구니 한 칸의 모양입니다. 메뉴에 count 가 하나 붙었습니다.

```
{ id: 1, name: "아메리카노", price: 4000, count: 2 }
```

```
const FIRST_ITEMS = [{ id: 1, name: "아메리카노", price: 4000, count: 2 }];
```

액션의 모양

12단원 개념04에서는 값을 payload 라는 이름 하나에 담았습니다. 이 파일은 개념04 [실수 6]에서 권한 대로 값마다 이름을 붙입니다. 무엇이 들어오는지 이름만 봐도 알 수 있어서 헷갈릴 일이 줄어듭니다.

{ type: "add", menu: { id, name, price } } 메뉴 하나를 담는다 { type: "increase", id: 1 } 그 칸의 수량을 1 올린다 { type: "decrease", id: 1 } 그 칸의 수량을 1 내린다 (1 아래로는 안 내려감) { type: "remove", id: 1 } 그 칸을 목록에서 뺀다 { type: "clear" } 전부 비운다

```
function cartReducer(state, action) {
```

state 는 장바구니 배열입니다. 07단원 불변성 규칙을 그대로 지킵니다.

```
switch (action.type) {
```

문제 1 답기 ★ 이 파일에서 가장 어렵습니다

두 가지 경우를 갈라야 합니다. ① 이미 담긴 메뉴다 → 그 칸의 count 만 1 올린 새 배열을 돌려준다

```
(map + { ...item, count: item.count + 1 })
```

② 처음 담는 메뉴다 → count 1 을 붙여 배열 끝에 넣는다

```
([...state, { ...action.menu, count: 1 }])
```

어느 쪽인지는 some 으로 확인합니다. "하나라도 있나" 를 알려 줍니다(JS자료 08단원).

★ state 를 직접 고치면 안 됩니다. push 도 item.count = ... 도 안 됩니다. 에러가 안 나고 화면만 조용히 안 바뀝니다(07단원 개념01).

기대 콘솔 아래 검산 첫 줄이 → 아메리카노 1개, 케이크 1개
빈 줄이면 아직 ② 를 안 한 것입니다.
"아메리카노 1개, 아메리카노 1개, 케이크 1개" 처럼 같은 메뉴가
두 줄이 되면 ① 을 빠뜨리고 무조건 새 칸을 만든 것입니다.

여기에 코드를 쓰세요

```
case "add":
  return state;
```

문제 2 수량 올리기 / 내리기

increase 는 그 칸의 count 를 1 올립니다. decrease 는 1 내리되 ★ 1 아래로는 안 내려갑니다. (0잔이 담겨 있는 것은 이상하니까요. 빼려면 아래 remove 를 씁니다) 1 일 때 [-] 를 눌러도 그대로 1 이면 맞게 한 것입니다.

둘 다 map 과 { ...item, count: ... } 형태입니다.

기대 화면 주문표의 [+] 를 누르면 "아메리카노 × 3" 이 됩니다.

- 를 두 번 누르면 × 1 까지만 내려가고 멈춥니다.

× 0 이나 음수가 나오면 아래 한계를 안 둔 것입니다.
(문제 4·7까지 해야 버튼이 살아납니다)

여기에 코드를 쓰세요

```
case "increase":
  return state;
```

여기에 코드를 쓰세요

```
case "decrease":  
    return state;
```

문제 3 빼기 / 비우기

remove 는 그 칸만 목록에서 지웁니다 (filter — 07단원 개념01). clear 는 전부 비웁니다. 빈 배열을 돌려주면 됩니다.

기대 화면 [빼기] 를 누르면 그 줄만 사라집니다.

전부 비우기 · 를 누르면 “장바구니가 비었습니다” 가 나오고

합계가 0원, 배지가 🛒 0 이 됩니다.
누른 것만 남고 나머지가 사라지면 filter 조건을 반대로 쓴 것입니다.

여기에 코드를 쓰세요

```
case "remove":  
    return state;
```

여기에 코드를 쓰세요

```
case "clear":  
    return state;  
  
default:
```

모르는 type 이 오면 아무 일도 하지 않고 지금 값을 그대로 돌려줍니다.

```
return state;
}
}
```

화면 없이 계산하기

reducer 는 컴포넌트 밖의 그냥 함수입니다. 그래서 브라우저를 열기 전에 콘솔로 먼저 시험해 볼 수 있습니다. 12단원 개념04 섹션2와 같은 방법입니다.

reduce 는 배열을 하나씩 접어 나가며 값을 만듭니다(JS자료 08단원). 액션 목록을 빈 장바구니에 차례로 적용한 결과가 나옵니다.

```
const testActions = [
  { type: "add", menu: MENU[0] }, // 아메리카노
  { type: "add", menu: MENU[0] }, // 아메리카노 한 잔 더 → 수량만 2
  { type: "add", menu: MENU[2] }, // 케이크
  { type: "decrease", id: 1 }, // 아메리카노를 1잔으로
];

const tested = testActions.reduce(cartReducer, []);

console.log("[검산] " + tested.map((item) => item.name + " " + item.count +
"개").join(", "));
```

문제 1~3을 다 풀면 이렇게 나옵니다 → [검산] 아메리카노 1개, 케이크 1개 지금은 reducer 가 아무 일도 안 하므로 "[검산] " 뒤가 비어 있습니다.

저장소 (Context)

상자를 만듭니다. 기본값을 null 로 두는 이유는 아래 useCart 에 있습니다.

```
const CartContext = createContext(null);

function CartProvider({ children }) {
```

문제 4 저장소 만들기

지금은 '고정된 값' 과 '아무 일도 안 하는 dispatch' 를 내려보내고 있습니다. 그래서 화면은 나오지만 버튼이 하나도 안 먹습니다.

useReducer 로 진짜 저장소를 만들어 내려보내세요.

```
const [items, dispatch] = useReducer(cartReducer, FIRST_ITEMS);
```

...

```
value={{ items: items, dispatch: dispatch }}
```

★ value 의 중괄호가 두 겹인 것에 주의하세요. 바깥 중괄호는 “여기부터 자바스크립트” 라는 뜻이고 (02단원 개념03), 안쪽 중괄호는 객체입니다. style={{ }} 와 같은 이야기입니다.

기대 화면 문제 1~3을 이미 풀었다면 이 문제를 푸는 순간

담기 · 버튼이 살아납니다. 아메리카노를 담으면 × 3 이 됩니다.

버튼이 여전히 안 먹으면 dispatch 를 바꾸지 않은 것입니다.
화면이 통째로 빨간 상자가 되면 value 의 중괄호를 한 겹만 쓴 것입니다.

아래 두 줄을 고치세요

```
const notYet = { items: FIRST_ITEMS, dispatch: () => {} };

return <CartContext.Provider value={notYet}>{children}</CartContext.Provider>;
}
```

값을 꺼내는 커스텀 훅입니다. 이미 만들어 두었습니다(12단원 개념04 섹션4). Provider 밖에서 부르면 조용히 틀리는 대신 그 자리에서 멈춥니다.

```
function useCart() {
  const cart = useContext(CartContext);

  if (cart === null) {
    throw new Error("useCart 는 <CartProvider> 안에서만 쓸 수 있습니다");
  }

  return cart;
}
```


문제 5 총액 구하기

items 를 받아 총액을 돌려주는 함수입니다. 한 칸의 값은 price × count 이고, 그것을 전부 더하면 총액입니다. reduce 를 씁니다(JS자료 08단원 개념05).

기대 화면 처음에 아메리카노 2잔이 담겨 있으므로 → 합계 8000원
0원 그대로면 아직 안 고친 것입니다.
4000원이 나오면 count 를 안 곱한 것입니다.
NaN 원이 나오면 reduce 의 시작값 0 을 빠뜨린 것입니다.

여기에 코드를 쓰세요

```
function getTotal(items) {
  return 0;
}
```

화면 조각

아래 네 조각은 props 를 하나도 받지 않습니다. 필요한 것은 각자 useCart() 로 꺼냅니다.

```
function MenuList() {
  const { dispatch } = useCart(); // items 는 안 쓰므로 dispatch 만 꺼냅니다
```

문제 6 담기 버튼

담기 · 를 누르면 그 메뉴를 장바구니에 넣으세요.

```
dispatch({ type: "add", menu: menu })
```

★ onClick 에 괄호를 붙이면 안 됩니다(04단원 개념01). 값을 같이 넘겨야 하므로 화살표 함수로 감쌉니다.

기대 화면 [케이크] 의 [담기] 를 누르면 주문표에 "케이크 × 1 = 6000원"
이 생기고 합계가 14000원, 배지가 🛒 3 이 됩니다.
아메리카노의 [담기] 를 누르면 새 줄이 아니라 × 3 이 되어야 합니다.
아무 반응이 없으면 dispatch 를 안 부른 것입니다.
화면을 열자마자 저절로 담기면 onClick 에 괄호를 붙인 것입니다.

아래 button 의 onClick 을 채우세요

```
return (
  <div className="output"> <strong>메뉴판</strong> <ul>
    {MENU.map((menu) => (
      <li key={menu.id}>
        {menu.name} {menu.price}원{ " "}
        <button>담기</button> </li>
      )))}
    </ul> </div>
);
}
```

```
function CartList() {
  const { items, dispatch } = useCart();
```

05단원 개념01의 '일찍 return' 입니다. 비었을 때는 여기서 끝냅니다.

```
if (items.length === 0) {
  return <div className="output">장바구니가 비었습니다</div>;
}
```

문제 7 수량 버튼과 빼기 버튼

버튼 세 개에 각각 알맞은 dispatch 를 붙이세요.

+ · { type: "increase", id: item.id }

- · { type: "decrease", id: item.id }

빼기 · { type: "remove", id: item.id }

기대 화면 아메리카노 줄의 [+] → × 3 = 12000원, 합계 12000원

- · 를 눌러 × 1 까지 내려간 뒤 한 번 더 눌러도 × 1 그대로

빼기 · → 그 줄이 사라지고 “장바구니가 비었습니다” 가 나옵니다

버튼을 눌렀는데 다른 줄이 바뀌면 item.id 를 안 넘긴 것입니다.

아래 button 세 개의 onClick 을 채우세요

```
return (
  <div className="output"> <strong>주문표</strong> <ul>
    {items.map((item) => (
      <li key={item.id}>
        {item.name} × {item.count} = {item.price * item.count}원{" "}
        <button>+</button> <button>-</button> <button>빼기</button> </li>
      )))}
    </ul> </div>
);
}

function CartTotal() {
  const { items, dispatch } = useCart();

  return (
    <div className="output"> <strong>합계 {getTotal(items)}원</strong>{" "}
    {/* 담긴 것이 있을 때만 비우기 버튼을 보여 줍니다 (05단원 개념01) */}
    {items.length > 0 && (
      <button onClick={() => dispatch({ type: "clear" })}>전부 비우기</button>
    )}
    </div>
  );
}
```

화면 맨 위 머리띠에 붙는 배지입니다. 일부러 깊은 곳에 두었습니다.

```
function CartBadge() {
  const { items } = useCart();
```

문제 8 담긴 잔 수 세기

배지에는 ‘종류’ 가 아니라 ‘전부 몇 개인가’ 를 보여 줍니다. 아메리카노 2잔 + 케이크 1개 = 3 입니다. items.length 는 2 라서 답이 아닙니다. count 를 전부 더하세요. reduce 를 다시 씁니다.

기대 화면 처음에 아메리카노 2잔이 담겨 있으므로 → 🛒 2
 케이크를 담으면 🛒 3 이 됩니다.
 케이크를 담았는데 🛒 2 로 나오면 items.length 를 쓴 것입니다.

아래 줄을 고치세요

```
const count = 0;

return <span>🛒 {count}</span>;
}
```

배지는 이렇게 세 겹 안쪽에 있습니다. 그런데 props 가 하나도 없습니다.

```
function HeaderRight() {
  return <CartBadge />;
}

function ShopHeader() {
  return (
    <div className="output"> <strong>민준이네 카페</strong> - <HeaderRight /> </div>
  );
}

function ShopScreen() {
```

이 컴포넌트도, ShopHeader 도, HeaderRight 도 장바구니를 모릅니다.

```
return (
  <div> <ShopHeader /> <MenuList /> <CartList /> <CartTotal /> </div>
);
}
```

정리

- 1 장바구니 한 칸은 { id, name, price, count } 입니다. 메뉴에 count 가 붙은 모양입니다.
- 2 바뀌는 방법이 다섯 가지(담기·올리기·내리기·빼기·비우기)라서 useReducer 를 씁니다.
- 3 reducer 는 컴포넌트 밖의 함수라 화면 없이 reduce 로 계산할 수 있습니다.
- 4 reducer 안에서도 07단원의 불변 규칙을 지킵니다. push 도, item.count = ... 도 안 됩니다.
- 5 Provider 를 CartProvider 컴포넌트로 감싸면 쓰는 쪽은 useReducer 를 몰라도 됩니다.
- 6 화면 조각 넷이 props 를 하나도 안 받습니다. 각자 useCart() 로 꺼냅니다.
- 7 총액과 담긴 개수는 state 가 아닙니다. items 에서 reduce 로 계산합니다.

```
export default function Project02CartApp() {
  return (
    <div> <h1>종합 02 - 장바구니</h1> <p className="guide"> <strong>Context +
    useReducer</strong> 로 만드는 카페 주문 화면입니다. 문제는{" "}
    <strong>8개</strong>입니다.
    <br /> <br /> <strong>가장 어려운 것은 문제 1(담기)</strong> 입니다. 처음
    담는 메뉴와 이미 담긴
    메뉴를 갈라야 합니다.
    <br /> <br />
    문제 1~3은 화면 없이 <strong>F12 → Console</strong> 의 계산 줄로 먼저 확인할
    수
    있습니다. 화면을 고치기 전에 규칙부터 맞추세요.
    <br /> <br />
    막히면 <strong>종합02_장바구니_정답.jsx</strong> 를 보세요.
    </p> <div className="demo"> <CartProvider> <ShopScreen
  /> </CartProvider> </div> </div>
  );
}
```

사용자 검색

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 여세요.

★ 이 앱은 JS자료 13단원 종합04와 같은 앱입니다. 연습용 서버에서 사람 10명을 받아 와서 검색 · 정렬 · 상세 보기를 합니다. 그때는 `fetch` 와 `DOM` 으로 만들었고, 이번에는 `useEffect` 와 `state` 로 만듭니다.

★★ 인터넷 연결이 필요합니다. 인터넷이 막힌 곳이라면 실습프로젝트 폴더의 `index.html` 을 열고 `<script src="/오프라인_대체.js">` 줄을 감싼 주석 기호만 지우세요. 인터넷 요청 0건으로 똑같은 값이 나옵니다. 흉내 내는 것은 `jsonplaceholder.typicode.com` 하나뿐입니다.

★ 콘솔에 빨간 줄이 나오는 것은 정상입니다. 이 실습에는 일부러 없는 주소를 부르는 [없는 주소] 버튼이 있습니다. 그것을 누르면 브라우저가 이렇게 적습니다.

Failed to load resource: the server responded with a status of 404 (Not Found)

우리 코드가 낸 에러가 아니라 브라우저가 알려 주는 줄입니다. 막을 방법이 없고, 막을 필요도 없습니다. 실무에서도 그냥 나옵니다. 우리가 낸 것은 한글로 나오고, 브라우저가 낸 것은 영어로 나옵니다.

서버에서 오는 데이터의 모양 · — 필요한 것만 적었습니다

```
{ id: 1, name: "Leanne Graham", email: "...", phone: "...",
```

```
  address: { city: "Gwenborough" }, company: { name: "..." } }
```

쓰는 단원 · 09(`useEffect` · `fetch` · 로딩 · 에러) · 05·06·07(리스트 · 입력 · 파생 값)

★ 이 실습에서 가장 어려운 것은 문제 2 입니다. 고칠 코드는 몇 줄 안 됩니다. 어려운 것은 '왜 그래야 하는가' 입니다. "한 번 받아온 화면을 어떻게 다시 받아오게 만드나" 라는 물음입니다.

— 푸는 법

1 아래로 내려가며 // TODO 를 찾아 코드를 고칩니다.

2 저장하면 화면이 저절로 바뀝니다(Vite).

3 화면 위쪽 [정상 주소] / [없는 주소] 버튼으로 성공과 실패를 둘 다 확인하세요.

★ 아직 아무것도 안 고친 지금도 화면은 나옵니다. 목록만 비어 있습니다.

```
import { useEffect, useState } from "react";
import Summary from "../_ui/Summary.jsx";

const BASE_URL = "https://jsonplaceholder.typicode.com";

function UserSearch() {
  const [users, setUsers] = useState([]); // 받아온 사람들
  const [loading, setLoading] = useState(true); // 기다리는 중인가
  const [error, setError] = useState(null); // 실패했다면 보여 줄 문구
  const [keyword, setKeyword] = useState(""); // 검색어 (06단원 제어 컴포넌트)
  const [sortBy, setSortByName] = useState(false); // 이름순인가
  const [selectedId, setSelectedId] = useState(null); // 상세로 볼 사람
```

일부러 실패시켜 보는 스위치입니다. 이미 만들어 두었습니다.

```
const [source, setSource] = useState("정상");
```

문제 2에서 씁니다. 지금은 아무 데서도 안 쓰이는 값입니다.

```
const [reloadCount, setReloadCount] = useState(0);
```

source 에 따라 부를 주소가 정해집니다. /nouses 는 서버에 없는 주소라 404 로 대답합니다.

```
const url = source === "정상" ? `${BASE_URL}/users` : `${BASE_URL}/nouses`;

useEffect(() => {
```

09단원 개념05의 것발입니다. 이미 넣어 두었습니다. 주소를 빨리 두 번 바꾸면 지난번 응답이 뒤늦게 도착할 수 있습니다. 그때 그 값을 버리기 위한 표시입니다.

```
let ignore = false;

async function loadUsers() {
  setLoading(true);
  setError(null);
```

이 줄은 미리 넣어 두었습니다. effect 가 몇 번 돌았는지 세어 볼 수 있습니다.

```
console.log("[요청] 보냅니다:", url);
```

```
F12 콘솔    [요청] 보냅니다: https://jsonplaceholder.typicode.com/users
```

개발 중에는 처음에 두 번 찍힙니다. StrictMode 때문이고 정상입니다(09단원 개념02).

문제 1 목록 받아오기

1 try 안에서 url 로 fetch 합니다

2 res.ok 가 false 면 throw new Error(서버 응답 오류 (\${res.status}))

★ 이 줄이 없으면 404 가 조용히 지나갑니다. fetch 는 404 를 실패로 치지 않습니다. 대답을 받았으니 성공으로 봅니다(09단원 개념04).

3 res.json() 으로 데이터를 꺼냅니다

4 몇 명을 받았는지 콘솔에 찍습니다

```
console.log("[도착] 받은 사람 수:", data.length);
```

5 if (ignore) return; 으로 오래된 응답이면 버리고, 아니면 setUsers(데이터)

6 catch 에서 setError("사용자 목록을 불러오지 못했습니다. 잠시 후 다시 시도해 주세요.")

콘솔에는 err.message 를 남깁니다 (사용자용 말과 개발자용 정보를 나눕니다)

7) finally 에서 setLoading(false)

★ try 안에 두면 실패했을 때 영원히 "불러오는 중..." 이 됩니다.

기대 화면 잠깐 뒤 목록에 사람 10명이 나옵니다.
첫 줄은 Leanne Graham / Gwenborough · Sincere@april.biz 입니다.
(문제 5·6까지 해야 목록이 눈에 보입니다)
콘솔에 "[도착] 받은 사람 수: 10" 이 찍히면 성공입니다.
개발 중에는 이 줄이 처음에 두 번 찍힙니다. StrictMode 때문이고
정상입니다(09단원 개념02).

없는 주소 · 를 눌렀을 때 아무 일도 없으면 res.ok 검사를 빠뜨린 것입니다.

여기에 코드를 쓰세요


```

setLoading(false); // ← 문제 1을 풀면 이 줄은 finally 안으로 들어갑니다
}

loadUsers();

return () => {
  ignore = true;
};

```

문제 2 다시 불러오기 ★ 이 파일에서 가장 어렵습니다

화면에 [다시 불러오기] 버튼이 있습니다. 지금 눌러도 아무 일도 안 일어납니다.

왜 그럴까요? useEffect 는 '의존성 배열 안의 값이 달라졌을 때' 만 다시 돕니다. 지금 의존성 배열에는 url 하나뿐입니다. 버튼을 눌러도 url 은 그대로이니 React 는 "달라진 게 없네" 하고 아무 일도 하지 않습니다.

그래서 '누를 때마다 달라지는 값' 을 하나 만들어 의존성에 넣습니다. 위에 만들어 둔 reloadCount 가 그 값입니다. ① [다시 불러오기] 버튼이 reloadCount 를 1 늘리게 하세요 (화면 아래쪽) ② 아래 의존성 배열에 reloadCount 를 넣으세요

★ 여기서 loadUsers() 를 버튼에서 직접 부르면 안 되나요? 이 함수는 useEffect 안에 있어서 밖에서는 부를 수 없습니다. 밖으로 꺼낼 수도 있지만, 그러면 '언제 받아오는가' 가 두 곳으로 흩어집니다. 받아오는 조건을 전부 의존성 배열 한 줄에 모아 두는 편이 읽기 쉽습니다.

기대 화면 [다시 불러오기] 를 누르면 "불러오는 중..." 이 잠깐 보였다가 다시 10명이 나옵니다.
콘솔에 "[요청] 보냅니다: ..." 가 누를 때마다 한 줄씩 늘어납니다.
이 줄이 안 늘어나면 effect 가 다시 안 돈 것입니다.

없는 주소 · 상태에서 누르면 빨간 안내가 다시 나옵니다.

아무 일도 안 일어나면 ② 를 빠뜨린 것입니다.
화면이 멈추고 콘솔이 끝없이 늘어나면(무한 루프) 의존성에 reloadCount 대신 users 같은 것을 넣은 것입니다(09단원 개념06).

아래 대괄호 안을 고치세요

```
}, [url]);
```

문제 3 검색으로 걸러내기

keyword 에 맞는 사람만 남기세요. - 검색어가 비어 있으면 전부 - 이름(name) 또는 도시(user.address.city)에 검색어가 들어 있으면 남김 - 대소문자를 구분하지 않습니다 (양쪽 다 toLowerCase)

★ 이것을 state 로 두지 마세요. users 와 keyword 로 언제든지 계산됩니다(07단원 개념05).

기대 화면 검색창에 gwen → 1명 (Leanne Graham)
이름이 아니라 도시 Gwenborough 로 걸린 것입니다. 대소문자도 무시됩니다.
검색창에 zzz → 0명
검색창을 비우면 다시 10명
대문자로 Gwen 을 쳤을 때만 나오면 toLowerCase 를 한쪽만 한 것입니다.

아래 줄을 고치세요

```
const searched = users;
```

문제 4 이름순 정렬

sortByName 이 true 면 이름순으로 정렬한 배열을, 아니면 받은 순서 그대로 쓰세요. - 글자 정렬은 a - b 가 아니라 localeCompare 입니다(JS자료 08단원) - ★ sort 는 원본을 바꿉니다. [...searched] 로 복사한 뒤에 정렬하세요(07단원 개념01) - 그리고 아래 버튼 글자도 바꾸세요. 글자는 '지금 누르면 무엇이 되는지' 를 뜻합니다.

정렬 안 된 상태 → "이름순 정렬"
정렬된 상태 → "원래 순서"

기대 화면 [이름순 정렬] 을 누르면 버튼 글자가 "원래 순서" 로 바뀌고
목록 맨 위가 Chelsey Dietrich 가 됩니다.
한 번 더 누르면 맨 위가 Leanne Graham 으로 돌아옵니다.
원래 순서로 못 돌아오면 복사하지 않고 원본을 정렬한 것입니다.

아래 줄을 고치세요

```
const visibleUsers = searched;

function handleSort() {
  setSortByName(!sortByName);
}
```

상세로 볼 사람을 찾습니다. 이것도 계산해서 씁니다. find 는 못 찾으면 `undefined` 를 돌려줍니다(JS자료 08단원).

```
const selectedUser = users.find((user) => user.id === selectedId);

return (
  <div className="demo"> <h3>사용자 검색</h3> <div> <input type="text" value=
{keyword} onChange={(e) => setKeyword(e.target.value)}
  placeholder="이름이나 도시로 검색"
  />{ " "}
  <button onClick={handleSort}>이름순 정렬</button>
  { /* 문제 2의 ① 을 이 버튼에 씁니다 */ }
  <button>다시 불러오기</button> </div> <div>
  주소 고르기{ " "}
  <button className={source === "정상" ? "on" : ""} onClick={() =>
setSource("정상")}>
  정상 주소
  </button> <button className={source === "정상" ? "" : "on"}
  onClick={() => setSource("없는주소")}
  >
  없는 주소 (404)
  </button> </div>
```

{ /* —— 문제 5 —— 화면 세 갈래

지금은 세 경우를 안 가르고 목록만 그립니다. 그래서 불러오는 동안에는
빈 화면이 잠깐 보이고, 실패해도 아무 말이 없습니다.

아래를 이렇게 갈라 주세요. 순서가 중요합니다.

① loading 이면 → "불러오는 중..." 만 보여 준다

② 아니고 error 가 있으면 → 빨간 안내만 보여 준다 (className="output error")

③ 아니면 → 목록을 그린다

기다리는 중에는 다른 것을 볼 필요가 없으니 loading 을 가장 먼저 봅니다. 순서를 바꾸면 로딩 중에 지난번 목록이 잠깐 다시 나타나는 이상한 화면이 됩니다.

기대 결과 (화면): 화면을 열면 "불러오는 중..." 이 잠깐 보였다가 목록이 나옵니다.

[없는 주소] 를 누르면 빨간 글씨로

"사용자 목록을 불러오지 못했습니다. 잠시 후 다시 시도해 주세요."

로딩 글자가 계속 남아 있으면 finally 에서 setLoading(false) 를 안 한 것입니다.

에러와 목록이 같이 보이면 ③ 에 error 검사를 안 넣은 것입니다.

TODO: 아래를 세 갈래로 고치세요 */}

```
{/* —— 문제 6 —— 목록 그리기
```

visibleUsers 를 map 으로 돌면서 li 를 그리세요. key 는 user.id 입니다.

li 하나의 모양:

```
<li key={...} onClick={...} style={{ cursor: "pointer" }}>
  <strong>{user.name}</strong>
  <div>{user.address.city} · {user.email}</div>
</li>
```

누르면 setSelectedId(user.id) 가 되게 하세요.

기대 결과 (화면): 사람 10명이 나옵니다. 첫 줄은

Leanne Graham

Gwenborough · Sincere@april.biz

key 를 빠뜨리면 화면은 나오지만 콘솔에 노란 경고가 뜹니다.

누르는데 아래 상세가 안 바뀌면 onClick 에 setSelectedId 를 안 넣은 것입니다.

TODO: 아래 ul 안을 고치세요 */}

```
<ul>
```

```
<li>여기에 목록이 나옵니다 (문제 6)</li>
```

```
</ul>
```

```
{/* —— 문제 7 —— 검색 결과가 없을 때
```

걸러낸 결과가 0명이면 "검색 결과가 없습니다" 를 보여 주세요.

* 로딩 중이거나 에러일 때는 이 문구가 나오면 안 됩니다.

"못 불러왔다"와 "검색 결과가 없다"는 전혀 다른 이야기입니다.

둘을 섞으면 사용자가 원인을 알 수 없습니다. 문제 5의 ③ 안쪽에 두면 됩니다.

기대 결과 (화면): 검색창에 zzz → 검색 결과가 없습니다

[없는 주소]를 눌렀을 때는 이 문구 대신 빨간 안내만 보여야 합니다.

둘 다 같이 보이면 문제 5의 ③ 바깥에 둔 것입니다.

TODO: 여기에 코드를 쓰세요 */}

{/* —— 문제 8 —— 상세 보기

selectedUser가 있으면 아래 다섯 줄을 보여 주세요. 없으면 안내 한 줄입니다.

이름 / 이메일 / 전화 / 도시(user.address.city) / 회사(user.company.name)

기대 결과 (화면): 목록에서 Chelsey Dietrich를 누르면

이름: Chelsey Dietrich

이메일: Lucio_Hettinger@annie.ca

전화: (254)954-1289

도시: Roscoeview

회사: Keebler LLC

"Cannot read properties of undefined"로 화면이 터지면

selectedUser가 있는지 확인하지 않고 바로 꺼내 쓴 것입니다(11단원 개념03 [실수 4]).

TODO: 아래 output 안을 고치세요 */}

<div className="output">목록에서 사람을 눌러 보세요 (문제 8)</div>

</div>

);

}

정리

- 1 데이터를 받아오는 화면은 로딩·에러·성공 세 가지 상태 중 하나입니다. 화면도 그 순서로 가릅니다.
- 2 fetch는 404를 실패로 치지 않습니다. if (!res.ok) throw 한 줄이 그것을 실패로 만들어 줍니다.
- 3 로딩을 끄는 일은 finally에 둡니다. try 안에 두면 실패했을 때 영원히 불러오는 중이 됩니다.
- 4 useEffect는 의존성 배열의 값이 달라졌을 때만 다시 돕니다. 다시 받아오려면 그 안의 값을 바꿉니다.
- 5 검색 결과와 정렬 결과는 state가 아닙니다. users와 keyword에서 계산합니다.
- 6 sort는 원본을 바꿉니다. [...배열]로 복사한 뒤에 정렬하세요.
- 7 '못 불러왔다'와 '검색 결과가 없다'는 다른 이야기입니다. 화면에서 섞이지 않게 하세요.

```

export default function Project03UserSearch() {
  return (
    <div> <h1>종합 03 - 사용자 검색</h1> <p className="guide"> <strong>인터넷 연결이
    필요합니다.</strong> JS자료 13단원 종합04와 같은 앱을
      React 로 다시 만듭니다. 문제는 <strong>8개</strong>입니다.
      <br /> <br /> <strong>가장 어려운 것은 문제 2(다시 불러오기)</strong> 입니다.
    고칠 코드는 몇 줄
      안 되지만 "왜 그래야 하는가" 가 어렵습니다.
      <br /> <br />
      인터넷이 막혀 있다면 실습프로젝트 폴더의 <code>index.html</code> 에서{" "}
      <code>오프라인_대체.js</code> 줄의 주석 기호만 지우세요. 인터넷 없이 똑같이
      동작합니다.
      <br /> <br /> <strong>[없는 주소 (404)]</strong> 를 누르면 콘솔에 빨간{" "}
      <code>Failed to load resource ... 404</code> 가 나옵니다. <strong>정상입니다.
    </strong>{" "}
      브라우저가 알려 주는 줄이지 우리 코드의 에러가 아닙니다.
      <br /> <br />
      막히면 <strong>종합03_사용자검색_정답.jsx</strong> 를 보세요.
    </p> <UserSearch /> </div>
  );
}

```

메모장

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 여세요.

이 자료의 마지막 실습입니다. 09 · 11 · 12단원을 한자리에 모읍니다.

서버에서 메모를 받아오고(09) → 보관소에 담고(12) → 목록 · 상세 · 새 메모 세 화면을 오갑니다(11)

— 쓰는 단원

09 useEffect · fetch · 로딩/에러 11 Router · Link · useParams · useNavigate 12 Context ·
useReducer 06 제어 컴포넌트 / 07 불변 갱신

★ 이 실습에서 가장 어려운 것은 문제 6 입니다. 고칠 글자는 여덟 자뿐입니다. 어려운 것은 **에러도 경고도 안 난다**는 점입니다. 화면은 멀쩡히 나오는데 메모만 안 찾아집니다. 원인을 스스로 찾아보세요.

★ 이 예제 안에서는 새로고침(F5)을 하지 마세요. 왼쪽 메뉴 선택이 처음으로 돌아가서 이 예제가 화면에서 사라집니다. 그때는 왼쪽 메뉴에서 이 예제를 다시 고르면 됩니다.

★★ 인터넷 연결이 필요합니다. 막힌 곳이라면 실습프로젝트 폴더의 index.html 을 열고 `<script src="/오프라인_대체.js">` 줄을 감싼 주석 기호만 지우세요. 인터넷 요청 0건으로 똑같은 값이 나옵니다.

— 푸는 법

1 아래로 내려가며 // TODO 를 찾아 고칩니다.

2 저장하면 화면이 저절로 바뀝니다(Vite). F5 를 누르지 마세요.

3 화면 위의 [메모 목록] / [새 메모] 를 눌러 확인하세요.

★ 아직 아무것도 안 고친 지금도 화면은 나옵니다.

메모 목록 · 을 누르면 “메모가 하나도 없습니다” 가 보입니다. 정상입니다.

```
import { createContext, useContext, useEffect, useReducer, useState } from "react";
import {
  BrowserRouter,
  Routes,
  Route,
  Link,
  useParams,
  useNavigate,
} from "react-router-dom";
import Summary from "../_ui/Summary.jsx";

const BASE_URL = "https://jsonplaceholder.typicode.com";
```

1. 메모 보관소 — Context + useReducer (12단원)

```
const MemoContext = createContext(null);
```

reducer 는 “지금 상태 + 무슨 일 = 새 상태” 를 적는 함수입니다. 규칙이 하나 있습니다. state 를 고치지 말고 **새로 만들어** 돌려주세요. (07단원)

```
function memoReducer(state, action) {
  switch (action.type) {
    case "채우기":
```

서버에서 받아온 것으로 통째로 갈아 끼웁니다. (이건 이미 되어 있습니다)

```
return action.memos;
```

문제 1 메모 추가하기

action 에는 { type: “추가”, title, body } 가 들어옵니다. 새 메모를 만들어 목록 맨 앞에 붙인 새 배열을 돌려주세요. - 번호(id)는 지금 있는 것 중 가장 큰 번호 + 1 로 정합니다. - 목록이 비어 있으면 1 번으로 합니다.

(Math.max() 를 빈 배열에 쓰면 -Infinity 가 나옵니다)

기대 화면 [새 메모] 에서 “케이크 사기” 를 저장하면
목록 맨 위에 “케이크 사기” 가 생기고 개수가 1 늘어납니다.
저장했는데 목록이 그대로면 이 문제를 아직 안 푼 것입니다.
맨 아래에 생겼다면 [...state, 새것] 순서로 쓴 것입니다.

```
case "추가":
```


여기를 고치세요

```
return state;
```

문제 2 메모 삭제하기

action 에는 { type: "삭제", id } 가 들어옵니다. 그 번호만 뺀 새 배열을 돌려주세요. (07단원 filter)

기대 화면 메모를 열어 [삭제] 를 누르면 목록에서 사라지고
개수가 1 줄어듭니다. 그대로면 아직 안 푼 것입니다.

```
case "삭제":
```

여기를 고치세요

```
return state;
```

문제 3 메모 수정하기

action 에는 { type: "수정", id, title, body } 가 들어옵니다. 번호가 같은 것 하나만 새 내용으로 바꾼 새 배열을 돌려주세요. - map 으로 전부 훑으면서, 번호가 같을 때만 바꿉니다. (07단원) - { ...m, title: ..., body: ... } 를 쓰면 나머지 칸은 그대로 둡니다.

기대 화면 목록에서 메모를 열어 제목을 고치고 [저장] 을 누르면
목록의 제목이 바뀌어 있습니다.
제목이 그대로면 이 문제를 아직 안 푼 것입니다.

```
case "수정":
```

여기를 고치세요

```
return state;

    default:
        return state;
    }
}

function MemoProvider({ children }) {
    const [memos, dispatch] = useReducer(memoReducer, []);
    const [loading, setLoading] = useState(true);
    const [error, setError] = useState(null);

    useEffect(() => {
        let ignore = false; // 09단원 개념05의 깃발입니다. 이미 넣어 두었습니다.
        async function loadMemos() {
```

문제 4 서버에서 메모 5개 받아오기

아래 순서로 채우세요. (09단원 개념03·04) ① ``${BASE_URL}/posts?_limit=5`` 를 `await fetch` 로 부릅니다. ② `if (!res.ok)` 이면 `throw new Error(...)` 로 실패로 만듭니다.

★ `fetch` 는 404 를 실패로 치지 않습니다. 이 줄이 있어야 실패가 됩니다.

③ `await res.json()` 으로 값을 꺼냅니다. ④ `if (ignore)` `return;` 으로 늦게 온 응답을 버립니다. ⑤ `dispatch({ type: "채우기", memos: ... })` 로 보관소에 넣습니다.

서버가 주는 모양은 `{ userId, id, title, body }` 입니다.
우리에게 필요한 것은 `id · title · body` 세 칸뿐이니 `map` 으로 골라 담으세요.

⑥ 실패하면 `catch` 에서 `setError(e.message)` 를 합니다. ⑦ 아래 `setLoading(false)` 를 `finally` 안으로 옮깁니다.

★ `try` 안에 두면 실패했을 때 영원히 "불러오는 중" 이 됩니다.

기대 화면 [메모 목록] 에 "메모 5개" 와 제목 5줄이 나옵니다.
 첫 줄은 sunt aut facere ... 로 시작합니다.
 기대 결과 (콘솔): 메모를 받아왔습니다: 5
 (개발 중에는 두 번 찍힐 수 있습니다. 09단원 개념02의 StrictMode 입니다)

여기에 코드를 쓰세요

```
setLoading(false); // ← 문제 4를 풀면 이 줄은 finally 안으로 들어갑니다
}

loadMemos();
return () => {
  ignore = true;
};
}, []);

return (
  <MemoContext.Provider value={{ memos, dispatch, loading, error }}>
    {children}
  </MemoContext.Provider>
);
}
```

Context 를 꺼내는 일을 커스텀 훅으로 감쌌습니다. (10단원)

```
function useMemos() {
  return useContext(MemoContext);
}
```

2. 화면들

목록 화면

```
function MemoListPage() {
  const { memos, loading, error } = useMemos();

  if (loading) return <p>불러오는 중...</p>;
  if (error) return <p className="error">못 불러왔습니다: {error}</p>;
  if (memos.length === 0) return <p>메모가 하나도 없습니다.</p>;

  return (
```


서버와 연동

OPEN 14_서버와_연동

```
function Section1Demo() {  
  useEffect(() => {  
    let 버렸나 = false;  
    (async () => {
```

01 내 서버에 붙기

02 폼을 useForm 으로

03 서버에 보내고 받기

04 파일 올리기

05 안 되는 경우들

내 서버에 붙기

RUN 터미널 두 개가 필요합니다.

① 실습프로젝트에서 `npm run dev`

② 실습프로젝트/14단원_서버에서 `node 서버.js`

②를 안 켜면 이 단원의 예제가 전부 “서버가 안 켜져 있습니다”를 보여 줍니다.

09단원에서 `fetch`로 데이터를 받아 봤습니다. 그런데 그건 **남의 서버**였습니다.

```
https://jsonplaceholder.typicode.com/posts
```

연습용으로 누가 열어 둔 곳이고, 우리가 고칠 수 없었습니다. 이번에는 **내 서버**에 붙습니다.

```
http://localhost:4000/api/v1/equipments
```

★ 부르는 방법은 09단원과 **똑같습니다**. `fetch` 그대로입니다. 달라지는 것은 세 가지입니다.

1 서버를 내가 켜야 합니다 (안 켜면 아무것도 안 됩니다)

2 ★ CORS 라는 것이 나옵니다 (섹션 5)

3 내가 데이터를 **보낼** 수도 있습니다 (개념03·04)

★★ 서버가 어떻게 만들어졌는지는 지금 몰라도 됩니다. `14단원_서버/서버.js`는 열어 보지 않아도 됩니다. PART 3에서 직접 만듭니다. 여기서는 **붙는** 쪽만 봅니다.

★ 콘솔에 같은 줄이 두 번씩 찍힙니다. 정상입니다(09단원 개념02의 StrictMode).

```
import { useState, useEffect } from "react";
import Summary from "../_ui/Summary.jsx";
import { 서버주소, 서버안내 } from "../_서버주소.js";
```

주소만 바꿉니다

섹션 1

09단원에서 이렇게 썼습니다.

```
const 응답 = await fetch("https://jsonplaceholder.typicode.com/posts?_limit=5");
const 자료 = await 응답.json();
```

이번에는 이렇게 씁니다.

```
const 응답 = await fetch("http://localhost:4000/api/v1/equipments");
const 자료 = await 응답.json();
```

★ 코드가 같습니다. 주소만 바뀌었습니다. React 입장에서는 “어디에 있는 서버냐”가 중요하지 않습니다.

★★ `localhost` 는 내 컴퓨터라는 뜻입니다. `4000` 은 그 안에서 이 서버가 쓰는 문 번호입니다. Vite 는 `5173` 을 씁니다. 그래서 둘은 서로 다른 곳입니다. 섹션 5에서 씁니다.

▹ 직접 해보기 1

브라우저 주소창에 `http://localhost:4000/api/v1/equipments` 를 직접 넣어 보세요. 무엇이 보입니까?
서버를 켜다 켜며 해 보세요.

```
function Section1Demo() {
  const [설비들, set설비들] = useState([]);
  const [상태, set상태] = useState("아직");

  useEffect(() => {
    let 버렸나 = false;

    (async () => {
      set상태("받는중");
      try {
        const 응답 = await fetch(`${서버주소}/equipments`);
        if (!응답.ok) throw new Error(`서버가 ${응답.status} 를 보냈습니다`);

        const 자료 = await 응답.json();
        if (버렸나) return;

        set설비들(자료.설비들);
        set상태("됨");
        console.log("받은 설비 수:", 자료.설비들.length);
      } catch (오류) {
        if (버렸나) return;
        set상태("안됨");
        console.log("못 받았습니다:", 오류.name);
      }
    })();

    return () => {
      버렸나 = true;
    };
  }, []);

  return (
    <div className="demo"> <h3>① 목록 받아 그리기</h3>

    {상태 === "받는중" && <div className="output">불러오는 중...</div>}
  )
}
```



```

{상태 === "안됨" && <div className="output">{서버안내}</div>}

{상태 === "됨" && (
  <ul>
    {설비들.map((하나) => (
      <li key={하나.번호}>
        {하나.이름} - {하나.상태} ({하나.담당})
      </li>
    ))}
  </ul>
)}
</div>
);
}

```

★ 버렸나 는 09단원 개념05에서 본 그것입니다. 화면이 사라진 뒤에 답이 오면 set 을 안 하려고 두는 표시입니다.

★★ if (!응답.ok) throw 를 빼먹으면 안 됩니다. fetch 는 404·500 을 **실패로 안 칩니다**. (JS자료 12단원 개념05) 그냥 “응답이 왔다” 로 봅니다. 그래서 직접 봐야 합니다.

◦ 직접 해보기 2

서버 터미널에서 Ctrl+C 로 서버를 끄고 이 예제를 새로고침하세요. 화면에 무엇이 나오니까? 콘솔에는 무엇이 찍힐니까?

불러오는 중과 안 됐을 때

섹션 3

위 예제의 상태 가 세 가지였습니다. 이게 중요합니다.

"받는중"	→ 불러오는 중... 을 보여 줍니다
"됨"	→ 목록을 그립니다
"안됨"	→ 안내를 보여 줍니다

★ 셋을 안 나누면 **빈 화면**이 뜹니다. 데이터가 없는 것과 아직 안 온 것과 못 받은 것은 전혀 다른 상황인데 화면에는 똑같이 “아무것도 없음” 으로 보이기 때문입니다.

서버에 일부러 느리게·고장 나게 시킬 수 있습니다.

```

/equipments?slow=1500 1.5초 뒤에 답합니다
/equipments?fail=yes    500 을 보냅니다

```

```

function Section2Demo() {
  const [상태, set상태] = useState("아직");
  const [말, set말] = useState("");

  async function 불러보기(뒤에붙일것) {
    set상태("받는중");
    set말("");

    try {
      const 응답 = await fetch(`${서버주소}/equipments${뒤에붙일것}`);
      if (!응답.ok) {
        const 몸 = await 응답.json().catch(() => ({}));
        throw new Error(몸.메시지 || `서버가 ${응답.status} 를 보냈습니다`);
      }
      const 자료 = await 응답.json();
      set상태("됨");
      set말(`${자료.설비들.length}건 받았습니다`);
    } catch (오류) {
      set상태("안됨");
      set말(오류.name === "TypeError" ? 서버안내 : 오류.message);
      console.log("실패:", 오류.name, "/", 오류.message);
    }
  }

  return (
    <div className="demo"> <h3>② 느릴 때 · 안 될 때</h3> <button onClick={() =>
불러보기("")}>보통</button>{" "}
    <button onClick={() => 불러보기("?slow=1500")}>느리게 (1.5초)</button>{" "}
    <button onClick={() => 불러보기("?fail=yes")}>고장 내기 (500)
</button> <div className="output">
      {상태 === "받는중" ? "불러오는 중..." : 말 || "버튼을 눌러 보세요"}
    </div> </div>
  );
}

```

★★ [느리게] 를 누르면 1.5초 동안 "불러오는 중..." 이 보입니다. 이걸 눈으로 봐야 왜 필요한지 압니다. 빠른 서버로만 개발하면 안 만들게 됩니다.

★★★ 오류가 두 종류인 것에 주의하세요.

TypeError	서버에 **닿지도 못했습니다** (안 켜졌거나 CORS)
그 밖	닿았는데 서버가 **잘못됐다고 답했습니다** (500 등)

앞의 것은 "서버를 켜세요", 뒤의 것은 "서버가 이상합니다" 입니다. ★ 사용자에게 보여 줄 말이 달라야 합니다.

▫ 직접 해보기 3

[고장 내기] 를 누르고 F12 → Network 에서 그 요청을 찾으세요. 상태가 무엇입니까? 응답 본문에는 무엇이 들어 있습니까?

▫ 직접 해보기 4

서버를 끈 채로 [보통] 을 누르고, 콘솔에 찍힌 오류 이름을 보세요. 위 표의 어느 쪽입니까?

주소는 영문으로 씁니다

섹션 4

이 자료는 코드 안의 이름을 한글로 씁니다. 그런데 **주소는 영문**입니다.

/api/v1/equipments	← 이렇게
/api/설비	← 이렇게 안 합니다

★★★ 왜냐하면 한글은 주소에서 바뀌어 나가기 때문입니다.

```
function Section3Demo() {
  const [보인것, set보인것] = useState("");

  function 보여주기() {
    const 한글주소 = new URL("http://localhost:4000/api/설비");
    const 영문주소 = new URL("http://localhost:4000/api/v1/equipments");

    const 줄들 = [
      `적은 것 : /api/설비`,
      `나가는 것: ${한글주소.pathname}`,
     ``,
      `적은 것 : /api/v1/equipments`,
      `나가는 것: ${영문주소.pathname}`,
    ];

    set보인것(줄들.join("\n"));
    console.log(한글주소.pathname);
  }

  return (
    <div className="demo"> <h3>③ 한글 주소는 이렇게 바뀝니다</h3> <button onClick=
    {보여주기}>바뀌는 것 보기</button> <pre className="output">{보인것 || "버튼을 눌러
    보세요"</pre> </div>
  );
}
```

★ `/api/%EC%84%A4%EB%B9%84` 가 됩니다. (JS자료 14단원 개념02에서 본 그것입니다) 서버에서 `길 === "/api/설비"` 로 비교하면 안 맞습니다.

★★ 이 자료의 서버를 만들면서 **실제로 이걸로 404 가 났습니다**. 풀어서 비교하게 고칠 수도 있지만, 주소는 그냥 영문으로 쓰는 게 맞습니다. PART 3(백엔드)도 `/api/v1/equipments` 로 씁니다. **같은 주소를 다시 만납니다**.

★★★ 규칙 하나로 — **주소(경로·쿼리)는 영문, 코드 안의 이름과 JSON의 키는 한글**. 실제로 이 서버가 돌려주는 JSON은 `{ 설비들: [...] }` 로 한글입니다. 그건 됩니다.

⇒ 직접 해보기

5

위 코드의 `/api/설비` 를 `/api/설비?이름=프레스` 로 바꿔 보고 `한글주소.search` 도 함께 찍어 보세요. 쿼리도 바뀔니까?

★★ CORS — 왜 나오나

섹션 5

화면은 `http://localhost:5173` (Vite) 에서 돕니다. 서버는 `http://localhost:4000` 에서 돕니다.

★ 포트가 다르면 ****다른 출처****입니다. 이름이 둘 다 `localhost` 라도 그렇습니다.

브라우저는 다른 출처에 요청을 보낼 때 그 서버에게 먼저 물어봅니다. “애가 나한테 요청해도 되나요?” 서버가 된다고 답해야 응답을 코드에 넘겨 줍니다.

`Access-Control-Allow-Origin: *`

이 헤더가 그 대답입니다. 우리 서버는 이렇게 답하도록 만들어져 있습니다.

```
function Section4Demo() {
  const [본것, set본것] = useState("");

  async function 헤더보기() {
    try {
      const 응답 = await fetch(`${서버주소}/equipments`);
      const 줄들 = [
        `상태: ${응답.status}`,
        `Content-Type: ${응답.headers.get("content-type")}`,
        `Access-Control-Allow-Origin: ${응답.headers.get("access-control-allow-origin")}`,
      ];
      set본것(줄들.join("\n"));
      console.log("CORS 헤더:", 응답.headers.get("access-control-allow-origin"));
    } catch {
      set본것(서버안내);
    }
  }
}
```

폼을 useForm 으로

RUN npm run dev → 왼쪽 목록에서 이 예제를 고르세요.

이 파일은 **서버가 없어도** 됩니다. 보내는 것은 개념03부터입니다.

06단원에서 폼을 만들었습니다. 이렇게요.

```
const [이름, set이름] = useState("");


```

이것을 **제어 컴포넌트**라고 불렀습니다. 잘 됩니다. 그런데 입력칸이 늘어나면 이렇게 됩니다.

```
const [이름, set이름] = useState("");
const [상태, set상태] = useState("가동");
const [담당, set담당] = useState("");
const [메모, set메모] = useState("");
... 그리고 검증 오류를 담을 state 가 또 그만큼
```

★ 06단원 개념02에서 객체 하나로 묶는 법을 배웠습니다. 그걸로 절반은 해결됩니다. 그런데 **검증**이 남습니다. “이름은 꼭 넣어야 한다”, “두 글자 이상이어야 한다” — 이걸 손으로 다 짜면 폼 하나에 100줄이 넘어갑니다.

★★ 그래서 실무에서는 폼 라이브러리를 씁니다. 제일 많이 쓰는 것이 **react-hook-form** 이고, 그 핵심이 `useForm` 입니다.

★★★ 이 단원의 목표는 “라이브러리를 외우는 것” 이 아닙니다. **무엇이 줄어드는지와 무엇을 잃는지를** 보는 것입니다. 섹션 5가 그것입니다.

```
import { useState } from "react";
import { useForm } from "react-hook-form";
import Summary from "../_ui/Summary.jsx";
```

손으로 만들면 이렇게합니다

섹션 1

비교하려고 06단원 방식으로 똑같은 폼을 먼저 만듭니다.

```
function Section1Demo() {
  const [값들, set값들] = useState({ 이름: "", 담당: "" });
  const [오류들, set오류들] = useState({});
  const [결과, set결과] = useState("");

  function 바뀌면(e) {
    set값들({ ...값들, [e.target.name]: e.target.value });
  }

  function 보내기(e) {
    e.preventDefault();
  }
}
```

검증을 손으로 합니다

```
const 새오류 = {};
if (!값들.이름.trim()) 새오류.이름 = "이름은 꼭 넣어야 합니다";
else if (값들.이름.trim().length < 2) 새오류.이름 = "두 글자 이상 넣으세요";
if (값들.담당 && 값들.담당.length > 10) 새오류.담당 = "10자까지만 됩니다";

set오류들(새오류);
if (Object.keys(새오류).length > 0) return;

set결과(JSON.stringify(값들));
console.log("손으로 만든 폼:", JSON.stringify(값들));
}

return (
  <div className="demo"> <h3>① 06단원 방식 (손으로)</h3> <form onSubmit=
{보내기}> <div>
  이름{""}
  <input name="이름" value={값들.이름} onChange={바뀌면} placeholder="3호
프레스" />
  {오류들.이름 && <span style={{ color: "#d9534f" }}> {오류들.이름}</span>}
</div> <div style={{ marginTop: 6 }}>
  담당{""}
  <input name="담당" value={값들.담당} onChange={바뀌면} placeholder="김철수"
/>
  {오류들.담당 && <span style={{ color: "#d9534f" }}> {오류들.담당}</span>}
</div> <button type="submit" style={{ marginTop: 8 }}>
  보내기
</button> </form> <div className="output">{결과 || "아직 안 보냈습니다"}
</div> </div>
);
}
```

★ 입력이 둘뿐인데도 state 가 셋(값들·오류들·결과)이고, 검증이 다섯 줄입니다. 입력이 여섯 개가 되면 검증만 스무 줄이 됩니다.

★★ 그리고 빠진 것이 많습니다. · 보내는 중에 버튼을 잠그기 · 칸을 벗어날 때(blur)만 검사하기 · 처음부터 빨강게 뜨지 않게 하기 전부 state 를 더 만들어야 합니다.

▫ 직접 해보기 1

위 폼에 메모 칸을 하나 더 넣어 보세요. 몇 군데를 고쳐야 합니까? 세어 보세요.

```

function Section2Demo() {
  const {
    register,
    handleSubmit,
    formState: { errors },
  } = useForm({
    defaultValues: { 이름: "", 담당: "" },
  });

  const [결과, set결과] = useState("");

  function 보내기(값들) {
    set결과(JSON.stringify(값들));
    console.log("useForm:", JSON.stringify(값들));
  }

  return (
    <div className="demo">
      <h3>② useForm 으로</h3>

      <form onSubmit={handleSubmit(보내기)}>
        <div>
          이름{" "}
          <input
            {...register("이름", {
              required: "이름은 꼭 넣어야 합니다",
              minLength: { value: 2, message: "두 글자 이상 넣으세요" },
            })}
            placeholder="3호 프레스"
          />
          {errors.이름 && <span style={{ color: "#d9534f" }}> {errors.이름.message}
        </span>}
        </div>
        <div style={{ marginTop: 6 }}>
          담당{" "}
          <input
            {...register("담당", {

```



```

    maxLength: { value: 10, message: "10자까지만 됩니다" },
    }}}
    placeholder="김철수"
  />
  {errors.담당 && <span style={{ color: "#d9534f" }}> {errors.담당.message}
</span>}
</div>
<button type="submit" style={{ marginTop: 8 }}>
  보내기
</button>
</form>

<div className="output">{결과 || "아직 안 보냈습니다"}</div>
</div>
);
}

```

★★★ 세 가지가 사라졌습니다.

· `useState` 가 없습니다 → `useForm` 이 값을 들고 있습니다 · `onChange` 가 없습니다 → `register` 가 붙여 줍니다 · `e.preventDefault()` 가 없습니다 → `handleSubmit` 이 해 줍니다

★ `{...register("이름")}` 는 **펼치기(스프레드)** 입니다. (JS자료 09단원) `register` 가 돌려주는 `{ name, onChange, onBlur, ref }` 를 `input` 에 한 번에 붙입니다. 직접 찍어 보면 무엇이 들어 있는지 보입니다. (🖋 2)

★★ `handleSubmit(보내기)` 는 **검증을 통과했을 때만 보내기** 를 부릅니다. 못 통과하면 `errors` 를 채우고 끝냅니다. 그래서 `보내기` 안에서 "값이 비었나" 를 다시 볼 필요가 없습니다.

◦ 직접 해보기 2

`console.log(register("이름"))` 를 컴포넌트 안에 넣고 무엇이 들어 있는지 콘솔에서 펼쳐 보세요.

◦ 직접 해보기 3

이름을 비운 채 [보내기] 를 눌러 보세요. 그 다음 한 글자만 넣고 눌러 보세요. 메시지가 바뀐니까?

검증 규칙

섹션 3

`register` 의 두 번째 인자에 규칙을 적습니다. 자주 쓰는 것들입니다.

```

function Section3Demo() {
  const {
    register,
    handleSubmit,
    formState: { errors },
  } = useForm({
    mode: "onBlur", // * 칸을 벗어날 때 검사합니다
    defaultValues: { 이름: "", 개수: "", 메일: "" },
  });

  const [결과, set결과] = useState("");

  return (
    <div className="demo">
      <h3>③ 검증 규칙</h3>

      <form onSubmit={handleSubmit((값) => set결과(JSON.stringify(값)))}>
        <div>
          이름{" "}
          <input
            {...register("이름", {
              required: "꼭 넣어야 합니다",
              pattern: {
                value: /^[0-9]+호 .+$/,
                message: "'3호 프레스' 처럼 적으세요",
              },
            })}
            placeholder="3호 프레스"
          />
          {errors.이름 && <span style={{ color: "#d9534f" }}> {errors.이름.message}
        </span>}
        </div>

        <div style={{ marginTop: 6 }}>
          개수{" "}
          <input
            type="number"

```

```

        {...register("개수", {
            required: "꼭 넣어야 합니다",
            min: { value: 1, message: "1 이상" },
            max: { value: 99, message: "99 이하" },
            valueAsNumber: true, // * 숫자로 바꿔 줍니다
        })}
    />
    {errors.개수 && <span style={{ color: "#d9534f" }}> {errors.개수.message}
</span>}
</div>

<div style={{ marginTop: 6 }}>
    메일{ " "}
    <input
        {...register("메일", {
            validate: (값) =>
                값 === "" || 값.includes("@") || "@ 가 있어야 합니다",
        })}
        placeholder="비워도 됩니다"
    />
    {errors.메일 && <span style={{ color: "#d9534f" }}> {errors.메일.message}
</span>}
</div> <button type="submit" style={{ marginTop: 8 }}>
    보내기
</button>
</form>

<div className="output">{결과 || "아직 안 보냈습니다"}</div>
</div>
);
}

```

★ 규칙 정리

required	비면 안 됨
minLength / maxLength	글자 수
min / max	숫자 크기 (type="number" 와 같이)
pattern	정규식
validate	* 내가 직접 판단. true 면 통과, 글자면 그게 오류 메시지

★★ valueAsNumber: true 를 안 붙이면 **글자로 옵니다**. type="number" 를 써도 그렇습니다. HTML 입력은 언제나 글자입니다. 개수 + 1 이 "5" + 1 = "51" 이 되는 그 함정입니다. (JS자료 01단원)

★★★ mode: "onBlur" 가 없으면 **보낼 때만** 검사합니다. 그러면 사용자가 다 채우고 누른 뒤에야 빨간 글씨를 봅니다. "onBlur" 는 칸을 벗어날 때, "onChange" 는 글자마다 검사합니다. ★ "onChange" 는 한 글자 칠 때마다 빨개져서 거슬립니다. 보통 "onBlur" 가 낫습니다.

▫ 직접 해보기 4

개수에 5 를 넣고 보내서 결과가 “개수”:5 인지 “개수”:“5” 인지 보세요. 그 다음 `valueAsNumber: true` 를 지우고 다시 해 보세요.

▫ 직접 해보기 5

`mode: “onBlur”` 를 `“onChange”` 로 바꾸고 이름 칸에 타자를 쳐 보세요. 느낌이 어떻게 다른지 확인했으면 되돌려 두세요.

폼의 지금 상태를 봅니다

섹션 4

```
function Section4Demo() {
  const {
    register,
    handleSubmit,
    reset,
    formState: { errors, isDirty, isValid, isSubmitting, submitCount },
  } = useForm({
    mode: "onChange",
    defaultValues: { 이름: "" },
  });

  const [결과, set결과] = useState("");

  async function 보내기(값들) {
```

보내는 데 걸리는 시간을 흉내 냅니다

```
    await new Promise((풀기) => setTimeout(풀기, 800));
    set결과(`보냈습니다: ${값들.이름}`);
    reset(); // * 성공하면 폼을 비웁니다
  }

  return (
    <div className="demo"> <h3>④ 폼의 상태</h3> <form onSubmit=
{handleSubmit(보내기)}> <input
  {...register("이름", { required: "꼭 넣어야 합니다" })}
  placeholder="아무거나"
/> <button type="submit" disabled={isSubmitting || !isValid} style=
{{ marginLeft: 6 }}>
  {isSubmitting ? "보내는 중..." : "보내기"}
</button>
  {errors.이름 && <span style={{ color: "#d9534f" }}> {errors.이름.message}
</span>}
    </div>
  );
}
```

서버에 보내고 받기

RUN 터미널 두 개 (npm run dev / node 서버.js)

개념01에서 **받아 왔고**, 개념02에서 **폼을 만들었습니다**. 이제 둘을 잇습니다. 폼에 쓴 것을 서버로 보냅니다.

★ 여기서 처음으로 **내가 데이터를 바꿉니다**. 지금까지는 읽기만 했습니다. 읽기는 틀려도 화면만 이상하지만, 쓰기는 틀리면 **자료가 잘못 남습니다**. 그래서 볼 것이 많습니다.

★★ 이 파일의 핵심은 하나입니다.

****화면에서 막았다고 끝이 아닙니다. 서버도 막습니다.****
그리고 서버가 막았을 때 그것을 ****폼에 되돌려 보여 줘야**** 합니다.

```
import { useState, useEffect } from "react";
import { useForm } from "react-hook-form";
import Summary from "../_ui/Summary.jsx";
import { 서버주소, 서버안내 } from "../_서버주소.js";
```

POST 로 보냅니다

섹션 1

받을 때와 다른 것이 셋입니다.

method: "POST"	어떤 일을 할지
headers: { "Content-Type": ... }	무엇을 보내는지
body: JSON.stringify(값)	★ 글자로 바꿔서 보냅니다

★ **body** 에 객체를 그냥 넣으면 안 됩니다. **[object Object]** 가 나옵니다. HTTP 로는 글자만 보낼 수 있습니다. (JS자료 12단원 개념03)

```
async function 설비추가(값들) {
  const 응답 = await fetch(`${서버주소}/equipments`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(값들),
  });

  const 몸 = await 응답.json().catch(() => ({}));
```

★ 실패해도 본문에 이유가 들어 있습니다. 버리지 말고 같이 돌려줍니다.

```

return { ok: 응답.ok, 상태: 응답.status, 몸 };
}

function Section1Demo() {
  const [결과, set결과] = useState("");

  async function 눌렀을때() {
    set결과("보내는 중...");
    try {
      const 답 = await 설비추가({ 이름: `시험설비 ${Date.now() % 10000}`, 담당:
"홍길동" });
      set결과(`${답.상태} ${JSON.stringify(답.몸)}`);
    } catch {
      set결과(서버안내);
    }
  }

  return (
    <div className="demo"> <h3>① 그냥 보내 보기</h3> <button onClick={눌렀을때}>설비
하나 추가</button> <pre className="output">{결과} || "버튼을 눌러 보세요"</pre> </div>
  );
}

```

★ 201 이 옳습니다. 200 이 아닙니다. 201 Created 는 “받아서 새로 만들었다” 는 뜻입니다. 200 은 그냥 “잘 됐다” 입니다. 만들었을 때는 201 을 쓰는 것이 관례입니다. ★★ 응답.ok 는 200~299 면 참이라 둘 다 통과합니다. 그래서 코드는 안 바꿉니다.

▫ 직접 해보기

1

버튼을 두 번 누르고, 개념01의 목록 예제를 새로고침해 보세요. 추가한 것이 보입니까?

폼에서 보냅니다

섹션 2

```
function Section2Demo() {
  const {
    register,
    handleSubmit,
    reset,
    formState: { errors, isSubmitting },
  } = useForm({
    mode: "onBlur",
    defaultValues: { 이름: "", 담당: "" },
  });

  const [알림, set알림] = useState("");

  async function 보내기(값들) {
    set알림("");
    try {
      const 답 = await 설비추가(값들);
      if (!답.ok) {
        set알림(`실패 (${답.상태}): ${답.몸.메시지}`);
        return;
      }
      set알림(`추가했습니다: ${답.몸.설비.이름} (번호 ${답.몸.설비.번호})`);
      reset();
    } catch {
      set알림(서버안내);
    }
  }

  return (
    <div className="demo">
      <h3>② 폼에서 보내기</h3>

      <form onSubmit={handleSubmit(보내기)}>
        <div>
          이름{" "}
          <input
```

```

        {...register("이름", {
            required: "이름은 꼭 넣어야 합니다",
            minLength: { value: 2, message: "두 글자 이상" },
        })}
        placeholder="3호 프레스"
    />
    {errors.이름 && <span style={{ color: "#d9534f" }}> {errors.이름.message}
</span>}
</div>
<div style={{ marginTop: 6 }}>
    담당 <input {...register("담당")} placeholder="김철수"
/> </div> <button type="submit" disabled={isSubmitting} style={{ marginTop: 8 }}>
    {isSubmitting ? "보내는 중..." : "추가"}
</button>
</form>

    <div className="output">{알림 || "이름을 넣고 [추가] 를 눌러 보세요"}</div>
</div>
);
}

```

★ `disabled={isSubmitting}` 이 두 번 누르기를 막습니다. (개념02 섹션4) 서버가 느릴 때 이게 없으면 같은 설비가 두 개 생깁니다.

★★ 성공하면 `reset()` 으로 폼을 비웁니다. 안 비우면 방금 넣은 값이 남아 있어서 “또 눌러도 되나?” 싶어집니다.

⇒ 직접 해보기 2

이름을 비운 채 [추가] 를 눌러 보세요. F12 → Network 에 요청이 갑니까? 안 갑니까? 왜 그렇습니까?

★★ 서버가 막았을 때

섹션 3

화면에서 “두 글자 이상” 을 막았습니다. 그런데 서버는 다른 것도 봅니다.

- 이름이 비었나 (화면에서도 봅니다)
- ★ 이미 있는 이름인가 ← 화면에서는 **알 수가 없습니다**

이미 있는지는 서버만 압니다. 그래서 서버가 막고, 그것을 폼에 되돌려야 합니다.

```

function Section3Demo() {
    const {
        register,
        handleSubmit,
    }

```



```

setError,
  formState: { errors, isSubmitting },
} = useForm({
  mode: "onBlur",
  defaultValues: { 이름: "3호 프레스" }, // 일부러 이미 있는 이름
});

const [알림, set알림] = useState("");

async function 보내기(값들) {
  set알림("");
  try {
    const 답 = await 설비추가(값들);

    if (!답.ok) {

```

★★★ 서버가 어느 칸이 문제인지 알려 주면 그 칸에 붙입니다

```

if (답.몸.칸) {
  setError(답.몸.칸, { type: "server", message: 답.몸.메시지 });
} else {
  set알림(답.몸.메시지 || `서버가 ${답.상태} 를 보냈습니다`);
}
return;
}

set알림(`추가했습니다: ${답.몸.설비.이름}`);
} catch {
  set알림(서버안내);
}
}

return (
  <div className="demo"> <h3>③ 서버가 막으면 폼에 붙입니다</h3> <form onSubmit=
{handleSubmit(보내기)}> <div>
  이름{" "}
  <input {...register("이름", { required: "꼭 넣어야 합니다" })} />
  {errors.이름 && <span style={{ color: "#d9534f" }}> {errors.이름.message}
</span>}
</div> <button type="submit" disabled={isSubmitting} style={{ marginTop: 8
}}>
  추가
</button> </form> <div className="output">
  {알림 || "이미 있는 이름(3호 프레스)이 들어 있습니다. 그대로 눌러 보세요."}
</div> </div>
);
}

```

★★★ `setError` 가 이 섹션의 핵심입니다.

서버가 이렇게 답합니다.

```
409 { "메시지": "이미 있는 이름입니다", "칸": "이름" }
```

칸 을 보고 그 입력칸 옆에 빨간 글씨를 붙입니다. 화면에서 검사한 것과 같은 자리에 같은 모양으로 보입니다. 사용자는 그게 어디서 온 판정인지 알 필요가 없습니다.

★★ 서버가 칸 을 안 알려 주면 폼 전체 알림으로 보여 줍니다. 그래서 위 코드가 `if (답.몸.칸)` 로 갈라져 있습니다.

★ `type: "server"` 는 그냥 이름표입니다. 아무 글자나 됩니다. 나중에 "서버에서 온 오류만 지우기" 같은 것을 할 때 씁니다.

★★★ 왜 서버도 검사해야 하나 — **화면 코드는 사용자가 고칠 수 있습니다.** F12 로 검사를 지우고 보낼 수 있습니다. 화면의 검사는 **진절**이고, 서버의 검사가 **진짜**입니다. (백엔드자료에서 이 이야기를 자세히 합니다)

▫ 직접 해보기 3

③번을 눌러 409 를 받아 보세요. 그 다음 이름을 다른 것으로 바꾸고 다시 눌러 보세요. 빨간 글씨가 언제 사라집니까?

▫ 직접 해보기 4

서버 코드에서 칸: "이름" 을 지우고 서버를 다시 켜 보세요. 같은 오류가 어디에 표시되니까? 확인했으면 되돌리세요.

보내고 나서 목록을 맞춥니다

섹션 4

추가했는데 목록이 그대로면 사용자는 "안 됐나?" 합니다. 방법이 둘입니다.

```

function Section4Demo() {
  const [설비들, set설비들] = useState([]);
  const [상태, set상태] = useState("아직");
  const { register, handleSubmit, reset, formState: { isSubmitting } } = useForm({
    defaultValues: { 이름: "" },
  });

  async function 목록받기() {
    set상태("받는중");
    try {
      const 응답 = await fetch(`${서버주소}/equipments`);
      const 자료 = await 응답.json();
      set설비들(자료.설비들);
      set상태("됨");
    } catch {
      set상태("안됨");
    }
  }

  useEffect(() => {
    목록받기();
  }, []);

  async function 보내기(값들) {
    if (!값들.이름.trim()) return;
    try {
      const 답 = await 설비추가(값들);
      if (답.ok) {
        reset();
        await 목록받기(); // ★ 다시 받아 옵니다
      }
    } catch {
      set상태("안됨");
    }
  }
}

```

```

return (
  <div className="demo"> <h3>④ 추가하면 목록도 바뀝니다</h3> <form onSubmit=
{handleSubmit(보내기)}> <input {...register("이름")} placeholder="새 설비 이름"
/> <button type="submit" disabled={isSubmitting} style={{ marginLeft: 6 }}>
  추가
</button> </form> <div className="output">
{상태 === "받는중" && "불러오는 중..."}
{상태 === "안됨" && "서버안내"}
{상태 === "됨" && (
  <ul>
    {설비들.map((하나) => (
      <li key={하나.번호}>
        {하나.이름} - {하나.상태}
      </li>
    ))}
  </ul>
)}
</div> </div>
);
}

```

★ 두 가지 방법

(가) 다시 받아 오기 (위 코드)

추가한 뒤 목록을 한 번 더 부릅니다.

- 좋은 점 - 서버와 ****확실히**** 같습니다
- 나쁜 점 - 요청이 한 번 더 갑니다. 느립니다

(나) 화면에만 더하기

```

`set설비들([...설비들, 답.몸.설비])`
· 좋은 점 - 즉시 반응합니다
· 나쁜 점 - ★ 서버가 정한 값(번호·기본 상태)을 놓칠 수 있습니다

```

★★ 이 서버는 응답에 만들어진 설비를 통째로 돌려줍니다(답.몸.설비). 그래서 (나) 도 안전합니다.
돌려주지 않는 서버라면 (가) 를 쓰세요.

★★★ 초보일 때는 (가) 를 쓰세요. 느려도 틀리지 않습니다. 화면과 서버가 어긋나는 버그는 찾기가 아주 어렵습니다.

◦ 직접 해보기

5

await 목록받기() 를 set설비들([...설비들, 답.몸.설비]) 로 바꿔 보세요. 잘 됩니까? 확인했으면 되돌려 두세요.

자주 하는 실수

섹션 5

이런 실수를 합니다

JSON.stringify 를 빼먹음

body: 값들 → 서버가 [object Object] 를 받습니다. ★ 서버는 "JSON 이 아닙니다" 라고 400 을 냅니다. 이 서버가 그렇게 답합니다.

이런 실수를 합니다

Content-Type 헤더를 빼먹음

서버가 무엇을 받았는지 몰라 본문을 안 읽는 경우가 있습니다. ★ 이 연습용 서버는 그래도 읽지만, Express 는 안 읽습니다. (PART 3)

이런 실수를 합니다

실패했는데 성공한 것처럼 처리

응답.ok 를 안 보고 바로 `reset()` 을 부릅니다. 사용자는 저장된 줄 압니다. 나중에 없어진 것을 알고 화냅니다.

이런 실수를 합니다

오류 본문을 버림

`if (!응답.ok) throw new Error("실패")` 로 뭉뚱그립니다. 서버가 "이미 있는 이름입니다" 라고 알려줬는데 그걸 버린 것입니다. (섹션 3)

이런 실수를 합니다

isSubmitting 없이 됨

느린 서버에서 두 번 눌러 두 개가 저장됩니다.

이런 실수를 합니다

서버 검사를 안 만들고 화면 검사만 함

화면 검사는 F12 로 지울 수 있습니다. 서버가 진짜 문지기입니다. (섹션 3)

화면 —————

정리

- 1 보낼 때는 method·headers·body 셋을 붙입니다. body 에는 `JSON.stringify` 로 글자를 만들어 넣습니다. 객체를 그냥 넣으면 `[object Object]` 가 나옵니다.
- 2 새로 만들면 서버가 201 을 보냅니다. 응답.ok 는 200~299 를 참으로 보므로 코드는 안 바뀝니다.
- 3 실패해도 본문에 이유가 들어 있습니다. 버리지 말고 화면에 쓰세요. 상태 코드만으로는 사용자에게 할 말이 안 나옵니다.
- 4 ★ 화면 검사는 친절이고 서버 검사가 진짜입니다. 화면 코드는 F12 로 고칠 수 있습니다.
- 5 ★ 서버가 어느 칸이 문제인지 알려 주면 `setError` 로 그 칸에 붙입니다. 사용자는 화면 검사와 서버 검사를 구분할 필요가 없습니다.
- 6 `isSubmitting` 으로 버튼을 잠급니다. 안 잠그면 느린 서버에서 두 개가 저장됩니다.
- 7 추가한 뒤에는 목록을 다시 받아 옵니다. 화면에만 더하는 방법은 빠르지만 서버가 정한 값을 놓칠 수 있습니다. 초보일 때는 다시 받아 오세요.

```
export default function Concept03Post() {
  const [restartKey, setRestartKey] = useState(0);

  return (
    <div> <h1>개념 03 - 서버에 보내고 받기</h1> <p className="guide"> <strong>서버를
    켜야 합니다.</strong> <code>실습프로젝트/14단원_서버</code> 에서 <code>node
    서버.js</code> <br /> <br />
    여기서 추가한 설비는 <strong>서버가 꺼지면 사라집니다.</strong> 메모리에만
    있습니다.
    마음껏 눌러 보세요.
    <br /> <br /> <strong>③번이 이 파일의 핵심입니다.</strong> 서버가 막은 것을
    폼에 되돌리는 방법입니다.
    </p> <button onClick={() => setRestartKey(restartKey + 1)}>
    이 예제를 처음부터 다시 그리기
    </button> <div key={restartKey}> <Section1Demo /> <Section2Demo
    /> <Section3Demo /> <Section4Demo /> </div> </div>
  );
}
```

직접 해보기 정답

1) 두 번 누르면 시험설비 1234, 시험설비 5678 처럼 다른 이름이 들어갑니다. (`Date.now()` % 10000 으로 이름을 다르게 만들어 중복을 피했습니다) 개념01의 목록을 새로고침하면 늘어나 있습니다. → ★ 두 예제가 같은 서버를 봅니다. 화면이 달라도 자료는 하나입니다.

서버를 켜다 켜면 처음 셋으로 돌아갑니다.

2) 화면: “이름은 꼭 넣어야 합니다” 가 뜹니다. Network: ★ 요청이 아예 안 갑니다. → `handleSubmit` 이 검증을 먼저 하고, 통과했을 때만 보내기 를 부르기 때문입니다.

★ 서버에 쓸데없는 요청을 안 보내는 것이 화면 검사의 이득입니다.
(그렇다고 서버 검사를 빼도 되는 것은 아닙니다. 섹션 3)

3) 화면: 이름칸 옆에 “이미 있는 이름입니다” 가 빨강게 붙습니다. 이름을 바꾸면 — → ★ 바로 안 사라집니다. 다시 [추가] 를 눌러야 사라집니다.

``setError`` 로 넣은 오류는 그 칸을 다시 검사할 때 지워지는데,
서버 검사는 보낼 때만 하기 때문입니다.

★★ 거슬리면 이렇게 지울 수 있습니다.

```
const { clearErrors } = useForm(...);


```

4) 서버에서 칸: “이름” 을 지우면 —

화면 빨간 글씨가 칸 옆이 아니라 **아래 알림 상자**에 나옵니다.

→ `if (답.몸.칸)` 이 거짓이 되어 `set알림` 쪽으로 갔기 때문입니다.

★ 서버가 어디가 문제인지 알려 주느냐 마느냐로 화면이 달라집니다.
서버를 만들 때 `칸` 을 같이 보내 주면 프론트가 훨씬 편해집니다.
PART 3 에서 서버를 만들 때 기억해 두세요.

5) `set설비들([...설비들, 답.몸.설비])` 로 바꾸면 —

화면 잘 됩니다. 그리고 **더 빠릅니다.** 요청이 한 번만 갑니다.

→ 이 서버가 만들어진 설비를 통째로 돌려주기 때문입니다.

``번호`` 도 ``상태`` 도 서버가 정한 값이 그대로 들어옵니다.

★ 만약 서버가 `{ ok: true }` 만 돌려줬다면 번호를 몰라서

``key={하나.번호}`` 가 깨졌을 것입니다. 그때는 다시 받아 와야 합니다.

확인했으면 되돌려 두세요.

파일 올리기

RUN 터미널 두 개 (npm run dev / node 서버.js)

개념03에서 글자를 보냈습니다. 이제 **파일**을 보냅니다.

현장에서 제일 많이 하는 일입니다. 점검일지 사진, 작업표준서 PDF, 자재 대장 엑셀 — 전부 파일로 올라옵니다.

★ 글자를 보낼 때와 두 가지가 다릅니다.

JSON 으로 못 보냅니다	파일은 글자가 아닙니다
→ FormData 를 씁니다	브라우저가 만들어 주는 답는 그릇

★★ 그리고 **Content-Type** 을 직접 쓰면 안 됩니다. 섹션 3의 그 함정입니다. 이걸 모르면 절대 못 고칩니다.

```
import { useState } from "react";
import { useForm } from "react-hook-form";
import Summary from "../_ui/Summary.jsx";
import { 서버주소, 서버안내 } from "../_서버주소.js";
```

파일을 고르는 칸

섹션 1

`<input type="file" />` 입니다. 다른 입력과 크게 다른 점이 하나 있습니다.

★ **값을 코드로 정할 수 없습니다.**

`value="C:/사진.jpg"` 라고 쓸 수 없습니다. 브라우저가 막습니다. 그게 되면 웹페이지가 몰래 내 파일을 올릴 수 있으니까요.

그래서 파일 칸은 언제나 **비제어**입니다. 06단원의 제어 컴포넌트가 안 됩니다.


```
function Section1Demo() {
  const [고른것, set고른것] = useState(null);

  function 골랐을때(e) {
    const 파일 = e.target.files[0];
    if (!파일) {
      set고른것(null);
      return;
    }
    set고른것({
      이름: 파일.name,
      크기: 파일.size,
      종류: 파일.type || "(모름)",
    });
    console.log("고른 파일 종류:", 파일.type || "(모름)");
  }

  return (
    <div className="demo"> <h3>① 파일을 고르면 무엇을 알 수 있나
    </h3> <input type="file" onChange={골랐을때} /> <div className="output">
      {고른것 ? (
        <> <div>이름: {고른것.이름}</div> <div>크기: {고른것.크기.toLocaleString()}
        바이트</div> <div>종류: {고른것.종류}</div> </>
      ) : (
        "파일을 골라 보세요"
      )}
    </div>
  </div>
  );
}
```

★ `e.target.files` 는 **배열이 아닙니다**. `FileList` 라는 것입니다. `[0]` 으로 첫 번째를 꺼냅니다. `.map()` 은 안 됩니다. 여러 개를 다루려면 `[...e.target.files]` 로 배열로 바꿉니다.

★★ `파일.type` 은 **브라우저가 확장자를 보고 짐작한 값**입니다. `.txt` 를 `.jpg` 로 이름만 바꾸면 `image/jpeg` 라고 합니다. ★ 그래서 이걸로 안전을 판단하면 안 됩니다. 서버가 진짜로 봐야 합니다.

★ 파일을 고르고 취소하면 `files[0]` 이 `undefined` 가 됩니다. 위 코드의 `if (!파일)` 이 그것을 막습니다. 없으면 그 다음 줄에서 터집니다.

▹ 직접 해보기

1

아무 `.txt` 파일의 확장자를 `.jpg` 로 바꾼 뒤 골라 보세요. **종류** 에 무엇이 나오니까?

JSON 대신 `FormData` 라는 그릇에 담습니다.

```
const 그릇 = new FormData();
그릇.append("파일", 파일);
그릇.append("메모", "8월 점검분");

await fetch(주소, { method: "POST", body: 그릇 });
```

★ 파일과 글자를 **같이** 담을 수 있습니다. 그게 `FormData` 의 쓸모입니다.

```
function Section2Demo() {
  const [결과, set결과] = useState("");
  const [보내는중, set보내는중] = useState(false);

  async function 올리기(e) {
    e.preventDefault();
    const 파일 = e.target.파일칸.files[0];

    if (!파일) {
      set결과("파일을 먼저 고르세요");
      return;
    }

    set보내는중(true);
    set결과("");

    try {
      const 그릇 = new FormData();
      그릇.append("파일", 파일);
      그릇.append("메모", e.target.메모칸.value);

      const 응답 = await fetch(`${서버주소}/files`, {
        method: "POST",
        body: 그릇,
      });
    } catch {
      // 에러 처리
    }
  }
}
```

★★★ headers 를 안 씁니다. 섹션 3을 보세요.

```

});

const 몸 = await 응답.json().catch(() => ({}));

if (!응답.ok) {
  set결과(`실패 (${응답.status}): ${몸.메시지}`);
  return;
}

set결과(
  `올렸습니다\n원래 이름: ${몸.원래이름}\n저장된 이름: ${몸.저장이름}\n` +
  `크기: ${몸.크기} 바이트\n메모: ${몸.메모} || "(없음)"`,
);
} catch {
  set결과(서버안내);
} finally {
  set보내는중(false);
}
}

return (
  <div className="demo"> <h3>② 올려 보기</h3> <form onSubmit=
{올리기}> <div> <input type="file" name="파일칸" /> </div> <div style={{ marginTop: 6
}}>
    메모 <input name="메모칸" placeholder="8월 점검분"
/> </div> <button type="submit" disabled={보내는중} style={{ marginTop: 8 }}>
    {보내는중 ? "올리는 중..." : "올리기"}
  </button> </form>

  <pre className="output">{결과} || "파일을 고르고 [올리기] 를 누르세요"</pre>
</div>
);
}

```

★★ 서버가 저장된 이름을 바꿔서 돌려줍니다.

원래 이름: 점검일지.txt
저장된 이름: 1787124281843_b1scby.txt

왜 바꾸냐 —

- 같은 이름을 올리면 **덮어써집니다**
- 이름에 `../` 같은 것을 넣어 엉뚱한 곳에 쓰게 만들 수 있습니다

★ 원래 이름은 따로 적어 두고 화면에만 보여 줍니다.

이건 서버 쪽 이야기라 PART 3(백엔드 09단원)에서 자세히 합니다.

▸ 직접 해보기 2

파일을 안 고르고 [올리기] 를 눌러 보세요. 무엇이 나오니까? 그 다음 서버 터미널을 보세요. 요청이 갔습니까?

★★★ Content-Type 을 쓰면 안 됩니다

섹션 3

개념03에서는 이렇게 썼습니다.

```
headers: { "Content-Type": "application/json" }
```

그래서 파일도 이렇게 쓰고 싶어집니다.

```
headers: { "Content-Type": "multipart/form-data" } ← ★ 이러면 깨집니다
```

```
function Section3Demo() {
  const [결과, set결과] = useState("");

  async function 해보기(헤더를쓸까) {
    set결과("보내는 중...");

    const 그릇 = new FormData();
    그릇.append(
      "파일",
      new File(["점검 결과: 이상 없음"], "시험.txt", { type: "text/plain" }),
    );
    그릇.append("메모", "헤더 시험");

    const 옵션 = { method: "POST", body: 그릇 };
    if (헤더를쓸까) {
      옵션.headers = { "Content-Type": "multipart/form-data" };
    }

    try {
      const 응답 = await fetch(`${서버주소}/files`, 옵션);
      const 몸 = await 응답.json().catch(() => ({}));
      set결과(`${응답.status} ${JSON.stringify(몸)}`);
      console.log(헤더를쓸까 ? "헤더 씬:" : "헤더 안 씬:", 응답.status);
    } catch {
      set결과(서버안내);
    }
  }

  return (
    <div className="demo"> <h3>③ 헤더를 쓰면 어떻게 되나</h3> <button onClick={() =>
해보기(false)}>헤더 없이 (맞는 방법)</button>{" "}
    <button onClick={() => 해보기(true)}>Content-Type 을 직접 쓰기
    </button> <pre className="output">{결과} || "두 버튼을 차례로 눌러 보세요"</pre>
    </div>
  );
}
```

★★★ 왜 깨지나

multipart 는 조각을 나누는 **경계 글자**가 필요합니다.

```
Content-Type: multipart/form-data; boundary=----WebKitFormBoundaryAbC123
      └────────── 이 부분 ─────────┘
```

이 경계 글자는 **브라우저가 매번 새로 만듭니다**. 그리고 본문 안에도 넣습니다. 내가 헤더를 직접 쓰면 그 **boundary=** 가 **빠집니다**. 서버는 어디서 조각이 나뉘는지 몰라 본문을 못 읽습니다.

★★ 그래서 규칙은 —

JSON 을 보낼 때 → Content-Type 을 ****직접 씁니다****
FormData 를 보낼 때 → *** **아무것도 쓰지 않습니다****

body 가 FormData 면 브라우저가 알아서 붙여 줍니다. 건드리지 마세요.

★ 이걸 “몰라서 안 쓴 것” 과 “알고 안 쓴 것” 이 결과가 같아서 모르고 지나가기 쉽습니다. 그러다 한 번 쓰면 반나절을 잃습니다.

▫ 직접 해보기 3

두 버튼의 결과를 비교하고, F12 → Network 에서 두 요청의 Request Headers 의 Content-Type 을 각각 보세요. **boundary=** 가 어느 쪽에 있습니까?

useForm 과 같이 쓰기

섹션 4

개념02의 **useForm** 으로 파일 칸도 다룰 수 있습니다.

```
function Section4Demo() {
  const {
    register,
    handleSubmit,
    reset,
    formState: { errors, isSubmitting },
  } = useForm({ defaultValues: { 메모: "" } });

  const [결과, set결과] = useState("");

  async function 올리기(값들) {
```

★ 파일은 **FileList** 로 옵니다. [0] 으로 꺼냅니다.

```

const 파일 = 값들.파일[0];

set결과("");
try {
  const 그릇 = new FormData();
  그릇.append("파일", 파일);
  그릇.append("메모", 값들.메모);

  const 응답 = await fetch(`${서버주소}/files`, { method: "POST", body: 그릇 });
  const 몸 = await 응답.json().catch(() => ({}));

  if (!응답.ok) {
    set결과(`실패 (${응답.status}): ${몸.메시지}`);
    return;
  }
  set결과(`올렸습니다: ${몸.원래이름} (${몸.크기} 바이트)`);
  reset();
} catch {
  set결과(서버안내);
}
}

return (
  <div className="demo">
    <h3>④ useForm 으로 (크기 검사까지)</h3>

    <form onSubmit={handleSubmit(올리기)}>
      <div>
        <input
          type="file"
          {...register("파일", {
            required: "파일을 골라야 합니다",
            validate: {
              크기: (목록) =>
                !목록?.[0] ||
                목록[0].size <= 2 * 1024 * 1024 ||

```

```

"2MB 까지만 됩니다",
    },
  })}
/>
{errors.파일 && <div style={{ color: "#d9534f" }}>{errors.파일.message}
</div>}
</div>

<div style={{ marginTop: 6 }}>
  메모 <input {...register("메모")} placeholder="8월 점검분"
/> </div> <button type="submit" disabled={isSubmitting} style={{ marginTop: 8 }}>
  {isSubmitting ? "올리는 중..." : "올리기"}
</button>
</form>

<div className="output">{결과 || "파일을 고르고 눌러 보세요"}</div>
</div>
);
}

```

★ `register("파일")` 이 돌려주는 값은 **FileList** 입니다. 파일 하나가 아닙니다. 그래서 `값들.파일[0]` 로 꺼내고, 검증에서도 `목록[0]` 을 봅니다.

★★ 화면에서 크기를 검사해도 **서버가 또 검사합니다**. 서버도 2MB 를 넘으면 413 을 보냅니다. 개념03 섹션3과 같은 이야기입니다. ★ 화면 검사는 사용자를 위한 것(요청을 아예 안 보냄),

서버 검사는 서버를 위한 것(디스크가 차는 것을 막음)입니다.

▫ 직접 해보기

4

2MB 가 넘는 파일(사진 등)을 골라 보세요. 요청이 갑니까? Network 를 확인하세요.

▫ 직접 해보기

5

`validate` 를 통째로 지우고 2MB 넘는 파일을 올려 보세요. 이번에는 무엇이 나오니까? 확인했으면 되돌려 두세요.

자주 하는 실수

섹션 5

이런 실수를 합니다

FormData 에 Content-Type 을 직접 씀 ★★ 섹션 3

제일 아픕니다. 원인을 짐작조차 못 합니다.

이런 실수를 합니다

`files[0]` 이 없을 수 있다는 것을 잊음

파일을 골랐다가 취소하면 `undefined` 입니다. 그 다음 줄에서 터집니다.

이런 실수를 합니다

`files` 를 배열로 봄

`e.target.files.map(...)` → `map` is not a function [`...e.target.files`] 로 바꾸세요.

이런 실수를 합니다

파일.type 을 믿음

확장자만 바꾸면 따라 바뀝니다. 안전 판단에 쓰면 안 됩니다. (섹션 1)

이런 실수를 합니다

올리는 중에 버튼을 안 잠금

큰 파일은 몇 초씩 걸립니다. 두 번 누르면 두 번 올라갑니다.

이런 실수를 합니다

서버 쪽 크기 제한을 안 둠

화면에서만 막으면 누군가 그냥 큰 파일을 올려 디스크를 채웁니다. (섹션 4)

화면 —————

정리

- 1 파일 칸은 value 를 코드로 정할 수 없습니다. 언제나 비제어입니다. e.target.files[0] 로 꺼냅니다.
- 2 files 는 배열이 아니라 FileList 입니다. map 을 쓰려면 [...files] 로 바꿉니다. 고르고 취소하면 undefined 가 되니 확인이 필요합니다.
- 3 파일.type 은 브라우저가 확장자를 보고 짐작한 값입니다. 확장자만 바뀌어도 따라 바뀌므로 안전 판단에 쓰면 안 됩니다.
- 4 파일은 FormData 에 담아 보냅니다. append 로 파일과 글자를 같이 담을 수 있습니다.
- 5 ★★ FormData 를 보낼 때 Content-Type 을 직접 쓰면 깨집니다. boundary 가 빠지기 때문입니다. 브라우저가 붙이게 두세요. JSON 일 때만 직접 씁니다.
- 6 useForm 으로도 됩니다. register 가 돌려주는 값이 FileList 라서 [0] 으로 꺼냅니다. validate 로 크기를 검사할 수 있습니다.
- 7 서버가 저장 이름을 바뀌서 돌려줍니다. 원래 이름을 그대로 쓰면 덮어쓰기와 경로 조작이 생깁니다. 크기 제한도 서버에 꼭 둡니다.

```
export default function Concept04Upload() {
  const [restartKey, setRestartKey] = useState(0);

  return (
    <div> <h1>개념 04 - 파일 올리기</h1> <p className="guide"> <strong>서버를 켜야
    합니다.</strong> <code>실습프로젝트/14단원_서버</code> 에서 <code>node
    서버.js</code> <br /> <br />
    올린 파일은 <code>14단원_서버/올라온파일</code> 에 쌓입니다.
    연습이 끝나면 그 폴더를 지워도 됩니다.
    <br /> <br /> <strong>③번을 꼭 눌러 보세요.</strong> 모르면 절대 못 고치는
    함정입니다.
    </p> <button onClick={() => setRestartKey(restartKey + 1)}>
    이 예제를 처음부터 다시 그리기
    </button> <div key={restartKey}> <Section1Demo /> <Section2Demo
    /> <Section3Demo /> <Section4Demo /> </div> </div>
  );
}
```

직접 해보기 정답

1) .txt 를 .jpg 로 바꾸고 고르면 —

화면 종류: image/jpeg

```
// 콘솔: 고른 파일 종류: image/jpeg
```

→ 내용은 그대로 글자인데 `image/jpeg` 라고 합니다.

★ 브라우저는 ****확장자만 보고**** 정합니다. 파일 안을 열어 보지 않습니다.
 그래서 "이미지만 받는다" 를 이 값으로 막으면 뚫립니다.
 진짜로 막으려면 서버가 파일 앞부분의 표식을 봐야 합니다. (PART 3)

2) 화면: "파일을 먼저 고르세요" 서버 터미널: ★ 아무것도 안 찍힙니다. **요청이 안 왔습니다.** → `if (!파일)`
`return` 이 먼저 막았기 때문입니다.

★ 이 확인을 빼면 ``그릇.append("파일", undefined)`` 가 되어
 서버가 "파일이 안 왔습니다" 로 400 을 보냅니다. 그것도 동작은 합니다.
 다만 쓸데없는 왕복이 한 번 생깁니다.

3) 헤더 없이 → 201, 잘 올라갑니다.

`Content-Type` 을 직접 쓰면 → 400 {"메시지": "multipart/form-data 가 아닙니다"}

Network 의 Request Headers 를 보면 —

```
헤더 없이: multipart/form-data; boundary=----WebKitFormBoundary...
직접 쓰면: multipart/form-data          ← ★ boundary 가 없습니다
```

→ 서버는 경계를 몰라 본문을 조각으로 나눌 수조차 없습니다.

그래서 "파일이 안 왔다" 보다 앞선 단계에서 걸러집니다.
 (서버가 "파일이 안 왔습니다" 를 주는 것은 `boundary` 는 멀쩡한데
 파일 칸만 비었을 때입니다 - 위 2번이 그 경우입니다)

★★ 이 메시지가 헛갈리는 이유 —

나는 분명히 `Content-Type` 에 `multipart/form-data` 라고 썼는데
 서버는 "multipart/form-data 가 아니다" 라고 합니다.
 빠진 것은 그 뒤에 브라우저가 붙여 줘야 할 `boundary=...` 입니다.
 원인은 헤더 한 줄입니다. 이래서 무서운 함정입니다.

4) 2MB 넘는 파일을 고르면 —

화면 "2MB 까지만 됩니다"

Network: ★ 요청이 안 갑니다. → `validate` 가 `handleSubmit` 단계에서 막았습니다.

큰 파일을 올리다 거절당하는 것보다 훨씬 낫습니다. 시간과 데이터를 아깁니다.

5) `validate` 를 지우고 올리면 —

화면 실패 (413): 파일이 너무 큼니다 (2MB 까지)

Network: 요청이 **갑니다**. 파일을 다 올린 뒤에 거절당합니다. → ★ 서버도 막고 있다는 뜻입니다. 화면 검사가 없어도 안전은 합니다.

다만 사용자는 다 올리고 나서 거절당합니다. 느린 인터넷이면 몇 분입니다.

★★ 그래서 **둘 다** 둡니다. 화면은 빠르라고, 서버는 안전하라고.

확인했으면 `validate` 를 되돌려 두세요.

안 되는 경우들

RUN 터미널 두 개 (npm run dev / node 서버.js)

이 파일은 일부러 실패시킵니다. 오류가 나는 게 정상입니다.

서버에 붙기 시작하면 안 되는 경우가 확 늘어납니다. 혼자 도는 화면은 내 코드만 맞으면 됐는데, 이제 상대가 생겼기 때문입니다.

- 상대가 안 켜져 있을 수 있고
- 켜져 있어도 안 받아 줄 수 있고 (CORS)
- 받아 줬는데 느릴 수 있고
- 답이 왔는데 **순서가 뒤바뀔** 수 있습니다

★ 이 파일의 목적은 **가르는 법**을 익히는 것입니다. 증상은 비슷한데 원인이 다릅니다. 원인을 못 가르면 엉뚱한 데를 고칩니다.

★★ 마지막 섹션의 “보는 순서” 를 종이에 적어 두세요. PART 3·4 에서도 그대로 씁니다.

```
import { useState, useEffect, useRef } from "react";
import Summary from "../_ui/Summary.jsx";
import { 서버주소 } from "../_서버주소.js";
```

닿지 못한 것과 거절당한 것

섹션 1

오류가 크게 두 갈래입니다. 이것 먼저 가릅니다.

```
function Section1Demo() {
  const [줄들, set줄들] = useState([]);

  async function 해보기(이름, 주소) {
    let 결과;
    try {
      const 응답 = await fetch(주소);
      const 몸 = await 응답.json().catch(() => ({}));
      결과 = `${이름} → 닿았습니다. 상태 ${응답.status} ${JSON.stringify(몸).slice(0, 40)}`;
    } catch (오류) {
      결과 = `${이름} → 못 닿았습니다. ${오류.name}: ${오류.message}`;
    }
    set줄들((전) => [...전, 결과]);
  }
}
```

```

}

return (
  <div className="demo"> <h3>① 두 갈래로 갈립니다</h3> <button onClick={() =>
해보기("정상", `${서버주소}/equipments`)}>정상</button>{" "}
    <button onClick={() => 해보기("500", `${서버주소}/equipments?fail=yes`)}>
      서버가 500
    </button>{" "}
    <button onClick={() => 해보기("없는 주소", `${서버주소}/없는곳`)}>없는 주소
  </button>{" "}
    <button onClick={() => 해보기("안 켜진 포트",
"http://localhost:4999/api/v1/equipments")}>
      안 켜진 포트
    </button>{" "}
    <button onClick={() => set줄들([])}>지우기</button> <pre className="output">
{줄들.join("\n")} || "버튼을 눌러 보세요"</pre> </div>
  );
}

```

★★★ 표로 정리하면 이렇습니다.

무엇 catch 로 가나? 무엇을 보나

정상 안 감 응답.status 200 서버가 500 ★ 안 감 응답.status 500 ← fetch 는 실패로 안칩니다 없는 주소 (404) ★ 안 감 응답.status 404 서버가 안 켜짐 감 TypeError CORS 에 막힘 감 ★ TypeError — 위와 똑같습니다

★★ 두 가지가 중요합니다.

(1) 404·500 은 catch 로 안 갑니다. 응답.ok 를 직접 봐야 합니다.

이걸 모르면 "오류 처리 했는데 왜 안 걸려요?" 가 됩니다.

(2) 서버가 안 켜진 것과 CORS 는 코드에서 구분이 안 됩니다.

둘 다 `TypeError: Failed to fetch` 입니다.
→ * **Network 탭**을 봐야 합니다. 요청이 아예 없으면 안 켜진 것,
요청은 갔는데 빨가면 CORS 입니다. (개념01 섹션5)

▫ 직접 해보기 1

네 버튼을 다 눌러 보고, 각각 Network 탭에 요청이 보이는지 확인하세요. "안 켜진 포트" 는 어떻게 보입니까?

두 번 눌러서 두 개가 저장됩니다

섹션 2

```
function Section2Demo() {
  const [보낸수, set보낸수] = useState(0);
  const [막을까, set막을까] = useState(false);
  const [보내는중, set보내는중] = useState(false);
  const [말, set말] = useState("");

  async function 보내기() {
    if (막을까 && 보내는중) return; // * 이 한 줄이 전부입니다
    set보내는중(true);

    try {
```

느린 서버를 흉내 냅니다

```
await fetch(`${서버주소}/equipments?slow=900`);
  set보낸수((전) => 전 + 1);
  set말("");
} catch {
  set말("서버가 안 켜져 있습니다");
} finally {
  set보내는중(false);
}
}

return (
  <div className="demo"> <h3>② 빨리 두 번 누르면
</h3> <label> <input type="checkbox" checked={막을까} onChange={(e) => set막을까
(e.target.checked)} />{" "}
    보내는 중이면 막기
    </label> <div style={{ marginTop: 6 }}> <button onClick={보내기}>보내기 (0.9초
    걸립니다)</button>{" "}
    <button onClick={() => set보낸수(0)}>세기 초기화
  </button> </div> <div className="output"> <div>{보내는중 ? "보내는 중..." : "쉬는
  중"}</div> <div>서버에 간 횟수: {보낸수}</div> <div style={{ fontSize: 13, color:
  "#666", marginTop: 6 }}>
    체크를 끈 채 [보내기] 를 빠르게 세 번 눌러 보세요. {말}
  </div> </div> </div>
);
}
```

★ 체크를 끄고 세 번 누르면 **세 번 다 갑니다**. 저장하는 요청이었다면 세 개가 생깁니다. 체크를 켜면 한 번만 갑니다.

★★ 개념02·03에서는 `disabled={isSubmitting}` 으로 막았습니다. 버튼 자체를 못 누르게요. 여기서는 함수 첫 줄에서 막았습니다. **둘 다 하는 것이 제일 좋습니다.** 버튼을 잠그는 것은 보기에 좋고, 함수에서 막는 것은 확실합니다. (키보드로도 누를 수 있고, 다른 곳에서 이 함수를 부를 수도 있으니까요)

★★★ 그런데 **정말 확실하게 하려면 서버가 막아야 합니다.** 화면을 아무리 잘 만들어도 요청을 두 번 보내는 방법은 있습니다. 같은 것을 두 번 받았을 때 하나만 만드는 것은 서버 몫입니다. (PART 3)

▫ 직접 해보기

2

체크를 끄고 빠르게 세 번 누른 뒤 “간 횟수” 를 보세요. 그 다음 체크를 켜고 다시 해 보세요.

화면이 사라진 뒤에 답이 옵니다

섹션 3

```

function Section3Demo() {
  const [보일까, set보일까] = useState(true);

  return (
    <div className="demo"> <h3>③ 꺾는데 답이 오면</h3> <button onClick={() =>
set보일까(!보일까)}>{보일까 ? "숨기기" : "보이기"}</button> <div style=
{{ fontSize: 13, color: "#666", margin: "6px 0" }}>
      [보이기] 를 누르고 <strong>1초 안에</strong> [숨기기] 를 누르세요. 콘솔을
      보세요.
    </div>

    {보일까 && <느린칸 />}
  </div>
  );
}

function 느린칸() {
  const [말, set말] = useState("받는 중...");

  useEffect(() => {
    const 끊개 = new AbortController();

    (async () => {
      try {
        const 응답 = await fetch(`${서버주소}/equipments?slow=1200`, {
          signal: 끊개.signal, // * 이걸로 도중에 끊습니다
        });
        const 자료 = await 응답.json();
        set말(`${자료.설비들.length}건 받았습니다`);
        console.log("답이 와서 화면을 고쳤습니다");
      } catch (오류) {
        if (오류.name === "AbortError") {
          console.log("* 화면이 사라져서 요청을 끊었습니다");
          return; // * set 을 안 부릅니다
        }

        set말("서버가 안 켜져 있습니다");
      }
    })();

    return () => 끊개.abort(); // * 화면이 사라질 때 부릅니다
  }, []);

  return <div className="output">{말}</div>;
}

```

★ `AbortController` 는 “이 요청 그만” 이라고 말하는 장치입니다. `signal` 을 `fetch` 에 넘겨 두고, 나중에 `abort()` 를 부르면 끊깁니다.

★★ `useEffect` 가 돌려주는 함수는 **화면이 사라질 때** 불립니다. (09단원 개념02) 거기서 `abort()` 를 부르면, 답이 와도 `AbortError` 로 빠집니다.

★★★ 안 끊으면 무슨 일이 생기나 — 사라진 컴포넌트에 `set`말 을 부릅니다. React 18 부터는 조용히 무시되지만, **쓸데없이 서버와 통신하고 결과를 버리는 것은 그대로입니다.** 목록 화면을 빠르게 오가면 요청이 계속 쌓입니다.

★ 09단원에서는 `버렸다` 라는 표시로 막았습니다. 그것도 맞습니다. 다만 `버렸다` 는 **결과만 버리고**, `abort` 는 **요청 자체를 끊습니다.** 끊는 쪽이 낫습니다.

▸ 직접 해보기 3

[보이기] 를 누르고 1초 안에 [숨기기] 를 누른 뒤 콘솔을 보세요. 어느 줄이 찍히니까? 1초를 넘겨 기다린 뒤 숨기면 어떻게 됩니까?

답이 뒤바뀝니다

섹션 4

검색창처럼 **입력할 때마다 요청**하는 화면에서 생깁니다. 먼저 보낸 요청이 나중에 도착할 수 있습니다.

```
function Section4Demo() {
  const [막을까, set막을까] = useState(false);
  const [보인값, set보인값] = useState("(아직)");
  const [기록, set기록] = useState([]);
  const 최신번호 = useRef(0);

  async function 요청(이름, 늦기) {
    const 내번호 = ++최신번호.current;
    set기록((전) => [...전, `${이름} 보냄 (${늦기}ms)`]);

    try {
      await fetch(`${서버주소}/equipments?slow=${늦기}`);
    } catch {
      return;
    }
  }
}
```

★ 내가 마지막 요청이 아니면 결과를 버립니다

```

if (막을까 && 내번호 !== 최신번호.current) {
  set기록((전) => [...전, `${이름} 도착 - 낚아서 버림`]);
  return;
}

set보인값(이름);
set기록((전) => [...전, `${이름} 도착 → 화면에 씬`]);
}

function 해보기() {
  set기록([]);
  set보인값("받는 중");
  최신번호.current = 0;
  요청("느린 것", 1500);
  setTimeout(() => 요청("빠른 것", 200), 100);
}

return (
  <div className="demo"> <h3>④ 먼저 보낸 게 나중에 옵니다
</h3> <label> <input type="checkbox" checked={막을까} onChange={e => set막을까
(e.target.checked)} />{" "}
    낚은 답 버리기
    </label> <div style={{ marginTop: 6 }}> <button onClick={해보기}>두 개를 겹쳐
    보내기</button> </div> <div className="output"> <div> <strong>화면에 남은 값:
    {보인값}</strong> </div> <pre style={{ margin: "6px 0 0", fontSize: 13 }}>
    {기록.join("\n")}</pre> </div>

    </div>
  );
}

```

★★★ 체크를 끄고 눌러 보세요.

“빠른 것” 이 먼저 도착해서 화면에 쓰이고, 나중에 “느린 것” 이 도착해서 덮어씁니다. → 화면에 낚은 값이 남습니다.

검색창이라면 “프” 를 치고 “프레스” 를 쳤는데 화면에는 “프” 의 결과가 보이는 것입니다. 아주 헛갈립니다.

★★ 체크를 켜면 “느린 것” 이 도착했을 때 내가 **마지막이 아니라서** 버립니다. `useRef` 에 번호를 두고 비교하는 방법입니다. (10단원의 `useRef`) ★ `useState` 로 하면 안 됩니다. 값이 바로 안 바뀌어서 비교가 어긋납니다.

★ 더 좋은 방법은 섹션 3의 `abort` 입니다. 새 요청을 보낼 때 옛것을 끊으면 낚은 답이 아예 안 옵니다. 09단원 개념05에서 본 그 방법입니다.

➡ 직접 해보기 4

체크를 끈 채로 눌러 “화면에 남은 값” 을 보세요. 그 다음 체크를 켜고 다시 눌러 비교하세요.

지금까지 나온 것을 순서로 묶습니다. **이대로만 하면 대부분 5분 안에 찾습니다.**

```
function Section5Demo() {
  const 순서 = [
    ["1", "서버 터미널을 본다", "요청이 찍히나? 안 찍히면 아예 안 간 것"],
    ["2", "F12 → Console 빨간 줄", "CORS policy 라고 쓰여 있나?"],
    ["3", "F12 → Network 에서 그 요청", "요청 자체가 없으면 주소·서버 문제"],
    ["4", "Status 를 본다", "4xx = 내가 잘못 보냄 / 5xx = 서버 문제"],
    ["5", "Response 본문을 본다", "서버가 이유를 적어 뉘을 수 있다"],
    ["6", "주소창에 직접 넣어 본다", "되면 서버는 멀쩡. React 쪽 문제"],
  ];

  return (
    <div className="demo"> <h3>⑤ 이 순서로 보세요</h3> <div className="output">
      {순서.map(([번, 무엇, 왜]) => (
        <div key={번} style={{ marginBottom: 4 }}> <strong>{번}. {무엇}</strong> <div style={{ fontSize: 13, color: "#666" }}>{왜}</div> </div>
      ))}
    </div> </div>
  );
}
```

★★★ 6번이 제일 강력합니다.

주소창에 `http://localhost:4000/api/v1/equipments` 를 넣어 보세요.

JSON 이 보인다 → 서버는 멀쩡합니다. **React 코드나 CORS** 문제입니다.
 안 보인다 → 서버가 안 켜졌거나 주소가 틀렸습니다.

★ 이 한 번으로 범위가 절반으로 줄입니다. 코드를 고치기 전에 이걸 하세요.

★★ 그리고 주소창은 CORS 가 안 걸립니다. 주소창은 “그 사이트로 가는 것” 이지 “다른 사이트에서 부르는 것” 이 아니기 때문입니다. ★ 그래서 주소창은 되는데 React 에서만 안 되면 CORS 일 가능성이 큼니다.

▸ 직접 해보기 5

서버를 끈 상태와 켜 상태에서 주소창에 넣어 보고 브라우저가 뭐라고 하는지 각각 확인하세요.

자주 하는 실수

섹션 6

이런 실수를 합니다

응답.ok 를 안 봄

404·500 은 catch 로 안 갑니다. (섹션 1)

이런 실수를 합니다

TypeError 를 보고 “서버가 안 켜졌다” 고 단정

CORS 도 똑같이 TypeError 입니다. Network 탭을 보세요. (섹션 1)

이런 실수를 합니다

오류를 `catch {}` 로 삼킴아무 일도 안 일어난 것처럼 보입니다. 원인을 찾을 단서가 없어집니다. ★ 최소한 `console.log(오류)` 는 남기세요.

이런 실수를 합니다

보내는 중에 안 막음 (섹션 2)

이런 실수를 합니다

useEffect 에서 끊지 않음 (섹션 3)

이런 실수를 합니다

낡은 답을 안 버림 (섹션 4)

빠른 컴퓨터에서는 거의 안 보입니다. 그래서 배포하고 나서 발견합니다.

화면

정리

- 1 오류가 두 갈래입니다. 404·500 은 catch 로 안 가고 응답.ok 로 봐야 합니다. 서버가 안 켜진 것과 CORS 는 둘 다 TypeError 라 코드로는 못 가릅니다.
- 2 ★ 서버 안 켜짐과 CORS 는 Network 탭으로 가릅니다. 요청이 아예 없으면 안 켜진 것, 요청은 갔는데 빨가면 CORS 입니다.
- 3 느린 서버에서 사용자는 반드시 두 번 누릅니다. 버튼을 잠그고 함수 첫 줄에서도 막으세요. 확실한 것은 서버가 막는 것입니다.
- 4 화면이 사라질 때 AbortController 로 요청을 끊습니다. useEffect 가 돌려주는 함수에서 `abort()` 를 부릅니다. 결과만 버리는 것보다 낫습니다.
- 5 ★ 요청을 겹쳐 보내면 답이 뒤바뀔 수 있습니다. 화면에 남은 값이 남습니다. useRef 에 번호를 두고 마지막 것만 쓰거나, 새 요청 때 옛것을 끊습니다.
- 6 안 될 때 보는 순서 — 서버 터미널 → Console → Network → Status → Response 본문 → 주소창에 직접.
- 7 ★ 주소창에 직접 넣어 보는 것이 제일 강력합니다. 되면 서버는 멀쩡하고 React 나 CORS 문제입니다. 범위가 절반으로 줄입니다.

```
export default function Concept05Trouble() {
  const [restartKey, setRestartKey] = useState(0);

  return (
    <div> <h1>개념 05 - 안 되는 경우들</h1> <p className="guide"> <strong>일부러
실패시키는 파일입니다.</strong> 오류가 나는 게 정상입니다.
    <strong>F12 → Console 과 Network 를 꼭 열고</strong> 누르세요.
    <br /> <br /> <code>실습프로젝트/14단원_서버</code> 에서 <code>node
서버.js</code> 를 켜 두세요.
    중간에 꺾다 켜 보는 실습도 있습니다.
    <br /> <br /> <strong>⑤번을 종이에 적어 두세요.</strong> PART 3·4 에서도
그대로 씁니다.
    <p> <button onClick={() => setRestartKey(restartKey + 1)}>
    이 예제를 처음부터 다시 그리기
    </button> <div key={restartKey}> <Section1Demo /> <Section2Demo
/> <Section3Demo /> <Section4Demo /> <Section5Demo /> </div> </div>
  );
}
```

직접 해보기 정답

1) Network 탭에서 —

```
정상 / 500 / 없는 주소 → 요청이 **보입니다.** 상태가 200 / 500 / 404
안 켜진 포트          → * 요청이 보이긴 하는데 `(failed)` 로 뜹니다
                        Status 칸이 비어 있습니다
```

```
// 콘솔: 안 켜진 포트 → 못 달았습니다. TypeError: Failed to fetch
```

→ 상태 코드가 **없다**는 것이 핵심입니다. 서버가 답을 안 했으니까요. ★ CORS 도 같은 모양입니다. 그래서 콘솔의 빨간 줄을 같이 봐야 합니다.

2) 체크를 끄고 세 번 누르면 —

```
화면      "서버에 간 횟수: 3"
```

체크를 켜면 —

```
화면      "서버에 간 횟수: 1"
```

→ 두 번째·세 번째 누름은 `if (막을까 && 보내는중) return` 에서 걸렸습니다. ★ 저장하는 요청이었다면 체크 없이는 **세 개가 저장됐**을 것입니다.

```
개념03의 폼에 `disabled={isSubmitting}` 이 있는 이유입니다.
```

3) 1초 안에 숨기면 —

```
// 콘솔: * 화면이 사라져서 요청을 끊었습니다
```

1초를 넘겨 기다린 뒤 숨기면 —

```
// 콘솔: 답이 와서 화면을 고쳤습니다
```

→ 이미 답이 온 뒤라 끊을 것이 없습니다. `abort()` 를 불러도 아무 일 없습니다. ★ Network 탭에서도 확인해 보세요. 끊긴 요청은 (canceled) 로 표시됩니다.

4) 체크를 끄면 — 화면에 남은 값: 느린 것 기록: 느린 것 보냄 → 빠른 것 보냄 → 빠른 것 도착 → 느린 것 도착 → **나중에 도착한 빠른 것이 이겼습니다.**

체크를 켜면 — 화면에 남은 값: 빠른 것 기록: ... → 느린 것 도착 — 낡아서 버림 → ★ 이 버그는 빠른 인터넷에서 거의 안 보입니다.

```
그래서 개발할 때는 멀쩡하다가 현장에서 나옵니다.
**일부러 느리게 만들어 시험해 보는 습관**을 들이세요.
(크롬 Network 탭의 Throttling 으로도 됩니다)
```

5) 서버가 켜져 있으면 —

JSON 이 그대로 보입니다. {"설비들":[{"번호":1,...}]}

꺼져 있으면 —

"사이트에 연결할 수 없음" / ERR_CONNECTION_REFUSED

→ ★ 이 확인이 5초면 끝납니다. React 코드를 읽기 전에 하세요.

"서버가 문제인가, 내가 문제인가" 가 바로 걸립니다.

14단원 ·

연습문제 (12문항)

RUN 터미널 두 개

- ① 실습프로젝트 에서 npm run dev
 - ② 실습프로젝트/14단원_서버 에서 node 서버.js
- F12 → Console 과 Network 를 함께 열어 두세요.

- 1 문제 설명을 읽습니다.
- 2 TODO 자리에 코드를 씁니다.
- 3 저장하면 화면이 저절로 새로 그려집니다. 기대 결과와 맞는지 봅니다.
- 4 막히면 개념 파일의 해당 섹션을 보세요. 문제마다 적어 두었습니다.

문제 1~9 기본 문제 10 [응용] 문제 11 [도전] 문제 12 에러 확인

★ 콘솔에 같은 줄이 두 번씩 찍히는 것은 정상입니다(StrictMode). 기대 결과에는 한 번만 적어 두었습니다.

★★ 서버를 껐다 켜면 설비 목록이 처음 셋으로 돌아갑니다. 연습하다 이름이 겹쳐 409 가 나면 서버를 다시 켜세요.

```
import { useState, useEffect } from "react";
import { useForm } from "react-hook-form";
import Summary from "../_ui/Summary.jsx";
import { 서버주소, 서버안내 } from "../_서버주소.js";
```

문제 1 (개념01 섹션2)

화면이 처음 나타날 때 설비 목록을 받아 와 이름만 로 그리세요. 주소는 \${서버주소}/equipments 입니다. 응답은 { 설비들: [...] } 모양입니다.

기대 화면 3호 프레스 / 5호 컨베이어 / 7호 절단기 세 줄
아무것도 안 나오면 useEffect 를 안 썼거나 .json() 을 빠뜨린 것입니다.
서버를 안 켜면 화면이 계속 비어 있습니다.

```
function Problem01() {
  const [설비들, set설비들] = useState([]);
```

여기에 `useEffect` 를 쓰세요

```
return (  
  <div className="demo"> <h3>문제 1 - 목록 받아 그리기</h3> <ul className="output">  
    {설비들.map((하나) => (  
      <li key={하나.번호}>{하나.이름}</li>  
    ))}  
  </ul> </div>  
);  
}
```

문제 2 (개념01 섹션3)

상태를 셋으로 나누세요. "받는중" 이면 "불러오는 중...", "안됨" 이면 서버안내, "됨" 이면 몇 건인지 보여주세요.

느리게 · 버튼은 `?slow=1200` 을 붙여 부릅니다.

기대 화면 [느리게] 를 누르면 1.2초 동안 "불러오는 중..." 이 보이고
 그 뒤에 "3건 받았습니다" 가 나옵니다.
 누르자마자 결과가 나오면 상태를 안 나눈 것입니다.

```
function Problem02() {  
  const [상태, set상태] = useState("아직");  
  const [말, set말] = useState("");  
  
  async function 불러오기() {
```

상태를 "받는중" 으로 바꾸고, 받아 온 뒤 "됨" 또는 "안됨" 으로 바꾸세요

```

    }

    return (
      <div className="demo"> <h3>문제 2 - 불러오는 중 보여 주기</h3> <button onClick=
{불러오기}><느리게 불러오기</button> <div className="output">
      {상태 === "받는중" ? "불러오는 중..." : 말 || "버튼을 눌러 보세요"}
    </div> </div>
    );
  }
}

```

문제 3 (개념01 섹션3, 개념05 섹션1)

?fail=yes 를 붙여 부르면 서버가 500 을 보냅니다. 그때 서버가 보낸 본문의 메시지를 화면에 보여 주세요.

기대 화면 서버가 잠깐 이상합니다
 "실패했습니다" 같은 내 마음대로 쓴 말이 나오면 본문을 안 읽은 것입니다.
 ★ 500 은 catch 로 안 갑니다. 응답.ok 를 봐야 합니다.

```

function Problem03() {
  const [말, set말] = useState("");

  async function 눌렀을때() {

```

?fail=yes 로 부르고, 응답.ok 가 거짓이면 본문의 메시지를 보여 주세요

```

  }

  return (
    <div className="demo"> <h3>문제 3 - 서버가 보낸 이유 보여 주기
    </h3> <button onClick={눌렀을때}>고장 내기</button> <div className="output">{말 ||
    "버튼을 눌러 보세요"}</div> </div>
  );
}

```

문제 4 (개념02 섹션2)

useForm 으로 이름 하나만 받는 폼을 만드세요. 보내면 콘솔에 값을 찍습니다. 화면 새로고침이 일어나면 안 됩니다.

기대 콘솔 {"이름": "3호 프레스"}
누를 때 페이지가 깜빡이면 handleSubmit 으로 안 감싼 것입니다.

```
function Problem04() {
```

useForm 에서 register 와 handleSubmit 을 꺼내세요

```
  return (  
    <div className="demo"> <h3>문제 4 - useForm 기본</h3>  
    { /* TODO: form 을 만들고 input 에 register 를 붙이세요 */}  
    <div className="output">콘솔(F12)을 보세요</div> </div>  
  );  
}
```

문제 5 (개념02 섹션3)

이름 칸에 규칙을 붙이세요. · 비면 "이름은 꼭 넣어야 합니다" · 두 글자 미만이면 "두 글자 이상 넣으세요" 오류 메시지를 칸 옆에 보여 주세요. 칸을 벗어날 때 검사되게 하세요.

기대 화면 빈 칸에서 다른 곳을 누르면 "이름은 꼭 넣어야 합니다"
한 글자만 넣고 벗어나면 "두 글자 이상 넣으세요"
누를 때까지 아무 말도 없으면 mode 를 안 정한 것입니다.

```
function Problem05() {  
  const {  
    register,  
    handleSubmit,  
    formState: { errors },  
  } = useForm({
```

mode 를 정하세요

```
defaultValues: { 이름: "" },
});

return (
  <div className="demo"> <h3>문제 5 – 검증 규칙 붙이기</h3> <form onSubmit=
{handleSubmit(() => {})}>
    /* TODO: register 에 required 와 minLength 를 넣으세요 */
    <input {...register("이름")} placeholder="3호 프레스" />
    /* TODO: errors.이름 이 있으면 메시지를 보여 주세요 */
    <button type="submit" style={{ marginLeft: 6 }}>
        보내기
    </button> </form> <div className="output">칸을 벗어나 보세요</div> </div>
);
}
```

문제 6 (개념02 섹션4)

보내는 동안 버튼을 잠그고 글자를 "보내는 중..." 으로 바꾸세요. 보내기 함수는 일부러 1초가 걸립니다.

기대 화면 누르면 1초 동안 버튼이 회색이 되고 "보내는 중..." 이 보입니다.
두 번 눌러도 한 번만 실행됩니다.
잠기지 않으면 isSubmitting 을 안 꺼낸 것입니다.

```
function Problem06() {
  const {
    register,
    handleSubmit,
    formState: {
      /* TODO: 여기서 isSubmitting 을 꺼내세요 */
    },
  } = useForm({ defaultValues: { 이름: "" } });

  const [센수, set센수] = useState(0);

  async function 보내기() {
    await new Promise((풀기) => setTimeout(풀기, 1000));
    set센수((전) => 전 + 1);
  }
}
```


부록 타입스크립트

OPEN 15_부록_타입스크립트

```
let count: number = 0;
count = 5; // 숫자니까 됩니다
console.log(count);
return price * count;
```

02 기본 타입.tsx

03 props에 타입 붙이기.tsx

04 useState와 이벤트 타입.tsx

- 연습문제

부록 개념 01 — 왜 타입인가

> 이 파일은 읽는 문서입니다. 코드는 다음 파일부터 나옵니다. > 실행할 것이 없으니 편하게 읽으세요.

13단원까지 오셨습니다. 여기까지 전부 자바스크립트였습니다.

이 부록은 **타입스크립트**를 맛만 봅니다. “이런 것이 있고, 이런 걸 막아 준다” 까지가 전부입니다.

1. 먼저 — 이 부록은 안 해도 됩니다

섹션 1

- 01~13단원 어디에도 타입은 한 번도 안 나왔습니다. **앞 단원은 이 내용을 전혀 전제하지 않습니다.** - 이 부록을 건너뛰어도 앞에서 만든 것은 전부 그대로 돌아갑니다. - 여기서 배운 것을 앞 단원 파일에 적용할 필요도 없습니다.

그럼 왜 있나요?

“다음에 뭘 배우면 되는지”를 한 번 보여 주는 것이 이 부록의 목적입니다. 회사에서 React를 쓰는 곳은 대부분 타입스크립트를 같이 씁니다. 채용 공고에 **React / TypeScript**라고 붙어 있는 것을 본 적이 있을 것입니다. 그때 “아, 그거 한 번 봤다”가 되도록 하는 것이 전부입니다.

분량은 예제 파일 3개와 연습문제 10문항입니다.

2. 이미 겪어 본 사고 세 가지

섹션 2

타입이 무엇을 막아 주는지 설명하기 전에, **여러분이 실제로 당해 본 일**을 먼저 꺼내겠습니다. 전부 JS자료에 있던 것입니다.

① “10” + 5가 105가 된다

JS자료 01단원 개념05(형변환)에 이 줄이 있었습니다.

```
console.log("10" + 5, "10" - 5);
```

```
출력      105 5
```

같은 두 값인데 +는 이어 붙이고 -는 뺍니다. 그때 “왜 10 + 5가 105가 되지?”하고 한참 헤맸을 것입니다.

이런 일이 생기는 이유는 하나입니다. **변수 안에 무엇이 들어 있는지 코드만 봐서는 알 수 없기** 때문입니다.


```
const price = 어딘가에서_받아온_값;
const total = price * 3; // price 가 문자열이면?
```

price 가 숫자인지 문자열인지는 **실행해 봐야** 합니다.

타입스크립트에서는 이렇게 됩니다.

```
const price: number = "10" + 5;
```

error TS2322: Type 'string' is not assignable to type 'number'.

> "string 타입은 number 타입 자리에 넣을 수 없습니다"

```
const priceText = "10";
const total = priceText * 5;
```

error TS2362: The left-hand side of an arithmetic operation must be of type 'any', 'number', 'bigint' or an enum type.

> "곱하기의 왼쪽은 숫자여야 합니다"

실행하기 전에 이 두 줄이 나옵니다. 브라우저를 열기 전입니다.

② dataset 으로 넣은 값이 문자열로 돌아온다

JS자료 11단원 개념05(이벤트 위임)에 이런 코드가 있었습니다.

```
li.dataset.id = nextId; // 숫자 3 을 넣었는데
console.log(li.dataset.id); // 출력: 3 ← 이걸 문자열 "3" 입니다
```

dataset 에 넣은 값은 HTML 속성이 되기 때문에 **무조건 문자열**로 저장됩니다. 숫자를 넣어도 꺼낼 때는 문자열입니다. 그래서 `li.dataset.count * 2` 같은 계산을 하면 조용히 이상한 값이 나옵니다.

타입스크립트에서는 이렇게 됩니다.

```
const total = box.dataset.price * 2;
```

error TS2362: The left-hand side of an arithmetic operation must be of type 'any', 'number', 'bigint' or an enum type. error TS18048: 'box.dataset.price' is possibly 'undefined'.

두 가지를 한 번에 알려 줍니다.

1 문자열이라 곱할 수 없다

2 그런 이름의 속성이 아예 없을 수도 있다(undefined)

②번은 특히 잘 잊어버리는 것입니다. `data-price` 라고 써야 하는데 `data-prices` 라고 쓰면 `undefined` 가 나오고, 그러면 화면에 `NaN` 이 찍힙니다. 에러는 안 납니다.

③ useParams 가 주는 값은 문자열이다

이건 이 자료 11단원 개념03(URL 파라미터) 섹션4에서 다룬 것입니다.

```
const { id } = useParams();
```

주소가 /users/3 이면 id 는 "3" 입니다. 숫자 3 이 아닙니다.

주소창에 있던 글자를 그대로 꺼내 준 것이라 언제나 문자열입니다. 그래서 `id === 3` 은 `false` 이고, `Number(id)` 로 바꿔야 했습니다.

타입스크립트에서는 이렇게 됩니다.

```
const params = useParams();  
const id: number = params.id;
```

error TS2322: Type 'string | undefined' is not assignable to type 'number'. Type 'undefined' is not assignable to type 'number'.

`string | undefined` 라는 표기가 보입니다. "문자열이거나, 아예 없거나 둘 중 하나" 라는 뜻입니다. 없는 주소로 들어오면 `undefined` 가 되기 때문입니다.

세 사고의 공통점은 이것입니다.

> 값의 종류를 잘못 알고 있어서 생긴 일이고, > 셋 다 **애러가 안 나고 조용히 틀렸습니다.**

타입은 이 세 가지를 전부 실행 전에 잡습니다.

3. 타입이 하는 일은 한 가지입니다

섹션 3

> "이 자리에 들어올 수 있는 값의 종류" 를 미리 적어 두는 것. > 그리고 어긋나면 실행하기 전에 알려 주는 것.

그게 전부입니다. 새로운 문법이 잔뜩 생기는 것이 아닙니다. 지금까지 쓰던 자바스크립트에 **설명을 덧붙이는** 것입니다.

```
const price: number = 4000;
```

← 이 부분만 새로 배웁니다

타입이 잡아 주는 것

| 잡아 주는 것 | 예 | --- | --- | | 종류 착각 | "4000" * 3 | | 이름 오타 | `props.nmae, e.preventDefault()` | | 빠뜨린 값 | `<MenuItem name="라떼" />` 인데 `price` 도 필요할 때 | | `null / undefined` | `document.querySelector(...)` 가 못 찾았을 때 |

타입이 못 잡는 것

- **계산 로직이 틀린 것.** 총액을 더하기 대신 빼기로 써도 둘 다 숫자라 타입은 통과합니다. - **화면이 이상한 것.** 글자가 겹쳐 보여도 타입은 아무 말 안 합니다. - **없는 값을 만들어 주지도 않습니다.** 서버가 안 보내 준 데이터가 생기지 않습니다.

덤으로 따라오는 것 — 자동완성

이게 실제로는 가장 크게 체감되는 이득입니다.

```
type Menu = { name: string; price: number };
```

이렇게 적어 두면 VS Code 에서 `menu.` 까지만 쳐도 `name` 과 `price` 가 목록으로 뜹니다. 오타가 날 자리가 없어집니다.

남이 만든 컴포넌트를 쓸 때도, 매개변수 한 줄만 보면 “이 컴포넌트는 무엇을 받는가” 가 그대로 보입니다. 03단원 개념03에서 “매개변수 자리가 설명서 역할을 한다” 고 했던 것, 그 설명서가 더 자세해지는 것입니다.

4. ★ 가장 중요 — 타입 에러는 화면에 안 나옵니다

섹션 4

이 절을 안 읽으면 반드시 해맵니다. 여기만은 꼭 읽으세요.

무슨 일이 일어나나

08단원에서 Vite 로 옮겼습니다. Vite 는 `.tsx` 파일을 브라우저에 보내기 전에 타입을 **지웁니다**. 검사하지 않고, 그냥 지웁니다.

여러분이 이렇게 썼다면

```
const price: number = 4000;
const menu: string[] = ["아메리카노", "라떼"];
```

브라우저로 가는 것은 이것입니다.

```
const price = 4000;
const menu = ["아메리카노", "라떼"];
```

: `number` 와 : `string[]` 은 흔적도 없이 사라집니다. 타입은 **브라우저에 존재하지 않는 것**입니다. 개발할 때만 쓰는 메모입니다.

그래서 이렇게 됩니다.

> **타입이 틀려도 화면은 그냥 돌아갑니다.** > 빨간 화면도, 콘솔 경고도, 아무것도 안 뜹니다.

01~13단원에서는 실수하면 화면이 깨지거나 콘솔에 빨간 줄이 나왔습니다. 그것에 익숙해져 있으면 “타입을 틀리게 썼는데 왜 아무 말도 없지?” 하고 해맽니다. 자기가 맞게 쓴 줄 알고 넘어갑니다. 여기가 초보자가 제일 많이 걸리는 자리입니다.

왜 그렇게 만들어 냈나

타입 검사는 프로젝트 전체를 한 번 훑어야 해서 파일이 많아지면 몇 초씩 걸립니다. 저장할 때마다 그걸 기다리면, 08단원에서 그렇게 좋아했던 “저장하면 바로 화면이 바뀌는 것” 이 느려집니다.

그래서 Vite 는 화면을 띄우는 일만 하고, 검사는 **다른 도구**에 맡깁니다.

그럼 타입 에러는 어디서 보나 — 두 곳뿐입니다

① VS Code 의 빨간 물결 밑줄

틀린 자리에 빨간 물결이 생깁니다. **저장하지 않아도** 바로 나옵니다. 그 위에 마우스를 올리면 메시지가 뜹니다.

```
Type 'string' is not assignable to type 'number'.
```

메모장이나 다른 편집기로 열면 이 밑줄이 안 보입니다. 이 부록은 **VS Code** 로 여는 것을 전제합니다.

② 터미널에서 `npm run typecheck`

실습프로젝트 폴더에서 이렇게 칩니다.

```
npm run typecheck
```

문제가 있으면 이렇게 나옵니다.

```
> -@0.0.0 typecheck
> tsc --noEmit

src/15_부록_타입스크립트/연습문제.tsx(12,23): error TS2322: Type 'string' is not assignable to type 'number'.
```

앞의 두 줄은 npm 이 “이 명령을 실행합니다” 라고 알려 주는 것입니다. 늘 나옵니다.

읽는 법은 이렇습니다.

```
연습문제.tsx(12,23)
|   └─ 23번째 글자
└─ 12번째 줄
```

문제가 없으면 **아무것도 안 나옵니다**.

```
> -@0.0.0 typecheck
> tsc --noEmit
```

조용하면 통과입니다. “성공” 같은 글자는 안 나옵니다. 처음 보면 당황스럽지만 정상입니다.

> `tsc` 는 타입스크립트 검사기이고 `--noEmit` 은 > “파일은 만들지 말고 검사만 해라” 라는 뜻입니다. > 화면을 만드는 일은 Vite 가 하니까 검사만 시키는 것입니다.

이 부록의 예제를 볼 때

타입 에러를 보여 주는 코드는 **전부 주석 처리**해 뒀습니다. 그대로 두면 `npm run typecheck` 가 깨지기 때문입니다.

```
const price: number = "4000";
```

← 주석을 풀면 VS Code 에 빨간 밑줄이 생기고 `npm run typecheck` 가 실패합니다

이런 표시가 보이면 **주석을 풀어서 직접 확인해 보세요**. 빨간 밑줄이 어떻게 생기는지 한 번 보는 것이 설명 열 줄보다 낫습니다. 확인했으면 **다시 //** 를 붙이세요.

5. 이 부록에서 배우는 것 / 안 배우는 것

섹션 5

배우는 것

| 파일 | 내용 | | --- | --- | | 개념02_기본_타입.tsx | `string · number · boolean · 배열 · 객체 · type` 별칭 · 선택 속성 ? · 유니온 | | 개념03_props에_타입_붙이기.tsx | `props` 에 타입 붙이기, 구조분해와 함께 쓰기, `children: React.ReactNode` | | 개념04_useState와_이벤트_타입.tsx | `useState<T>`, 타입 추론이 되는 경우와 안 되는 경우, 이벤트 타입, 자주 나는 에러 메시지 | | 연습문제.tsx / 연습문제_정답.tsx | 10문항 |

안 배우는 것

- 제네릭 직접 만들기 (`function f<T>(x: T)`) - 유틸리티 타입 (`Partial · Pick · Omit · Record`) - `interface` 와 상속 - `enum`, `as` 단언, 타입 좁히기 심화

일부러 뺐습니다. 맛보기에 필요 없는 것들입니다. 실무에서 필요해지면 그때 찾아보면 됩니다. 지금 다 외워도 쓸 자리가 없어서 그냥 잊어버립니다.

6. 파일과 준비물

섹션 6

확장자가 바뀝니다

| 지금까지 | 타입을 쓰려면 | | --- | --- | | `.js` | `.ts` | | `.jsx` | `.tsx` |

이 부록의 예제는 전부 `.tsx` 입니다. **메뉴에는 `.jsx` 와 똑같이 나타납니다**. 왼쪽 목록에서 그냥 고르면 됩니다.

이미 준비돼 있는 것

- 실습프로젝트/tsconfig.json — 타입 검사 설정입니다. **고치지 마세요.** 안에 “strict”: true 가 켜져 있습니다. “엄하게 검사해라” 라는 뜻입니다. - typescript 와 @types/react — npm install 로 이미 깔려 있습니다. - @types/react 는 “React 의 함수들이 무엇을 받고 무엇을 돌려주는지” 를

적어 둔 설명서입니다. 이게 있어서 `useState` 나 이벤트에도 타입이 붙습니다.

- 실습프로젝트/src/15_부록_타입스크립트/_타입선언.d.ts - 정리 상자(_ui/Summary.jsx)가 자바스크립트 파일이라 타입이 없습니다.

“그건 그냥 컴포넌트다” 라고 한 줄 알려 주는 파일입니다.

- 읽지 않아도 됩니다. 없으면 예제가 검사에서 걸려서 넣어 둔 것입니다.

진행 순서

개념02_기본_타입.tsx → 개념03_props에_타입_붙이기.tsx
→ 개념04_useState와_이벤트_타입.tsx
→ 연습문제.tsx

켜는 법

터미널 하나에서 화면을 띄웁니다. 08단원에서 하던 그대로입니다.

```
cd 실습프로젝트
npm run dev
```

터미널을 하나 더 열어서 검사를 돌립니다.

```
cd 실습프로젝트
npm run typecheck
```

npm run dev 는 켜 둔 채로 두고, 검사는 필요할 때마다 다른 창에서 돌리면 됩니다. 지금 한 번 돌려 보세요. 아무것도 안 나오면 준비가 끝난 것입니다.

정리

섹션 7

- 1 이 부록은 안 해도 됩니다. 01~13단원은 전부 자바스크립트로 끝났고, 앞 단원은 타입을 전혀 전제하지 않습니다. “다음에 뭘 배우면 되는지” 를 보여 주는 것이 목적입니다.
- 2 타입은 “이 자리에 들어올 값의 종류” 를 미리 적어 두는 것입니다. 어긋나면 실행하기 전에 알려 줍니다.
- 3 “10” + 5 가 105 가 되던 것, dataset 값이 문자열이던 것, useParams 가 문자열을 주던 것 — 전부 에러 없이 조용히 틀리던 사고였고, 타입은 이 셋을 실행 전에 잡습니다.
- 4 타입이 못 잡는 것도 분명합니다. 계산 로직이 틀린 것, 화면이 이상한 것은 그대로 통과합니다.
- 5 Vite 는 타입을 검사하지 않고 지웁니다. 그래서 타입이 틀려도 화면은 그냥 돌아갑니다. 아무 경고도 안 뜹니다.
- 6 타입 에러를 보는 곳은 두 곳뿐입니다. ① VS Code 의 빨간 물결 밑줄 ② npm run typecheck. 검사는 조용히 끝나면 통과입니다.
- 7 확장자는 .jsx → .tsx 로 바뀝니다. 설정과 라이브러리는 이미 준비돼 있습니다.

기본 타입.tsx

RUN 실습프로젝트에서 npm run dev → 왼쪽 목록에서 이 예제를 고르세요.

15단원(부록) · 개념 02 — 기본 타입

F12 → Console 도 함께 보세요.

개념01에서 읽은 것을 한 줄로 줄이면 이렇습니다.

타입 = "이 자리에 들어올 수 있는 값의 종류" 를 미리 적어 두는 것

이 파일에서는 그 "종류" 를 적는 법을 봅니다. 새로 배우는 문법은 여섯 개뿐입니다.

```
: string      : number      : boolean
: string[]    { name: string } type 별칭
```

★ 이 파일은 VS Code 로 열어 두세요. 타입 에러는 화면에 안 나옵니다. 빨간 물결 밑줄로만 보입니다.
(개념01 4절)

★ 주석으로 막아 둔 코드가 여러 번 나옵니다. "주석을 풀면 ..." 이라고 적힌 것은 직접 풀어서 밑줄을
확인해 보세요. 확인했으면 다시 // 를 붙이세요. 안 붙이면 npm run typecheck 가 실패합니다.
01~07단원과 달리 주석을 풀어도 화면은 멀쩡합니다. Vite 가 타입을 지우고 보내기 때문입니다.

```
import Summary from "../_ui/Summary.jsx";
```

타입을 적는 자리

섹션 1

모양은 이것 하나입니다. 변수 이름 뒤에 콜론을 찍고 종류를 적습니다.

```
const 이름: 타입 = 값;
    └──┘
    여기서 새로 배우는 부분입니다
```

종류 이름은 전부 소문자입니다.

```
const menuName: string = "아메리카노"; // 글자
const menuPrice: number = 4000; // 숫자
const isHot: boolean = true; // 참/거짓 console.log(menuName, menuPrice, isHot);
```

```
F12 콘솔    아메리카노 4000 true
```


값은 지금까지 쓰던 것과 똑같습니다. 앞에 표시만 붙었습니다. JS자료 01단원에서 배운 자료형 세 가지가 그대로 이름이 된 것입니다.

```
string - 따옴표로 감싼 글자
number - 숫자. 정수와 소수를 구분하지 않습니다
```

boolean — true / false

전부 소문자입니다. String 처럼 대문자로 시작하는 것도 있지만 다른 물건이니 쓰지 마세요.

이제 종류가 어긋나면 실행하기 전에 걸립니다.

```
const wrongPrice: number = "4000";
```

← 주석을 풀면 VS Code 에 빨간 밑줄이 생기고 npm run typecheck 가 실패합니다

```
error TS2322: Type 'string' is not assignable to type 'number'.
"string 타입은 number 타입 자리에 넣을 수 없습니다"
```

한 번 정해진 종류는 나중에도 안 바뀝니다.

```
let count: number = 0;
count = 5; // 숫자니까 됩니다
```

```
count = "다섯";
```

← 주석을 풀면 밑줄이 생깁니다

```
error TS2322: Type 'string' is not assignable to type 'number'.
```

```
console.log(count);
```

```
F12 콘솔 5
```

종류를 알면 쓸 수 있는 것도 정해집니다. menuPrice 는 숫자니까 toUpperCase()(대문자 만들기)가 없습니다.

```
console.log(menuPrice.toUpperCase());
```

← 주석을 풀면 밑줄이 생깁니다

```
error TS2339: Property 'toUpperCase' does not exist on type 'number'.
"number 타입에는 toUpperCase 라는 것이 없습니다"
```

JS자료에서 이런 실수를 하면 실행한 뒤에야 TypeError: x.toUpperCase is not a function 을 봤습니다. 이제는 치는 순간 밑줄이 생깁니다. 그 차이 하나가 전부입니다.

▫ 직접 해보기 1

`const menuKcal: number = 0;` 을 만들고 값을 “없음” 으로 바꿔 보세요. 어떤 밑줄이 생기는지 확인한 뒤 다시 숫자로 되돌리세요.

안 적어도 알아냅니다 — 타입 추론

섹션 2

사실 위 세 줄은 `: string` 을 안 적어도 됩니다. 오른쪽 값을 보면 종류가 뻔하기 때문입니다. 이렇게 타입스크립트가 알아서 알아내는 것을 **타입 추론** 이라고 합니다.

```
const cakeName = "케이크"; // 알아서 글자
const cakePrice = 6000; // 알아서 숫자
const cakeTotal = cakePrice * 2; // 알아서 숫자 console.log(cakeName, cakePrice, cakeTotal);
```

F12 콘솔 케이크 6000 12000

적은 것과 똑같이 취급됩니다. 아래도 그대로 걸립니다.

```
console.log(cakeTotal.trim());
```

← 주석을 풀면 밑줄이 생깁니다

```
error TS2339: Property 'trim' does not exist on type 'number'.
```

한 가지 더 · VS Code 에서 각 변수 위에 마우스를 올려 보면 조금씩 다르게 뜹니다.

```
const cakePrice: 6000      ← const 는 다시 대입할 수 없으니 "그 값" 으로 좁게 잡습니다
const cakeTotal: number   ← 계산 결과는 얼마가 나올지 모르니 넓게 잡습니다
```

숫자로 쓰는 데는 아무 차이가 없습니다. 섹션 6의 유니온이 이 성질을 이용한 것입니다.

어쨌든 실무 코드는 생각보다 타입을 덜 적습니다. 값이 바로 옆에 있으면 안 적는 편이 깔끔합니다.

그럼 어디에 적어야 할까요? **값이 옆에 없는 자리** 입니다.

1 함수의 매개변수 — 무엇이 들어올지 함수 안에서는 알 수 없습니다

2 빈 배열이나 `null` 로 시작하는 값 — 볼 값이 없습니다 (개념04에서 다시 봅니다)

1번이 가장 중요합니다. 매개변수는 추론이 안 됩니다.

```
function getTotal(price: number, count: number) {
  return price * count;
}

console.log(getTotal(4000, 3));
```

F12 콘솔 12000

돌려주는 값의 타입은 안 적어도 됩니다. price * count 가 숫자니까 number 로 추론됩니다.

이제 개념01에서 본 그 사고가 여기서 걸립니다.

```
console.log(getTotal("10", 5));
```

← 주석을 풀면 밑줄이 생깁니다

```
error TS2345: Argument of type 'string' is not assignable to parameter of type
'number'.
"string 을 number 자리(매개변수)에 넘길 수 없습니다"
```

매개변수 자리에 타입을 안 적으면 이런 밑줄이 생깁니다.

```
error TS7006: Parameter 'price' implicitly has an 'any' type.
```

“이 매개변수가 뭔지 못 알아내겠다” 는 뜻입니다. tsconfig.json 의 “strict”: true 가 이걸 잡아 줍니다.

▸ 직접 해보기 2

getTotal 을 불러 라떼(4500) 두 잔의 값을 콘솔에 찍어 보세요.

배열

섹션 3

배열은 “무엇이 들어 있는 배열인지” 까지 적습니다. 뒤에 대괄호를 붙입니다.

```
const menuList: string[] = ["아메리카노", "라떼", "케이크"];
const priceList: number[] = [4000, 4500, 6000];

console.log(menuList);
```

F12 콘솔 ['아메리카노', '라떼', '케이크']

```
console.log(priceList);
```

F12 콘솔 [4000, 4500, 6000]

string[] 는 “글자만 들어 있는 배열” 이라는 뜻입니다. 그래서 숫자를 끼워 넣으면 걸립니다.

```
const mixed: string[] = ["아메리카노", 4000];
```

← 주석을 풀면 4000 자리에 밑줄이 생깁니다

```
error TS2322: Type 'number' is not assignable to type 'string'.
```

여기서 진짜 이득이 나옵니다. **꺼내 쓸 때도 종류를 압니다.**

```
const lengths = menuList.map((name) => name.length);  
  
console.log(lengths);
```

```
F12 콘솔    [5, 2, 3]
```

map 의 name 에는 타입을 안 적었는데도 string 입니다. menuList 가 string[] 이니 안에 든 것은 string 일 수밖에 없기 때문입니다. VS Code 에서 name 위에 마우스를 올리면 (parameter) name: string 이라고 뜹니다.

반대로 priceList 는 숫자 배열이라 글자 메소드가 없습니다.

```
const trimmed = priceList.map((n) => n.trim());
```

← 주석을 풀면 밑줄이 생깁니다

```
error TS2339: Property 'trim' does not exist on type 'number'.
```

▹ 직접 해보기 3

priceList 의 값을 전부 10% 올린 배열을 만들어 콘솔에 찍어 보세요. (JS자료 08단원 map 그대로입니다)

객체와 type 별칭

섹션 4

객체는 속성마다 종류를 적습니다. 중괄호 안에 줄줄이 적으면 됩니다.

```
const americano: { name: string; price: number } = {  
  name: "아메리카노",  
  price: 4000,  
};  
  
console.log(americano);
```

```
F12 콘솔    { name: '아메리카노', price: 4000 }
```

그런데 메뉴가 세 개면 이 긴 것을 세 번 적어야 합니다. 그래서 **이름을 붙여 줍니다**. 이것을 타입 별칭이라고 합니다.

```
type Menu = { name: string; price: number };
```

읽는 법: "앞으로 Menu 라고 쓰면 { name: string; price: number } 를 뜻한다"

```
type Menu = { ... };
```

키워드 이름 내용

이름은 대문자로 시작하는 것이 관례입니다. 컴포넌트 이름과 같은 규칙입니다.

```
const latte: Menu = { name: "라떼", price: 4500 };
const cake: Menu = { name: "케이크", price: 6000 };

console.log(latte, cake);
```

```
F12 콘솔 { name: '라떼', price: 4500 } { name: '케이크', price: 6000 }
```

배열에도 그대로 씁니다. Menu[] 는 "Menu 가 여러 개 든 배열" 입니다.

```
const menus: Menu[] = [
  { name: "아메리카노", price: 4000 },
  { name: "라떼", price: 4500 },
  { name: "케이크", price: 6000 },
];

const sum = menus.reduce((acc, m) => acc + m.price, 0);

console.log(sum);
```

```
F12 콘솔 14500
```

reduce 의 m 도 자동으로 Menu 입니다. m. 까지만 쳐도 name·price 가 목록으로 뜹니다.

어긋나면 세 가지 밑줄이 생깁니다. 메시지가 각각 다릅니다.

```
const noPrice: Menu = { name: "물" };
```

← 주석을 풀면 밑줄이 생깁니다 — 빠뜨렸을 때

```
error TS2741: Property 'price' is missing in type '{ name: string; }' but required in
type 'Menu'.
```

```
const textPrice: Menu = { name: "물", price: "0" };
```

← 주석을 풀면 밑줄이 생깁니다 — 종류가 다를 때

```
error TS2322: Type 'string' is not assignable to type 'number'.
```

```
const extra: Menu = { name: "물", price: 0, hot: true };
```

← 주석을 풀면 밑줄이 생깁니다 — 없는 속성을 넣었을 때

```
error TS2353: Object literal may only specify known properties, and 'hot' does not exist in type 'Menu'.
```

세 번째가 특히 고맙습니다. 03단원 개념03 [실수 2] 에서 nmae 라고 잘못 써도 아무 말 없이 `undefined` 가 됐던 것, 그게 이제 밑줄로 잡힙니다.

⇒ 직접 해보기

4

`type User = { name: string; age: number };` 를 만들고 김민준(20) 을 담은 변수를 하나 만들어 콘솔에 찍어 보세요.

선택 속성 — 있어도 되고 없어도 되는 것

섹션 5

03단원 개념03에서 `props` 에 기본값을 줬습니다. 안 넘어와도 되는 값이 있었습니다. 타입에서도 “이건 없어도 된다” 를 표시할 수 있습니다. 이를 뒤에 물음표를 붙입니다.

```
type Person = {  
  name: string;  
  nickname?: string; // ← 있어도 되고 없어도 됩니다  
};  
  
const minjun: Person = { name: "김민준" }; // 없어도 통과  
const seoyeon: Person = { name: "이서연", nickname: "서니" }; // 있어도  
통과 console.log(minjun.nickname);
```

```
F12 콘솔    undefined
```

```
console.log(seoyeon.nickname);
```

```
F12 콘솔    서니
```

물음표를 붙이면 그 속성의 종류가 이렇게 바뀝니다.

```
nickname?: string → string | undefined
"글자이거나, 아예 없거나"
```

개념01에서 useParams 가 주던 string | undefined 와 같은 모양입니다.

그래서 그냥 쓰면 안 됩니다. 없을 수도 있으니까요.

```
console.log(minjun.nickname.length);
```

← 주석을 풀면 밀줄이 생깁니다

```
error TS18048: 'minjun.nickname' is possibly 'undefined'.
"이거 없을 수도 있는데요?"
```

이 밀줄이 막아 주는 것이 바로 JS자료에서 제일 많이 보던 그 에러입니다.

```
TypeError: Cannot read properties of undefined (reading 'length')
```

확인하고 쓰면 밀줄이 사라집니다. 방법은 JS자료 02단원 개념04에서 배운 ?? 그대로입니다.

```
const nick1 = minjun.nickname ?? "별명 없음"; // ?? 는 null·undefined 일 때만 오른쪽
(JS자료 02단원 개념04)
const nick2 = seoyeon.nickname ?? "별명 없음";

console.log(nick1, nick2);
```

```
F12 콘솔    별명 없음 서니
```

⇒ 직접 해보기 5

Person 에 age?: number 를 추가하고 박지훈(28) 을 담은 변수를 만들어 콘솔에 찍어 보세요.

유니온 — 정해진 것 중 하나

섹션 6

사이즈처럼 “이 셋 중 하나여야 하는” 값이 있습니다. string 이라고 적으면 아무 글자나 다 들어갑니다. 오타를 못 잡습니다. 세로줄(|)로 이어 쓰면 그 목록 중 하나만 받게 됩니다.

```
type Size = "S" | "M" | "L";

const mySize: Size = "M";

console.log(mySize);
```

```
F12 콘솔    M
```

세로줄은 “또는” 이라고 읽습니다. JS자료 02단원 개념04의 || 와 같은 뜻입니다. 값 자체를 타입으로 쓴 것이라 다른 글자는 못 들어옵니다.

```
const bigSize: Size = "XL";
```

← 주석을 풀면 밑줄이 생깁니다

```
error TS2322: Type '"XL"' is not assignable to type 'Size'.
```

소문자 “m” 도 안 됩니다. 적어 둔 셋과 글자 하나까지 같아야 합니다.

유니온은 이런 자리에 씁니다. - 버튼 종류: “primary” | “danger” - 화면 상태: “로딩” | “성공” | “실패” ← 09단원 개념04에서 state 세 개로 나눠 두던 것 - 정렬 방향: “asc” | “desc”

VS Code 에서 따옴표를 열면 셋이 목록으로 떠서 고르기만 하면 됩니다.

세로줄로는 종류도 섞을 수 있습니다. string | number 는 “글자이거나 숫자” 입니다. 개념01에서 본 string | undefined 도 같은 모양이었습니다.

타입 별칭과 함께 쓰면 이런 것도 됩니다.

```
type Order = { menu: Menu; size: Size; memo?: string };
```

```
const myOrder: Order = { menu: latte, size: "L" };
```

```
console.log(myOrder);
```

```
F12 콘솔 { menu: { name: '라떼', price: 4500 }, size: 'L' }
```

▹ 직접 해보기

6

type Pay = “카드” | “현금”, 을 만들고 “카드” 를 담은 변수를 만들어 콘솔에 찍어 보세요. 그 다음 “포인트” 로 바꿔 보고 밑줄을 확인하세요.

자주 하는 실수

섹션 7

이런 실수를 합니다

물음표와 undefined 를 헷갈림

nickname?: string 은 “속성이 아예 없어도 된다” 는 뜻입니다. { name: “김민준” } 처럼 아예 안 적는 것이 정상이고, { name: “김민준”, nickname: undefined } 라고 적을 필요는 없습니다.

이런 실수를 합니다

선택 속성을 확인 없이 씀 ★ 가장 자주 납니다

섹션 5의 minjun.nickname.length 가 그것입니다. error TS18048: '...' is possibly 'undefined'. ?? 로 기본값을 주거나 if 로 확인하고 쓰세요. 밑줄이 사라집니다.

이런 실수를 합니다

타입이 실행 중에도 검사해 줄 거라고 믿음 ★ 에러 없이 조용히 틀립니다

개념01 4절에서 본 그대로입니다. 타입은 브라우저에 존재하지 않습니다. 아래 코드는 밑줄이 하나도 없고 typecheck 도 통과합니다. 그런데 결과가 틀립니다.

```
function addTip(m: Menu): number {
  return m.price + 500; // 타입만 보면 4500 + 500 = 5000 이어야 합니다
}
```

서버에서 받은 값이라고 칩시다. `JSON.parse` 는 무엇이 나올지 모르니 검사하지 않습니다.

```
const fromServer = JSON.parse('{"name":"라떼","price":"4500"}');

console.log(addTip(fromServer));
```

F12 콘솔 4500500

price 가 문자열 "4500" 이라 + 가 이어 붙이기가 됐습니다. JS자료 01단원 그대로입니다. 타입은 "여기 숫자가 온다" 고 적어 둔 메모일 뿐, 실행 중에 진짜 숫자인지 확인해 주지는 않습니다.

그래서 이렇게 정리하면 됩니다.

내가 쓴 코드끼리 어긋나는 것 → 타입이 잡아 줍니다
바깥에서 들어온 값 → 타입은 못 잡습니다. 직접 확인해야 합니다

화면에 그리기 —————

정리

- 1 타입은 변수 이름 뒤에 콜론을 찍고 적습니다. `const price: number = 4000`; 종류 이름은 전부 소문자입니다.
- 2 값이 옆에 있으면 안 적어도 알아냅니다(타입 추론). 꼭 적어야 하는 곳은 함수 매개변수입니다.
- 3 배열은 `string[] · number[]` 처럼 씁니다. 꺼내 쓸 때도 종류를 알아서 `map` 안의 값에 자동완성이 붙습니다.
- 4 객체 타입에 이름을 붙인 것이 `type` 별칭입니다. `type Menu = { name: string; price: number };` 처럼 씁니다.
- 5 이름 뒤에 `?` 를 붙이면 없어도 되는 속성이 됩니다. 대신 종류가 `string | undefined` 가 되어 확인 없이 못 씁니다.
- 6 세로줄(`|`)로 이으면 정해진 것 중 하나만 받습니다. `type Size = "S" | "M" | "L";`
- 7 타입은 브라우저에 존재하지 않습니다. 서버에서 온 값처럼 바깥에서 들어온 것은 타입이 못 잡습니다.

```
export default function Concept02BasicTypes() {
  const sizeLabel: Size = "L";

  return (
    <div> <h1>개념 02 – 기본 타입</h1> <p className="guide">
      이 화면은 위 코드가 만든 값을 보여 줄 뿐입니다. <strong>진짜 볼 것은
      코드입니다.</strong> <br />
      VS Code 로 이 파일을 열고, 주석으로 막아 둔 줄을 하나씩 풀어 보세요. 빨간
      물결
      밑줄이 어떻게 생기는지 보는 것이 이 파일의 목적입니다.
      <br /> <br /> <strong>주석을 풀어도 이 화면은 안 깨집니다.</strong> Vite 가
      타입을 지우고 보내기
      때문입니다. 확인이 끝나면 다시 <code>///</code> 를 붙이세요.
      </p> <div className="demo"> <h3>① 기본 타입 세 가지
    </h3> <div className="output">
      {menuName} / {menuPrice}원 / 뜨거움: {String(isHot)}
    </div> </div> <div className="demo"> <h3>② 배열 – string[] 과 number[]
    </h3> <div className="output">
      메뉴: {menuList.join(", ")}
      <br />
      가격: {priceList.join(", ")}
      <br />
      이름 길이: {lengths.join(", ")}
    </div> </div>
```

```

<div className="demo">
  <h3>③ 객체와 type 별칭</h3> <ul>
    {menus.map((m) => (
      <li key={m.name}>
        {m.name} - {m.price}원
      </li>
    ))}
  </ul> <div className="output">합계: {sum}원</div>
</div>

<div className="demo"> <h3>④ 선택 속성 - 없어도 되는 값
</h3> <div className="output">
  {minjun.name} ({nick1})
  <br />
  {seoyeon.name} ({nick2})
</div> </div> <div className="demo"> <h3>⑤ 유니온 - 정해진 것 중 하나
</h3> <div className="output">
  주문: {myOrder.menu.name} / 사이즈 {myOrder.size} / 내 사이즈 {sizeLabel}
</div> </div> <div className="demo"> <h3>⑥ 실수 3 - 타입은 실행 중에
검사하지 않습니다</h3> <div className="output">
  addTip 이 number 를 돌려준다고 적혀 있는데 화면에는 <strong>
{addTip(fromServer)}</strong>{" "}
  이 나옵니다.
  <br />
  서버에서 온 price 가 문자열이라 <code>+</code> 가 이어 붙이기가 됐습니다.
</div>

</div>
);
}

```

직접 해보기 정답

```
1) const menuKcal: number = 0;
```

menuKcal 을 "없음" 으로 바꾸면(let 으로 바꾼 뒤 menuKcal = "없음");

```
// error TS2322: Type 'string' is not assignable to type 'number'.
```

→ const 인 채로 초기값만 "없음" 으로 바뀌어도 같은 밑줄이 생깁니다.

메시지가 안 뜨면 VS Code 가 아니라 다른 편집기로 연 것입니다.

```
2) console.log(getTotal(4500, 2));  
// 콘솔: 9000
```

→ getTotal("4500", 2) 라고 따옴표를 붙이면 이 밀줄이 생깁니다.

```
error TS2345: Argument of type 'string' is not assignable to parameter of type  
'number'.
```

```
3) console.log(priceList.map((p) => p * 1.1));  
// 콘솔: [4400, 4950, 6600.000000000001]
```

→ 마지막 값만 소수점이 지저분한 것은 타입과 상관없는 일입니다.

JS자료 01단원에서 본 0.1 + 0.2 문제와 같은 이유입니다.
p 에 타입을 안 적어도 number 로 자동 추론됩니다. priceList 가 number[] 이기
때문입니다.

```
4) type User = { name: string; age: number };  
const minjunUser: User = { name: "김민준", age: 20 };  
console.log(minjunUser);  
// 콘솔: { name: '김민준', age: 20 }
```

→ age 를 빼면 이 밀줄이 생깁니다.

```
error TS2741: Property 'age' is missing in type '{ name: string; }' but required in  
type 'User'.
```

```
5) type Person = {
```

```
  name: string;  
  nickname?: string;  
  age?: number;
```

```
};  
const jihun: Person = { name: "박지훈", age: 28 };  
console.log(jihun);  
// 콘솔: { name: '박지훈', age: 28 }
```

→ nickname 을 안 적어도 통과합니다. 물음표가 붙어 있기 때문입니다.

```
jihun.age * 2 를 하면 밀줄이 생깁니다. age 가 number | undefined 이기 때문입니다.  
error TS18048: 'jihun.age' is possibly 'undefined'.
```

```
6) type Pay = "카드" | "현금";  
const myPay: Pay = "카드";  
console.log(myPay);  
// 콘솔: 카드
```

→ "포인트" 로 바꾸면 이 밑줄이 생깁니다.

```
error TS2322: Type '"포인트"' is not assignable to type 'Pay'.  
화면은 그대로 잘 돌아갑니다. 밑줄로만 보입니다.
```

props에 타입 붙이기.tsx

RUN 실습프로젝트에서 npm run dev → 왼쪽 목록에서 이 예제를 고르세요.

15단원(부록) · 개념 03 — props 에 타입 붙이기

F12 → Console 도 함께 보세요.

03단원 개념03에서 이렇게 썼습니다.

```
function MenuItem({ name, price }) {
  return <p>{name} - {price}원</p>;
}
```

그때 이런 말을 했습니다.

"매개변수만 봐도 이 컴포넌트가 무엇을 받는지 보입니다.
남이 만든 컴포넌트를 읽을 때 이 한 줄이 설명서 역할을 합니다."

그런데 이 설명서에는 빠진 것이 있습니다. **이름만 있고 종류가 없습니다.** price 가 숫자인지 문자열인지, 안 넘겨도 되는지는 안 적혀 있습니다.

이 파일은 그 한 줄을 완성합니다. 새로 배우는 것은 두 개뿐입니다.

```
type Props = { ... }   그리고   ({ name, price }: Props)
```

★ VS Code 로 열어 두세요. 타입 에러는 화면에 안 나옵니다. (개념01 4절) ★ "주석을 풀면 ..." 이라고 적힌 것은 직접 풀어서 밑줄을 확인하고 다시 붙이세요.

```
import { useState } from "react";
import Summary from "../_ui/Summary.jsx";
```

타입을 안 붙이면 어떻게 되나

섹션 1

.tsx 파일에서 03단원 코드를 그대로 쓰면 밑줄이 생깁니다.

```
function NoTypeItem({ name, price }) {
  return (
```

```
<p>
  {name} - {price}원
</p>
```

```
);
}
```

← 주석을 풀면 name 과 price 에 각각 밑줄이 생깁니다

```
error TS7031: Binding element 'name' implicitly has an 'any' type.
error TS7031: Binding element 'price' implicitly has an 'any' type.
```

읽는 법은 이렇습니다.

Binding element 구조분해로 꺼낸 것 implicitly 내가 안 적었는데 저절로 has an 'any' type 아무거나 다 되는 종류가 됐다

합치면 “구조분해로 꺼낸 name 이 뭔지 못 알아내겠다” 입니다.

왜 못 알아낼까요? 개념02 섹션2에서 본 그대로입니다. **함수 매개변수는 추론이 안 됩니다.** 무엇이 들어올지 함수 안에서는 볼 수가 없습니다.

any 는 “아무거나” 라는 뜻의 특별한 타입입니다. any 가 되면 검사를 아예 안 합니다. 타입스크립트를 쓰는 의미가 사라집니다. 그래서 tsconfig.json 의 “strict”: true 가 “그건 안 된다” 고 막아 주는 것입니다.

▫ 직접 해보기 1

위 NoTypeItem 의 주석을 풀고, name 에 마우스를 올려 무슨 메시지가 뜨는지 읽어 보세요. 확인했으면 다시 주석을 붙이세요.

type 으로 설명서 적기

섹션 2

개념02 섹션4의 타입 별칭을 그대로 씁니다. 달라지는 것은 쓰는 자리뿐입니다.

```
type MenuItemProps = {
  name: string;
  price: number;
};

function MenuItem({ name, price }: MenuItemProps) {
  return (
    <p>
      {name} - {price}원
    </p>
  );
}
```

모양을 뜯어 보면 이렇습니다.

```
function MenuItem({ name, price }: MenuItemProps) {
  //
  03단원에서 배운 구조분해      여기가 새로
  구조분해 그대로 붙은 부분
}
```

구조분해는 하나도 안 바뀌었습니다. 닫는 중괄호 뒤에 콜론과 타입 이름만 붙었습니다.

props 는 객체 하나라고 했습니다(03단원 개념02). 그러니 “그 객체가 어떻게 생겼는지” 를 적어 주는 것이고, 그게 개념02에서 배운 객체 타입 별칭입니다. 새 문법이 아닙니다.

쓰는 쪽은 이렇게 됩니다.

```
<MenuItem name="아메리카노" price={4000} />
```

넘기는 쪽 코드는 03단원과 완전히 똑같습니다. 한 글자도 안 바뀝니다.

이제 세 가지가 잡힙니다. 메시지가 각각 다릅니다.

```
<MenuItem name="라떼" />
```

← 빠뜨렸을 때

```
error TS2741: Property 'price' is missing in type '{ name: string; }' but required in
type 'MenuItemProps'.
```

```
<MenuItem name="라떼" price="4500" />
```

← 종류가 다를 때 (숫자 자리에 문자열)

```
error TS2322: Type 'string' is not assignable to type 'number'.
```

```
<MenuItem nmae="라떼" price={4500} />
```

← 이름을 틀렸을 때

```
error TS2322: Type '{ nmae: string; price: number; }' is not assignable to type
'IntrinsicAttributes & MenuItemProps'.
  Property 'nmae' does not exist on type 'IntrinsicAttributes & MenuItemProps'.
```

세 번째가 03단원 개념03 [실수 2] 바로 그것입니다. 그때는 nmae 라고 잘못 써도 아무 말 없이 화면에 빈칸만 나왔습니다. 이제는 넘기는 쪽에 밑줄이 생깁니다.

(IntrinsicAttributes 는 React 가 key 같은 것을 위해 몰래 끼워 넣는 자리입니다. 우리가 적은 것이 아니니 그냥 넘어가세요. & 는 “그리고” 라는 뜻입니다.)


```
console.log("MenuItem 은 name(string) 과 price(number) 를 받습니다");
```

```
F12 콘솔    MenuItem 은 name(string) 과 price(number) 를 받습니다
```

▹ 직접 해보기 2

MenuItemProps 에 kcal: number 를 추가해 보세요. 화면에 그려 둔 <MenuItem ... /> 세 곳에 전부 밀줄이 생깁니다. 확인했으면 되돌리세요.

기본값과 함께 쓰기

섹션 3

03단원 개념03 섹션3의 기본값도 그대로 됩니다. 타입은 그 뒤에 붙습니다.

```
type GreetingProps = {
  name: string;
};

function Greeting({ name = "손님" }: GreetingProps) {
  return <p>{name}님, 어서 오세요.</p>;
}
```

그런데 여기에 함정이 하나 있습니다. 기본값을 줬는데도 **안 넘기면 밀줄이 생깁니다.**

```
<Greeting />
error TS2741: Property 'name' is missing in type '{}' but required in type
'GreetingProps'.
```

기본값과 타입은 서로 모르는 사이이기 때문입니다.

```
기본값  - 안 넘어왔을 때 함수 안에서 대신 쓸 값
타입    - 넘기는 쪽이 지켜야 할 약속
```

기본값을 줬다는 것은 “안 넘겨도 된다” 는 뜻이니, 타입에도 그렇게 적어야 합니다. 그 표시가 개념02 섹션5의 물음표입니다.

```
type GreetingProps2 = {
  name?: string; // ← 안 넘겨도 됩니다
};

function Greeting2({ name = "손님" }: GreetingProps2) {
  return <p>{name}님, 어서 오세요.</p>;
}
```

이제 둘 다 됩니다.

```
<Greeting2 name="이서연" /> → 이서연님, 어서 오세요.  
<Greeting2 /> → 손님님, 어서 오세요.
```

그리고 여기에 이득이 하나 더 있습니다. 물음표를 붙이면 name 은 string | undefined 가 되는데, 기본값이 있으니 함수 안에서는 그냥 string 입니다. 밀줄이 안 생깁니다. 기본값을 안 주면 함수 안에서 name.length 같은 것을 쓸 때 밀줄이 생깁니다.

▫ 직접 해보기

3

MenuItemProps 의 price 를 price?: number 로 바꾸고 함수 쪽도 { name, price = 0 } 으로 고쳐 보세요. 그러면 <MenuItem name="물" /> 이 밀줄 없이 통과합니다. 확인했으면 되돌리세요.

안 넘겨도 되는 props 를 다루는 법

섹션 4

물음표를 붙였지만 기본값은 안 주고 싶을 때가 있습니다. "넘어오면 보여 주고, 안 넘어오면 아무것도 안 그린다" 같은 경우입니다.

```
type BadgeProps = {  
  text: string;  
  note?: string; // 안 넘겨도 됩니다  
};  
  
function Badge({ text, note }: BadgeProps) {  
  return (  
    <span>  
      [{text}]{note ? ` (${note})` : ""}  
    </span>  
  );  
}
```

note 는 string | undefined 입니다. 그래서 그냥 note.length 라고 쓰면 밀줄이 생깁니다.

```
function BadBadge({ note }: BadgeProps) {  
  return <span>{note.length}</span>;  
}
```

← 주석을 풀면 밀줄이 생깁니다

```
error TS18048: 'note' is possibly 'undefined'.
```

위 Badge 처럼 삼항으로 확인하고 쓰면 밀줄이 사라집니다. 05단원 개념01의 조건부 렌더링을 그대로 쓴 것뿐입니다. 타입스크립트는 "if 나 삼항으로 확인한 뒤" 라는 것을 알아봅니다.


```

    {children}
  </div>
);
}

```

▫ 직접 해보기

5

화면 ㉔ 에 <Card title="빈 상자" /> 를 하나 더 넣어 보세요. 밀줄 없이 통과하는지 확인하세요.

객체·배열·함수를 props 로 받기

섹션 6

03단원 개념03 섹션5에서 객체를 통째로 넘겼습니다. 타입도 그대로 씁니다.

```

type Menu = { name: string; price: number };

type MenuCardProps = {
  menu: Menu; // 개념02에서 만든 타입을 그대로 씁니다
};

function MenuCard({ menu }: MenuCardProps) {
  return (
    <div className="output"> <strong>{menu.name}</strong> - {menu.price}원
    </div>
  );
}

```

목록을 통째로 넘길 때는 대괄호를 붙입니다. 05단원 개념02의 map 그대로입니다.

```

type MenuListProps = {
  menus: Menu[];
};

function MenuList({ menus }: MenuListProps) {
  return (
    <ul>
      {menus.map((m) => (
        <li key={m.name}>
          {m.name} - {m.price}원
        </li>
      ))}
    </ul>
  );
}

```

map 안의 m 에는 타입을 안 적었는데도 Menu 입니다. menus 가 Menu[] 이기 때문입니다. m. 까지만 쳐 보세요. name 과 price 가 목록으로 뜹니다. 그래서 m.name 같은 오타가 그 자리에서 밀줄로 잡힙니다.

07단원 개념04에서 함수를 props 로 내려보냈습니다. 그것도 타입을 적을 수 있습니다.

```
type AddButtonProps = {
  label: string;
  onAdd: () => void; // "받는 것 없고 돌려주는 것도 없는 함수"
};

function AddButton({ label, onAdd }: AddButtonProps) {
  return (
    <button type="button" onClick={onAdd}>
      {label}
    </button>
  );
}
```

() => void 는 "()" 매개변수 없음 + "=>" 화살표 함수 모양 + "void" 돌려주는 값 없음 입니다. 값을 하나 받는 함수라면 (name: string) => void 라고 적습니다. 이 정도면 충분합니다.

▸ 직접 해보기 6

AddButtonProps 의 onAdd 를 (name: string) => void 로 바꾸고 아래 handleAdd 도 이름을 받아 콘솔에 찍게 고쳐 보세요.

자주 하는 실수

섹션 7

이런 실수를 합니다

타입을 매개변수 안쪽에 붙임 ★ 가장 자주 납니다

```
function Bad1({ name: string, price: number }) { ... }
```

밀줄이 잔뜩 생깁니다. 이건 타입이 아니라 03단원 개념03 섹션4의 **이름 바꾸기** 입니다. "name 을 string 이라는 변수로 받겠다" 가 됩니다. 컴포넌트가 받는 것은 객체 하나이니(03단원 개념02) 타입도 객체 하나에 대해 적습니다. 그러니 닫는 중괄호 **바깥**에 붙입니다. ({ name, price }: MenuItemProps) └ 여기입니다

이런 실수를 합니다

기본값을 줬으니 안 넘겨도 되는 줄 알 ★ 자주 납니다

섹션 3에서 본 그대로입니다. 기본값과 물음표는 다른 것입니다. 기본값을 줬으면 타입에도 물음표를 붙이세요.

이런 실수를 합니다

children 을 타입에 안 적음

```
type Bad3Props = { title: string };  
function Bad3({ title, children }: Bad3Props) { ... }
```

children 자리에 밑줄이 생깁니다. error TS2339: Property 'children' does not exist on type 'Bad3Props'. React 가 알아서 넣어 주지 않습니다. 쓸 것이면 직접 적어야 합니다.

이런 실수를 합니다

타입만 고치고 넘기는 쪽을 안 고침 ★ 에러가 두 곳에 생깁니다

Props 에 속성을 추가하면, 그 컴포넌트를 쓰는 모든 자리에 밑줄이 생깁니다. 놀라지 마세요. 그게 타입의 목적입니다. “여기도 고쳐야 한다” 를 전부 찾아 주는 것이라, 오히려 빠뜨릴 일이 없어집니다. VS Code 왼쪽 아래 문제 탭을 열면 고칠 곳이 목록으로 나옵니다.

화면에 그리기

```
const menus: Menu[] = [  
  { name: "아메리카노", price: 4000 },  
  { name: "라떼", price: 4500 },  
  { name: "케이크", price: 6000 },  
];
```

정리

- 1 props 타입은 type Props = { ... } 로 적고, 구조분해 뒤에 붙입니다. ({ name, price }: MenuItemProps) 처럼 닫는 중괄호 바깥입니다.
- 2 .tsx 에서 타입을 안 붙이면 밑줄이 생깁니다. error TS7031: Binding element 'name' implicitly has an 'any' type.
- 3 넘기는 쪽 코드는 03단원과 한 글자도 안 바뀝니다. 받는 쪽에 설명서 한 줄이 붙는 것뿐입니다.
- 4 빠뜨림·종류 다름·이름 오타가 전부 넘기는 쪽에서 잡힙니다. 03단원에서 조용히 undefined 가 되던 오타가 밑줄이 됩니다.
- 5 기본값과 물음표는 다른 것입니다. 기본값을 줬으면 타입에도 name?: string 처럼 물음표를 붙여야 안 넘길 수 있습니다.
- 6 children 은 React.ReactNode 로 적습니다. ‘화면에 그릴 수 있는 것 아무거나’ 라는 뜻입니다. 안 적으면 children 을 못 씁니다.
- 7 객체는 Menu, 목록은 Menu[], 함수는 () ⇒ void 로 적습니다. map 안의 값에는 타입이 저절로 붙습니다.

```
export default function Concept03PropsTypes() {
```

useState 에 타입을 붙이는 것은 개념04에서 봅니다. 여기서는 그냥 씁니다.

```
const [cart, setCart] = useState(0);
```

담기를 누르면 콘솔에 이렇게 찍힙니다.

```
function handleAdd() {  
  setCart((prev) => prev + 1);  
  console.log("담기를 눌렀습니다");  
}
```

```
F12 콘솔    담기를 눌렀습니다
```

```

}

return (
  <div>
    <h1>개념 03 – props 에 타입 붙이기</h1>

    <p className="guide">
      화면은 03단원 개념03·04와 거의 같습니다. <strong>달라진 것은 코드뿐입니다.
    </strong>
    <br />
    VS Code 로 이 파일을 열고, 컴포넌트 이름 위에 마우스를 올려 보세요. 무엇을
    받는지
    그대로 뜹니다.
  </p>

  <div className="demo">
    <h3>① type 으로 설명서를 적은 컴포넌트</h3>
    <div className="output">
      <MenuItem name="아메리카노" price={4000} />
      <MenuItem name="라떼" price={4500} />
      <MenuItem name="케이크" price={6000} />
    </div>
  </div>

  <div className="demo">
    <h3>② 기본값 – 물음표가 있어야 안 넘길 수 있습니다</h3>
    <div className="output">
      <Greeting name="이서연" />
      <Greeting2 name="김민준" />
      <Greeting2 />
    </div>
  </div>

  <div className="demo">
    <h3>③ 선택 속성 – 넘어왔을 때만 보이기</h3>
    <div className="output">
      <Badge text="아메리카노" note="뜨거움" />
      <br />

```



```

        <Badge text="삼각김밥" />
      </div>
    </div>

    <div className="demo"> <h3>④ children – 태그 사이에 넣은 것
  </h3> <Box title="오늘의 메뉴"> <p>아메리카노가 제일 잘 나갑니다.
  </p> </Box> <Card title="안이 비어도 되는 상자"
  /> </div> <div className="demo"> <h3>⑤ 객체·배열·함수를 props 로
  </h3> <MenuCard menu={menus[1]} /> <MenuList menus={menus}
  /> <div className="output"> <AddButton label="담기" onAdd={handleAdd} />
      장바구니: {cart}개
    </div> </div>

  </div>
);
}

```

직접 해보기 정답

1) 주석을 풀면 name 과 price 양쪽에 빨간 물결이 생깁니다. 마우스를 올리면 이렇게 뜹니다.

Binding element 'name' implicitly has an 'any' type.

→ 화면은 멀쩡합니다. 이 상태로 npm run typecheck 를 돌리면 실패합니다.

다시 주석을 붙이면 통과합니다.

2) type MenuItemProps = {

```

name: string;
price: number;
kcal: number;

```

```
};
```

→ 화면 ① 의 <MenuItem /> 세 개에 전부 이 밑줄이 생깁니다.

error TS2741: Property 'kcal' is missing in type '{ name: string; price: number; }' but required in type 'MenuItemProps'.

실수 4에서 말한 그것입니다. 고칠 곳을 전부 찾아 준 것입니다.

kcal?: number 로 물음표를 붙이면 밑줄이 한 번에 사라집니다.

3) type MenuItemProps = {

```
name: string;
price?: number;
```

```
};
function MenuItem({ name, price = 0 }: MenuItemProps) { ... }
// 화면: 물 - 0원
```

→ price = 0 을 안 주고 price?: number 만 붙이면

{price}원 자리는 밑줄 없이 통과하지만(그냥 그리기만 하니까)
화면에는 "물 - 원" 이 나옵니다. undefined 는 화면에 안 그려집니다(02단원).

4) type BadgeProps = {

```
text: string;
note?: string;
count?: number;
```

```
};
function Badge({ text, note, count }: BadgeProps) {
```

```
return (
  <span>
```

```
{text} · {note ? ({note}) : ""}{count ? x${count} : ""}
```

```
</span>
);
```

```
}
```

쓸 때: <Badge text="아메리카노" note="뜨거움" count={3} />

```
// 화면: [아메리카노] (뜨거움) x3
```

→ count 를 안 넘기면 x 부분이 안 나옵니다.

count && `x\${count}` 라고 쓰면 count 가 0 일 때 화면에 0 이 찍힙니다.
05단원 개념05의 falsy 함정 그대로입니다. 삼항이 안전합니다.

5) <Card title="빈 상자" />

```
// 화면: 빈 상자 (제목만 나오고 아래는 비어 있습니다)
```

→ CardProps 의 children 에 물음표가 있어서 통과합니다.

Box 로 바꿔서 <Box title="빈 상자" /> 라고 하면 밀줄이 생깁니다.
 error TS2741: Property 'children' is missing in type '{ title: string; }' but required in type 'BoxProps'.

6) type AddButtonProps = {

label: string;
 onAdd: (name: string) => void;

};
 function AddButton({ label, onAdd }: AddButtonProps) {

return (
 <button type="button" onClick={() => onAdd("아메리카노")}>
 {label}
 </button>
);

}
 function handleAdd(name: string) {

setCart((prev) => prev + 1);
 console.log("담기를 눌렀습니다:", name);

}
 // 콘솔(누르면): 담기를 눌렀습니다: 아메리카노

→ onClick={onAdd} 로 그대로 두면 밀줄이 생깁니다.

onClick 은 함수에 이벤트 객체를 넘기는데, onAdd 는 문자열을 기다리기 때문입니다.
 그래서 () => onAdd("아메리카노") 로 감싸야 합니다. 04단원 개념01에서 배운 그
 모양입니다.

useState와 이벤트 타입.tsx

RUN 실습프로젝트에서 npm run dev → 왼쪽 목록에서 이 예제를 고르세요.

15단원(부록) · 개념 04 — useState 와 이벤트 타입

F12 → Console 도 함께 보세요.

개념02에서 값에, 개념03에서 props 에 타입을 붙였습니다. 남은 것은 화면을 움직이게 하는 두 가지입니다. state 와 이벤트입니다.

이 파일에서 배우는 것은 세 가지입니다.

- 1 useState 는 대부분 알아서 알아냅니다 — 안 알아내는 두 경우가 있습니다
- 2 못 알아낼 때 useState<타입> 으로 알려 줍니다
- 3 이벤트 매개변수에는 이름이 긴 타입을 적습니다 — 외우지 않고 찾는 법을 봅니다

★ VS Code 로 열어 두세요. 타입 에러는 화면에 안 나옵니다. (개념01 4절) ★ “주석을 풀면 ...” 이라고 적힌 것은 직접 풀어서 밑줄을 확인하고 다시 붙이세요.

```
import { useState } from "react";
import Summary from "../_ui/Summary.jsx";

type Menu = { name: string; price: number };

const menus: Menu[] = [
  { name: "아메리카노", price: 4000 },
  { name: "라떼", price: 4500 },
  { name: "케이크", price: 6000 },
];
```

대부분은 알아서 알아냅니다

섹션 1

04단원에서 쓰던 그대로입니다. 타입을 한 글자도 안 적어도 됩니다.

```
const [count, setCount] = useState(0);
```

개념02 섹션2의 타입 추론이 여기서도 일합니다. 초기값이 0 이니 count 는 number 이고, setCount 는 number 를 받는 함수가 됩니다.

VS Code 에서 count 위에 마우스를 올려 보세요.

```
const count: number
```

라고 씁니다. 안 적었는데도 이미 정해져 있습니다.

그래서 이런 것이 잡힙니다. (실제 코드는 아래 CounterDemo 안에 있습니다)

```
setCount("3");
error TS2345: Argument of type 'string' is not assignable to parameter of type
'SetStateAction<number>'.
```

SetStateAction<number> 라는 낯선 이름이 보입니다. "setCount 에 넣을 수 있는 것" 이라는 뜻이고, 두 가지를 뜻합니다.

```
setCount(3)          숫자를 그대로
setCount((prev) => prev+1)  이전 값을 받아 새 값을 돌려주는 함수 ← 04단원 개념05
```

04단원에서 배운 두 가지 방식이 그대로 한 이름에 들어 있는 것입니다. 이름이 길어서 그렇지 새로운 이야기가 아닙니다.

```
function CounterDemo() {
  const [count, setCount] = useState(0); // 초기값 0 → number
  function handlePlus() {
    setCount((prev) => prev + 1);
```

```
setCount("3");
```

← 주석을 풀면 밑줄이 생깁니다

```
error TS2345: Argument of type 'string' is not assignable to parameter of type
'SetStateAction<number>'.
```

```
}

return (
  <div className="output">
    담긴 잔 수: {count}
    <br /> <button type="button" onClick={handlePlus}>
      한 잔 담기
    </button> </div>
  );
}
```

문자열·불리언도 똑같습니다.

```
const [text, setText] = useState("");          → string
const [isOpen, setIsOpen] = useState(false); → boolean
```

초기값만 보고 다 알아냅니다. 여기까지는 타입스크립트를 쓴다는 느낌도 안 납니다.

▫ 직접 해보기 1

CounterDemo 안에 `const [isHot, setIsHot] = useState(true);` 를 넣고 `setIsHot("뜨거움")` 을 써 보세요. 어떤 밀줄이 생기는지 확인하세요.

못 알아내는 두 경우

섹션 2

문제는 초기값에 볼 것이 없을 때 생깁니다. 두 가지가 있습니다.

경우 1 · 빈 배열로 시작할 때

```
const [items, setItems] = useState([]);
```

06단원 개념05에서 목록을 만들 때 늘 이렇게 시작했습니다. 그런데 빈 배열에는 안에 아무것도 없습니다. 무엇이 들어올 배열인지 알 수가 없습니다.

이때 타입스크립트는 `never[]` 라고 정합니다. `never` 는 “아무것도 될 수 없는 것” 이라는 뜻입니다. 아무것도 못 넣는 배열이 되어 버립니다.

```
function BrokenList() {
  const [items, setItems] = useState([]);
  function add() {

    setItems(["아메리카노"]);

  }
  return <p>{items.map((item) => item.name)}</p>;
}
```

← 위 함수의 주석을 풀면(아래 두 줄은 그대로 두세요) 밀줄이 두 군데 생깁니다

```
"아메리카노" 자리: error TS2322: Type 'string' is not assignable to type 'never'.
item.name 자리:    error TS2339: Property 'name' does not exist on type 'never'.
```

“type ‘never’” 라는 말이 나오면 십중팔구 `useState([])` 입니다. 처음 보면 뜻을 알 수 없는 메시지인데, 원인은 늘 같습니다.

경우 2 · null 로 시작할 때

```
const [user, setUser] = useState(null);
```

09단원에서 “아직 안 받아온 상태” 를 `null` 로 두던 그 코드입니다. 초기값이 `null` 이니 타입스크립트는 “이건 언제나 `null` 인 값” 이라고 정합니다. 그래서 나중에 진짜 값을 넣으면 걸립니다.

```
function BrokenUser() {
  const [user, setUser] = useState(null);
  function load() {
```

```
    setUser({ name: "김민준" });
```

```
  }
  return <p>{String(user)}</p>;
}
```

← 위 함수의 주석을 풀면 { name: “김민준” } 자리에 밑줄이 생깁니다

```
error TS2353: Object literal may only specify known properties, and 'name' does not
exist in type '(prevState: null) => null'.
```

두 경우의 공통점은 하나입니다. 초기값만 봐서는 앞으로 무엇이 들어올지 알 수 없다. 그러면 우리가 직접 말해 주면 됩니다. 그게 다음 섹션입니다.

⇒ 직접 해보기 2

위 BrokenList 의 주석을 풀고 밑줄 두 개를 눈으로 확인하세요. 확인했으면 다시 주석을 붙이세요.

useState<타입> 으로 알려 주기

섹션 3

useState 이름 뒤에 꺾쇠를 붙이고 그 안에 타입을 적습니다.

```
useState<string[]>([])
  └──┬──┘
      "글자가 들어올 배열이다"
```

꺾쇠 안에 타입을 적는 이 모양은 다른 데서도 자주 보게 됩니다. 지금은 “useState 에 알려 주는 자리” 로만 알아 두면 됩니다.

```
function TodoDemo() {
  const [todos, setTodos] = useState<string[]>([]); // 글자 목록
  const [text,
```

```

setText] = useState(""); // 이걸 추론됩니다 const [picked, setPicked] = useState<Menu
| null>(null); // 메뉴 하나 또는 아직 없음 function handleAdd() {
  if (text.trim() === "") return;
  setTodos([...todos, text]); // 07단원 개념01 그대로 setText("");
}

function handleReset() {
  setTodos([]);
  setPicked(null);
  console.log("비웠습니다");
}

```

F12 콘솔 비웠습니다

```

}

return (
  <div className="output"> <input value={text} onChange={e =>
setText(e.target.value)}
  placeholder="할 일을 적으세요"
  />
  <button type="button" onClick={handleAdd}>
    추가
  </button> <button type="button" onClick={handleReset}>
    비우기
  </button> <ul>
    {todos.map((todo, index) => (
      <li key={index}>{todo}</li>
    ))}
  </ul> <div>
    {menus.map((m) => (
      <button key={m.name} type="button" onClick={() => setPicked(m)}>
        {m.name}
      </button>
    ))}
  </div>

  {/* picked 는 Menu | null 이라 확인하고 써야 합니다 */}
  {picked === null ? <p>고른 메뉴가 없습니다.</p> : <p>고른 메뉴: {picked.name}</p>}
</div>
);
}

```

todos 는 이제 string[] 입니다. map 안의 todo 도 string 이라 todo.toUpperCase() 가 됩니다. 숫자를 넣으려 하면 밀줄이 생깁니다.


```
setTodos([...todos, 4000]);
error TS2322: Type 'string | number' is not assignable to type 'string'.
  Type 'number' is not assignable to type 'string'.
error TS2322: Type 'number' is not assignable to type 'string'.
```

밀줄이 두 군데 생깁니다. 배열 전체에 하나, 4000 자리에 하나입니다. 고칠 곳은 한 곳(4000)이고, 나머지 하나는 그 때문에 배열 전체가 어긋난 것입니다.

picked 는 Menu | null 입니다. “메뉴 하나 또는 아직 없음” 입니다. 개념02 섹션6의 유니온이 여기서 제 일을 합니다. 그리고 확인 없이 쓰면 막습니다.

```
<p>{picked.name}</p>
error TS18047: 'picked' is possibly 'null'.
```

이 밀줄이 막아 주는 것이 JS자료 10단원에서 제일 많이 보던 그 예러입니다.

```
TypeError: Cannot read properties of null (reading 'name')
```

위 코드처럼 picked === null 을 먼저 확인하면 밀줄이 사라집니다. 삼항의 오른쪽에서는 “여기까지 왔으면 null 이 아니다” 를 타입스크립트가 알아봅니다.

```
console.log("todos 는 string[] 이고 picked 는 Menu | null 입니다");
```

```
F12 콘솔    todos 는 string[] 이고 picked 는 Menu | null 입니다
```

▹ 직접 해보기 3

TodoDemo 안에 `const [prices, setPrices] = useState<number[]>([]);` 를 넣고 `setPrices([...prices, "4000"])` 을 써 보세요. 밀줄을 확인하세요.

입력칸의 이벤트 타입

섹션 4

06단원 개념01에서 제어 컴포넌트를 배웠습니다.

```
<input value={text} onChange={handleChange} />
```

핸들러를 따로 빼면 매개변수 e 에 타입이 필요합니다.

```
function handleChange(e) {
  setText(e.target.value);
}
```

← 주석을 풀면 밀줄이 생깁니다

error TS7006: Parameter 'e' implicitly has an 'any' type.
개념03 섹션1의 그 밑줄과 같은 이야기입니다. 매개변수는 추론이 안 됩니다.

그럼 뭐라고 적어야 할까요? 이렇게 적습니다.

```
React.ChangeEvent<HTMLInputElement>  
| | |  
React 의 바뀔 이벤트 어느 태그에서 났나
```

이름이 길니다. **외우지 마세요.** 찾는 방법이 있습니다.

1. 일단 화살표 함수로 그 자리에 바로 씁니다. `onChange={(e) => ...}`

이러면 `React` 가 무슨 이벤트인지 아니까 타입이 저절로 붙습니다.

2 e 위에 마우스를 올립니다. 타입 이름이 그대로 뜹니다.

3 그걸 복사해서 함수를 밖으로 뺄 때 붙입니다.

실무에서도 이렇게 합니다. 아무도 안 외웁니다.

```
function InputDemo() {  
  const [text, setText] = useState("");  
  const [size, setSize] = useState("M");
```

입력칸에서 난 바뀔 이벤트

```
function handleText(e: React.ChangeEvent<HTMLInputElement>) {  
  setText(e.target.value);  
}
```

고르기 칸에서 난 바뀔 이벤트 — 태그 이름이 다릅니다

```
function handleSize(e: React.ChangeEvent<HTMLSelectElement>) {  
  setSize(e.target.value);  
}  
  
return (  
  <div className="output"> <input value={text} onChange=  
    {handleText} placeholder="이름을 적으세요" /> <select value={size} onChange=  
    {handleSize}> <option value="S">S</option> <option value="M">M</option> <option value=  
    "L">L</option> </select> <p>  
    {text === "" ? "(비어 있음)" : text} / 사이즈 {size}  
  </p> </div>
```

```
);
}
```

꺾쇠 안이 왜 필요할까요? **e.target** 이 무엇인지 정해 주기 때문입니다.

HTMLInputElement → e.target.value 가 있습니다. 입력칸이니까요 HTMLSelectElement → 고르기 칸입니다 HTMLButtonElement → 버튼입니다

태그를 잘못 적으면 넘기는 자리에서 걸립니다.

```
<select onChange={handleText}>
error TS2322: Type '(e: ChangeEvent<HTMLInputElement, Element>) => void' is not
assignable to type 'ChangeEventHandler<HTMLSelectElement, HTMLSelectElement>'.
```

이 메시지는 아래로 몇 줄 더 이어집니다. **첫 줄만 읽으면 됩니다.** "input 용 함수를 select 자리에 넣었다" 는 뜻입니다. 태그 이름을 HTMLSelectElement 로 바꾸면 사라집니다.

◀ 직접 해보기 4

InputDemo 의 <select> 에 onChange={handleText} 를 넣어 보세요. 첫 줄만 읽어 보고 다시 handleSize 로 되돌리세요.

form 과 버튼의 이벤트 타입

섹션 5

06단원 개념04의 품입니다. submit 이벤트에는 다른 이름을 씁니다.

```
React.FormEvent
```

입력칸의 e 는 "무엇이 입력됐나" 를 알아야 해서 태그까지 적었지만, submit 은 e.preventDefault() 만 부르는 경우가 대부분이라 꺾쇠를 안 붙여도 됩니다.

```
function FormDemo() {
  const [name, setName] = useState("");
  const [saved, setSaved] = useState<string[]>([]);

  function handleSubmit(e: React.FormEvent) {
    e.preventDefault(); // 06단원 개념04 · JS자료 11단원 console.log("form 01 submit
    됐습니다");
  }
}
```

```
F12 콘솔    form 01 submit 됐습니다
```

```
if (name.trim() === "") return;
setSaved([...saved, name]);
setName("");
}
```

버튼 클릭은 MouseEvent 입니다

```
function handleClear(e: React.MouseEvent<HTMLButtonElement>) {  
  console.log("누른 버튼의 글자:", e.currentTarget.textContent);
```

F12 콘솔 누른 버튼의 글자: 지우기

```
setSaved([]);  
}  
  
return (  
  <form onSubmit={handleSubmit}> <input value={name} onChange={(e) =>  
setName(e.target.value)}  
  placeholder="이름을 적으세요"  
  />  
  <button type="submit">저장</button> <button type="button" onClick=  
{handleClear}>  
    지우기  
  </button> <ul>  
    {saved.map((s, index) => (  
      <li key={index}>{s}</li>  
    ))}  
  </ul> </form>  
)  
);  
}
```

FormEvent 에서 e.target.value 를 꺼내려고 하면 걸립니다.

```
function BadSubmit(e: React.FormEvent) {  
  console.log(e.target.value);  
}
```

← 주석을 풀면 밀줄이 생깁니다

```
error TS2339: Property 'value' does not exist on type 'EventTarget'.
```

submit 이벤트의 target 은 form 태그이지 입력칸이 아니기 때문입니다. 06단원 개념04에서 “입력값은 state 에서 꺼내 쓴다” 고 한 이유가 이것입니다. 위 handleSubmit 도 e 가 아니라 name 을 씁니다.

오타도 잡아 줍니다. 이건 특히 고맙습니다.

```
e.preventDefault();  
error TS2551: Property 'preventDefault' does not exist on type 'FormEvent<Element>'.  
Did you mean 'preventDefault'?
```

맨 뒤에 “Did you mean ‘preventDefault’?” 까지 붙습니다. 06단원에서 이 오타를 내면 새로그침이 일어나 화면이 초기화됐고, 원인을 찾느라 한참 걸렸습니다. 이제는 치는 순간 밀줄입니다.

👉 직접 해보기 5

FormDemo 의 `e.preventDefault()` 를 `e.preventDefualt()` 로 바꿔 보세요. Did you mean 이 뜨는지 확인하고 되돌리세요.

자주 하는 실수

섹션 6

이런 실수를 합니다

useState([]) 로 시작하고 이유를 모름 ★ 가장 자주 납니다

섹션 2의 그것입니다. "type 'never'" 가 보이면 `useState([])` 를 찾으세요. `useState<string[]>([])` 처럼 무엇이 들어올 배열인지 적어 주면 끝납니다.

이런 실수를 합니다

useState(null) 로 시작하고 이유를 모름

섹션 2의 두 번째입니다. `useState<Menu | null>(null)` 이라고 적으세요. "메뉴 하나 또는 아직 없음" 이라는 뜻입니다.

이런 실수를 합니다

null 검사를 안 하고 씬

error TS18047: 'picked' is possibly 'null'. 귀찮아 보이지만 이게 09단원에서 "받아오기 전에 화면이 먼저 그려져서" 터지던 그 사고를 통째로 막아 줍니다. if 나 삼항으로 먼저 확인하세요.

이런 실수를 합니다

입력값이 문자열이라는 것을 잊음 ★ 개념01의 그 사고입니다

`e.target.value` 는 언제나 문자열입니다. 숫자 입력칸이어도 문자열입니다.

```
function NumberDemo() {
  const [count, setCount] = useState(0); // number
  function handleCount(e:
    React.ChangeEvent<HTMLInputElement>) {
```

```
    setCount(e.target.value);
```

← 주석을 풀면 밑줄이 생깁니다

```
error TS2345: Argument of type 'string' is not assignable to parameter of type
  'SetStateAction<number>'.
```

```

setCount(Number(e.target.value)); // 이렇게 바꿔서 넣어야 합니다
}

return (
  <div className="output"> <input type="number" value={count} onChange=
{handleCount} /> <p>두 배: {count * 2}</p> </div>
);
}

```

type="number" 를 써도 e.target.value 는 문자열입니다. HTML 속성이라 그렇습니다. 개념01 2절의 dataset 이야기와 완전히 같은 이유입니다. JS자료에서는 이걸 잊으면 화면에 "00" 이나 NaN 이 조용히 찍혔습니다.

이런 실수를 합니다

메시지가 길다고 안 읽음

타입 에러 메시지는 아래로 길게 이어질 때가 많습니다. 거의 언제나 첫 줄에 답이 있습니다. 아래는 자세한 설명입니다. 첫 줄을 읽고 모르겠으면 그때 아래를 보세요.

화면에 그리기

정리

- 1 useState 는 초기값을 보고 타입을 알아냅니다. useState(0) 이면 number, useState("") 면 string 입니다.
- 2 빈 배열 useState([]) 는 never[] 가 되어 아무것도 못 넣습니다. 'type never' 가 보이면 이걸 의심하세요.
- 3 null 로 시작한 useState(null) 도 마찬가지입니다. 나중에 진짜 값을 넣을 때 걸립니다.
- 4 못 알아낼 때는 useState([]) · useState(null) 처럼 꺾쇠 안에 적어 줍니다.
- 5 Menu | null 로 적으면 확인 없이 못 씁니다. error TS18047: 'picked' is possibly 'null'. 09단원에서 터지던 사고를 막아 줍니다.
- 6 이벤트 타입은 React.ChangeEvent 처럼 겁니다. 외우지 말고 화살표 함수로 먼저 쓴 뒤 e 에 마우스를 올려 복사하세요.
- 7 submit 은 React.FormEvent 입니다. e.target.value 는 없습니다. 입력값은 state 에서 꺼내 씁니다.
- 8 e.target.value 는 type='number' 여도 문자열입니다. Number() 로 바꿔서 넣어야 합니다.

```
export default function Concept04StateAndEvents() {
  return (
    <div> <h1>개념 04 - useState 와 이벤트 타입</h1> <p className="guide">
      화면은 04·06단원에서 만든 것과 같습니다. <strong>달라진 것은 코드뿐입니다.
    </strong> <br />
      VS Code 로 이 파일을 열고
    <code>count</code>·<code>todos</code>·<code>picked</code> 위에
      마우스를 올려 보세요. 타입이 어떻게 정해졌는지 그대로 뜹니다.
    </p> <div className="demo"> <h3>① 추론되는 state - useState(0)
  </h3> <CounterDemo /> </div> <div className="demo"> <h3>② 알려 줘야 하는 state -
  useState<string[]>([]) 와 useState<Menu | null>(null)</h3> <TodoDemo
  /> </div> <div className="demo"> <h3>③ 입력칸과 고르기 칸의 이벤트</h3> <InputDemo
  /> </div> <div className="demo"> <h3>④ form 의 submit 과 버튼의 클릭</h3> <FormDemo
  /> </div> <div className="demo"> <h3>⑤ 실수 4 - 입력값은 언제나 문자열입니다
  </h3> <NumberDemo /> </div>

    </div>
  );
}
```

직접 해보기 정답

```
1) const [isHot, setIsHot] = useState(true);
setIsHot("뜨거움");
// error TS2345: Argument of type '"뜨거움"' is not assignable to parameter of type
'SetStateAction<boolean>'.
```

→ 초기값이 true 라서 boolean 으로 정해졌습니다.

섹션 1의 number 짜리와 메시지 모양이 같고 뒤쪽 이름만 boolean 으로 바뀌었습니다.
 앞쪽이 'string' 이 아니라 '"뜨거움"' 으로 나오는 것은
 내가 적은 그 값을 그대로 보여 주는 것뿐입니다. 읽는 데는 차이가 없습니다.

2) 밑줄 두 개가 이렇게 뜹니다.

```
// error TS2322: Type 'string' is not assignable to type 'never'.
// error TS2339: Property 'name' does not exist on type 'never'.
```

→ 고치는 법은 useState<string[]>([]) 처럼 무엇이 들어올 배열인지 적는 것입니다.

그러면 두 밑줄이 한 번에 사라집니다.
 (두 번째 줄은 item.name 을 쓰려던 것이니 Menu[] 가 맞겠지요)

```
3) const [prices, setPrices] = useState<number[]>([]);
setPrices([...prices, "4000"]);
// error TS2322: Type 'string | number' is not assignable to type 'number'.
//   Type 'string' is not assignable to type 'number'.
// error TS2322: Type 'string' is not assignable to type 'number'.
```

→ 섹션 3에서 본 그대로 밀줄이 두 군데 생깁니다. 고칠 곳은 “4000” 하나뿐입니다.

따옴표를 빼고 `setPrices([...prices, 4000])` 이라고 하면 둘 다 사라집니다.
개념01 2절의 “10” + 5 사고를 state 에서 막아 주는 것이 이것입니다.

```
4) <select value={size} onChange={handleText}>
// error TS2322: Type '(e: ChangeEvent<HTMLInputElement, Element>) => void' is not
assignable to type 'ChangeEventHandler<HTMLSelectElement, HTMLSelectElement>'.
```

→ 아래로 네 줄쯤 더 이어지는데, 마지막 줄에는

`HTMLSelectElement` 에 없는 속성 이름이 잔뜩 나옵니다. 읽을 필요 없습니다.
첫 줄의 `HTMLInputElement` 와 `HTMLSelectElement` 두 단어만 보면 원인이 보입니다.

```
5) e.preventDefault();
// error TS2551: Property 'preventDefault' does not exist on type
'FormEvent<Element>'. Did you mean 'preventDefault'?
```

→ 화면에서 저장을 누르면 페이지가 새로고침되면서 입력한 것이 다 날아갑니다.

06단원에서 이 사고가 났을 때는 원인을 찾기가 어려웠습니다.
이제는 저장 버튼을 누르기 전에 밀줄로 먼저 보입니다.

.tsx

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

15단원(부록) · 연습문제 (10문항)

F12 → Console 도 함께 여세요.

- 1 아래로 내려가며 // TODO 를 찾아 코드를 고칩니다.
- 2 저장하면 화면이 저절로 바뀝니다(Vite). F5 를 누르지 않아도 됩니다.
- 3 각 문제의 '기대 결과' 와 화면을 비교합니다.
- 4 막히면 개념01~04 파일의 해당 섹션을 다시 보세요.

순서는 기본 7문제 → [응용] → [도전] → 에러 확인 입니다. 뒤로 갈수록 어렵습니다. 정답은 연습문제_정답.tsx 에 있습니다. 먼저 스스로 해 보세요.

★ 이 단원만의 두 가지를 꼭 기억하세요.

1. 타입 에러는 화면에 안 나옵니다. 화면이 멀쩡해도 틀렸을 수 있습니다.

VS Code 의 빨간 물결 밑줄과 아래 명령으로만 보입니다.

```
npm run typecheck
```

이 파일은 ****고치기 전** 지금 상태에서 이미 통과**합니다.
고치다가 밑줄이 생기면 그게 힌트입니다. 메시지를 읽고 고치세요.

2. 문제를 다 풀고 나면 반드시 `npm run typecheck` 를 한 번 더 돌려세요.

아무것도 안 나오면 통과입니다.

★ 고치기 전 지금 화면에는 "(아직 안 고쳤습니다)" 같은 글자가 보입니다. 정상입니다.

```
import { useState } from "react";
import Summary from "../_ui/Summary.jsx";
```

문제 1 (개념02)

가격이 문자열이라 더하기가 이어 붙이기가 되고 있습니다. JS자료 01단원의 그 사고입니다. `q1Price`의 타입을 `number` 로 바꾸고 값도 숫자로 고치세요.

기대 화면 $4000 + 500 = 4500$
4000500 이 그대로면 아직 문자열입니다.
타입만 number 로 바꾸고 값에 따옴표를 남겨 두면 밀줄이 생깁니다.
error TS2322: Type 'string' is not assignable to type 'number'.

아래 두 곳(타입과 값)을 고치세요

```
const q1Price: string = "4000";  
  
const q1Total = q1Price + 500;
```

문제 2 (개념02)

q2Menu 에 “글자가 들어 있는 배열” 이라는 타입을 붙이고, “케이크” 를 하나 더 넣으세요.

기대 화면 아메리카노 / 라떼 / 케이크 (3개)
2개로 나오면 아직 “케이크” 를 안 넣은 것입니다.
타입을 제대로 붙였는지 확인하려면 배열에 숫자 4000 을 잠깐 넣어 보세요.
밀줄이 생기면 잘 붙인 것입니다. 확인했으면 다시 지우세요.
error TS2322: Type 'number' is not assignable to type 'string'.

타입을 붙이고 “케이크” 를 추가하세요

```
const q2Menu = ["아메리카노", "라떼"];
```

문제 3 (개념02)

타입 별칭 Q3Menu 에 가격을 추가하세요. 가격은 숫자이고 반드시 있어야 합니다.

기대 화면 라떼 - 4500원
 "라떼 - 0원" 이 그대로면 아직 안 고친 것입니다.
 타입에만 추가하고 q3Latte 에 값을 안 넣으면 밑줄이 생깁니다.
 error TS2741: Property 'price' is missing in type '{ name: string; }' but required in type 'Q3Menu'.

타입에 price 를 추가하고, q3Latte 에도 4500 을 넣고,

아래 q3PriceText 를 q3Latte 의 가격으로 바꾸세요

```
type Q3Menu = { name: string };

const q3Latte: Q3Menu = { name: "라떼" };

const q3PriceText = 0;
```

문제 4 (개념02)

별명은 있어도 되고 없어도 되는 값입니다. 지금은 반드시 있어야 하게 돼 있어서 김민준에게 억지로 빈 문자열을 넣어 뒀습니다. nickname 을 선택 속성으로 바꾸고, 김민준에게서는 nickname 을 아예 지우세요.

기대 화면 김민준(별명 없음) / 이서연(서니)
 "김민준()" 처럼 괄호 안이 비면 아직 빈 문자열이 들어 있는 것입니다.
 ?? 는 값이 null 이나 undefined 일 때만 오른쪽을 씁니다.
 빈 문자열은 "있는 값" 이라 그대로 씁니다. (03단원 개념03 섹션3과 같은 함정)

nickname 을 선택 속성으로 바꾸고 q4Minjun 에서 nickname 을 지우세요

```
type Q4Person = { name: string; nickname: string };

const q4Minjun: Q4Person = { name: "김민준", nickname: "" };
const q4Seoyeon: Q4Person = { name: "이서연", nickname: "서니" };
```

문제 5 (개념02)

사이즈는 S · M · L 셋 중 하나여야 합니다. 지금은 string 이라 아무 글자나 들어갑니다. 유니온으로 바꾸세요. 그리고 q5MySize 를 "L" 로 고치세요.

기대 화면 내 사이즈: L
타입만 바꾸고 "라지" 를 그대로 두면 밑줄이 생깁니다.
error TS2322: Type '"라지"' is not assignable to type 'Q5Size'.

Q5Size 를 "S" | "M" | "L" 로 바꾸고 q5MySize 도 고치세요

```
type Q5Size = string;  
  
const q5MySize: Q5Size = "라지";
```

문제 6 (개념03)

Q6Item 이 가격도 받아서 보여 주게 하세요. 고칠 곳은 세 군데입니다. 타입 · 컴포넌트 · 쓰는 쪽입니다.

기대 화면 아메리카노 - 4000원
라떼 - 4500원
타입에만 price 를 추가하면 쓰는 쪽 두 군데에 밑줄이 생깁니다.
error TS2741: Property 'price' is missing in type '{ name: string; }' but required in type 'Q6Props'.

Q6Props 에 price: number 를 추가하고 화면에도 보이게 하세요

```

type Q6Props = { name: string };

function Q6Item({ name }: Q6Props) {
  return (
    <p>
      {name} - (아직 안 고쳤습니다)
    </p>
  );
}

```

문제 7 (개념03)

Q7Box 가 태그 사이에 넣은 내용을 보여 주게 하세요. children 의 타입 이름은 개념03 섹션5에 있습니다.

기대 화면 오늘의 메뉴
아메리카노가 제일 잘 나갑니다.
타입에 children 을 안 적고 컴포넌트에서 꺼내 쓰면 밑줄이 생깁니다.
error TS2339: Property 'children' does not exist on type 'Q7Props'.

Q7Props 에 children 을 추가하고, 제목 아래에 children 을 그리세요.

그리고 아래 화면 그리는 곳의 <Q7Box ... /> 안에 내용을 넣으세요.

```

type Q7Props = { title: string };

function Q7Box({ title }: Q7Props) {
  return (
    <div className="output"> <strong>{title}</strong> <p>(아직 안 고쳤습니다)
  </p> </div>
  );
}

```

문제 8 [응용] (개념04)

장바구니를 완성하세요. 두 군데를 고칩니다.

(1) q8Change 의 매개변수 e 가 지금 any 로 돼 있습니다.

any 는 "아무거나" 라서 검사를 아예 안 합니다.
입력칸의 바뀜 이벤트 타입으로 바꾸세요. (개념04 섹션4)

(2) q8Add 가 아무 일도 안 합니다.

q8Text 가 비어 있지 않으면 목록에 넣고 입력칸을 비우세요.

기대 화면 입력칸에 "아메리카노" 를 적고 담기를 누르면
아래 목록에 "아메리카노" 한 줄이 늘어나고 입력칸이 비워집니다.
목록이 안 늘면 setQ8Items 를 안 부른 것입니다.
입력칸이 안 비면 setQ8Text("") 를 빠뜨린 것입니다.
(1)을 고쳤는지 확인하려면 e.target.valu 처럼 오타를 내 보세요.
any 인 채로는 아무 밑줄도 안 생깁니다. 그게 any 의 무서운 점입니다.

```
function Q8Cart() {  
  const [q8Text, setQ8Text] = useState("");  
  const [q8Items, setQ8Items] = useState<string[]>([]);
```

TODO (1): any 를 제대로 된 이벤트 타입으로 바꾸세요

```
function q8Change(e: any) {  
  setQ8Text(e.target.value);  
}
```

TODO (2): 목록에 넣고 입력칸을 비우세요

```
function q8Add() {  
  console.log("담기 - 아직 안 고쳤습니다");  
}  
  
return (  
  <div className="output"> <input value={q8Text} onChange=  
{q8Change} placeholder="메뉴를 적으세요" /> <button type="button" onClick={q8Add}>  
    담기  
  </button> <ul>  
    {q8Items.map((item, index) => (  
      <li key={index}>{item}</li>  
    ))}  
  </ul> <p>담긴 개수: {q8Items.length}</p> </div>  
);  
}
```

문제 9 [도전] (개념02 · 03 · 04)

메뉴를 고르면 아래에 이름과 가격이 나오게 하세요. 세 군데를 고칩니다.

(1) 고른 메뉴를 담은 state 를 만드세요.

아직 안 골랐을 때는 null 입니다. 초기값이 null 이면 타입을 알아내지 못하니
꺾쇠로 알려 줘야 합니다. (개념04 섹션3)

(2) 버튼을 누르면 그 메뉴가 state 에 담기게 하세요. (3) 골랐으면 "라떼 — 4500원", 아직이면 "고른
메뉴가 없습니다" 가 보이게 하세요.

기대 화면 처음에는 "고른 메뉴가 없습니다"
라떼를 누르면 "라떼 - 4500원"
케이크를 누르면 "케이크 - 6000원"
확인 없이 q9Picked.name 이라고 쓰면 밑줄이 생깁니다.
error TS18047: 'q9Picked' is possibly 'null'.
삼항이나 if 로 먼저 null 인지 확인하면 사라집니다.

```
type Q9Menu = { name: string; price: number };

const q9Menus: Q9Menu[] = [
  { name: "아메리카노", price: 4000 },
  { name: "라떼", price: 4500 },
  { name: "케이크", price: 6000 },
];

function Q9Picker() {
```

TODO (1): 아래 줄을 state 로 바꾸세요

```
const q9Picked: Q9Menu | null = null;

return (
  <div className="output">
    {q9Menus.map((m) => (
      <button
        key={m.name}
        type="button"
```

TODO (2): 누르면 이 메뉴가 담기게 하세요

```

        onClick={() => console.log("고르기 - 아직 안 고쳤습니다:", m.name)}
      >
        {m.name}
      </button>
    )})

    {/* TODO (3): 골랐으면 이름과 가격을, 아니면 안내 문구를 보여 주세요.
      지금은 q9Picked 를 쓰지 않고 고정된 글자만 그리고 있습니다. */}
    <p>(아직 안 고쳤습니다) - 담긴 값: {String(q9Picked)}</p>
  </div>
);
}

```

문제 10 에러 확인

아래 다섯 개는 전부 타입 에러가 나는 코드입니다. **한 번에 하나씩만** 주석을 풀고, 터미널에서 `npm run typecheck` 를 돌려세요.

- 1 에러 번호(TS로 시작하는 숫자)와 첫 줄을 적어 보세요.
- 2 무슨 뜻인지 한국어로 한 줄 적어 보세요.
- 3 확인이 끝나면 반드시 다시 주석을 붙이세요.

화면은 다섯 개 다 풀어도 멀쩡합니다. 그것이 이 단원의 핵심입니다. 실제 메시지는 연습문제_정답.tsx 에 그대로 적어 뒀습니다.

① 값의 종류가 다를 때

```
const q10a: number = "4000";
```

② props 에 타입을 안 붙였을 때

```
function Q10b({ name }) {
  return <p>{name}</p>;
}
```

③ 빈 배열로 state 를 시작했을 때

```
function Q10c() {
  const [items, setItems] = useState([]);
  setItems(["아메리카노"]);
  return <p>{items.length}</p>;
}
```


④ 없어도 되는 값을 확인 없이 썼을 때

```
type Q10dUser = { name: string; nickname?: string };
const q10dUser: Q10dUser = { name: "박지훈" };
const q10dLength = q10dUser.nickname.length;
```

⑤ 이벤트 메소드 이름을 틀렸을 때

```
function q10eSubmit(e: React.FormEvent) {
  e.preventDefault();
}
```

화면에 그리기

정리

- 1 이 연습문제는 화면과 타입 검사를 함께 봐야 합니다. 화면이 맞아도 밑줄이 남아 있을 수 있습니다.
- 2 다 풀고 나면 실습프로젝트 폴더에서 npm run typecheck 를 돌려세요. 아무것도 안 나오면 통과입니다.
- 3 막히면 개념02(값의 타입) · 개념03(props) · 개념04(state 와 이벤트)의 해당 섹션을 다시 보세요.
- 4 문제 10의 주석은 하나씩만 풀고, 확인이 끝나면 반드시 다시 붙이세요.

```

export default function Exercise14() {
  return (
    <div>
      <h1>15단원 부록 - 연습문제</h1>

      <p className="guide">
        고치기 전에는 "(아직 안 고쳤습니다)" 가 보입니다. 정상입니다.
        <br />
        <strong>이 단원은 화면만 보고 판단하면 안 됩니다.</strong> 타입 에러는 화면에
안
        나옵니다. VS Code 의 빨간 밑줄과 <code>npm run typecheck</code> 를 함께
보세요.
      </p>

      <div className="demo">
        <h3>문제 1 - 가격이 문자열이면</h3>
        <div className="output">4000 + 500 = {q1Total}</div>
      </div>

      <div className="demo">
        <h3>문제 2 - 배열 타입</h3>
        <div className="output">
          {q2Menu.join(" / ")} ({q2Menu.length}개)
        </div>
      </div>

      <div className="demo">
        <h3>문제 3 - type 별칭에 속성 추가</h3>
        <div className="output">
          {q3Latte.name} - {q3PriceText}원
        </div>
      </div>

      <div className="demo">
        <h3>문제 4 - 선택 속성</h3>
        <div className="output">
          {q4Minjun.name}({q4Minjun.nickname ?? "별명 없음"}) / {q4Seoyeon.name}(
            {q4Seoyeon.nickname ?? "별명 없음"})
        </div>
      </div>
    </div>
  )
}

```

```

    </div>
  </div>

  <div className="demo"> <h3>문제 5 – 유니온</h3> <div className="output">내
  사이즈: {q5MySize}</div> </div> <div className="demo"> <h3>문제 6 – props 에 타입
  불이기</h3> <div className="output"> <Q6Item name="아메리카노" /> <Q6Item name="라떼"
  /> </div> </div> <div className="demo"> <h3>문제 7 – children</h3>
  {/* TODO(문제 7): 아래 태그 사이에 내용을 넣으세요 */}
  <Q7Box title="오늘의 메뉴" /> </div> <div className="demo"> <h3>문제 8 [응용]
  – 장바구니</h3> <Q8Cart /> </div> <div className="demo"> <h3>문제 9 [도전] – 메뉴
  고르기</h3> <Q9Picker /> </div>

  <div className="demo">
    <h3>문제 10 – 에러 확인</h3>
    <div className="output">
      이 문제는 화면이 아니라 <code>npm run typecheck</code> 로 확인합니다. 코드
      아래쪽의
      주석 다섯 개를 하나씩 풀어 보세요.
    </div>
  </div>

  </div>
);
}

```


부 록 가

연습문제·종합연습 정답

먼저 스스로 풀어 본 다음에 보세요. 정답과 다르게 썼더라도 기대 출력이 같으면 맞은 것입니다.
다만 “왜 그렇게 되는지” 설명은 꼭 읽으세요.

11단원 연습문제 정답.jsx 정답 React Router

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 파일을 고르세요.

먼저 스스로 풀어 본 다음에 비교하세요.

답이 하나뿐인 문제는 아닙니다. 결과가 같으면 다른 방법도 맞습니다.

★ 이 파일이 쓰는 주소 앞머리는 `/a` 입니다. 연습문제 파일(/q)과 겹치지 않게 나눴습니다.

주소를 옮긴 뒤에는 새로고침(F5)을 하지 마세요.

왼쪽 메뉴 선택이 풀려서 다른 예제가 열립니다. 그때는 메뉴에서 다시 고르면 됩니다.

```
import { useState } from "react";
import {
  BrowserRouter,
  Routes,
  Route,
  Link,
  NavLink,
  Outlet,
  useParams,
  useNavigate,
  useSearchParams,
} from "react-router-dom";
import Summary from "../_ui/Summary.jsx";

const menuItems = [
  { id: 1, name: "아메리카노", price: 4000 },
  { id: 2, name: "라떼", price: 4500 },
  { id: 3, name: "케이크", price: 6000 },
  { id: 4, name: "삼각김밥", price: 1200 },
];
```

문제 1 정답 (개념01)

Routes 안에 이 한 줄을 넣습니다. 위치는 어디든 상관없습니다.

```
<Route path="/a/about" element={<AboutPage />} />
```

화면 '소개' 를 누르면 "2020년에 문을 연 작은 카페입니다"

→ Routes 는 순서대로 훑는 것이 아니라 가장 잘 맞는 것을 고릅니다.

```
function AboutPage() {
  return <p>2020년에 문을 연 작은 카페입니다.</p>;
}
```

문제 2 정답 (개념01)

element 에는 컴포넌트가 아니라 JSX 를 바로 넣어도 됩니다.

```
<Route path="/a/hours" element={<p>10시부터 22시까지 엽니다</p>} />
```

화면 '영업시간' 을 누르면 "10시부터 22시까지 엽니다"

→ 화면이 짧을 때는 이렇게 쓰는 것이 간단합니다. 길어지면 컴포넌트로 빼는 것이 읽기 좋습니다.

문제 3 정답 (개념02)

```
<a href="/a/menu">메뉴(a)</a> → <Link to="/a/menu">메뉴</Link>
```

화면 카운터를 3으로 올린 뒤 눌러도 3 그대로입니다

→ <a> 는 페이지를 서버에서 새로 받아 오므로 React 가 다시 시작합니다. 그러면 state 인 카운터도, 왼쪽 메뉴 선택도 전부 처음으로 돌아갑니다. <Link> 는 주소만 바꾸고 그 자리만 다시 그립니다.

문제 4 정답 (개념02)

```
<NavLink to="/a" end className={({ isActive }) => (isActive ? "on" : "")}>
  홈
</NavLink>
```

화면 주소가 /a 일 때만 '홈' 에 파란 배경이 들어옵니다

→ end 가 없으면 /a/about 처럼 /a 로 시작하는 모든 주소에서 켜집니다. isActive 는 NavLink 가 함수를 부르면서 넘겨 주는 값이라 함수로 받아야 합니다.

문제 5 정답 (개념03)

```
function MenuList() {
  return (
    <div> <p>메뉴를 눌러 보세요.</p> <ul>
```

```

    {menuItems.map((item) => (
      <li key={item.id}>
        {/* 값을 끼워 넣을 때는 백틱 문자열을 씁니다. 큰따옴표로는 안 됩니다. */}
        <Link to={`/a/menu/${item.id}`}>
          {item.name} {item.price}원
        </Link> </li>
      )))
  </ul> </div>
);
}

```

화면 네 줄이 나오고, '라떼' 를 누르면 주소가 /a/menu/2 가 됩니다

→ key 는 05단원에서 배운 그대로입니다. id 가 있으니 id 를 씁니다.

문제 6 정답 (개념03)

```
<Route path="/a/menu/:id" element={<MenuDetail />} />
```

화면 목록에서 아무 메뉴나 누르면 상세 화면이 나옵니다

→ :id 의 콜론이 “여기는 아무 값이나 온다” 는 표시입니다. 콜론을 빼면 /a/menu/id 라는 주소일 때만 맞습니다.

문제 7 · 문제 8 정답 (개념03)

```
function MenuDetail() {
```

문제 7 — useParams 로 꺼냅니다. 이름은 path 에 적은 :id 와 같아야 합니다.

```
const { id } = useParams();
```

문제 8 — menuItems 의 id 는 숫자라서 Number 로 바꿔서 비교합니다.

```

const found = menuItems.find((item) => item.id === Number(id));

return (
  <div> <h4>상세 화면</h4> <p>
    받은 id: <strong>{id}</strong> (타입: {typeof id})
  </p>

  {/* 못 찾았을 때를 먼저 처리합니다. 안 하면 found.name 에서 화면이 터집니다.

```



```

{found ? (
  <p> <strong>
    {found.name} {found.price}원
  </strong> </p>
) : (
  <p>그런 메뉴가 없습니다.</p>
)}

<p> <Link to="/a/menu">- 목록으로</Link> </p> </div>
);
}

```

화면 '라떼' → 받은 id: 2 (타입: string) / 라떼 4500원
 '없는 메뉴(99)' → 받은 id: 99 (타입: string) / 그런 메뉴가 없습니다

→ 주소는 글자라서 useParams 가 주는 값은 언제나 문자열입니다. item.id === id 로 비교하면 2 와 "2" 를 건주게 되어 늘 못 찾습니다.

문제 9 정답 (개념 04)

```

function ShopLayout() {
  return (
    <div> <h4>민준이네 카페</h4> <nav> <Link to="/a/shop">가게 홈</Link> |
    <Link to="/a/shop/notice">공지</Link> </nav> <div className="output">
      {/* 자식 Route 의 화면이 이 자리에 들어옵니다 */}
      <Outlet /> </div> </div>
  );
}

```

화면 '공지' 를 누르면 꺾데기 아래에 "이번 주 화요일은 쉽니다"

→ Outlet 이 없으면 꺾데기만 계속 나옵니다. 에러도 경고도 없어서 찾기 어렵습니다.

문제 10 정답 (개념 04)

```

<Route path="/a/shop" element={<ShopLayout />}>
  <Route index element={<ShopHome />} /> <Route path="notice" element={<ShopNotice
  />} />
</Route>

```

화면 '가게' 를 누르면 "어서 오세요. 위에서 골라 보세요"

→ index 는 "부모 주소와 딱 맞을 때" 라는 뜻입니다. path 를 적지 않습니다. index 가 없으면 /a/shop 에서 꺾데기만 나오고 가운데가 빕니다.

```
function ShopHome() {
  return <p>어서 오세요. 위에서 골라 보세요.</p>;
}

function ShopNotice() {
  return <p>이번 주 화요일은 쉽니다.</p>;
}
```

문제 11 정답 [응용] (개념05)

```
function OrderScreen() {
  const [item, setItem] = useState("");
  const [error, setError] = useState("");
```

혹은 컴포넌트 맨 위에서 부릅니다. handleOrder 안에서 부르면 안 됩니다.

```
const navigate = useNavigate();

function handleOrder() {
```

먼저 검사하고, 문제가 있으면 이동하지 않고 끝냅니다.

```
if (item.trim() === "") {
  setError("무엇을 주문할지 적어 주세요");
  return;
}
setError("");
navigate("/a/done");
}

return (
  <div> <h4>주문 화면</h4> <input value={item} onChange={(e) =>
setItem(e.target.value)}
  placeholder="아메리카노"
  />
  <button onClick={handleOrder}>주문하기</button> <p className="error">{error}</p> </div>
);
}

function DoneScreen() {
  return <p>주문이 끝났습니다. 감사합니다.</p>;
}
```

화면 빈칸에서 누르면 주소는 /a/order 그대로이고 "무엇을 주문할지 적어 주세요"
'라떼' 를 적고 누르면 주소가 /a/done 이 되고 "주문이 끝났습니다"

→ 이것이 <Link> 로는 할 수 없는 일입니다. Link 는 누르면 무조건 갑니다. trim() 을 쓴 이유는 공백만 친 경우도 막기 위해서입니다(06단원).

문제 12 정답 [도전] (개념05 + 개념03)

```
function SearchScreen() {
  const [searchParams] = useSearchParams();
  const keyword = searchParams.get("keyword");
```

검색어가 없으면 null 입니다. 빈 문자열이 아닙니다.

```
if (keyword === null) {
  return (
    <div> <h4>검색 화면</h4> <p>검색어가 없습니다.</p> </div>
  );
}
```

includes 는 그 글자가 들어 있으면 true 입니다(JS자료 06단원).

```
const found = menuItems.filter((item) => item.name.includes(keyword));

return (
  <div> <h4>검색 화면</h4> <p>'{keyword}' 로 찾은 결과</p>

  {found.length === 0 ? (
    <p>찾은 메뉴가 없습니다.</p>
  ) : (
    <ul>
      {found.map((item) => (
        <li key={item.id}>
          {item.name} {item.price}원
        </li>
      ))}
    </ul>
  )}
</div>
);
}
```

화면 '검색(라)' → 라떼 4500원 한 줄
 '검색(김)' → 삼각김밥 1200원 한 줄
 '검색(없이)' → 검색어가 없습니다

→ 검색어는 Route 에 등록하지 않았습니다. Route 는 /a/search 하나뿐입니다. 조건만 바뀌는 것이라 쿼리스트링으로 받는 것이 맞습니다. found.length === 0 을 && 대신 삼항으로 쓴 이유는 05단원의 0 함정 때문입니다.

```
function NoRoute() {
  return <p>이 예제가 아는 주소가 아닙니다. 위의 링크를 눌러 보세요.</p>;
}

function AnswerApp() {
  const [count, setCount] = useState(0);

  return (
    <BrowserRouter>
      <div className="demo">
        <h3>정답 화면</h3>

        <p>
          카운터: <strong>{count}</strong>{" "}
          <button onClick={() => setCount(count + 1)}>+1</button>
        </p>

        <nav>
          {/* 문제 4 - 홈만 NavLink 입니다 */}
          <NavLink to="/a" end className={({ isActive }) => (isActive ? "on" : "")}>
            홈
          </NavLink>{" "}
          | <Link to="/a/about">소개</Link> | <Link to="/a/hours">영업시간</Link>
          |{" "}

          {/* 문제 3 - <a> 를 Link 로 바꿨습니다 */}
          <Link to="/a/menu">메뉴</Link> | <Link to="/a/menu/99">없는 메뉴(99)
        </Link>

        <br />
        <Link to="/a/shop">가게</Link> | <Link to="/a/order">주문</Link> |{" "}
        <Link to="/a/search?keyword=라">검색(라)</Link> |{" "}
        <Link to="/a/search?keyword=김">검색(김)</Link> |{" "}
        <Link to="/a/search">검색(없어)</Link> |{" "}
        <Link to="/a/nope">없는 주소</Link>
      </nav>

      <div className="output">
        <Routes>
          <Route path="/a" element={<p>홈 - 어서 오세요.</p>} />
        </Routes>
      </div>
    </BrowserRouter>
  );
}
```

```

    <Route path="/a/menu" element={<MenuList />} />

    {/* 문제 1 */}
    <Route path="/a/about" element={<AboutPage />} />
    {/* 문제 2 */}
    <Route path="/a/hours" element={<p>10시부터 22시까지 엽니다</p>} />
    {/* 문제 6 */}
    <Route path="/a/menu/:id" element={<MenuDetail />} />

    <Route path="/a/shop" element={<ShopLayout />>
      {/* 문제 10 */}
      <Route index element={<ShopHome />} />
      <Route path="notice" element={<ShopNotice />} />
    </Route> <Route path="/a/order" element={<OrderScreen />} />
  </Route> <Route path="/a/done" element={<DoneScreen />} /> <Route path="/a/search" element=
  {<SearchScreen />} /> <Route path="*" element={<NoRoute />} />
</Routes>
</div>
</div>
</BrowserRouter>
);
}

```

정리

- 1 Route 는 path 와 element 의 짝입니다. element 에는 JSX 를 넣습니다.
- 2 앱 안에서 옮길 때는 , 다른 사이트로 갈 때는 입니다.
- 3 NavLink 는 isActive 를 함수로 넘겨 줍니다. 짧은 주소에는 end 를 붙입니다.
- 4 useParams 가 주는 값은 언제나 문자열입니다. 숫자와 비교하려면 Number() 로 바꿉니다.
- 5 중첩 라우트는 부모의 자리에 자식이 들어갑니다. 부모 주소용 화면은 index 입니다.
- 6 검사한 뒤에 옮기려면 useNavigate 를 씁니다. Link 는 누르면 무조건 갑니다.
- 7 조건만 바뀌는 값은 쿼리스트링으로 받고 useSearchParams 로 꺼냅니다. 없으면 null 입니다.

```

export default function Practice11RouterAnswer() {
  return (
    <div> <h1>11단원 연습문제 정답 – React Router</h1> <p className="guide">
      먼저 스스로 풀어 본 다음에 보세요. 각 문제의 풀이는 이 파일의 주석에 문제
      번호
      순서로 적어 두었습니다.
      <br /> <br /> <strong>주소를 옮긴 뒤 새로고침(F5)을 하지 마세요.</strong>
      왼쪽 메뉴 선택이 풀려서
    </div>
  );
}

```

다른 예제가 열립니다. 그때는 왼쪽 메뉴에서 다시 고르면 됩니다.

```
</p> <AnswerApp /> </div>
);
}
```

문제 13 정답 (에러 확인)

(가) 자식 Route 의 path 를 "/notice" 로 바꾸면

- 화면이 통째로 빨간 에러 상자가 됩니다.
- 콘솔: Absolute route path "/notice" nested under path "/a/shop" is not valid.
An absolute child route path must start with the combined path of all its parent routes.
- 자식은 부모 주소 뒤에 이어 붙여야 하는데, 슬래시로 시작하면 "앱의 맨 처음부터" 라는 뜻이라 부모와 말이 안 맞습니다.
자식 path 에는 슬래시를 붙이지 않습니다.

(나) element={AboutPage} 로 화살괄호를 지우면

- 화면: '소개' 를 눌러도 그 자리가 텅 빕니다. 빨간 에러 상자는 안 나옵니다.
다른 화면은 멀쩡합니다. 그래서 더 찾기 어렵습니다.
- 콘솔: Functions are not valid as a React child.
This may happen if you return AboutPage instead of <AboutPage /> from render. Or maybe you meant to call this function rather than return it.
- element 에는 '함수' 가 아니라 '만들어진 화면' 을 넣습니다.
onClick 과 반대라고 기억하면 편합니다.
onClick 은 나중에 부를 함수를, element 는 지금 그릴 화면을 받습니다.

(다) Link 의 to 를 href 로 바꾸면

- 화면: 글자는 그대로 나오고, 눌러도 아무 일이 안 일어납니다.
- 콘솔: 아무것도 안 나옵니다. 에러도 경고도 없습니다.
- Link 가 아는 이름은 to 입니다. href 는 모르는 이름이라 그냥 무시합니다.
이런 실수는 콘솔이 조용해서 가장 찾기 어렵습니다.
"눌러도 반응이 없다" 면 속성 이름부터 확인하세요.

12단원 ·

12단원 연습문제 정답.jsx 정답 Context와 useReducer

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 여세요.

먼저 연습문제.jsx 를 스스로 풀어 보세요. 여기는 답을 맞춰 보는 곳입니다. 답이 하나만 있는 것은 아닙니다. 화면이 기대 결과와 같으면 맞은 것입니다.

각 문제마다 '왜 그런지' 를 같이 적어 두었습니다. 그 부분을 읽는 것이 더 중요합니다.

```
import { createContext, useContext, useReducer, useState } from "react";
import Summary from "../_ui/Summary.jsx";
```

문제 1 정답 (개념01)

props 를 한 단계씩 이어서 넘깁니다. 이름을 넘길 때와 받을 때가 같아야 합니다.

```
function Q1GrandChild({ nickname }) {
  return <div className="output">별명: {nickname}</div>;
}

function Q1Child({ nickname }) {
```

Q1Child 는 nickname 을 쓰지 않습니다. 받아서 넘기기만 합니다. 이것이 개념01에서 본 props drilling 입니다.

```
return (
  <div className="output"> <Q1GrandChild nickname={nickname} /> </div>
);
}

function Q1Parent() {
  const nickname = "민준";
  return (
    <div className="output"> <strong>문제 1</strong> <Q1Child nickname={nickname} /> </div>
  );
}
```

화면 별명: 민준

중간에서 이름을 바꿔 `<Q1GrandChild nick={nickname} />` 라고 쓰면 받는 쪽 `{ nickname }` 이 `undefined` 가 되어 "별명: " 만 나옵니다. 에러는 안 납니다.

문제 2 정답 (개념02)

`createContext` 의 인자가 곧 기본값입니다.

```
const Q2Context = createContext("라이트");

function Q2Label() {
  const theme = useContext(Q2Context);
  return <div className="output">테마: {theme}</div>;
}
```

화면 테마: 라이트

인자를 안 주면 기본값이 `undefined` 가 되고, JSX 는 `undefined` 를 아무것도 안 그립니다. 그래서 "테마:" 뒤가 비어 보입니다. 에러가 안 나는 것이 함정입니다.

문제 3 정답 (개념02)

`Provider` 로 감싼 안쪽에서는 기본값 대신 `value` 가 나옵니다.

```
const Q3Context = createContext("라이트");

function Q3Label() {
  const theme = useContext(Q3Context);
  return <div className="output">테마: {theme}</div>;
}

function Q3Demo() {
  return (
    <div className="output"> <strong>문제
3</strong> <Q3Context.Provider value="다크"> <Q3Label /> </Q3Context.Provider> </div>
  );
}
```

화면 테마: 다크

`Provider` 를 `<Q3Label />` 아래에 두면 소용이 없습니다. 반드시 위를 감싸야 합니다.

문제 4 정답 (개념02)

Provider 안: Provider 가 준 값 Provider 밖: 기본값입니다

useContext 는 자기 위로 올라가며 같은 상자의 Provider 를 찾습니다. 못 찾으면 createContext 에 적어둔 기본값을 돌려줍니다. 밖에 있는 라벨은 화면상 Provider 바로 아래에 그려지지만, 컴포넌트 관계로는 밖입니다. '화면에서 아래' 와 '컴포넌트 안' 은 다른 이야기입니다.

```
const Q4Context = createContext("기본값입니다");

function Q4Label({ where }) {
  const value = useContext(Q4Context);
  return (
    <div className="output">
      {where}: {value}
    </div>
  );
}

function Q4Demo() {
  return (
    <div className="output"> <strong>문제
4</strong> <Q4Context.Provider value="Provider 가 준 값"> <Q4Label where="Provider
안" /> </Q4Context.Provider> <Q4Label where="Provider 밖" /> </div>
  );
}
```

화면	Provider 안: Provider 가 준 값
	Provider 밖: 기본값입니다

문제 5 정답 (개념03)

```
function q5Reducer(state, action) {
  switch (action.type) {
    case "add":
      return state + 1;
    case "remove":
      if (state <= 0) return state;
      return state - 1;
    default:
      return state;
  }
}
```

```

}

function Q5Demo() {
  const [count, dispatch] = useReducer(q5Reducer, 0);
  return (
    <div className="output"> <strong>문제 5</strong> <p>아메리카노 {count}잔</p>
    <button onClick={() => dispatch({ type: "add" })}>담기</button> <button onClick=
    {() => dispatch({ type: "remove" })}>빼기</button> </div>
  );
}

```

화면(누르면): [담기] 세 번 → 아메리카노 3잔 / [빼기] 를 계속 눌러도 0잔에서 멈춤

remove 에서 return state 를 쓴 것이 핵심입니다. 아무것도 return 하지 않으면 새 state 가 undefined 가 되어 화면이 터집니다. `Math.max(state - 1, 0)` 으로 써도 같은 결과입니다.

문제 6 정답 (개념03)

```

function q6Reducer(state, action) {
  switch (action.type) {
    case "add":
      return state + 1;
    case "reset":

```

지금 값과 상관없이 0을 돌려줍니다

```

      return 0;
    default:
      return state;
  }
}

function Q6Demo() {
  const [count, dispatch] = useReducer(q6Reducer, 0);
  return (
    <div className="output"> <strong>문제 6</strong> <p>아메리카노 {count}잔</p>
    <button onClick={() => dispatch({ type: "add" })}>담기</button> <button onClick=
    {() => dispatch({ type: "reset" })}>비우기</button> </div>
  );
}

```

화면(누르면): [담기] 두 번 → 2잔 → [비우기] → 0잔

dispatch 의 type 과 case 의 이름이 글자 하나라도 다르다면 default 로 갑니다. 그러면 에러 없이 아무 일도 안 일어납니다. 개념03 [실수 4] 입니다.

문제 7 정답 (개념03)

```
function q7Reducer(state, action) {
  switch (action.type) {
    case "addMany":
      return state + action.payload;
    default:
      return state;
  }
}

function Q7Demo() {
  const [count, dispatch] = useReducer(q7Reducer, 0);
  return (
    <div className="output"> <strong>문제 7</strong> <p>아메리카노 {count}잔</p>
    <p><button onClick={() => dispatch({ type: "addMany", payload: 1 })}>1잔 담기</button>
    <button onClick={() => dispatch({ type: "addMany", payload: 3 })}>3잔 담기</button> </div>
  );
}
```

화면(누르면): [3잔 담기] 두 번 → 아메리카노 6잔

payload 라는 이름은 규칙이 아닙니다. { type: "addMany", n: 3 } 이라고 해도 됩니다. 다만 보내는 쪽과 읽는 쪽의 이름이 같아야 합니다. 이름이 다르면 action.payload 가 undefined 가 되고 state + undefined 는 NaN 이 됩니다.

문제 8 정답 (개념03)

답은 5 입니다.

```
function q8Reducer(state, action) {
  switch (action.type) {
    case "add":
      return state + 1;
    case "double":
      return state * 2;
    case "reset":
      return 0;
    default:
      return state;
  }
}
```

```

}

const q8Steps = [
  { type: "add" },
  { type: "add" },
  { type: "double" },
  { type: "add" },
  { type: "모르는것" },
];

console.log(q8Steps.reduce(q8Reducer, 0));

```

F12 콘솔 5

한 바퀴씩 보면 이렇습니다. JS자료 08단원 개념05에서 쓴 방법 그대로입니다.

```

q8Steps.reduce((state, action) => {
  const next = q8Reducer(state, action);
  console.log(`state=${state}, action=${action.type}, 결과=${next}`);
  return next;
}, 0);

```

F12 콘솔

```

state=0, action=add, 결과=1
state=1, action=add, 결과=2
state=2, action=double, 결과=4
state=4, action=add, 결과=5
state=5, action=모르는것, 결과=5

```

마지막 "모르는것" 은 default 로 가서 state 를 그대로 돌려줍니다. 그래서 값이 안 바뀝니다. 실제 화면에서도 이런 일이 생기면 버튼이 먹통이 됩니다.

문제 9 정답 (개념04)

```

const Q9_MENU = [
  { id: 1, name: "아메리카노", price: 4000 },
  { id: 2, name: "라떼", price: 4500 },
];

function q9Reducer(state, action) {
  switch (action.type) {
    case "add":
      if (state.some((item) => item.id === action.payload.id)) {
        return state.map((item) =>
          item.id === action.payload.id ? { ...item, count: item.count + 1 } : item
        );
      }
      return [...state, { ...action.payload, count: 1 }];
    default:
      return state;
  }
}

const Q9Context = createContext(null);

function Q9Provider({ children }) {
  const [items, dispatch] = useReducer(q9Reducer, []);
  return (
    <Q9Context.Provider value={{ items: items, dispatch: dispatch }}>
      {children}
    </Q9Context.Provider>
  );
}

function useQ9Cart() {
  const cart = useContext(Q9Context);
  if (cart === null) {
    throw new Error("useQ9Cart 는 <Q9Provider> 안에서만 쓸 수 있습니다");
  }
  return cart;
}

```

```

}

function Q9MenuList() {
  const { dispatch } = useQ9Cart();
  return (
    <div className="output"> <ul>
      {Q9_MENU.map((menu) => (
        <li key={menu.id}>
          {menu.name} {menu.price}원{" "}
          <button onClick={() => dispatch({ type: "add", payload: menu })}>담기
        </button> </li>
      ))}
    </ul> </div>
  );
}

function Q9CartList() {
  const { items } = useQ9Cart();
  if (items.length === 0) {
    return <div className="output">장바구니가 비어 있습니다</div>;
  }
  return (
    <div className="output"> <ul>
      {items.map((item) => (
        <li key={item.id}>
          {item.name} {item.count}개
        </li>
      ))}
    </ul> </div>
  );
}

```

화면(누르면): 아메리카노 [담기] → “아메리카노 1개”, 한 번 더 → “아메리카노 2개”

onClick={dispatch({ type: "add", payload: menu })} 처럼 괄호를 붙이면 안 됩니다.

그리는 중에 dispatch 가 실행되어 “Too many re-renders” 에러가 납니다. 04단원 개념01에서 배운 그대로입니다. 화살표로 감싸세요.

문제 10 정답 (개념 04)

```

function Q10Total() {
  const { items } = useQ9Cart();

```

JS자료 08단원 개념05 섹션4와 같은 모양입니다. 시작값 0 을 꼭 주세요.

```
const total = items.reduce((acc, item) => acc + item.price * item.count, 0);
return (
  <div className="output"> <strong>합계 {total}원</strong> </div>
);
}

function Q9Q10Demo() {
  return (
    <div className="output"> <strong>문제 9 · 10</strong> <Q9Provider> <Q9MenuList
  /> <Q9CartList /> <Q10Total /> </Q9Provider> </div>
  );
}
```

화면(누르면): 아메리카노 두 번, 라떼 한 번 담으면 "합계 12500원"

$$(4000 \times 2 + 4500 \times 1 = 12500)$$

item.count 를 안 곱하면 8500 이 나옵니다. 단가만 더한 것입니다. 시작값 0 을 빼먹으면 빈 배열에서 TypeError 가 납니다(JS자료 08단원 섹션3).

문제 11 정답 [응용] (개념04)

```
const Q11_MENU = { id: 1, name: "아메리카노", price: 4000 };

function q11Reducer(state, action) {
  switch (action.type) {
    case "add":
      if (state.some((item) => item.id === action.payload.id)) {
        return state.map((item) =>
          item.id === action.payload.id ? { ...item, count: item.count + 1 } : item
        );
      }
      return [...state, { ...action.payload, count: 1 }];
    case "decrease":
```

먼저 map 으로 수량을 1 줄이고, 그 다음 filter 로 0이 된 칸을 뺍니다. 순서가 반대면 안 됩니다. 먼저 걸러 버리면 줄일 대상이 사라집니다.

```
return state
  .map((item) =>
    item.id === action.payload ? { ...item, count: item.count - 1 } : item
  )
  .filter((item) => item.count > 0);
default:
```

```

}
}

function Q11Demo() {
  const [items, dispatch] = useReducer(q11Reducer, []);
  return (
    <div className="output"> <strong>문제 11 [응용]</strong> <button onClick={() =>
dispatch({ type: "add", payload: Q11_MENU })}>담기</button>
      {items.length === 0 ? (
        <div className="output">비어 있습니다</div>
      ) : (
        <ul>
          {items.map((item) => (
            <li key={item.id}>
              {item.name} {item.count}개{" "}
              <button onClick={() => dispatch({ type: "decrease", payload: item.id
            }}}>
              -
            </button> </li>
          ))}
        </ul>
      )}
    </div>
  );
}

```

화면(누르면): [담기] 두 번 → “아메리카노 2개” → [-] → “1개” → [-] → “비어 있습니다”

{ ...item, count: item.count - 1 } 로 새 객체를 만드는 것이 중요합니다. item.count = item.count - 1 처럼 직접 고치면 07단원의 얇은 복사 함정에 빠집니다.

add 와 decrease 의 payload 모양이 다른 점도 보세요. add 는 메뉴 객체, decrease 는 id 하나입니다. 개념04 [실수 6] 에서 짚은 부분입니다.

문제 12 정답 [도전] (개념05)

상자를 둘로 나누고 Provider 를 겹쳐 씁니다.

```

const Q12UserContext = createContext(null);
const Q12ThemeContext = createContext(null);

function Q12UserName() {
  console.log("[문제12] Q12UserName 실행");
}

```

F12 콘솔 [문제12] Q12UserName 실행


```
const user = useContext(Q12UserContext);
return <div className="output">사용자: {user}</div>;
}
```

```
function Q12ThemeLabel() {
  console.log("[문제12] Q12ThemeLabel 실행");
}
```

F12 콘솔 [문제12] Q12ThemeLabel 실행

```
const theme = useContext(Q12ThemeContext);
return <div className="output">테마: {theme}</div>;
}

function Q12Demo() {
  const [user, setUser] = useState("김민준");
  const [theme, setTheme] = useState("light");

  return (
    <div className="output"> <strong>문제 12 [도전]
    </strong> <Q12UserContext.Provider value={user}> <Q12ThemeContext.Provider value=
    {theme}> <button onClick={() => setTheme(theme === "light" ? "dark" : "light")}>
      테마만 바꾸기
    </button> <button onClick={() => setUser(user === "김민준" ? "이서연" :
    "김민준")}>
      사용자만 바꾸기
    </button> <Q12UserName /> <Q12ThemeLabel
    /> </Q12ThemeContext.Provider> </Q12UserContext.Provider> </div>
  );
}
```

콘솔을 지우고 [테마만 바꾸기] 를 한 번 누른 결과입니다. 실제로 세어서 확인했습니다.

문제12 · Q12ThemeLabel 실행 ← 두 줄 (StrictMode)

Q12UserName 은 안 나옵니다

반대로 [사용자만 바꾸기] 를 누르면 Q12UserName 만 나옵니다.

한 상자였을 때는 어느 쪽을 눌러도 둘 다 나왔습니다. value 를 객체로 담으면 그 안의 한 속성만 바뀌어도 객체 전체가 새것이 되기 때문입니다. React 는 “이 컴포넌트는 user 만 읽더라” 를 구분하지 못합니다.

문제 13 정답 에러 확인 (개념02 · 개념04)

고치기 전에는 이런 에러가 났습니다.

TypeError: Cannot read properties of null (reading 'name')

읽는 법: "null 의 name 을 읽을 수 없다" 입니다. useContext 가 Provider 를 못 찾아 기본값 null 을 돌려줬고, 그 null 에서 .name 을 읽으려다 멈춘 것입니다. 에러는 user.name 을 읽는 Q13Badge 에서 났지만, 진짜 원인은 Provider 로 감싸지 않은 Q13Demo 에 있습니다.

고치는 방법은 Provider 로 감싸는 것입니다.

```
const Q13Context = createContext(null);

function Q13Badge() {
  const user = useContext(Q13Context);
  return <div className="output">{user.name} 님</div>;
}

function Q13Demo() {
  return (
    <div className="output"> <strong>문제 13 (에러 확인)
</strong> <Q13Context.Provider value={{ name: "김민준" }}> <Q13Badge
/> </Q13Context.Provider> </div>
  );
}
```

화면 김민준 님

개념04에서 배운 대로 useCart 같은 커스텀 훅으로 감싸 두면 "Cannot read properties of null" 대신 "useCart 는 <CartProvider> 안에서만 쓸 수 있습니다" 라는 문장이 나옵니다. 같은 실수인데 고치기가 훨씬 쉬워집니다.

정리

- 1 props 는 넘기는 이름과 받는 이름이 같아야 합니다. 다르면 에러 없이 undefined 가 됩니다.
- 2 createContext 의 인자가 기본값입니다. Provider 를 만나면 기본값은 쓰이지 않습니다.
- 3 reducer 의 case 이름과 dispatch 의 type 이 다르면 default 로 가서 조용히 아무 일도 안 합니다.
- 4 reducer 는 순수한 함수라 reduce 에 넣어 화면 없이 계산할 수 있습니다.
- 5 배열 state 는 map 으로 고치고 filter 로 걸러 새 배열을 만듭니다. 원본을 고치지 마세요.
- 6 상자를 쪼개면 다시 그려지는 범위가 줄어듭니다. 콘솔로 직접 세어서 확인하세요.
- 7 Provider 없이 useContext 를 쓰면 기본값이 나옵니다. null 이면 그것을 읽는 자리에서 터집니다.

```

export default function Exercise12Answer() {
  return (
    <div> <h1>12단원 연습문제 - 정답</h1> <p className="guide"> <strong>F12 →
    Console</strong> 을 함께 여세요. 문제 8의 계산 과정과 문제 12의
      다시 그려지는 범위를 콘솔에서 확인할 수 있습니다.
    <br />
    StrictMode 때문에 같은 줄이 두 번씩 찍힙니다. <strong>두 줄이 한 번</strong>
    입니다.
    </p> <div className="demo"> <Q1Parent /> </div> <div className="demo"> <strong>
    문제 2</strong> <Q2Label /> </div> <div className="demo"> <Q3Demo
    /> </div> <div className="demo"> <Q4Demo /> </div> <div className="demo"> <Q5Demo
    /> </div> <div className="demo"> <Q6Demo /> </div>

    <div className="demo">
      <Q7Demo />
    </div>

    <div className="demo"> <strong>문제 8</strong> <p>콘솔에서 확인하세요. 답은 5
    입니다.</p> </div> <div className="demo"> <Q9Q10Demo
    /> </div> <div className="demo"> <Q11Demo /> </div> <div className="demo"> <Q12Demo
    /> </div> <div className="demo"> <Q13Demo /> </div>

    </div>
  );
}

```

13단원 종합 01 정답 할 일 목록

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 여세요.

먼저 스스로 만들어 본 다음에 보세요. 추가 · 완료 토글 · 삭제 · 필터 · 개수가 전부 동작합니다.

★ 이 앱은 JS자료 13단원 종합03과 '완전히 같은 앱' 입니다. 요구사항이 하나도 다르지 않습니다. 다른 것은 만드는 방법뿐입니다.

— 두 코드는 무엇이 다른가

JS 판에서 우리는 화면을 '직접' 고쳤습니다. `createElement` 로 `li` 를 만들고, `appendChild` 로 붙이고, `innerHTML = ""` 로 지우고, `classList.add("done")` 로 취소선을 그었습니다. 그리고 그 일을 모아 둔 `render()` 함수를 직접 만들고, 무슨 일이 생길 때마다 잊지 않고 다시 불러야 했습니다(JS 판 문제 8). 이 파일에는 그런 코드가 한 줄도 없습니다. `createElement` 도, `appendChild` 도, `innerHTML` 도, `classList` 도, `render()` 를 다시 부르는 줄도 없습니다. 우리는 "todos 가 이르면 화면은 이렇게 생겼다" 만 적었습니다. `setTodos` 로 데이터를 바꾸면 화면을 고치는 일은 React 가 합니다. 01단원 개념01에서 "React 는 그 일을 대신해 줍니다" 라고 했던 이야기가 여기서 끝납니다. 아래 섹션 '나란히 놓고 보기' 에 두 코드를 붙여 두었습니다.

★ 이 실습에서 가장 어려운 것은 문제 5(완료 토글) 입니다. 배열도 새것, 그 안의 객체도 새것 — 두 겹을 동시에 지켜야 하기 때문입니다.

```
import { useState } from "react";
import Summary from "../_ui/Summary.jsx";

const FIRST_TODOs = [
  { id: 1, text: "장보기", done: false },
  { id: 2, text: "설거지", done: true },
];
```

새 항목에 붙일 번호입니다. 화면에 보이는 값이 아니므로 `state` 가 아니어도 됩니다(07단원 개념05). 07단원 개념01의 `nextCartId` 와 같은 방식입니다.

```
let nextId = 3;
```

브라우저를 열기 전에 검사하기

이 앱이 배열에 하는 일은 세 가지뿐입니다. 추가 · 수정 · 삭제. 화면을 만들기 전에 그 세 줄이 정말 '새것'을 만드는지 콘솔로 확인해 둡니다. 12단원 개념04에서 reducer 를 검사했던 것과 같은 방법입니다.

```
const sample = FIRST_TODOs;
```

(1) 추가 — 대괄호를 새로 열었으니 결과는 반드시 새 배열입니다

```
const afterAdd = [...sample, { id: 3, text: "운동하기", done: false }];
```

```
console.log(afterAdd.length);
```

F12 콘솔 3

```
console.log(afterAdd !== sample);
```

F12 콘솔 true

(2) 수정 — map 은 새 배열을 돌려주고, 스프레드는 새 객체를 만듭니다

```
const afterToggle = sample.map((todo) =>
  todo.id === 1 ? { ...todo, done: !todo.done } : todo
);
```

```
console.log(afterToggle[0].done);
```

F12 콘솔 true

코드

```
console.log(afterToggle[0] !== sample[0]);
```

```
console.log(afterToggle[1] === sample[1]);
```

▶ F12 콘솔

▶ true

▶ true

마지막 줄을 눈여겨보세요. 손대지 않은 줄은 '원래 객체 그대로' 입니다. map 은 목록 전체를 새로 만드는 것이 아니라 필요한 줄만 새로 만듭니다.

(3) 삭제 — filter 는 '남길 것' 을 고릅니다

```
const afterDelete = sample.filter((todo) => todo.id !== 1);
```

```
console.log(afterDelete.map((todo) => todo.text).join(", "));
```

F12 콘솔 설거지

```
console.log(sample.length);
```

지우고 나서도 원본 sample 은 그대로 2개입니다. filter 는 원본을 안 건드립니다. splice 였다면 원본까지 함께 줄어듭니다. 그래서 splice 를 안 씁니다.

화면 부품

할 일 한 줄입니다. props 로 값 하나와 함수 둘을 받습니다. 함수를 props 로 내려보내는 것은 07단원 개념04에서 배웠습니다.

```
function TodoItem({ todo, onToggle, onDelete }) {
  return (
    <li> <span className={todo.done ? "done" : ""}
      style={{ cursor: "pointer" }}
      onClick={() => onToggle(todo.id)}
    >
      {todo.text}
    </span>{" "}
    <button onClick={() => onDelete(todo.id)}>삭제</button> </li>
  );
}
```

— 이 부품이 JS 판과 다른 점

JS 판에서는 li · span · button 을 createElement 로 세 번 만들고 appendChild 를 세 번 불러 조립했습니다. 여기서는 생긴 모양을 그대로 적었습니다. 그리고 JS 판은 어느 항목을 눌렀는지 알아내려고 li 에 data-id 를 심고 e.target.closest("li") 로 거슬러 올라가 dataset 을 Number 로 바꿔야 했습니다. 여기서는 그 줄이 통째로 사라졌습니다. 각 줄이 자기 todo 를 이미 알고 있으니 onToggle(todo.id) 라고 그냥 부르면 됩니다. id 는 처음부터 숫자입니다.

```
function FilterButton({ value, label, filter, onChange }) {
```

문제 7

지금 고른 필터와 같은 버튼에만 "on" 클래스를 붙입니다.

```
return (
  <button className={filter === value ? "on" : ""} onClick={() => onChange(value)}>
    {label}
  </button>
);
}
```

— 왜 이렇게 했나

JS 판에서는 버튼 세 개를 전부 돌며 `classList.remove("active")` 한 뒤 눌린 것에만 `add("active")` 를 해야 했습니다. '떼고 붙이는' 두 단계였습니다. React 에서는 각 버튼이 "나는 지금 고른 값인가?" 만 스스로 답하면 끝입니다. filter 가 바뀌면 세 버튼이 모두 다시 그려지면서 저절로 정리됩니다.

`className` 은 02단원 개념03에서 배운 대로 `class` 가 아니라 `className` 입니다. 삼항 연산자로 값을 고르는 것은 05단원 개념01입니다.

앱 본체

```
function TodoApp() {
  const [todos, setTodos] = useState(FIRST_TODOs);
  const [text, setText] = useState(""); // 입력칸 (06단원 제어 컴포넌트)
  const [error, setError] = useState(""); // 빨간 안내 문구
  const [filter, setFilter] = useState("all"); // "all" | "active" | "done"
```

— state 를 왜 이렇게 넷으로 나눴나

넷 다 '서로 계산해 낼 수 없는 값' 입니다. 그래서 각자 필요합니다. `todos` 는 진짜 데이터, `text` 는 입력 중인 글자, `error` 는 안내 문구, `filter` 는 사용자가 고른 보기 방식입니다. 반대로 아래 `visibleTodos` · `total` · `doneCount` · `leftCount` 는 `todos` 와 `filter` 에서 계산해 낼 수 있으므로 state 로 두지 않습니다. 07단원 개념05의 '파생 state 금지' 가 그 이야기입니다.

문제 1 보여 줄 목록 고르기

```
const visibleTodos = todos.filter((todo) => {
  if (filter === "active") return !todo.done;
  if (filter === "done") return todo.done;
  return true; // "all"
});
```

— 왜 이렇게 했나

filter 콜백이 `true` 를 돌려주면 그 항목이 남습니다. "all" 일 때는 전부 남겨야 하므로 그냥 `true` 입니다. 조기 반환을 쓰면 else 없이 읽기 좋습니다(05단원·JS자료 05단원).

이 값을 state 로 두면 어떻게 될까요? 할 일을 추가할 때마다 `todos` 와 `visibleTodos` 를 둘 다 고쳐야 합니다. 한쪽을 잊으면 "추가했는데 목록에 안 보이는" 상태가 됩니다. 에러는 안 납니다. 계산해서 쓰면 그런 어긋남이 아예 생길 수 없습니다.

문제 3 개수 세기

```
const total = todos.length;
const doneCount = todos.filter((todo) => todo.done).length;
const leftCount = total - doneCount;
```

— 왜 전체 todos 로 세나

상태줄은 '지금 보이는 것' 이 아니라 '내가 가진 할 일 전부' 를 알려 주는 줄입니다. 그래서 visibleTodos 가 아니라 todos 를 셉니다. leftCount 는 뺄셈으로 구했습니다. 또 한 번 filter 를 돌 이유가 없습니다.

문제 4 추가

```
function handleSubmit(e) {
  e.preventDefault(); // * 없으면 페이지가 새로고침되어 담아 둔 것이 전부
  날아갑니다 const trimmed = text.trim(); // 공백만 친 경우도 걸러집니다 if (trimmed
  === "") {
    setError("할 일을 입력해 주세요");
    return;
  }

  if (todos.some((todo) => todo.text === trimmed)) {
    setError("이미 있는 항목입니다");
    return;
  }

  setError(""); // 통과했으니 지난 안내를 지웁니다 setTodos([...todos, { id:
  nextId, text: trimmed, done: false }]);
  nextId = nextId + 1;

  setText(""); // 입력칸 비우기
}
```

화면(누르면): 빈 칸에서 [추가] → 할 일을 입력해 주세요 화면(누르면): “장보기” 를 넣고 [추가] → 이미
있는 항목입니다 화면(누르면): “운동하기” 를 넣고 [추가] → 목록 3줄, 입력칸이 비고 빨간 글자도
사라집니다

— 왜 이렇게 했나

· 조기 반환을 세 번 씩입니다. 조건마다 return 으로 끊으면 else 가 필요 없습니다. · setError("") 를 성공
쪽에 둔 이유: 안 지우면 한 번 뜬 빨간 글자가

그 뒤로 계속 남습니다. 에러는 안 나고 화면만 조용히 틀립니다.

· `{ id: nextId, text: trimmed, done: false }` 는

`{ id: nextId, text, done: false }` 로 줄여 쓸 수도 있습니다.
키 이름과 변수 이름이 같으면 한 번만 적어도 되는 줄임 문법입니다.
(여기서는 변수 이름이 `trimmed` 라 줄일 수 없습니다)

· `nextId` 는 state 가 아니라 그냥 변수입니다. 화면에 안 보이는 값이라

바뀌어도 다시 그릴 필요가 없기 때문입니다.

— JS 판과 비교

JS 판에서는 여기서 `todoInput.value = ""` 로 입력칸을 직접 비우고 `render()` 를 직접 불렀습니다.
여기서는 `setText("")` 와 `setTodos(...)` 로 '데이터' 만 바꿉니다. 화면은 저절로 따라옵니다.

문제 5 완료 토글 ★ 가장 어려운 문제

```
function handleToggle(id) {
  setTodos(
    todos.map((todo) => (todo.id === id ? { ...todo, done: !todo.done } : todo))
  );
}
```

화면(누르면): "장보기" 글자를 누르면 회색 취소선이 생깁니다 화면(누르면): 상태줄이 "할 일 2개 (완료 2개, 남은 일 0개)" 로 바뀝니다

— 왜 이 문제가 가장 어려운가

지켜야 할 것이 두 겹이기 때문입니다.

- ① 배열이 새것이어야 한다 → `map` 이 새 배열을 만들어 줍니다
- ② 바꾸는 그 칸도 새 객체여야 한다 → `{ ...todo, done: !todo.done }`

① 만 지키면 어떻게 될까요? 이렇게 쓰는 실수가 아주 흔합니다.

```
todos.map((todo) => {
  if (todo.id === id) todo.done = !todo.done; // ← 객체를 직접 고침
  return todo;
})
```

이 코드는 이 화면에서는 '되는 것처럼' 보입니다. 배열이 새것이니까요. 하지만 원래 있던 객체를 그 자리에서 고쳐 버렸습니다. 그 객체를 다른 곳에서도 보고 있으면 그쪽까지 조용히 바뀝니다. 07단원 개념02의 얀은 복사 함정입니다.

② 만 지키고 ① 을 어기면(= 배열을 그대로 두고 안의 객체만 새로 넣으면) 화면이 아예 안 바뀝니다. 07단원 개념01 데모 ① 에서 본 그 상황입니다.

— JS 판과 비교

JS 판의 문제 5도 가장 어려웠습니다. 하지만 어려운 이유가 전혀 달랐습니다. 그쪽은 ui 하나에 이벤트를 붙이고(위임), e.target.closest("li") 로 항목을 찾고, dataset 값이 문자열이라 Number() 로 바꿔야 하는 것이 어려웠습니다. React 판에는 위임도 closest 도 dataset 도 없습니다. 대신 '불변' 이 어렵습니다.

문제 6 삭제

```
function handleDelete(id) {  
  setTodos(todos.filter((todo) => todo.id !== id));  
}
```

화면(누르면): "설거지" 의 [삭제] 를 누르면 그 줄만 사라집니다

— 왜 이렇게 했나

filter 는 '지울 것' 이 아니라 '남길 것' 을 고릅니다. 그래서 !== 입니다. === 로 쓰면 누른 것만 남고 나머지가 전부 사라집니다. 에러가 안 나고 화면만 반대로 나오므로 알아채기 어렵습니다.

문제 8 완료 항목 한꺼번에 지우기

```
function handleClearDone() {  
  setTodos(todos.filter((todo) => !todo.done));  
}
```

화면(누르면): [완료 항목 삭제] → "설거지" 가 사라지고 "장보기" 한 줄만 남습니다

— 왜 이렇게 했나

문제 6과 같은 filter 인데 조건만 다릅니다. "done 이 아닌 것만 남긴다" 이므로 !todo.done 입니다. 지운 개수를 따로 셀 필요도, 반복문을 돌 필요도 없습니다.

```

return (
  <div className="demo"> <h3>할 일 목록</h3> <form onSubmit=
{handleSubmit}> <input type="text" value={text} onChange={
(e) =>
setText(e.target.value)}
  placeholder="할 일을 입력하고 Enter"
  />{ " "}
  <button type="submit">추가</button> </form> <div className="error">{error}
</div> <div> <FilterButton value="all" label="전체" filter={filter} onChange=
{setFilter} /> <FilterButton value="active" label="미완료" filter={filter} onChange=
{setFilter} /> <FilterButton value="done" label="완료" filter={filter} onChange=
{setFilter} />{ " "}
  <button onClick={handleClearDone}>완료 항목 삭제</button> </div>

  { /* — 문제 2 — 목록 그리기 */ }
  <ul>
    {visibleTodos.map((todo) => (
      <TodoItem key={todo.id} todo={todo} onToggle={handleToggle} onDelete=
{handleDelete}
      />
    ))}
  </ul>

  { /* 05단원 개념05 - length 를 그대로 && 앞에 쓰면 0이 화면에 찍힙니다.
  그래서 === 0 으로 비교해 true/false 를 만든 뒤 씁니다. */ }
  {visibleTodos.length === 0 && <div className="output">항목이 없습니다</div>}

  <div className="output">
    할 일 {total}개 (완료 {doneCount}개, 남은 일 {leftCount}개)
  </div>
</div>
);
}

```

화면 목록에 "장보기" 와 취소선 그어진 "설거지" 두 줄,

맨 아래 "할 일 2개 (완료 1개, 남은 일 1개)"

화면(누르면): [미완료] → "장보기" 한 줄만 남고 [미완료] 버튼이 파랗게 됩니다 화면(누르면): [완료] → "설거지" 한 줄만 남습니다 화면(누르면): [완료 항목 삭제] 를 [완료] 필터 상태에서 누르면

목록이 비면서 "항목이 없습니다" 가 나옵니다

나란히 놓고 보기 — JS 판과 React 판 —

같은 일을 하는 코드를 붙여 두었습니다. 줄 수보다 '무엇을 적었는가' 를 보세요.

[1] 화면 그리기

JS 판 React 판

```
todoList.innerHTML = "";
visible.forEach(({ id, text, done }) => { {visibleTodos.map((todo) => (

const li = document.createElement("li");  <TodoItem key={todo.id} ... />
li.dataset.id = id;
if (done) li.classList.add("done");
const span = document.createElement("span");
span.classList.add("text");
span.textContent = text;
... appendChild 세 번 ...

});
```

JS 판은 '만드는 순서'를 적었고, React 판은 '생긴 모양'을 적었습니다. innerHTML = ""로 먼저 비우는 줄이 React 판에는 없습니다. 지난번에 그린 것을 치우는 일도 React가 합니다.

[2] 완료 토글

JS 판 React 판

```
const li = e.target.closest("li");
const id = Number(li.dataset.id);
todos = todos.map((todo) =>

function handleToggle(id) {
  setTodos(todos.map((todo) =>
    todo.id === id

    todo.id === id ? { ...todo, done: !todo.done } : todo));
  : todo);
}

render();
```

가운데 map 줄은 똑같습니다. 앞뒤가 사라졌을 뿐입니다. 앞의 두 줄(누른 항목 찾기)과 마지막 줄(render 다시 부르기)이 없어졌습니다.

[3] 처음 화면 그리기

JS 판 React 판

```
render(); ← 이 한 줄을 빠뜨리면 (없음)

화면이 통째로 비었습니다
```

JS 판 문제 8이 통째로 사라졌습니다. React는 컴포넌트를 처음 그릴 때부터 todos를 보고 그리므로, "처음 한 번은 직접 불러야 한다"는 일이 없습니다.

[4] 그래서 무엇이 남았나

사라진 것 : createElement · appendChild · innerHTML · classList · dataset ·

closest · 이벤트 위임 · render() 를 다시 부르기

새로 생긴 것 : 불변하게 바꾸기(스프레드 · map · filter) · key

공짜는 아닙니다. 하지만 새로 생긴 것은 '데이터를 다루는 규칙' 하나뿐이고, 사라진 것은 '화면을 손보는 방법' 여덟 가지입니다. 화면이 복잡해질수록 이 차이가 커집니다.

정리

- 1 이 앱은 JS자료 종합03과 같은 앱입니다. 요구사항이 하나도 다르지 않습니다.
- 2 JS 판의 createElement · appendChild · innerHTML · classList · dataset · closest 가 전부 사라졌습니다.
- 3 직접 만들던 render() 와 '바뀔 때마다 다시 부르기' 도 사라졌습니다. setTodos 가 그 일을 합니다.
- 4 대신 새로 지킬 것이 하나 생겼습니다. state 는 고치지 말고 새것을 만들어 넣습니다.
- 5 완료 토글은 배열도 새것(map), 그 안의 객체도 새것({ ...todo }) 이어야 합니다. 두 겹입니다.
- 6 보여 줄 목록과 개수는 state 가 아닙니다. todos 와 filter 에서 계산합니다(07단원 개념05).
- 7 여기까지가 04~07단원의 전부입니다. 다음 종합02부터는 09~12단원을 씁니다.

```
export default function Project01TodoAnswer() {
  return (
    <div> <h1>종합 01 정답 - 할 일 목록</h1> <p className="guide">
      먼저 스스로 만들어 본 다음에 보세요. <strong>추가 · 완료 · 삭제 · 필터
    </strong> 가
      전부 동작합니다.
      <br /> <br /> <strong>F12 → Console</strong> 도 함께 보세요. 화면을 만들기
      전에 배열 세 줄을
      검사해 둔 결과가 찍혀 있습니다.
      <br /> <br />
      코드 아래쪽 <strong>'나란히 놓고 보기'</strong> 에 JS자료 종합03과 같은
      자리의
      코드를 붙여 두었습니다. 두 번 만들어 본 사람만 볼 수 있는 비교입니다.
    </p> <TodoApp /> </div>
  );
}
```

13단원 종합 02 정답 장바구니

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 여세요.

먼저 스스로 만들어 본 다음에 보세요. 담기 · 수량 조절 · 빼기 · 총액 · 비우기가 전부 동작합니다.

— 이 파일의 순서

- 1 액션의 모양을 정한다 — 무엇이 일어날 수 있는지 목록을 만듭니다
- 2 reducer 를 만든다 — 그 일이 일어나면 값이 어떻게 되는지 적습니다
- 3 콘솔로 검사한다 — 화면을 열기 전에 규칙이 맞는지 확인합니다
- 4 Provider 로 감싼다 — 어느 깊이에서든 꺼내 쓸 수 있게 합니다
- 5 화면 조각을 만든다 — 조각들은 props 를 하나도 안 받습니다

화면을 먼저 만들고 규칙을 나중에 붙이는 것보다 이 순서가 훨씬 편합니다. 규칙이 컴포넌트 밖의 그냥 함수라서 브라우저 없이도 시험할 수 있기 때문입니다.

★ 이 실습에서 가장 어려운 것은 문제 1(담기) 입니다. '담기' 하나가 상황에 따라 완전히 다른 두 가지 일을 하기 때문입니다.

```
import { createContext, useContext, useReducer } from "react";
import Summary from "../_ui/Summary.jsx";

const MENU = [
  { id: 1, name: "아메리카노", price: 4000 },
  { id: 2, name: "라떼", price: 4500 },
  { id: 3, name: "케이크", price: 6000 },
  { id: 4, name: "삼각김밥", price: 1200 },
];
```

장바구니 한 칸의 모양입니다. 메뉴에 count 가 하나 붙었습니다.

```
const FIRST_ITEMS = [{ id: 1, name: "아메리카노", price: 4000, count: 2 }];
```

액션의 모양

12단원 개념04에서는 값을 payload 라는 이름 하나에 담았습니다. 이 파일은 개념04 [실수 6]에서 권한 대로 값마다 이름을 붙였습니다.

```
{ type: "add", menu: { id, name, price } }
{ type: "increase", id: 1 }
{ type: "decrease", id: 1 }
{ type: "remove", id: 1 }
{ type: "clear" }
```

이름을 붙이면 reducer 를 읽을 때 action.menu 인지 action.id 인지 헷갈리지 않습니다. payload 하나로 쓰면 액션마다 그 안에 무엇이 들었는지 따로 기억해야 합니다.

```
function cartReducer(state, action) {
  switch (action.type) {
```

문제 1 답기 ★ 가장 어려운 문제

```
case "add": {
```

some 은 “조건에 맞는 게 하나라도 있나” 를 true/false 로 알려 줍니다(JS자료 08단원).

```
const already = state.some((item) => item.id === action.menu.id);

if (already) {
```

① 이미 담긴 메뉴 — 그 칸만 새 객체로 바꾼 새 배열

```
return state.map((item) =>
  item.id === action.menu.id ? { ...item, count: item.count + 1 } : item
);
}
```

② 처음 담는 메뉴 — count 1 을 붙여 끝에 넣습니다

```
return [...state, { ...action.menu, count: 1 }];
}
```

문제 2 수량 올리기 / 내리기

```
case "increase":
  return state.map((item) =>
    item.id === action.id ? { ...item, count: item.count + 1 } : item
  );

case "decrease":
  return state.map((item) => {
    if (item.id !== action.id) return item; // 다른 칸은 손대지 않습니다 if
    (item.count <= 1) return item; // * 1 아래로는 안 내려갑니다 return { ...item, count:
    item.count - 1 };
  });
```

문제 3 빼기 / 비우기

```
case "remove":
  return state.filter((item) => item.id !== action.id);

case "clear":
  return [];

default:
```

모르는 type 이 오면 아무 일도 하지 않고 지금 값을 그대로 돌려줍니다.

```
return state;
}
}
```

— 왜 이렇게 했나 — 문제 1

‘담기’ 는 이름이 하나인데 하는 일이 둘입니다. 이것이 어려운 이유입니다. 먼저 some 으로 “이미 있나?” 를 묻고, 답에 따라 갈라야 합니다. ① 쪽에서 map 을 쓴 이유: 목록 전체를 새 배열로 만들면서 그 칸만 새 객체로 바꿉니다. ② 쪽에서 { ...action.menu, count: 1 } 을 쓴 이유: 메뉴 객체를 그대로 넣으면

장바구니와 메뉴판이 같은 객체를 보게 됩니다. 나중에 수량을 올리면 메뉴판까지 함께 바뀔 수 있습니다. 복사해서 넣어야 안전합니다.

case 에 종괄호 { } 를 씌운 것은 그 안에서 const already 를 선언하기 때문입니다.

종괄호가 없으면 다른 case 와 이름이 겹쳐서 에러가 납니다.

— 왜 이렇게 했나 — 문제 2

decrease 는 map 콜백을 여러 줄로 썼습니다. 조건이 둘이라 삼항으로는 읽기 나쁩니다. “다른 칸이면 그대로 / 1잔 이하면 그대로 / 아니면 하나 줄인 새 객체” 세 갈래입니다. ★ map 안에서 아무것도 안 돌려주는 갈래가 있으면 그 자리가 `undefined` 가 되어

화면이 터집니다(07단원 개념01 [실수 5]). 세 갈래 모두 `return` 이 있는지 보세요.

12단원 개념04의 장바구니는 수량이 0이 되면 목록에서 저절로 빠졌습니다. 이 파일은 1에서 멈추고, 빼는 것은 [빼기] 버튼으로만 하게 했습니다. 어느 쪽이 옳은 것은 아닙니다. 다만 ‘저절로 사라지는’ 화면은 사용자가 놀랍니다.

— 왜 이렇게 했나 — 문제 3

remove 는 filter 입니다. ‘지울 것’ 이 아니라 ‘남길 것’ 을 고르므로 `!==` 입니다. clear 는 [] 입니다. `state.length = 0` 처럼 원본을 고치면 안 됩니다. 빈 배열을 ‘새로’ 만들어 돌려줘야 React 가 바뀐 것을 알아챱니다.

화면 없이 검산하기

reducer 는 컴포넌트 밖의 그냥 함수입니다. 브라우저를 열기 전에 시험할 수 있습니다. reduce 는 액션 목록을 하나씩 접어 나가며 최종 장바구니를 만듭니다(JS자료 08단원).

```
const testActions = [
  { type: "add", menu: MENU[0] }, // 아메리카노
  { type: "add", menu: MENU[0] }, // 아메리카노 한 잔 더 → 수량만 2
  { type: "add", menu: MENU[2] }, // 케이크
  { type: "decrease", id: 1 }, // 아메리카노를 1잔으로
];

const tested = testActions.reduce(cartReducer, []);

console.log("[검산] " + tested.map((item) => item.name + " " + item.count +
"개").join(", "));
```

F12 콘솔 [검산] 아메리카노 1개, 케이크 1개

같은 메뉴를 두 번 담았는데 줄이 하나뿐입니다. 문제 1의 ㉠ 이 동작한 것입니다.

```
const twiceDown = [
  { type: "decrease", id: 1 },
  { type: "decrease", id: 1 },
].reduce(cartReducer, tested);

console.log("[검산] 1개에서 두 번 더 내리면 " + twiceDown[0].count + "개");
```

F12 콘솔 [검산] 1개에서 두 번 더 내리면 1개

1 아래로 안 내려간다는 규칙도 화면 없이 확인했습니다.

```
const cleared = cartReducer(tested, { type: "clear" });

console.log("[검산] 비운 뒤 칸 수: " + cleared.length + " / 원래 칸 수: " +
  tested.length);
```

F12 콘솔 [검산] 비운 뒤 칸 수: 0 / 원래 칸 수: 2

비운 뒤에도 원래 배열 `tested` 는 2칸 그대로입니다. 원본을 안 건드렸다는 뜻입니다.

저장소 (Context) —————

상자를 만듭니다. 기본값은 `null` 입니다. 이유는 `useCart` 에 있습니다.

```
const CartContext = createContext(null);

function CartProvider({ children }) {
```

문제 4 저장소 만들기

```
const [items, dispatch] = useReducer(cartReducer, FIRST_ITEMS);

return (
  <CartContext.Provider value={{ items: items, dispatch: dispatch }}>
    {children}
  </CartContext.Provider>
);
}
```

— 왜 이렇게 했나

`Provider` 를 직접 쓰지 않고 컴포넌트로 감쌌습니다. 그래서 쓰는 쪽이 이렇게 짧아집니다.

```
<CartProvider>
  <ShopScreen />
</CartProvider>
```

`useReducer` 도, `cartReducer` 도, 초기값도 쓰는 쪽에서는 안 보입니다. `children` 은 03단원 개념04에서 배웠습니다. 태그 사이에 넣은 것이 그대로 옵니다.

`value={{ ... }}` 의 중괄호가 두 겹인 이유:

바깥 중괄호는 “여기부터 자바스크립트” 라는 뜻이고(02단원 개념03), 안쪽 중괄호는 객체입니다. `style={{}}` 와 똑같은 이야기입니다. 한 겹만 쓰면 `items` 가 무슨 뜻인지 몰라서 그 자리에서 멈춥니다.

```
function useCart() {
  const cart = useContext(CartContext);
```

Provider 밖에서 부르면 조용히 틀리는 대신 그 자리에서 알려 줍니다.

```
if (cart === null) {
  throw new Error("useCart 는 <CartProvider> 안에서만 쓸 수 있습니다");
}

return cart;
}
```

문제 5 총액 구하기

```
function getTotal(items) {
  return items.reduce((sum, item) => sum + item.price * item.count, 0);
}
```

— 왜 이렇게 했나

한 칸의 값은 `price × count` 입니다. 그것을 전부 더하면 총액입니다. `reduce` 의 마지막 인자 0 이 ‘시작값’ 입니다. 이것을 빠뜨리면 빈 배열일 때 에러가 나고, 첫 칸이 숫자가 아니라 객체라서 결과가 `NaN` 이 됩니다.

총액을 state 로 두지 않은 이유가 중요합니다. `items` 가 바뀔 때마다 총액도 같이 고쳐야 하고, 한 군데라도 잊으면 “화면에 담긴 것과 합계가 안 맞는” 상태가 됩니다. 에러는 안 납니다. `items` 하나에서 계산해 내면 어긋날 수가 없습니다(07단원 개념05).

화면 조각

아래 네 조각은 props 를 하나도 받지 않습니다. 필요한 것은 각자 `useCart()` 로 꺼냅니다.

```
function MenuList() {
  const { dispatch } = useCart(); // items 는 안 쓰므로 dispatch 만 꺼냅니다
```

문제 6 담기 버튼

```
return (
  <div className="output"> <strong>메뉴판</strong> <ul>
    {MENU.map((menu) => (
```

13단원 종합 03 정답 사용자 검색

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 여세요.

먼저 스스로 만들어 본 다음에 보세요. 받아오기 · 로딩 · 에러 · 검색 · 정렬 · 상세 보기가 전부 동작합니다.

★ 이 앱은 JS자료 13단원 종합04와 같은 앱입니다. 그때는 `fetch` 와 `DOM` 으로 만들었고, 이번에는 `useEffect` 와 `state` 로 만듭니다.

★★ 인터넷 연결이 필요합니다. 인터넷이 막힌 곳이라면 실습프로젝트 폴더의 `index.html` 을 열고 `<script src="/오프라인_대체.js">` 줄을 감싼 주석 기호만 지우세요. 인터넷 요청 0건으로 똑같은 값이 나옵니다. 흉내 내는 것은 `jsonplaceholder.typicode.com` 하나뿐입니다.

★ 콘솔에 빨간 줄이 나오는 것은 정상입니다.

없는 주소 · 를 누르면 브라우저가 이렇게 적습니다.

Failed to load resource: the server responded with a status of 404 (Not Found)

우리 코드가 낸 에러가 아니라 브라우저가 알려 주는 줄입니다. 막을 방법이 없고, 막을 필요도 없습니다. 실무에서도 그냥 나옵니다.

★ 이 실습에서 가장 어려운 것은 문제 2(다시 불러오기) 입니다. 고칠 코드는 두 곳뿐인데, 왜 그래야 하는지가 어렵습니다.

```
import { useEffect, useState } from "react";
import Summary from "../_ui/Summary.jsx";

const BASE_URL = "https://jsonplaceholder.typicode.com";

function UserSearch() {
  const [users, setUsers] = useState([]); // 받아온 사람들
  const [loading, setLoading] = useState(true); // 기다리는 중인가
  const [error, setError] = useState(null); // 실패했다면 보여 줄 문구
  const [keyword, setKeyword] = useState(""); // 검색어 (06단원 제어 컴포넌트)
  const [sortBy, setSortByName] = useState(false); // 이름순인가
  const [selectedId, setSelectedId] = useState(null); // 상세로 볼 사람
  const [source, setSource] = useState("정상"); // 일부러 실패시켜 보는 스위치
  const [reloadCount, setReloadCount] = useState(0); // 문제 2에서 씁니다
```

— state 를 왜 이렇게 일곱 개나 두나

일곱 개 다 서로 계산해 낼 수 없는 값입니다. 특히 `users · loading · error` 셋은 09단원 개념04에서 본 그대로입니다. `users` 가 비어 있다는 것만으로는 '아직 기다리는 중' 인지 '실패했는지' '정말 0명인지' 를 구별할 수 없습니다. 그래서 셋이 다 필요합니다. 반대로 아래 `searched · visibleUsers · selectedUser` 는 계산해서 씁니다.

`source` 에 따라 부를 주소가 정해집니다. `/nouses` 는 서버에 없는 주소라 404 로 대답합니다.

```
const url = source === "정상" ? `${BASE_URL}/users` : `${BASE_URL}/nouses`;

useEffect(() => {
```

09단원 개념05의 것발입니다. 주소를 빨리 두 번 바꾸면 지난번 응답이 뒤늦게 도착할 수 있습니다. 그때 그 값을 버리기 위한 표시입니다.

```
let ignore = false;
```

문제 1 목록 받아오기

```
async function loadUsers() {
  setLoading(true);
  setError(null); // 새로 시작하니 지난번 에러를 지웁니다 console.log("[요청]
보냅니다:", url);
```

F12 콘솔 [요청] 보냅니다: <https://jsonplaceholder.typicode.com/users>

이 줄이 몇 번 찍히는지 세어 보세요. `effect` 가 몇 번 돌았는지와 같습니다. 개발 중에는 `StrictMode` 때문에 처음에 두 번 찍힙니다. 정상입니다(09단원 개념02).

```
try {
  const res = await fetch(url);
```

★ 이 세 줄이 이 파일의 핵심입니다. `fetch` 는 404 를 실패로 치지 않습니다. 서버가 대답을 했으니까요. 이 검사가 없으면 없는 주소를 불러도 조용히 지나갑니다(09단원 개념04).

```
if (!res.ok) {
  throw new Error(`서버 응답 오류 (${res.status})`);
}

const data = await res.json();
```

요청은 되돌릴 수 없습니다. 버릴 응답도 도착은 합니다. 그래서 이 줄은 버릴 응답일 때도 찍힙니다(09단원 개념05).

```
console.log("[도착] 받은 사람 수:", data.length);
```

F12 콘솔 [도착] 받은 사람 수: 10

```
if (ignore) return; // 옛날 요청의 응답이면 화면에 안 넣고 여기서  
끝냅니다 setUsers(data);  
} catch (err) {  
  if (ignore) return;
```

사용자에게는 다음에 무엇을 하면 되는지 알려 주는 우리말로

```
setError("사용자 목록을 불러오지 못했습니다. 잠시 후 다시 시도해 주세요.");
```

개발자에게는 자세히 (09단원 개념04 '좋은 에러 처리' 1번)

```
console.log("[실패]", err.message);
```

F12 콘솔 [실패] 서버 응답 오류 (404)

```
} finally {
```

★ 성공하든 실패하든 로딩 표시는 반드시 꺼야 합니다. try 안에 두면 실패한 사용자의 화면이 영원히 "불러오는 중..." 입니다. 단, 옛날 요청이 뒤늦게 끝난 경우에는 건드리지 않습니다.

```
if (!ignore) setLoading(false);  
}  
}  
  
loadUsers();  
  
return () => {  
  ignore = true;  
};
```

문제 2 다시 불러오기 ★ 가장 어려운 문제

useEffect 는 '의존성 배열 안의 값이 달라졌을 때' 만 다시 돕니다. 그래서 "다시 받아오게 하라" 는 요구는 React 에서 이렇게 바뀝니다.

"다시 받아오게 하라" → "의존성 배열 안의 값을 달라지게 하라"

reloadCount 는 그것만을 위해 만든 값입니다. 화면에 보이지도 않습니다. 버튼을 누를 때마다 1씩 늘어나므로 언제나 '달라진 값' 이 됩니다.

— 왜 `loadUsers()` 를 버튼에서 직접 부르지 않나

부를 수 없습니다. `loadUsers` 는 `useEffect` 안에 있어서 밖에서 안 보입니다. 밖으로 꺼낼 수도 있지만, 그러면 ‘언제 받아오는가’가 두 곳으로 흩어집니다. 지금은 이 한 줄만 보면 됩니다. “url 이 바뀌거나 다시 불러오기를 누르면 받아온다.”

— 의존성에 `users` 를 넣으면 안 되는 이유

받아오면 `users` 가 바뀌고 → effect 가 또 돌고 → 또 받아오고 ... 무한 루프입니다. 09단원 개념06에서 본 그것입니다. 의존성에는 ‘언제 다시 받아올지를 정하는 값’만 넣습니다.

```
}, [url, reloadCount]);
```

문제 3 검색으로 걸러내기

```
const searched = users.filter((user) => {
  if (keyword.trim() === "") return true; // 검색어가 없으면 전부 남깁니다
  const word = keyword.trim().toLowerCase();
  const nameMatch = user.name.toLowerCase().includes(word);
  const cityMatch = user.address.city.toLowerCase().includes(word);

  return nameMatch || cityMatch;
});
```

— 왜 양쪽 다 `toLowerCase` 인가

한쪽만 소문자로 바꾸면 반쪽짜리가 됩니다. 검색어만 소문자로 바꾸면 “Gwen” 을 찾을 때 못 찾습니다. 데이터만 소문자로 바꾸면 “GWEN” 을 찾을 때 못 찾습니다.

— `address.city` 를 꺼낼 때

`address` 는 객체 안의 객체입니다. `user.address.city` 로 두 번 들어갑니다. 서버가 준 데이터라 모양이 늘 같으므로 그대로 꺼내 씁니다.

문제 4 이름순 정렬

```
const visibleUsers = sortByNames
  ? [...searched].sort((a, b) => a.name.localeCompare(b.name))
  : searched;

function handleSort() {
  setSortByName(!sortByNames);
}
```

왜 [...searched] 로 복사했나]

sort 는 원본을 그 자리에서 정렬하고 자기 자신을 돌려줍니다(07단원 개념01). 복사하지 않으면 searched 가, 따라서 그 안의 users 순서까지 바뀝니다. 그러면 [원래 순서] 로 되돌릴 방법이 없어집니다. 받은 순서를 잃어버린 것입니다.

— 왜 localeCompare 인가

a - b 는 숫자끼리만 됩니다. 글자를 빼면 NaN 이 나와서 정렬이 안 됩니다. localeCompare 는 두 글자의 앞뒤를 알려 줍니다(JS자료 08단원 개념06).

상세로 볼 사람을 찾습니다. 이것도 state 가 아니라 계산해서 씁니다. find 는 못 찾으면 undefined 를 돌려줍니다(JS자료 08단원).


```
const selectedUser = users.find((user) => user.id === selectedId);

return (
  <div className="demo"> <h3>사용자 검색</h3> <div> <input type="text" value=
{keyword} onChange={(e) => setKeyword(e.target.value)}
    placeholder="이름이나 도시로 검색"
  />{" "}
  {/* 버튼 글자는 '지금 누르면 무엇이 되는지' 를 뜻합니다 */}
  <button onClick={handleSort}>{sortByName ? "원래 순서" : "이름순 정렬"}
</button>
  {/* —— 문제 2 의 ① —— 누를 때마다 달라지는 값을 만듭니다 */}
  <button onClick={() => setReloadCount(reloadCount + 1)}>다시 불러오기
</button> </div> <div>
    주소 고르기{" "}
    <button className={source === "정상" ? "on" : ""} onClick={() =>
setSource("정상")}>
      정상 주소
    </button> <button className={source === "정상" ? "" : "on"}
      onClick={() => setSource("없는주소")}
    >
      없는 주소 (404)
    </button> </div>

    {/* —— 문제 5 —— 화면 세 갈래 (로딩 → 에러 → 성공) */}
    {loading && <p className="output">불러오는 중...</p>}

    {!loading && error !== null && <p className="output error">{error}</p>}
```

```

{!loading && error === null && (
  <div>
    {/* —— 문제 6 —— 목록 그리기 */}
    <ul>
      {visibleUsers.map((user) => (
        <li key={user.id} onClick={() => setSelectedId(user.id)}
          style={{ cursor: "pointer" }}
        >
          <strong>{user.name}</strong> <div>
            {user.address.city} · {user.email}
          </div> </li>
        </div>
      ))}
    </ul>

    {/* —— 문제 7 —— 검색 결과가 없을 때 */}
    {visibleUsers.length === 0 && (
      <div className="output">검색 결과가 없습니다</div>
    )}
  </div>
)}

{/* —— 문제 8 —— 상세 보기 */}
<div className="output">
  {selectedUser ? (
    <div> <div>이름: {selectedUser.name}</div> <div>이메일:
    {selectedUser.email}</div> <div>전화: {selectedUser.phone}</div> <div>도시:
    {selectedUser.address.city}</div> <div>회사: {selectedUser.company.name}</div> </div>

    ) : (
      <div>목록에서 사람을 눌러 보세요</div>
    )}
</div>
</div>
);
}

```

화면 잠깐 "불러오는 중..." 이 보였다가 사람 10명이 나옵니다.

첫 줄은 Leanne Graham / Gwenborough · Sincere@april.biz

화면(누르면): [이름순 정렬] → 버튼 글자가 "원래 순서" 로 바뀌고

맨 위가 Chelsey Dietrich 가 됩니다

화면(누르면): [없는 주소 (404)] → 빨간 글씨로

"사용자 목록을 불러오지 못했습니다. 잠시 후 다시 시도해 주세요."

화면(누르면): 목록에서 Chelsey Dietrich → 아래 상자에 다섯 줄이 나옵니다

화면을 가르는 순서가 왜 중요한가

위 세 갈래를 이 순서로 썼습니다.

```
loading → error → 성공
```

순서를 바꿔 성공을 먼저 보면 이런 화면이 됩니다.

다시 불러오기 · 를 누른 직후, 아직 새 데이터가 안 왔는데

지난번 목록이 그대로 남아 있습니다. 사용자는 버튼이 안 먹었다고 생각합니다.

error 를 성공보다 뒤에 두면 이런 화면이 됩니다.

없는 주소 · 로 실패했는데, users 에는 아까 받아 둔 10명이 남아 있으니

목록과 빨간 안내가 같이 보입니다. 무엇을 믿어야 할지 알 수 없습니다.

세 상태는 동시에 참일 수 없습니다. 화면도 그렇게 그려야 합니다.

JS 판과 비교

[1] 상세 보기

JS 판은 li 에 data-id 를 심고, 눌린 뒤 closest("li") 로 항목을 되찾고, dataset 값을 Number() 로 바꿔야 했습니다. 그 파일에서 가장 어려운 문제였습니다. 여기서는 onClick={() => setSelectedId(user.id)} 한 줄입니다. 각 줄이 자기 user 를 이미 알고 있고, id 는 처음부터 숫자입니다.

[2] '못 불러왔다' 와 '검색 결과가 없다' 구분

JS 판은 hasError 라는 변수를 따로 두고, render 안에서 "실패한 상태면 안내를 건드리지 말 것" 을 손으로 지켜야 했습니다. 여기서는 화면을 세 갈래로 가른 것만으로 저절로 해결됩니다. "검색 결과가 없습니다" 가 성공 갈래 안쪽에 있기 때문입니다.

[3] 다시 불러오기

JS 판은 loadUsers() 를 그냥 한 번 더 부르면 됐습니다. React 에서는 그 대신 의존성 배열 안의 값을 바꿉니다. 이것이 이 파일에서 가장 낯선 부분이고, 그래서 문제 2가 가장 어렵습니다.

[4] 실시간 검색

JS 판은 input 이벤트에 render 를 붙였습니다.
여기서는 keyword 가 state 라서 글자가 바뀌면 화면이 저절로 다시 그려집니다.
"검색할 때 화면을 다시 그려라" 라는 코드가 아예 없습니다.

정리

- 1 받아오는 화면은 로딩·에러·성공 셋 중 하나입니다. 화면도 그 순서로 가릅니다. 순서를 바꾸면 조용히 이상해집니다.
- 2 fetch 는 404 를 실패로 치지 않습니다. if (!res.ok) throw 한 줄이 그것을 실패로 만들어 줍니다.
- 3 로딩을 끄는 일은 finally 에 둡니다. try 안에 두면 실패했을 때 영원히 불러오는 중이 됩니다.
- 4 useEffect 를 다시 돌리는 방법은 하나뿐입니다. 의존성 배열 안의 값을 바꾸는 것입니다.
- 5 검색·정렬·상세는 전부 계산해서 씁니다. state 는 users·keyword·sortByName·selectedId 로 충분합니다.
- 6 sort 는 원본을 바꿉니다. [...배열] 로 복사한 뒤에 정렬해야 원래 순서로 돌아올 수 있습니다.
- 7 JS 판의 dataset·closest·hasError 가 통째로 사라졌습니다. 화면을 상태로 가르니 저절로 정리됐습니다.

```
export default function Project03UserSearchAnswer() {
  return (
    <div> <h1>종합 03 정답 - 사용자 검색</h1> <p className="guide">
      먼저 스스로 만들어 본 다음에 보세요. <strong>인터넷 연결이 필요합니다.
    </strong> <br /> <br /> <strong>F12 → Console</strong> 을 함께 열어 두세요. 받아온
    사람 수와 실패 이유가
      찍힙니다.
    <br /> <br />
    인터넷이 막혀 있다면 실습프로젝트 폴더의 <code>index.html</code> 에서{" "}
    <code>오프라인_대체.js</code> 줄의 주석 기호만 지우세요.
    <br /> <br /> <strong>[없는 주소 (404)]</strong> 를 누르면 콘솔에 빨간{" "}
    <code>Failed to load resource ... 404</code> 가 나옵니다. <strong>정상입니다.
  </strong>{" "}
    브라우저가 알려 주는 줄이지 우리 코드의 에러가 아닙니다. 우리가 낸 것은
    한글로,
    브라우저가 낸 것은 영어로 나옵니다.
    </p> <UserSearch /> </div>
  );
}
```

13단원 종합 04 정답 메모장 (정답)

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 여세요.

★ 먼저 종합04_메모장.jsx 를 스스로 풀어 본 뒤에 이 파일을 여세요.

이 파일은 완성본입니다. 각 문제 자리에 [문제 N 정답] 표시를 해 두었고, 왜 그렇게 했는지도 함께 적었습니다. **답보다 이유를 읽으세요.**

— 쓰는 단원

09 useEffect · fetch · 로딩/에러 11 Router · Link · useParams · useNavigate 12 Context · useReducer 06 제어 컴포넌트 / 07 불변 갱신

★ 이 예제 안에서는 새로고침(F5)을 하지 마세요. 왼쪽 메뉴 선택이 처음으로 돌아가서 이 예제가 화면에서 사라집니다. 주소를 되돌리려면 왼쪽 메뉴에서 이 예제를 다시 고르면 됩니다.

★★ 인터넷 연결이 필요합니다. 막힌 곳이라면 실습프로젝트 폴더의 index.html 을 열고 `<script src="/오프라인_대체.js">` 줄을 감싼 주석 기호만 지우세요.

```
import { createContext, useContext, useEffect, useReducer, useState } from "react";
import {
  BrowserRouter,
  Routes,
  Route,
  Link,
  useParams,
  useNavigate,
} from "react-router-dom";
import Summary from "../_ui/Summary.jsx";

const BASE_URL = "https://jsonplaceholder.typicode.com";
```

1. 메모 보관소 — Context + useReducer (12단원)

```
const MemoContext = createContext(null);
```

문제 1·2·3 정답 · reducer 세 갈래

reducer 는 “지금 상태 + 무슨 일 = 새 상태” 를 적는 함수입니다. 규칙이 하나 있습니다. **state 를 고치지 말고 새로 만들어 돌려줍니다.** (07단원) `state.push(...)` 를 쓰면 화면이 안 바뀝니다.

```
function memoReducer(state, action) {
  switch (action.type) {
    case "채우기":
```

서버에서 받아온 것으로 통째로 갈아 끼웁니다.

```
return action.memos;

    case "추가": {
```

— 문제 1 정답

새 번호는 지금 있는 것 중 가장 큰 번호 + 1로 정합니다. 목록이 비어 있으면 `Math.max()`가 `-Infinity`를 주므로 따로 처리합니다.

```
const nextId = state.length === 0 ? 1 : Math.max(...state.map((m) => m.id)) + 1;
```

새 메모를 맨 앞에 붙입니다. [새것, ...기존]이라 원본은 그대로입니다.

```
return [{ id: nextId, title: action.title, body: action.body }, ...state];
}

    case "삭제":
```

문제 2 정답 · `filter`는 조건에 맞는 것만 남긴 '새 배열'을 줍니다.

```
return state.filter((m) => m.id !== action.id);

    case "수정":
```

— 문제 3 정답

`map`으로 전부 훑으면서 번호가 같은 것만 새 객체로 바꿔 넣습니다. `{ ...m, title: ... }`은 나머지는 그대로 두고 두 칸만 바꾼 새 객체입니다. (07단원)

```
return state.map((m) =>
  m.id === action.id ? { ...m, title: action.title, body: action.body } : m
);

default:
```

모르는 이름이 오면 아무것도 하지 않습니다.

```

return state;
}
}

function MemoProvider({ children }) {
  const [memos, dispatch] = useReducer(memoReducer, []);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

```

문제 4 정답 · 처음 한 번만 서버에서 메모 5개를 받아옵니다. (09단원)

```

useEffect(() => {
  let ignore = false; // 09단원 개념05의 깃발
  async function loadMemos() {
    try {
      const res = await fetch(`${BASE_URL}/posts?_limit=5`);

```

★ fetch 는 404 를 실패로 치지 않습니다. 이 한 줄이 실패로 만들어 줍니다. (09단원)

```

    if (!res.ok) throw new Error(`서버가 ${res.status} 로 답했습니다`);

    const data = await res.json();
    if (ignore) return;

```

서버가 주는 모양은 { userId, id, title, body } 입니다. 우리에게 필요한 세 칸만 골라 담습니다.

```

    dispatch({
      type: "채우기",
      memos: data.map((post) => ({
        id: post.id,
        title: post.title,
        body: post.body,
      })),
    });

    console.log("메모를 받아왔습니다:", data.length);

```

F12 콘솔 메모를 받아왔습니다: 5

```

  } catch (e) {
    if (!ignore) setError(e.message);
  } finally {

```

★ 로딩을 끄는 일은 finally 에 둡니다. try 안에 두면 실패했을 때 영원히 "불러오는 중" 이 됩니다.

```

    if (!ignore) setLoading(false);
  }
}

loadMemos();
return () => {
  ignore = true;
};
}, []);

```

Provider 로 감싼 안쪽이면 어디서든 이 값을 꺼내 쓸 수 있습니다.

```

return (
  <MemoContext.Provider value={{ memos, dispatch, loading, error }}>
    {children}
  </MemoContext.Provider>
);
}

```

Context 를 꺼내는 일을 커스텀 훅으로 감쌌습니다. (10단원) 이러면 쓰는 쪽에서 useContext 와 MemoContext 를 몰라도 됩니다.

```

function useMemos() {
  return useContext(MemoContext);
}

```

2. 화면들

목록 화면

```

function MemoListPage() {
  const { memos, loading, error } = useMemos();

  if (loading) return <p>불러오는 중...</p>;
  if (error) return <p className="error">못 불러왔습니다: {error}</p>;
  if (memos.length === 0) return <p>메모가 하나도 없습니다.</p>;

  return (
    <div> <p>메모 {memos.length}개</p> <ul>
      {memos.map((memo) => (
        <li key={memo.id}>
          {/* 목록에서 상세로 가는 링크입니다. 주소에 번호를 끼워 넣습니다. */}
          <Link to={`/m/${memo.id}`}>{memo.title}</Link> </li>
        )))}
    </ul> </div>
  );
}

```



```
);
}
```

상세 · 수정 화면

```
function MemoDetailPage() {
  const { memos, dispatch, loading } = useMemos();
  const { id } = useParams();
  const navigate = useNavigate();
```

★★ [문제 6 정답] 이 파일에서 가장 어려운 곳입니다.

useParams 가 주는 값은 **언제나 문자열**입니다. 주소는 글자니까요. 반면 memo.id 는 숫자입니다. 그래서 이렇게 쓰면 영원히 못 찾습니다.

```
memos.find((m) => m.id === id)    ← "3" === 3 은 false
```

Number(id) 로 숫자로 바꿔서 비교해야 합니다. JS자료 11단원에서 dataset 값이 문자열이라 계산이 틀렸던 것과 같은 함정입니다.

```
const memo = memos.find((m) => m.id === Number(id));
```

(상세 화면에 들어가면) 콘솔: 주소에서 받은 id: 3 string

```
console.log("주소에서 받은 id:", id, typeof id);
```

수정용 입력칸입니다. 처음 값은 찾은 메모에서 가져옵니다. (06단원 제어 컴포넌트)

```
const [title, setTitle] = useState("");
const [body, setBody] = useState("");
```

메모를 찾은 뒤에 입력칸을 채웁니다. memo 가 바뀔 때만 돌아야 하므로 의존성에 memo 를 적었습니다. (09단원)

```
useEffect(() => {
  if (memo) {
    setTitle(memo.title);
    setBody(memo.body);
  }
}, [memo]);
```

아직 받아오는 중이면 “없는 메모” 라고 단정하면 안 됩니다.

```
if (loading) return <p>불러오는 중...</p>;
```

문제 8 정답 · 없는 번호로 들어왔을 때

```
if (!memo) {
  return (
    <div> <p className="error">{id}번 메모는 없습니다.</p> <Link to="/m">목록으로
    돌아가기</Link> </div>
  );
}

function handleSave(e) {
  e.preventDefault(); // 06단원 - 새로그침 막기 dispatch({ type: "수정", id:
memo.id, title: title, body: body });
}
```

문제 7 정답 · 저장한 뒤 목록으로 보냅니다. (11단원 useNavigate)

Link 는 사용자가 '누르는' 것이고, navigate 는 코드가 '보내는' 것입니다.

```
navigate("/m");
}

function handleDelete() {
  dispatch({ type: "삭제", id: memo.id });
  navigate("/m");
}

return (
  <form onSubmit={handleSave}> <p> <input value={title} onChange={(e) =>
setTitle(e.target.value)}
    placeholder="제목"
    size="40"
  />
  </p> <p> <textarea value={body} onChange={(e) => setBody(e.target.value)}
    rows="4"
    cols="45"
  />
  </p> <button type="submit">저장</button> <button type="button" onClick=
{handleDelete}>
    삭제
  </button>
  { /* * type="button" 을 빼면 이 버튼도 폼을 제출합니다. (06단원 개념04) */ }
</form>
);
}
```

새 메모 화면

```
function MemoNewPage() {
  const { dispatch } = useMemos();
  const navigate = useNavigate();

  const [title, setTitle] = useState("");
  const [body, setBody] = useState("");
  const [message, setMessage] = useState("");

  function handleSubmit(e) {
    e.preventDefault();
```

공백만 친 것도 빈 것으로 봅니다. (06단원 trim)

```
if (title.trim() === "") {
  setMessage("제목을 입력해 주세요");
  return;
}
```

여기서 { type, title, body } 처럼 쓰지 않고 이름을 또박또박 적었습니다. 같은 이름일 때 title: title 을 title 하나로 줄여 쓸 수 있는데, 처음 보면 헷갈리므로 이 자료에서는 줄이지 않았습니다.

```
dispatch({ type: "추가", title: title.trim(), body: body.trim() });
navigate("/m");
}

return (
  <form onSubmit={handleSubmit}> <p> <input value={title} onChange={(e) =>
setTitle(e.target.value)}
  placeholder="제목"
  size="40"
  />
  </p> <p> <textarea value={body} onChange={(e) => setBody(e.target.value)}
  rows="4"
  cols="45"
  placeholder="내용"
  />
  </p> <div className="error">{message}</div> <button type="submit">저장
</button> </form>
);
}
```

어느 주소에도 안 맞을 때 나오는 화면입니다. 이 예제는 왼쪽 메뉴 안에서 도는 작은 앱이라, 처음 골랐을 때의 주소가 이 앱이 아는 주소가 아닙니다. 그래서 이 화면이 먼저 보입니다.

```
function StartHere() {
  return <p>위의 [메모 목록] 을 눌러 시작하세요.</p>;
}
```

3. 조립

```
function MemoApp() {
  return (
```

예제가 메뉴 안에서 돌기 때문에 파일마다 자기 BrowserRouter 를 가집니다. 진짜 앱에서는 맨 바깥에 딱 한 번만 둡니다. (11단원 개념01)

```
<BrowserRouter>
  <MemoProvider> <div className="demo"> <p> <Link to="/m">메모 목록</Link> ·
  <Link to="/m/new">새 메모</Link> </p>

    {/* [문제 5 정답] 주소 세 개 + 나머지 전부 */}
    <Routes> <Route path="/m" element={<MemoListPage />} />
      <Route path="/m/new" element={<MemoNewPage />} />
      {/* * /m/new 를 /m/:id 보다 먼저 써야 합니다.
        순서가 바뀌면 "new" 가 :id 로 잡혀서 새 메모 화면이 안 나옵니다. */}
      <Route path="/m/:id" element={<MemoDetailPage />} />
      <Route path="*" element={<StartHere />} />
    </Routes> </div> </MemoProvider>
</BrowserRouter>
);
}
```

정리

- 1 Context 는 값을 내려보내는 통로, useReducer 는 그 값을 바꾸는 규칙입니다. 둘을 한 Provider 에 묶으면 전역 보관소가 됩니다.
- 2 reducer 는 state 를 고치지 않고 새로 만들어 돌려줍니다. push 를 쓰면 화면이 안 바뀝니다.
- 3 받아온 데이터로 화면을 만들 때는 로딩·에러·성공 세 갈래를 먼저 가릅니다.
- 4 useParams 가 주는 값은 언제나 문자열입니다. 숫자 id 와 비교하려면 Number() 로 바꿉니다.
- 5 Link 는 사용자가 누르는 것, useNavigate 는 코드가 보내는 것입니다. 저장한 뒤 이동은 navigate 입니다.
- 6 /m/new 처럼 고정된 주소는 /m/:id 보다 먼저 적어야 합니다. 순서가 바뀌면 new 가 id 로 잡힙니다.
- 7 없는 번호로 들어왔을 때의 화면을 반드시 만들어 두세요. 안 만들면 빈 화면이 됩니다.

```

export default function Project04MemoPadAnswer() {
  return (
    <div> <h1>종합 04 - 메모장 (정답)</h1> <p className="guide"> <strong>정답
파일입니다.</strong> 먼저 <strong>종합04_메모장.jsx</strong> 를 스스로
풀어 본 뒤에 보세요.
    <br /> <br /> <strong>가장 어려운 것은 문제 6(상세 화면에서 메모 찾기)
</strong> 입니다.
    <code>useParams</code> 가 주는 값이 <strong>문자열</strong>이라는 것 하나
때문에
    에러도 경고도 없이 "없는 메모" 가 나옵니다.
    <br /> <br />
    이 예제 안에서 <strong>새로고침(F5)</strong>을 하지 마세요.</strong> 왼쪽 메뉴 선택이
처음으로 돌아가 예제가 사라집니다. 그때는 메뉴에서 다시 고르면 됩니다.
    </p> <MemoApp /> </div>
  );
}

```

14단원 연습문제 정답.jsx 정답 서버와 연동

RUN 터미널 두 개

- ① 실습프로젝트 에서 npm run dev
- ② 실습프로젝트/14단원_서버 에서 node 서버.js

★ 코드만 보지 말고 각 문제 아래의 설명을 읽으세요. 왜 그렇게 쓰는지가 답보다 중요합니다.

★★ 그리고 정답을 본 뒤에 연습문제 파일에서 다시 손으로 써 보세요.

```
import { useState, useEffect, useRef } from "react";
import { useForm } from "react-hook-form";
import Summary from "../_ui/Summary.jsx";
import { 서버주소, 서버안내 } from "../_서버주소.js";
```

문제 1 정답

```
function Problem01() {
  const [설비들, set설비들] = useState([]);

  useEffect(() => {
    (async () => {
      try {
        const 응답 = await fetch(`${서버주소}/equipments`);
        if (!응답.ok) return;
        const 자료 = await 응답.json();
        set설비들(자료.설비들);
      } catch {
```

서버가 안 켜졌으면 그냥 빈 채로 둡니다 (문제 2에서 다룹니다)

```
    }
  })();
}, []);

return (
  <div className="demo"> <h3>문제 1 - 목록 받아 그리기</h3> <ul className="output">
    {설비들.map((하나) => (
      <li key={하나.번호}>{하나.이름}</li>
```

```

    ))}
  </ul> </div>
);
}

```

★ `useEffect` 안에서 바로 `async` 를 못 씁니다. `useEffect(async () => {...})` 는 안 됩니다. `useEffect` 는 **치우는 함수**를 돌려받길 기대하는데, `async` 함수는 `Promise` 를 돌려주기 때문입니다. 그래서 안에서 `async` 함수를 만들어 바로 부릅니다. (09단원 개념03)

★★ `[]` 를 빼먹으면 받아올 때마다 다시 그려지고, 다시 그려지니 또 받아옵니다. 무한히 돕니다. Network 탭이 요청으로 가득 찹니다.

문제 2 정답

```

function Problem02() {
  const [상태, set상태] = useState("아직");
  const [말, set말] = useState("");

  async function 불러오기() {
    set상태("받는중");
    set말("");

    try {
      const 응답 = await fetch(`${서버주소}/equipments?slow=1200`);
      if (!응답.ok) throw new Error(`서버가 ${응답.status} 를 보냈습니다`);

      const 자료 = await 응답.json();
      set상태("됨");
      set말(`${자료.설비들.length}건 받았습니다`);
    } catch (오류) {
      set상태("안됨");
      set말(오류.name === "TypeError" ? 서버안내 : 오류.message);
    }
  }

  return (
    <div className="demo"> <h3>문제 2 - 불러오는 중 보여 주기</h3> <button onClick=
    {불러오기}>느리게 불러오기</button> <div className="output">
      {상태 === "받는중" ? "불러오는 중..." : 말 || "버튼을 눌러 보세요"}
    </div> </div>
  );
}

```

★★ **set**상태(“받는중”)을 **try** 밖 맨 위에 둡니다. try 안에 두면 오류가 났을 때 “받는중”인 채로 멈출 수 있습니다.

★ 오류를 두 갈래로 갈랐습니다. `TypeError`면 “서버를 켜세요”, 그 밖이면 서버가 알려 준 말. 사용자에게 할 말이 다릅니다. (개념01 섹션3)

문제 3 정답

```
function Problem03() {
  const [말, set말] = useState("");

  async function 눌렀을때() {
    set말("");
    try {
      const 응답 = await fetch(`${서버주소}/equipments?fail=yes`);
      const 몸 = await 응답.json().catch(() => ({}));

      if (!응답.ok) {
        set말(몸.메시지 || `서버가 ${응답.status} 를 보냈습니다`);
        return;
      }
      set말("이번에는 성공했습니다");
    } catch {
      set말(서버안내);
    }
  }

  return (
    <div className="demo"> <h3>문제 3 - 서버가 보낸 이유 보여 주기
    </h3> <button onClick={눌렀을때}>고장 내기</button> <div className="output">{말 ||
    "버튼을 눌러 보세요"></div> </div>
  );
}
```

★★★ `.json()`을 **응답.ok**를 보기 전에 부른 것에 주목하세요. 실패한 응답에도 본문이 들어 있기 때문입니다. `if (!응답.ok) throw new Error("실패")`로 끝내면 그 본문을 버리게 됩니다.

★ `.catch(() => ({}))`를 붙인 이유 — 서버가 JSON이 아닌 것(HTML 오류 페이지 등)을 보낼 수도 있습니다. 그때 `.json()`이 터지면 진짜 원인이 가려집니다. (JS자료 14단원 개념05)

문제 4 정답

```
function Problem04() {
  const { register, handleSubmit } = useForm({ defaultValues: { 이름: "" } });
```


15단원 연습문제 정답.tsx 정답 부록 타입스크립트

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

15단원(부록) · 연습문제 정답 (10문항)

먼저 스스로 풀어 보고 나서 여세요.

이 파일은 `npm run typecheck` 를 통과합니다. 밀줄이 하나도 없습니다. 문제 10의 에러 코드는 **실제로 돌려서 나온 메시지를 그대로 옮긴 것**입니다.

타입은 정답이 하나가 아닐 때가 많습니다. 화면이 같고 밀줄이 없으면, 아래와 조금 달라도 맞은 것입니다.

```
import { useState } from "react";
import Summary from "../_ui/Summary.jsx";
```

문제 1 (개념02)

가격이 문자열이라 더하기가 이어 붙이기가 되던 것을 고칩니다.

```
const q1Price: number = 4000; // string → number, "4000" → 4000
const q1Total = q1Price + 500;
```

화면 `4000 + 500 = 4500`

고치기 전에는 `4000500` 이었습니다. JS자료 01단원 개념05의 “10” + 5 와 같은 일입니다. 타입만 `number` 로 바꾸고 값에 따옴표를 남기면 이 밀줄이 납니다. `error TS2322: Type 'string' is not assignable to type 'number'.` 그래서 “고쳐야 할 곳이 한 군데 더 있다” 는 것을 타입이 알려 준 셈입니다.

문제 2 (개념02)

글자만 들어 있는 배열이라고 적고, 항목을 하나 더 넣습니다.

```
const q2Menu: string[] = ["아메리카노", "라떼", "케이크"];
```

화면 아메리카노 / 라떼 / 케이크 (3개)

타입을 붙였는지 확인하려고 숫자를 하나 넣어 보면 이 밀줄이 납니다. `error TS2322: Type 'number' is not assignable to type 'string'.`

사실 타입을 안 적어도 오른쪽 값을 보고 string[] 로 추론됩니다(개념02 섹션2). 그래도 적어 두면 “여기에는 글자만 넣는다” 는 약속이 눈에 보입니다.

문제 3 (개념02)

타입 별칭에 속성을 추가합니다. 타입과 값을 같이 고쳐야 합니다.

```
type Q3Menu = { name: string; price: number };

const q3Latte: Q3Menu = { name: "라떼", price: 4500 };

const q3PriceText = q3Latte.price;
```

화면 라떼 - 4500원

타입에만 price 를 추가하면 값 쪽에 이 밑줄이 납니다. error TS2741: Property 'price' is missing in type '{ name: string; }' but required in type 'Q3Menu'. “Q3Menu 는 price 가 있어야 하는데 그 객체에는 없다” 는 뜻입니다.

문제 4 (개념02)

별명을 선택 속성으로 바꾸고, 김민준에게서는 아예 지웁니다.

```
type Q4Person = {
  name: string;
  nickname?: string; // 있어도 되고 없어도 됩니다
};

const q4Minjun: Q4Person = { name: "김민준" }; // nickname 을 아예 안 적습니다
const q4Seoyeon: Q4Person = { name: "이서연", nickname: "서니" };
```

화면 김민준(별명 없음) / 이서연(서니)

고치기 전에는 “김민준()” 이었습니다. nickname 이 빈 문자열이었기 때문입니다. ?? 는 null 이나 undefined 일 때만 오른쪽을 씁니다. 빈 문자열은 “있는 값” 이라 그대로 쓰이고, 화면에는 아무것도 안 보입니다. 03단원 개념03 섹션3에서 본 “기본값은 undefined 일 때만” 과 완전히 같은 이야기입니다.

nickname?: string 은 string | undefined 라는 뜻이라, 확인 없이 쓰면 막습니다.

```
const n = q4Minjun.nickname.length;
```

error TS18048: 'q4Minjun.nickname' is possibly 'undefined'.

문제 5 (개념02)

정해진 셋 중 하나만 받게 합니다.

```
type Q5Size = "S" | "M" | "L";

const q5MySize: Q5Size = "L";
```

화면 내 사이즈: L

타입만 바꾸고 "라지" 를 그대로 두면 이 밀줄이 납니다. error TS2322: Type '"라지"' is not assignable to type 'Q5Size'. 소문자 "l" 도 안 됩니다. 적어 둔 셋과 글자 하나까지 같아야 합니다.

string 이라고만 적었을 때와 비교해 보세요. 그때는 "라지" 도 "엘" 도 다 통과했습니다. 통과했지만 나중에 화면이 이상해졌을 것입니다. 유니온은 그걸 미리 막습니다.

문제 6 (개념03)

props 타입에 가격을 추가하고, 컴포넌트와 쓰는 쪽을 함께 고칩니다.

```
type Q6Props = {
  name: string;
  price: number;
};

function Q6Item({ name, price }: Q6Props) {
  return (
    <p>
      {name} - {price}원
    </p>
  );
}
```

쓸 때: <Q6Item name="아메리카노" price={4000} />

화면 아메리카노 - 4000원

라떼 - 4500원

타입에만 price 를 추가하면 쓰는 쪽 두 군데에 이 밀줄이 납니다. error TS2741: Property 'price' is missing in type '{ name: string; }' but required in type 'Q6Props'. 밀줄이 두 개 생겨서 놀랐을 수 있지만, 고칠 곳을 전부 찾아 준 것입니다. 03단원에서는 이런 것을 눈으로 찾아야 했습니다.

price 를 숫자가 아니라 문자열로 넘기면 다른 밀줄이 납니다.

```
<Q6Item name="라떼" price="4500" />
```

error TS2322: Type 'string' is not assignable to type 'number'.

문제 7 (개념03)

children 을 타입에 적고 화면에 그림니다.

```
type Q7Props = {
  title: string;
  children: React.ReactNode; // 화면에 그릴 수 있는 것 아무거나
};

function Q7Box({ title, children }: Q7Props) {
  return (
    <div className="output"> <strong>{title}</strong> <div>{children}</div> </div>
  );
}
```

쓸 때: <Q7Box title="오늘의 메뉴">

```
<p>아메리카노가 제일 잘 나갑니다.</p>
```

```
</Q7Box>
```

화면 오늘의 메뉴

아메리카노가 제일 잘 나갑니다.

타입에 안 적고 컴포넌트에서 꺼내 쓰면 이 밑줄이 납니다. error TS2339: Property 'children' does not exist on type 'Q7Props'. React 가 알아서 넣어 주지 않습니다. 쓸 것이면 직접 적어야 합니다.

children?: React.ReactNode 로 물음표를 붙이면 안이 비어도 됩니다. 붙이지 않으면 <Q7Box title="가" /> 처럼 비워 뒀을 때 이 밑줄이 납니다. error TS2741: Property 'children' is missing in type '{ title: string; }' but required in type 'Q7Props'.

문제 8 [응용] (개념04)

이벤트 타입을 붙이고 장바구니를 완성합니다.

```
function Q8Cart() {
  const [q8Text, setQ8Text] = useState("");
  const [q8Items, setQ8Items] = useState<string[]>([]);
```

any 를 지우고 입력칸의 바뀜 이벤트 타입을 적었습니다

```
function q8Change(e: React.ChangeEvent<HTMLInputElement>) {
  setQ8Text(e.target.value);
}

function q8Add() {
  if (q8Text.trim() === "") return; // 06단원 개념05의 공백
  검사 setQ8Items([...q8Items, q8Text]); // 07단원 개념01의 불변 갱신 setQ8Text(""); //
  입력칸 비우기
}

return (
  <div className="output"> <input value={q8Text} onChange=
{q8Change} placeholder="메뉴를 적으세요" /> <button type="button" onClick={q8Add}>
    담기
  </button> <ul>
    {q8Items.map((item, index) => (
      <li key={index}>{item}</li>
    ))}
  </ul> <p>담긴 개수: {q8Items.length}</p> </div>
);
}
```

화면(적고 담기를 누르면): 목록에 한 줄이 늘고 입력칸이 비워집니다.

이벤트 타입 이름은 외워서 적은 것이 아닙니다.

```
onChange={(e) => ...} 처럼 그 자리에 바로 쓴 뒤 e 에 마우스를 올리면
```

React.ChangeEvent<HTMLInputElement> 가 그대로 뜹니다. 그걸 복사한 것입니다.

any 인 채로 두면 아무 밀줄도 안 생깁니다. e.target.valu 라고 오타를 내도, e.tagret 이라고 써도 조용히 통과합니다. 그리고 화면에서 입력이 안 되는 것만 보게 됩니다. any 를 피해야 하는 이유입니다.

목록에 숫자를 넣으려 하면 밀줄이 납니다. q8Items 가 string[] 이기 때문입니다.

```
setQ8Items([...q8Items, 4000]);
```

error TS2322: Type 'string | number' is not assignable to type 'string'.

```
Type 'number' is not assignable to type 'string'.
```

error TS2322: Type 'number' is not assignable to type 'string'.

문제 9 [도전] (개념 02 · 03 · 04)

고른 메뉴를 담은 state 를 만들고, `null` 을 확인하고 씁니다.

```
type Q9Menu = { name: string; price: number };

const q9Menus: Q9Menu[] = [
  { name: "아메리카노", price: 4000 },
  { name: "라떼", price: 4500 },
  { name: "케이크", price: 6000 },
];

function Q9Picker() {
```

“메뉴 하나 또는 아직 없음” — 초기값이 `null` 이라 꺾쇠로 알려 줘야 합니다

```
const [q9Picked, setQ9Picked] = useState<Q9Menu | null>(null);

return (
  <div className="output">
    {q9Menus.map((m) => (
      <button key={m.name} type="button" onClick={() => setQ9Picked(m)}>
        {m.name}
      </button>
    ))}

    {q9Picked === null ? (
      <p>고른 메뉴가 없습니다.</p>
    ) : (
      <p>
        {q9Picked.name} - {q9Picked.price}원
      </p>
    )}
  </div>
);
}
```

화면 처음에는 "고른 메뉴가 없습니다"

화면(라떼를 누르면): 라떼 — 4500원

세 군데가 서로 이어져 있습니다. `useState<Q9Menu | null>(null)` → 답을 그릇의 모양을 정하고
`setQ9Picked(m)` → `m` 이 `Q9Menu` 라서 그대로 들어가고 `q9Picked === null ? ... : ...` → 확인했으니
아래에서 `.name` 을 쓸 수 있습니다

확인을 빼면 이 밑줄이 납니다. error TS18047: 'q9Picked' is possibly 'null'. 09단원에서 "받아오기 전에 화면이 먼저 그려져서" 터지던 사고를 통째로 막아 주는 것입니다.

useState<Q9Menu>(null) 이라고 null 을 빼먹으면 초기값 쪽에서 걸립니다. "아직 없음" 도 담을 수 있어야 하니 | null 을 꼭 같이 적어야 합니다.

onClick={() => setQ9Picked(m)} 에서 화살표로 감싼 이유는 04단원 개념01 그대로입니다. onClick={setQ9Picked(m)} 라고 쓰면 그리는 도중에 실행돼 버립니다.

문제 10 에러 확인

아래는 실제로 npm run typecheck 를 돌려서 나온 메시지를 그대로 옮긴 것입니다. 여러분이 적은 것과 비교해 보세요.

① `const q10a: number = "4000";`

error TS2322: Type 'string' is not assignable to type 'number'. 뜻: 글자를 숫자 자리에 넣었습니다. → 가장 기본이 되는 메시지입니다. "A is not assignable to B" 는

"A 를 B 자리에 못 넣는다" 로 읽으면 됩니다. 앞으로 제일 자주 보게 됩니다.

② `function Q10b({ name }) { ... }`

error TS7031: Binding element 'name' implicitly has an 'any' type. 뜻: 구조분해로 꺼낸 name 이 무엇인지 알 수 없습니다. → props 에 타입을 안 붙인 것입니다. type 을 만들어 붙이면 사라집니다.

implicitly 는 "내가 안 적었는데 저절로" 라는 뜻입니다.

③ `const [items, setItems] = useState([]);`
`setItems(["아메리카노"]);`

error TS2322: Type 'string' is not assignable to type 'never'. 뜻: 아무것도 넣을 수 없는 배열에 글자를 넣으려 했습니다. → 빈 배열에는 볼 것이 없어서 never[] 가 된 것입니다(개념04 섹션2).

"type 'never'" 가 보이면 거의 언제나 useState([]) 입니다.
useState<string[]>([]) 로 고치면 사라집니다.

④ `const q10dLength = q10dUser.nickname.length;`

error TS18048: 'q10dUser.nickname' is possibly 'undefined'. 뜻: 그 값은 없을 수도 있는데 확인 없이 썼습니다. → nickname?: string 이라 string | undefined 입니다.

?? 로 기본값을 주거나 if 로 확인하고 쓰면 사라집니다.
이 밑줄이 막아 주는 것이
TypeError: Cannot read properties of undefined (reading 'length') 입니다.

⑤ e.preventDefault();

error TS2551: Property 'preventDefault' does not exist on type 'FormEvent<Element>'. Did you mean 'preventDefault'? 뜻: 그런 이름의 메소드가 없습니다. preventDefault 를 말한 건가요? → 오타입니다. 맨 뒤에 고칠 이름까지 알려 줍니다.

06단원에서 이 오타를 내면 새로고침이 일어나 입력한 것이 다 날아갔고,
원인을 찾기가 어려웠습니다.

다섯 개의 공통점을 보세요. ①③ 은 종류가 다른 것, ②④ 는 무엇인지 모르거나 없을 수 있는 것, ⑤ 는 오타입니다. 타입이 잡아 주는 것이 딱 이 세 가지입니다. 계산이 틀린 것이나 화면이 이상한 것은 여전히 여러분이 찾아야 합니다.

화면에 그리기

정리

- 1 타입은 정답이 하나가 아닙니다. 화면이 같고 밑줄이 없으면 조금 달라도 맞은 것입니다.
- 2 문제 1·3·5·6 은 '타입만 고치고 값을 안 고쳤을 때' 밑줄이 남습니다. 타입과 값은 함께 고칩니다.
- 3 문제 4·9 는 없을 수 있는 값입니다. ?? 나 삼항으로 먼저 확인하고 써야 밑줄이 사라집니다.
- 4 문제 8 의 any 는 검사를 아예 끕니다. 오타를 내도 조용히 통과합니다. 타입을 적어 두는 편이 낫습니다.
- 5 타입이 잡아 주는 것은 종류 착각 · 모르거나 없을 수 있는 값 · 오타 세 가지입니다. 계산이 틀린 것은 못 잡습니다.


```

export default function Exercise14Answer() {
  return (
    <div>
      <h1>15단원 부록 - 연습문제 정답</h1>

      <p className="guide">
        이 파일은 <code>npm run typecheck</code> 를 통과합니다. 밑줄이 하나도
        없습니다.
      <br />
      문제 10의 에러 메시지는 실제로 돌려서 나온 것을 그대로 옮긴 것입니다.
    </p>

    <div className="demo">
      <h3>문제 1 - 가격이 문자열이면</h3>
      <div className="output">4000 + 500 = {q1Total}</div>
    </div>

    <div className="demo">
      <h3>문제 2 - 배열 타입</h3>
      <div className="output">
        {q2Menu.join(" / ")} ({q2Menu.length}개)
      </div>
    </div>

    <div className="demo">
      <h3>문제 3 - type 별칭에 속성 추가</h3>
      <div className="output">
        {q3Latte.name} - {q3PriceText}원
      </div>
    </div>

    <div className="demo">
      <h3>문제 4 - 선택 속성</h3>
      <div className="output">
        {q4Minjun.name}({q4Minjun.nickname ?? "별명 없음"}) / {q4Seoyeon.name}(
        {q4Seoyeon.nickname ?? "별명 없음"})
      </div>
    </div>
  )
}

```

```

    </div>

    <div className="demo"> <h3>문제 5 – 유니온</h3> <div className="output">내
사이즈: {q5MySize}</div> </div> <div className="demo"> <h3>문제 6 – props 에 타입
붙이기</h3> <div className="output"> <Q6Item name="아메리카노" price={4000}
/> <Q6Item name="라떼" price={4500} /> </div> </div> <div className="demo"> <h3>문제
7 – children</h3> <Q7Box title="오늘의 메뉴"> <p>아메리카노가 제일 잘 나갑니다.
</p> </Q7Box> </div> <div className="demo"> <h3>문제 8 [응용] – 장바구니</h3> <Q8Cart
/> </div> <div className="demo"> <h3>문제 9 [도전] – 메뉴 고르기</h3> <Q9Picker
/> </div>

    <div className="demo">
      <h3>문제 10 – 에러 확인</h3>
      <div className="output">
        이 문제의 답은 코드 위쪽 주석에 있습니다. 실제 메시지를 그대로 옮겨
        뒀습니다.
      </div>
    </div>

  </div>
);
}

```

부 록 나

심화문제 정답

심화문제는 선택 과제입니다. 풀지 않고 넘어가도 다음 단원에 지장이 없습니다.